FMSS M2 hardware reference

iPod nano 4G / S5L8720

19 chapters. Device observations, source evidence, inference and simulation are distinguished throughout. Includes linked contents and chapter/section bookmarks.

Contents

nano 4G FMSS ("M2"): sequencer, NAND controller and ECC engine **10**

Chapters . . . 11

Evidence labels . . . 11

Chapter 1 — Overview and terms **13**

1.1 Evidence classes . . . 13

1.2 Numbering, names and notation . . . 13

1.3 Terms . . . 15

1.4 The three blocks . . . 16

1.5 Named parts of the data path . . . 16

1.5.1 System memory . . . 16

1.5.2 Data buffer 0 and data buffer 1 . . . 16

1.5.3 Parity buffer 0 and parity buffer 1 . . . 17

1.5.4 Slot . . . 17

1.5.5 Chunk . . . 17

1.5.6 ECC area . . . 17

1.5.7 Metadata registers . . . 17

1.5.8 FIFO port (FMC_FIFO) . . . 17

1.5.9 Status register . . . 18

1.5.10 Chip page . . . 18

1.5.11 What the ECC engine touches . . . 18

1.6 Scope of the device evidence . . . 18

Chapter 2 — Architecture: execution model, start, stop and fault handling **19**

2.1 How the sequencer is started and stopped . . . 19

2.2 Execution model . . . 21

2.3 END and fault . . . 21

2.4 Start word behaviour . . . 22

2.5 Execution and time . . . 23

2.6 Instruction completion versus peripheral data . . . 23

Chapter 3 — The register window as the ARM CPU sees it **24**

3.1 Offsets and names . . . 24

3.2 Memory map: the register window . . . 25

3.3 CPU-only rules . . . 29

3.4 CPU-visible interrupt lines . . . 29

3.5 Starting and finishing a run . . . 30

3.6 Instruction and wait behaviour seen through the window . . . 30

Chapter 4 — The register window as the sequencer sees it **32**

4.1 Window offsets and target widths . . . 32

4.2 Segments and mirrors, sequencer view . . . 32

4.3 Register file r0..r7, PC and extended register roles . . . 33

4.4 Parameter RAM . . . 34

4.5 System memory and the program . . . 34

4.6 What the sequencer sees differently from the CPU . . . 34

Chapter 5 — Instruction encoding **36**

5.1 Encoding . . . 36

5.2 Operand map . . . 36

5.3 Opcodes . . . 38

5.4 Unassigned opcodes . . . 39

5.5 ALU operand rule . . . 39

5.6 Branch and jump targets . . . 40

5.7 Register write ordering . . . 40

5.8 Instruction cost . . . 40

5.9 Byte order . . . 40

Chapter 6 — Opcode reference **42**

6.1 Conventions and measurement notes . . . 42

6.2 Opcodes . . . 42

00 end . . . 42

01 wr . . . 42

02 st . . . 43

03 ldi . . . 43

04 ld . . . 44

05 movi . . . 44

06 mov . . . 44

07 wait . . . 45

08 nop . . . 46

09 wait2 . . . 46

0a and . . . 47

0b or . . . 47

0c add . . . 48

0d sub . . . 48

0e bnz . . . 48

0f jr . . . 49

10 sig . . . 49

11 sti . . . 49

12 pause . . . 50
13 shl . . . 50
14 shr . . . 50
15 tmwait . . . 51
16 evjmp . . . 51
17 bz . . . 52
18 pld . . . 52
19 pst . . . 53
1a..fe and other values without an established meaning . . . 53
ff nop2 . . . 53
6.3 Test coverage . . . 54

Chapter 7 — Events, waits and the timeout **55**

7.1 The wait instruction . . . 55
7.1.1 Idle behaviour . . . 55
7.2 Wait events . . . 56
7.3 Consumption and acknowledgement . . . 57
7.4 Wait event 4 (ECC) . . . 57
7.5 Wait event 5 and the status latch . . . 58
7.6 Wait event 12 (masked status) . . . 59
7.7 wait2 and the interrupt request lines . . . 59
7.8 Events that nothing releases . . . 60
7.9 Timeout length . . . 61
7.10 Other event-like opcodes . . . 62

Chapter 8 — Sequencer control registers **64**

8.1 Sequencer control block . . . 64
8.2 FMSS_CTRL (0xc00): start word and control . . . 65
8.3 FMSS_PROG (0xc04) . . . 66
8.4 CS_STATUS (0xc08) . . . 66
8.5 CS_IRQ (0xc0c) . . . 67
8.6 FMSS_C10 (0xc10) . . . 67
8.7 FMSS_C14 (0xc14): counter/timer . . . 68
8.8 r0 to r7 (0xc18 to 0xc34) . . . 68
8.9 FMSS_PC (0xc38) . . . 68
8.10 FMSS_C3C (0xc3c) . . . 69
8.11 0xc40 and 0xc44: the held instruction . . . 69
8.12 0xc48 and 0xc54 . . . 69
8.13 FMSS_C4C (0xc4c): the fault vector . . . 69
8.14 FMSS_C50 (0xc50): completion bits . . . 69

8.15 FMSS_C58 (0xc58): fault routing . . . 70
8.16 FMSS_C5C (0xc5c) . . . 70
8.17 Buffer handshake (0xc60 to 0xc6c) . . . 70
8.18 Parameter RAM (0xd00 to 0xd7c) . . . 71
8.19 Program lifecycle . . . 71
8.20 pause (12) and CPU resume . . . 71
8.21 Reset . . . 71
8.22 States that block the reset and their release . . . 72

Chapter 9 — NAND controller, buffers and DMA **74**

9.1 Register map . . . 75
9.2 FMC_CTRL0 (0x000): configuration word . . . 78
9.3 FMCTRL1 (0x004): operation codes . . . 78
9.4 Command, address and data transfer . . . 79
9.5 DMA . . . 79
9.6 Status latch and FMC_NSTAT (0x04c) . . . 83
9.7 FMC_STATUS flags . . . 84
9.8 Interrupt . . . 85
9.9 FIFO and READ ID . . . 86
9.10 The +0x400 range . . . 86
9.11 Bus timing . . . 87
9.12 The 0x100 block . . . 87

Chapter 10 — ECC engine and codeword layout **88**

10.1 How error correction works . . . 88
10.2 Registers . . . 89
10.3 ECC_CTRL (0x804) . . . 90
10.4 Stream on the chip . . . 90
10.5 The codes . . . 90
10.5.1 BCH8 . . . 90
10.5.2 BCH16 . . . 91
10.6 Encode and the parity registers . . . 91
10.7 Decode and correction . . . 91
10.8 Error locations (0x820..) . . . 92
10.9 The result ring (0x80c) . . . 92
10.10 ECC_STATUS (0x810) . . . 93
10.11 Corrected totals (0x818) . . . 93
10.12 WAIT4 and acknowledgement . . . 93
10.13 Timing . . . 94
10.14 Data-path hazard . . . 94

10.15 Interrupt . . . 94
10.16 Limits of the evidence and the model . . . 94

Chapter 11 — Original M2 programs and operation sequences 96

11.1 The original programs . . . 96
11.2 Reading a page (program READ) . . . 96
11.3 End and fault results . . . 97
11.4 Completion and acknowledgement rules . . . 97
11.5 Reset, READ ID, program and erase as stimuli . . . 98
11.6 nano4g and nano5g . . . 99

Chapter 12 — CPU interface and software ABI 100

12.1 Parameter words (READ / READID) . . . 100
12.2 Parameter lifetime and reset . . . 100
12.3 Completion and coherency . . . 101
12.4 Result layers . . . 101
12.5 Reset verification and recovery . . . 101
12.6 Recovery diagnostic records . . . 102
12.7 Controller access by the CPU alone . . . 102

Chapter 13 — M2 experiment coverage 104

13.1 Additional matrices . . . 115

Chapter 14 — Evidence: outcomes 120

14.1 Physical capture: p06 . . . 120
14.2 Selected observed values . . . 124
14.3 Original-code observations . . . 125
14.4 Emulated checks . . . 125
14.5 Physical capture: p07 . . . 125
14.6 Physical capture: p09 . . . 129
14.7 p10: M1 register-role comparisons on M2 . . . 130
14.8 p11: full selector comparisons and source-10 difference . . . 130
14.9 p12: source 10 and isolated reset phases . . . 130
14.10 P13: full attempt, state-b reset failure . . . 130
14.11 P14: no candidates execute; forced reset was skipped . . . 130
14.12 P15: effective reset and independent execution . . . 130
14.13 P16: full campaign with effective recovery . . . 130
14.14 P16 offline re-analysis . . . 131
14.15 P17: DMA acknowledgement and overlap . . . 131
14.16 P18: timeout, +400 words, BCH8 correction . . . 131
14.17 P19: ECC status, BCH16, status during busy . . . 131

14.18 P20: result ring and BCH16 correction . . . 132
14.19 P21: ECC status bits, WAIT4 release, ECC_CTRL . . . 132
14.20 P22: decode WAIT4, BCH16 bit 5, ECC_CTRL, timing, FMC 018/05c . . . 132
14.21 P23: jump targets and WAIT vs ready/busy . . . 132
14.22 P24: register hazards and JR . . . 132
14.23 P25 bundle . . . 133
14.24 P26 bundle (partial) . . . 133
14.25 P43 bundle . . . 133
14.26 Reboot sessions: unmapped bus accesses . . . 134
14.27 P48 bundle (round 6) . . . 134
14.27.1 Harness correction and model evidence . . . 135
14.28 P49 bundle . . . 135
14.29 P50: clock-gate scan . . . 135
14.30 P51 and the p50/p51 journals . . . 136

Chapter 15 — Interpreting M2 evidence 137

15.1 Required distinctions . . . 137
15.2 Order and state contamination . . . 137
15.3 M1 comparison discipline . . . 137
15.4 P16 examples of interpretation boundaries . . . 138
15.5 P43 examples of interpretation boundaries . . . 138

Chapter 16 — Physical M2 findings through P43 139

16.1 What the completed campaign contains . . . 139
16.2 Execution chronology and pause (12) . . . 139
16.3 Positive architectural observations . . . 140
16.4 Results that cannot answer their labels . . . 140
16.5 p07: pause recovery and the next failure . . . 141
16.5.1 pause cleanup and CPU resume work in the measured controls . . . 141
16.5.2 tmwait, faults and event controls . . . 141
16.5.3 First failure: status-latch experiment, then reset . . . 142
16.6 Reproduction after p07 (historical model hypothesis) . . . 142
16.7 p09: status recovery, opcode controls and communication loss . . . 142
16.8 p10: M1 register-role comparisons on M2 . . . 143
16.9 p11: full selector comparisons and source-10 difference . . . 143
16.10 p12: source 10 and isolated reset phases . . . 143
16.11 P13: full attempt, state-b reset failure . . . 143
16.12 P14: no candidates execute; forced reset was skipped . . . 143
16.13 P15: effective reset and independent execution . . . 144
16.14 P16: all families execute; data-path questions remain . . . 144

16.15 P16 offline: serial DMA and BCH8 settled from retained records . . . 144
16.16 P17: acknowledgement confirmed; same-slot overlap corrupts . . . 144
16.17 P18: timeout rule, +400 words, BCH8 radius . . . 144
16.18 P19: BCH16, status byte and ECC status interface . . . 144
16.19 P20: result ring and BCH16 radius . . . 144
16.20 P21: ECC status bits and WAIT4 . . . 145
16.21 P22: WAIT4 sources, engine clock . . . 145
16.22 P23: branch targets, JR hazard, no ready/busy WAIT . . . 145
16.23 P25: JR, flags, DMA timing, prescaler, parity registers . . . 145
16.24 P43: round 5 and the CPU-only NAND path . . . 145
16.25 Reboot sessions: unmapped bus accesses . . . 145
16.26 P48: round 6 . . . 145
16.27 P49: CPU-issued command sequences . . . 145
16.28 P50: clock gates and VIC0 line 5 . . . 146
16.29 P51: gate3 request and entry history . . . 146

Chapter 17 — Open questions **147**

17.1 Instruction set and sequencer behaviour . . . 148
17.2 Events and the status latch . . . 150
17.3 Interrupts . . . 151
17.4 NAND controller registers . . . 152
17.5 ECC engine . . . 153
17.6 Original programs and the CPU interface . . . 154
17.7 Reset stages . . . 154
17.8 Settled since the first M2 survey . . . 154

Chapter 18 — M2 finding and documentation ledger **159**

18.1 P06 documentation topic review (historical) . . . 160
18.2 p07 additions . . . 161
18.3 p09 additions and corrections . . . 161
18.4 p10: qualified comparisons with M1 . . . 162
18.5 p11: M2 selector table comparison . . . 163
18.6 p12: resolved source-10 and reset questions . . . 164
18.7 p13: evjmp and the reset failure . . . 164
18.8 P14: status checks versus clearing a blocked instruction . . . 164
18.9 P15: reset while held and independent execution . . . 165
18.10 P16: full execution and qualified hardware findings . . . 166
18.11 P16 offline re-analysis . . . 167
18.12 P18 . . . 168
18.13 P19 . . . 168

18.14 P20 . . . 169
18.15 P21 . . . 169
18.16 P22 . . . 170
18.17 P23 . . . 170
18.18 P24 . . . 170
18.19 P25 . . . 171
18.20 P26 . . . 171
18.21 P28 to P36 . . . 172
18.22 P37 and the power-on capture . . . 185
18.23 P38 and P39 . . . 195
18.24 P42 . . . 201
18.25 P43 . . . 207
18.26 Reboot sessions . . . 213
18.27 P48 . . . 214
18.28 P49 . . . 216
18.29 P49 gate completion and worker dispatch . . . 217
18.30 P50 . . . 218
18.31 P51 journals and checkpoint validation . . . 219

Chapter 19 — Differences between M1 and M2 220

19.1 Substantive differences . . . 220
Instruction set . . . 220
Registers and sources . . . 221
Events and waits . . . 221
Interrupts and wait2 . . . 222
Control registers and reset . . . 222
NAND controller . . . 223
ECC . . . 223
Timing . . . 224
Original programs and CPU interface . . . 224
19.2 Finding by finding . . . 225
19.3 Counts . . . 271
19.4 Experiment keys . . . 272

nano 4G FMSS ("M2"): sequencer, NAND controller and ECC engine

These documents describe the FMSS engine of the iPod nano 4G (S5L8720): the sequencer coprocessor inside the NAND controller, the NAND controller (FMC) it drives, and the ECC engine. They cover the engine's behaviour, registers, instruction set, memory maps, ECC, events, timing and interfaces, and how the original firmware uses it. Each statement about the engine carries its evidence class and a finding ID.

The documents form a book of numbered chapters with the same chapter numbers, section structure, mnemonics and operand notation as the M1 book of the nano 3G (S5L8702). Chapter 1 defines the evidence classes, the numbering and the terms (conventions). Chapters 2 to 12 are the reference chapters that describe the engine. Chapter 13 describes the experiments. Chapters 14 to 16 hold the evidence: the outcomes, their interpretation and the device findings. Chapter 17 lists the open questions, chapter 18 is the finding list with the `M2-nn` IDs cited throughout, and chapter 19 compares M2 with M1 finding by finding.

The compiled book FMSS_M2.pdf contains the same 19 chapters with a linked contents page and chapter/section bookmarks.

Chapters

chapter	title	content
1	Overview and terms	evidence classes, numbering, terms, the three blocks, the named parts of the data path, scope of the device evidence
2	Architecture: execution model, start, stop and fault handling	start, pause, hold, STOP, abort and reset, state codes, execution model, END and fault, start word, timing, completion versus data
3	The register window as the ARM CPU sees it	offsets and names, memory map, CPU-only rules, interrupt lines, how the original CPU driver runs a program, wait sources
4	The register window as the sequencer sees it	window offsets and widths, segments and mirrors, register file and extended register roles, parameter RAM, system memory, program fetch
5	Instruction encoding	instruction word layout, operand map, opcode summary table, ALU operand rule, byte order
6	Opcode reference	one entry per established or candidate mnemonic, plus unclassified opcode values
7	Events, waits and the timeout	`wait` events, `wait2` conditions, `tmwait` and `evjmp`, `C14`, the timeout length
8	Sequencer control registers	the control block `0xc00..0xc6c` and the start-word bits
9	NAND controller (FMC)	`FMC_CTRL0`, the `FMCTRL1` codes, the other registers, `FMC_STATUS`, buffers, DMA, the `+0x400` range, the `0x100` block
10	ECC engine	codeword layout, `ECC_CTRL`, the BCH8 and BCH16 codes, correction, status, result ring and error locations
11	NAND operations	how the original firmware performs reset, ID read, page read, page program, block erase and fault handling
12	CPU interface: parameter words, result words and timing word	parameter words and their lifetime, completion and coherency, result layers, reset verification and recovery
13	Experiments: what was done and why	the experiment families, their purposes and their M1-to-M2 coverage
14	Evidence: outcomes	the outcomes of the device experiments and of the reverse engineering of the original firmware
15	Evidence interpretation	what the outcomes are taken to show, by topic
16	Device findings: risks and what only a device can settle	the risks and their status, and the results settled on a device
17	Open questions	every point marked open, not measured or unexplained
18	Finding list	M2 findings with evidence and class
19	Differences between M1 and M2	every M1 finding: on M1 / on M2, verdict and evidence

Evidence labels

- **Device / device-observed:** a physical capture with execution and initial state qualified for the case.

- **Observed / source-observed:** an original-code finding with image and address, from the Ghidra analyses; not a physical measurement.
- **Inferred:** an explanation or mapping supported by the stated observations.
- **3p:** third-party driver evidence, with its reported testing scope.
- **Open:** not settled by the available M2 evidence.
- **Emulated:** behaviour of the named model and build; no claim about unobserved silicon behaviour or calibrated hardware timing.

M1 results are comparison hypotheses on M2 until an M2 measurement confirms them; the M1 channel-1 routing, power-on masks and RS modes are not imported.

Chapter 1 — Overview and terms

Summary: what the M2 FMSS engine of the S5L8720 (iPod nano 4G) is, the evidence classes and numbering used in this book, the terms, the three hardware blocks (sequencer, NAND controller (FMC), ECC engine) and the named parts of the data path.

The M2 FMSS engine is the NAND hardware of the S5L8720: a small sequencer coprocessor that runs programs of 8-byte instructions, the NAND controller (FMC) it drives, and the ECC engine. All three share one register window at `mem:0x38a00000`.

1.1 Evidence classes

The statements in the chapters rest on three sources:

- static analysis of the original programs and of the original CPU driver in the bootloader (`bl_NandReadOnly`), the restore OS (`fw_restore`) and the operating system (`osos`);
- device tests: small programs run on the sequencer of one nano 4G unit (chapter 13, outcomes in chapter 14);
- the M2 simulator, which runs the sequencer programs and the actual ARM driver and probe code.

Each statement carries one of the following evidence classes:

- **observed** (source-observed): seen in the original programs or in the original CPU driver, with image and address.
- **device** (device-observed): measured on the unit, with execution and initial state qualified for the case.
- **inferred**: an interpretation of observed use or of device results.
- **3p**: a third-party driver, reported by its authors as tested.
- **open**: not settled by the available M2 evidence.
- **emulated**: behaviour of the named model and build; it makes no claim about unobserved silicon behaviour or calibrated hardware timing.

The finding list (chapter 18) abbreviates these as D (device), S (original-code use), I (interpretation) and M (model validation). An M1 measurement does not become an M2 device result because its name is retained: M1 results are hypotheses on M2 until an M2 measurement confirms them (for example, the 78 M1-comparison expectations confirmed only for their tested forms and states, M2-34, M2-65).

1.2 Numbering, names and notation

Chapters are numbered 1 to 19 and headings inside chapter N are numbered N.1, N.2, N.2.1 and so on, as in the M1 book. The entries of chapter 6 (opcode reference) are the exception: each heading is the hexadecimal opcode and its lowercase mnemonic, without a section number. A section number identifies the M2 organisation; it does not imply that the section has the same contents as the M1 section with the same number.

Established mnemonics are retained for matching operations: `end`, `wr`, `st`, `ldi`, `ld`, `movi`, `mov`, `wait`, `nop`, `and`, `or`, `add`, `sub`, `bnz`, `sti`, `pause`, `shl`, `shr`, `bz`, `pld` and `pst`. `wait2`, `jr`, `sig`, `tmwait`, `evjmp` and `nop2` are the names of their M1 comparison candidates; unresolved M2 behaviour is stated separately. A mnemonic is not a claim that every operand, timing or side effect is identical across generations.

Conventions used throughout:

- Opcode values are hexadecimal (12 means `0x12`), matching the M1 tables. Event numbers, register indices and bit numbers are decimal unless prefixed.
- Bits are written "bit N" or "bits H:L", high bit first. Register offsets and values are hexadecimal. `C58`, `C4C` and `C14` abbreviate the window offsets `0xc58`, `0xc4c` and `0xc14`; they are not absolute CPU addresses.
- `FMSS_CTRL`, `FMSS_PROG`, `CS_STATUS`, `CS_IRQ`, `SEQ_RESULT` and `FMSS_PC` name the corresponding control registers (chapter 8).

• Instruction fields are R, `arg` and `imm`. rR names register R, rA the register selected by arg, `[REG]` a window word and `#imm` an immediate. Branch targets and the PC are byte offsets from the program start.

• A `wait` **event N** and a `wait2` **condition N** belong to separate selector spaces; the same N in both names unrelated things.

• Times are in us and ms; "about" or "≈" marks a measured approximation. Raw capture values keep their original notation.

• "Original firmware", "original programs" and "original CPU driver" name Apple's firmware, its FMSS programs and its ARM NAND driver. FMSS is the firmware's own name; an expansion of the acronym is not established.

• Source locations distinguish `file:<image>+0xOFF` from `mem:0xADDR`.

• M2 findings are cited by their stable `M2-nn` IDs from chapter 18; M1 findings by `M1-Fnnn`. Experiment outcomes are in chapter 14.

1.3 Terms

term	meaning	source of the name
NAND controller (FMC)	the SoC's flash memory controller at `0x38a00000`. It produces command, address and data cycles on the NAND bus and contains the data buffers and the DMA (chapter 9). `0x400..0x7fc` reads 0 (M2-155) and holds no second register set: writes are not stored (M2-82), no `FMC_CTRL0` bit enables it (M2-139), and commands and DMAs through it do nothing (M2-183, M2-185)	register names of the SoC family (`FMCTRL0`, `FMCTRL1`, ...)
FMSS engine	the whole NAND hardware described here: sequencer, NAND controller and ECC engine (section 1.4)	the original firmware's own name
sequencer	the coprocessor inside the NAND controller that runs programs of 8-byte instructions (chapter 5)	name used in these documents
M1 engine	the engine of the nano 3G (S5L8702), described in the M1 book	name used in these documents
M2 engine	the engine of the nano 4G (S5L8720) described here; the nano 5G firmware carries the same programs	the 4G/5G firmware
ECC engine	error-correcting code hardware at offset `0x800` that generates BCH8/BCH16 parity for 512-byte sectors and corrects them (chapter 10)	name used in these documents
program	M2 microcode, not ARM code	name used in these documents
window offset	a register address relative to `0x38a00000`	name used in these documents
CE	chip enable: the signal that selects one NAND chip (`FMC_CTRL0` bits 8:1)	NAND standard
CS_IRQ	the sequencer's flag word at `0xc0c`: bit 0 END, bit 1 `sig` (M2-34, M2-65), bit 2 fault, bit 3 pause, bit 4 routed FMC flag (M2-200)	the original firmware calls it `FC_CS_IRQ`; "CS" is not expanded
CS_STATUS (`C08`)	the sequencer's state code at `0xc08` as the CPU reads it (0 idle, 4 waiting, a paused, 1, 2 or 9 while a program runs, others in chapter 2)	name used in these documents
event	one of the status flags that `wait` waits for, selected by a number: "`wait` event N" (chapter 7)	name used in these documents
condition	the number N of `wait2`: VIC0 hardware interrupt line N (chapter 3, section 3.4; M2-140)	name used in these documents
VIC0, VIC1	the CPU's two vectored interrupt controllers	SoC register names
parameter RAM	32 words at `0xd00..0xd7c` that the CPU and the program both access; they hold a program's inputs and outputs (chapter 4, section 4.4; M2-03)	name used in these documents
sector	the 512-byte ECC and data unit of the original programs	name used in these documents
slot	one of the two ECC/buffer selections the original programs alternate (section 1.5.4)	name used in these documents
chunk	the 2 KiB DMA grouping of the original driver (section 1.5.5)	name used in these documents
page	the NAND transaction unit	NAND standard

Metadata, parity, the corrected-sector status, the sequencer IRQ flags and a software return value are different objects; none is another name for the same status.

1.4 The three blocks

The NAND hardware at `0x38a00000` has three parts that share one register window (chapter 3, section 3.2):

block	registers	what it does	who drives it
sequencer	control block `0xc00..0xc6c`, registers `0xc18..0xc34`, parameter RAM `0xd00..0xd7c` (chapter 8)	runs a program of 8-byte instructions fetched from system memory	the CPU starts and stops it through FMSS_CTRL (`0xc00`)
NAND controller (FMC)	`0x000..0x0fc` (chapter 9)	puts commands, addresses and data on the NAND bus and moves data by DMA	whoever writes its registers: the sequencer program or the CPU
ECC engine	`0x800..0x8fc` (chapter 10)	generates BCH parity for 512-byte sectors, checks and corrects them, and reports error counts and locations	the same: the sequencer program or the CPU

The sequencer does no NAND work itself. Its program writes the NAND controller's and the ECC engine's registers and waits for their completion flags (`wait` events; M2-19). How the sequencer is started and stopped, and how the CPU and the sequencer interact, is in chapter 2. The original programs (RESET, READID and READ in the bootloader; WRITE and ERASE in the restore OS) are described in chapter 11.

1.5 Named parts of the data path

Data passes through the following parts between system memory and a NAND chip. "Device" marks what was measured and "inferred" what follows from it.

The data and parity buffers are storage inside the NAND controller. M2 evidence establishes no window word that reads or writes a data buffer directly. Transfers fill and empty the buffers: chip transfers and DMAs of the NAND controller (FMCTRL1), ECC operations, and the FIFO/metadata latch path (`0x040`). The ECC engine exposes two results as registers: the parity of the last encode at `0x840..0x85c` (M2-116) and the error locations of the last decode at `0x820..` (M2-128).

1.5.1 System memory

- **What:** the SoC's SDRAM and internal SRAM on the CPU bus, addressed with 32-bit bus addresses.
- **Used by the FMSS engine for:** the program (fetched from the bus address in FMSS_PROG, `0xc04`), `ldi`/`sti` data, and the data of every DMA (bus address in FMC_DMA_ADDR, `0x034`).
- **Details:** words are little-endian and `ldi`/`sti` addresses align down to a multiple of 4 (device, M2-05), as do DMA RAM addresses in both directions (device, M2-664). SRAM and SDRAM accesses cost the same in the measured forms (M2-102). Chapter 4, section 4.5.

1.5.2 Data buffer 0 and data buffer 1

- **What:** the 512-byte sector buffers between the NAND bus, the ECC engine and the DMA.
- **Selected by:** `0x014`; the original READ program alternates the selections `0x102` and `0x201` and uses `0x10` for the FIFO (observed; meaning of the fields inferred). `0x014` stores 16 bits; `0x058` is its bits 7:0 and `0x05c` reads its bits 15:8 in place, so a later `0x058` write replaces the low byte (device, M2-83, M2-643). A DMA in writes data buffer 0 only with `0x014` = 1; with 0 and the other tested non-source values it completes and writes no buffer (device, M2-668).
- **Filled and emptied by:** NAND data transfers and DMA. Memory → buffer → memory DMA is byte exact for both selections (device, M2-76). A DMA moves 512 bytes whatever FMC_DNUM holds (device, M2-189). A DMA in flight follows a rewrite of `0x014` and of FMC_DMA_ADDR at the same byte offsets (device, M2-603).

1.5.3 Parity buffer 0 and parity buffer 1

• **What:** the storage for metadata and parity that travels with a sector, separate from the 512-byte data buffer (observed in the original programs).

• **Selected by:** `0x014` = `0x10` for a DMA (device, M2-232) and the ECC slot (section 1.5.4). The original programs also write the transfer word `0x058` (values 1, `0x10`, `0x20`; observed); `0x058` is `0x014` bits 7:0, so such a write sets the low byte of the selection, and a later `0x014` write replaces it (device, M2-597, M2-643).

• **Contents:** after an encode, the parity is readable at `0x840..0x85c` with the word order reversed and big-endian words (device, M2-116).

• **Size:** a 512-byte DMA into parity buffer 0 (`0x014` = `0x10`) leaves the source's last 64 bytes, which a DMA out returns repeated over 512 bytes and a 128-byte program writes twice (device, M2-232): the buffer wraps at 64 bytes (inferred).

1.5.4 Slot

• **What:** the ECC buffer index 0 or 1 in bit 0 of `0x01c`; `0x01c` stores only that bit, and an encode uses the data buffer of the slot it selects (device, M2-83, M2-417).

• **Behaviour:** the original pipeline uses two sector/ECC selections (observed, M2-21). Of two encode kicks back to back the second replaces the first, and changing `0x01c` during an encode gives a parity that matches neither slot (device, M2-418, M2-419). Serial DMA/ECC sequences and overlap on different slots are byte exact for BCH8 and BCH16. A DMA into or out of a buffer while its own ECC operation is running loses bytes in runs of 4..32 bytes and sets `FMC_STATUS` bit 25 (device; meaning of the flag inferred, M2-80, M2-399, M2-401).

1.5.5 Chunk

• **What:** the original driver's 2 KiB DMA grouping: one DMA pointer per 2 KiB in parameter word `d20`, and `d28` = sectors per page divided by four (chapter 12). The ECC engine works on 512-byte sectors. A chunk is a software transfer grouping, not a physical NAND page.

1.5.6 ECC area

• **What:** the metadata and parity bytes stored with the sectors of a page.

• **Layout used by the original driver:** for sector 0, ten metadata bytes, the parity and the 512 data bytes; for each further sector, parity and data. BCH8 uses 13 parity bytes per sector and BCH16 26 (observed, M2-22): 114 metadata/parity bytes per 4 KiB page with BCH8, 218 with BCH16 (chapter 10, section 10.4).

• **Codeword:** the sector-0 message is the data followed by the metadata bytes, MSB first (device, M2-77, M2-86). `ECC_CTRL` bits 13:8 set the metadata length; bits 15:14 are ignored (device, M2-98, M2-101, M2-191, M2-197). A length up to 32 covers that many stream bytes (the twelve bytes of `0x060..0x068`, then `0xff`); 33 and above give the parity of the data alone, as length 0 (device, M2-662).

• **Not a window range:** the M1 ECC register area `0x900..0x9fc` reads 0 on M2 (device, M2-171).

1.5.7 Metadata registers

• **What:** `0x060`, `0x064` and `0x068`, the FIFO/metadata data words, loaded by latch commands written to `0x040`: `0x52` ID latch, `0xc2` corrected metadata latch, `0xc1` metadata input (observed).

• **Device:** a READ ID through latch `0x52` appears in `0x060`/`0x064`; the ID repeats after five bytes (M2-124).

1.5.8 FIFO port (`FMC_FIFO`)

• **What:** `0x080`, a CPU read port. The restore OS reads it repeatedly and each read consumes FIFO data (observed, `fw_restore` `mem:0x0802a640`).

• **Device:** after a READ ID through latch `0x52`, `0x080` reads 0 (M2-124). Sequencer reads return 0 from DMA-filled data and parity buffers (M2-207). CPU reads return page data after `FMCTRL1` = `0x1c2` (chip read into the buffer) and `0x340`, 32 bytes per transfer (M2-436). Sequencer writes to it never reach a data buffer: without `0x340` they leave the buffer unchanged (M2-207); `0x340` with `0x014` = 1 writes 512 zero bytes to `FMC_DMA_ADDR`

and is followed by a transfer of the RAM after that output into data buffer 0, which a page programmed from the buffer shows, and no written value appears (M2-341, M2-617).

1.5.9 Status register

• **What:** `0x04c`, the latched NAND chip status, distinct from the sequencer's `CS_IRQ` and from the controller's completion flags at `0x048`.

• **Device:** after one status latch, `0x04c` follows the chip: `0x80` while busy, `0xe0` when ready; without a latch it is not updated. With no CE it reads `0x70`, the command echo (M2-87, M2-136).

1.5.10 Chip page

• **What:** the NAND page as stored on the chip: data sectors and the ECC area (section 1.5.6). The measured chip uses 4 KiB pages with a 128-byte spare (section 1.6).

1.5.11 What the ECC engine touches

• **Encode:** reads the selected buffer's data and metadata and produces parity, readable at `0x840..0x85c` (M2-116).

• **Decode:** corrects data, metadata and parity: up to 8 bit errors with BCH8 and up to 16 with BCH16; beyond that the data is left unchanged (device, M2-84, M2-92). Error locations are numbered over data (0..4095), then metadata and parity (M2-132, M2-138); per-decode results go to the 8-slot ring at `0x80c` (M2-91).

1.6 Scope of the device evidence

The device results come from one nano 4G unit with the Hynix chip `AD D5 14 B6` (M2-124), 4 KiB pages with a 128-byte spare and BCH8 in the original format. The 4 KiB/eight-sector programs and the CE0 and BCH8 defaults of the tests are a workload; they do not establish the maximum page size, CE count, address width, supported ECC modes or pipeline capacity of the engine.

Three kinds of behaviour are kept apart:

layer	covers
sequencer	operand rules, register widths and aliases, arithmetic, branches, RAM access, PC, faults, pause and resume, reset and start state
SoC integration	WAIT sources, interrupt routing and acknowledgement, DMA ordering, buffers, ECC commands, clocks
attached NAND	ID, address cycles, ready/status behaviour, page layout, program and erase response

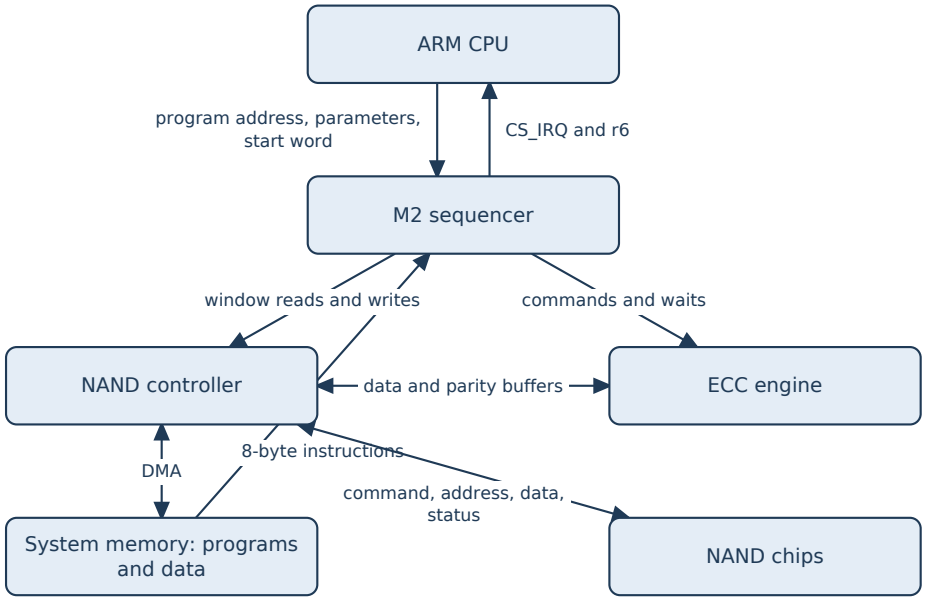
NAND chip latency is not instruction latency. Shared Apple programs do not by themselves make the nano 4G and nano 5G register maps, clocks, interrupts or ISA edge cases identical; each finding names the SoC it was observed on.

A full execution of all 172 families (31,001 sampled native records) had no setup, reset or cleanup failure (M2-64), and a later full execution of all families reproduces the earlier captures with 79 of 31,974 outcomes differing (M2-144). One capture begins at a power cycle, so its first access reads the power-on window (M2-430); every other case starts from the ordinary reset. Execution does not decode every tested word; hardware faults and observation timeouts are expected outcomes of discovery cases. A program that reaches END does not certify the bytes its peripherals moved (M2-71, M2-78). The device findings and their limits are in chapter 16, their interpretation in chapter 15 and the open points in chapter 17.

Chapter 2 — Architecture: execution model, start, stop and fault handling

Summary: how the sequencer runs a program, how the CPU starts, pauses, holds, stops and resets it, what happens at END and at a fault, and how instruction completion relates to peripheral data. The three blocks are described in chapter 1, section 1.4.

M2 is a small programmable sequencer attached to the NAND controller of the S5L8720. It executes its own instruction format, not ARM instructions. The ARM CPU supplies the program and the parameters; the sequencer drives the NAND controller, the DMA and the ECC engine and reports completion through registers (observed in the original CPU driver and programs, chapter 14, section 14.3).



2.1 How the sequencer is started and stopped

Starting (original CPU driver, `bl_NandReadOnly` `mem:0x51b8`; details in chapter 3, section 3.5):

1. The engine is reset (section "Stopping" below), then the parameters are written to parameter RAM and the program is in system memory. A reset after the parameters are written clears them (M2-54, M2-55).

2. `FMSS_PROG` (`0xc04`) = the program's bus address; `CS_IRQ` = `0xff`.

3. Buffer handshake: `c6c` = 0, `c60` = 1, wait for `c64` ≠ 0, `c60` = 0; or `c6c` = `0x10` on the alternate path without the handshake (observed, M2-20).

4. `FMSS_CTRL` (`0xc00`) = `0xffb5` (bootloader) or `0xfff5` (restore OS). Bit 0 starts the program at its first instruction; r0..r7 keep the values the CPU wrote (device, M2-02, M2-157). The first start word after the ordinary reset runs the program even when its instruction 0 is not END (M2-151), and so does the first start after power-on (M2-431).

Stopping:

- **END:** the program executes end. `CS_IRQ` bit 0 is set and r6 is visible as `SEQ_RESULT` at `0xc30` (device, M2-02).
- **Fault:** an unmet `wait` or an illegal opcode sets `CS_IRQ` bit 2 after the timeout; the sequencer then vectors to `C4C` or continues, as `C58` bit 2 selects (M2-11, section 2.3). The program must still reach end.
- **STOP (`C00` = 8):** from a running program, STOP sets PC = 0 and `C08` = 0 and preserves all eight GPRs, all 32 parameter words, `C10`, the program base, the instruction latches and the fault vector (device, M2-53, M2-54).
- **Abort (`C00` = 2):** the original reset writes it when `C08` = 4 and waits for `C08` = 0 or 3 (observed, `mem:0x502c`).

• **Clearing step (C00 = 0x80000):** alone, after STOP or abort, or followed by either, it clears the GPRs, the parameter RAM and C04, C10, C40, C44, C4C and leaves C00 = 0x7f60 (0 when STOP follows); STOP, abort and STOP then abort clear none of them, and no C00 sequence changes FMC 0x00c..0x05c or ECC_CTRL (device, M2-54, M2-422). The complete reset sequence leaves FMC 00c..05c, ECC_CTRL and the parameter RAM zero and FMC_DMA_CTRL = 7 (M2-123): FMCTRL1 bit 31 zeroes the FMC words and sets FMC_DMA_CTRL to 7, 0x81c = 1 zeroes ECC_CTRL, and FMCTRL1 0x800 changes none of them (M2-645). The per-case reset also sets the phase of the timeout prescaler (M2-631). Separate buffer, FMC and ECC reset steps preserve GPR and parameter values (M2-54).

• **Pause (**pause**, opcode** 12**):** the two measured forms stop at the instruction with C08 = a, CS_IRQ = 8 and PC at the pause (M2-14). The original CPU handler advances PC by 8, acknowledges bit 3 and sets C00 bit 0 (observed, mem:0x4d08); with these CPU writes the program continues to the following instruction and END (device, M2-25). Setting FMSS_PROG to an END program, PC = 0, acknowledging and resuming ends a paused run so that ordinary reset completes; a pause inherited from an earlier run needs the abort in addition (device, M2-26).

• **Held by the buffer request:** a CPU write C60 = 1 while a program runs holds the engine: C08 = 1, C00 = 0x407b4, C60 = C64 = 1; STOP and the buffer handshake no longer reset it (M2-121). Writing C60 = 0 releases the hold (M2-126).

• **Blocked** evjmp**:** a zero-field evjmp started with C00 = 1 stays at C08 = b, PC = the evjmp offset (0x20 and 0x38 measured), CS_IRQ = 0; STOP leaves it there (M2-57, M2-62, M2-392). Abort gives C08 = 3, and the clearing step issued before another STOP gives 0; ordinary reset then completes and the next program runs (device, M2-62, M2-63).

• **Active status latch:** a NAND status-latch transaction survives END and blocks the FMC acknowledgement step of the reset (FMCTRL1 = 2). FMCTRL1 = 0x30020 with acknowledgement 0x1800000 releases it (device, M2-31, M2-181).

State codes read by the CPU in C08:

C08	state
0	idle (after END, STOP or reset)
1, 2, 9	a program running: a delay loop, a loop after sig, a program running on after a timeout fault with C58 = 0 and a fault handler after a vectored fault; the value differs between identical runs (M2-384); 1 also while held by C60 = 1 (M2-121)
3	after abort from state b or from a running program (M2-62, M2-422); a running program frozen by start-word bit 1 (M2-158)
4	waiting: unmet wait or wait2, illegal opcode, tmwait in progress, a wait held by start word 1 (M2-13, M2-56, M2-384)
a	stopped at pause, CS_IRQ bit 3 (M2-14)
b	blocked zero-field evjmp started with C00 = 1 (M2-57)

A program reading C08 itself reads 5 (M2-70). Raw C00 readbacks are the start word with bit 18 set and bit 0 clear while running or waiting (0x4ffb4 for 0xffb5, M2-384), 0x40000 stopped, 0x40002 after STOP and abort and 0x7f60 after the clearing step; C14 stays 0x7f in these steps (chapter 8; M2-54, M2-281). After every END, a handler END included, C08 reads 0 (M2-384).

Power-on state. At the first access after a power cycle the window equals the window after the ordinary reset in every word except C00 (0x7f60 at power-on, the last start word after a reset) and C04 (0 at power-on, the program address after a reset); C08 is 0 and C14 reads 0x7f (M2-430). The first program after power-on runs on its first start, also when its instruction 0 is not END (M2-431).

Transitions:

from	event	to
idle (0)	start word bit 0	running
running	end; STOP (C00 = 8)	idle (0)
running	unmet wait	waiting (4)

from	event	to
waiting (4)	event; timeout fault	running
waiting (4)	timeout fault	running: the fault handler with C58 = 4, the next instruction with C58 = 0 (C08 1, 2 or 9; M2-384)
waiting (4)	abort (C00 = 2)	idle (0)
running	pause	paused (a)
paused (a)	PC + 8, acknowledge bit 3, C00 bit 0	running
running	zero-field evjmp with C00 = 1	blocked (b)
blocked (b)	abort (C00 = 2)	3
3	C00 = 0x80000	idle (0)
running	start word with bit 1	frozen (3); a later start resumes it (M2-158)
running	start word with bit 3 or 16	restarts at instruction 0 after the release (M2-158)
running	CPU writes C60 = 1	held (1)
held (1)	C60 = 0	released (M2-126)

2.2 Execution model

• The sequencer fetches 8-byte little-endian instructions (arg:u16, R:u8, opcode:u8, imm:u32) from system memory, starting at the bus address in FMSS_PROG (M2-01; chapter 5).

• It has eight 32-bit registers r0..r7, readable and writable by the CPU at 0xc18 + 4*n; r6 (0xc30) is the result word (M2-02). There are no flags or condition codes: bz/bnz test a register, and jr jumps to r[arg] iff r[R] ≠ 0 (M2-110).

• The PC (FMSS_PC, 0xc38) is a byte offset from the program start. Branch and jr targets are byte offsets with the low 3 bits ignored and no wrap between 2^15 and 2^24; the PC has at least 25 bits (M2-105, M2-122, M2-725). A write of r8 is a local jump (M2-43).

• A register written by one instruction is seen by the next instruction at distance 0 for add, mov, the ldi address, the sti data and bnz (M2-108): there is no register-write hazard.

• Register-access instructions (wr, st, ld) address window offsets: 0x000.. NAND controller, 0x800.. ECC engine, 0xc00.. sequencer control, 0xd00.. parameter RAM. ld masks the word by AND. pld/pst (18/19) reach the same window through a register, using offset bits 11:0 only; unaligned offsets read 0 and drop writes (M2-08, M2-175; chapter 4). A write of 0xc38 is a jump (M2-162).

• ldi/sti use a 32-bit bus address held in a register; words are little-endian and the address aligns down to a multiple of 4 (M2-05). No cache snooping by the sequencer or the DMA is established (chapter 12, section 12.3).

• ALU operations wrap at 32 bits; an immediate of 0 selects the register form; shifts are logical, count 31 gives 0x80000000 (SHL of 1) and 1 (SHR), and every count ≥ 32 gives 0 (M2-06, M2-07, M2-146). Source selector 10 is the destination register itself in every MOV/ALU form (M2-141).

• The original programs establish end, immediate and register stores, RAM load/store, masked load, immediate and register move, AND/OR/ADD/SUB, shifts, conditional branches and wait (observed, M2-19). The zero-field nop (08) lets the following instruction and END execute (M2-17); with R = 0..7 it continues and with R = 8..0xff it holds until the timeout fault like an unmet wait, which no tested pending flag releases (M2-625). The bounded opcode scans add no opcode beyond those of chapter 6 (M2-65).

2.3 END and fault

• **END:** CS_IRQ bit 0 is set and r6 is visible at 0xc30. The value in r6 is defined by the program; it is not a hardware error code. Non-zero R, arg and imm fields of end are ignored (M2-164). No C58 value diverts an END to the fault vector (M2-148, M2-149). While C10 is non-zero an END, a fault-handler END included, sets VIC1 raw bit 22 (M2-182). The original CPU driver polls CS_IRQ (bits 0, 2, 3) rather than a VIC line (observed, mem:0x51f4).

• **Fault:** an unmet `wait` and an illegal opcode fault at the same time (M2-81), every value `1a..fe` also with a pending flag (M2-583), and set `CS_IRQ` bit 2. With `C58` bit 2 set the sequencer vectors to the byte offset in `C4C`; with it clear it continues at the next instruction (M2-11). `C58` reads back as the written value AND `0xd6` from the CPU and from the sequencer (M2-70); bit 0 does not start a program at the vector (M2-12). A handler that reaches END leaves `CS_IRQ` = 5, fault plus END, which is not a clean result. In the handler `C3C` holds the faulting offset + 8, also for a nested fault in the handler (M2-179). `C5C` reads 1 after a timeout vector and 0 after continuation (chapter 8). While the handler runs, and while a program continues after a fault with `C58` = 0, the CPU reads `C08` = 1, 2 or 9 as for any running program, the value differing between identical runs (M2-384). `C58` bits 1, 4, 6 and 7 are stored and change neither path (M2-612).

• **Fault-handler registers:** the original handlers read `C3C` and test `C50` bit 0, writing 1 to acknowledge (observed). `C50` bits 0..5 mirror the pending flags of command, address, DMA, data/parity read, ECC and status latch and clear with their acknowledgements; bits 11..12 read 1 while the ECC engine is idle (M2-112, M2-152, M2-209). Faulting and no-fault controls read `C50` = `0x00ff1800` before and after the write of 1 (M2-18, M2-28).

• **Waits in the idle state:** only `wait` 11 passes; events 0..10 and 12..15 fault at about 126.6 ms in the idle baseline (M2-10); `wait` 11 passes at once in every tested state and operand form (M2-387). No `wait` event follows NAND ready/busy (M2-107). An `evjmp` vectors for six primary FMC/ECC stimuli in every tested operand form and faults when idle (M2-58, M2-391). A `tmwait` (15) with count = start-word bits 15:8 ≥ 1 ends at the (count + 1)-th timeout instant after its entry, 492.747 us × count + 450 us after the start when entered at the start (126.1 ms for `0xff`), whatever its fields, and leaves `C14` = 0; with count 0 it takes the timeout fault at the first instant (M2-27, M2-373, M2-584); consecutive `tmwaits` each wait a full period (M2-150); C14 writes do not change its end (M2-629). Any of `C48` bits 1..7 makes a `tmwait` fault at the wait timeout instead, bit 0 ends a count-1 `tmwait` one instant earlier, and bits 8..31 change nothing (M2-179, M2-627; inferred: it ends when `C14` equals `C48` bits 7:0).

• **Self-chosen handler path:** a program can jump to its own handler, for example when the NAND status reports FAIL; that path does not set `CS_IRQ` bit 2 (chapter 11).

• **Fault handling by the original CPU driver:** `CS_IRQ` bit 2 is a failed run, followed by the original reset (observed, `mem:0x51f4`, `mem:0x502c`).

2.4 Start word behaviour

bits	measured effect
0	starts the program at instruction 0; ignored while a program runs (M2-158)
1, 3, 16	hold the program at instruction 0 until a start without them; no single-step (M2-153, M2-157); while running, bit 1 freezes the program (`C08` = 3) and bits 3, 16 restart it (M2-158)
2	timeout enable (M2-56)
7:4 = n	timeout step of 2^3 cycles for n = 2, 2^5 for n = 3 and 2^(n+3) for n = 4..11 engine cycles; saturates at n = 11; n = 0 faults at once (0..4 us for every tested count); n = 1 is consistent with 2 cycles per count within the 2-us resolution (2, 4, 8 and 14..16 us for counts `0x0f`, `0x40`, `0x80`, `0xff`) (M2-115, M2-216)
15:8 = count	with bits 7:4 = `0xb`, a wait entered at the start faults at 492.747 us × (count + 1) + 447.5 us, counts 8..256 including multi-bit values (M2-81) and counts 1..3 (M2-584); the faults fall on timeout instants whose phase the per-case reset sets before the start (M2-433, M2-588, M2-631); counted from the wait's entry a wait of count ≥ 1 faults at the (count + 2)-th instant and a count-0 wait at the first (M2-584); `tmwait` ends one instant earlier than the wait (M2-373, M2-584)
19	inhibits the start; `C00` reads `0x7f60` (M2-153)
17, 18, 20..31	no observed effect (M2-201)

The step of 492.747 us per count equals 2^14 cycles of the inferred 33.25 MHz engine clock (inferred, M2-103). With start word `C00` = 1, an unmet WAIT3 and the illegal opcode 1a stay at `C08` = 4 without a fault (M2-56). No priming start is needed after a reset or after power-on (M2-151, M2-431), and no `C58` value makes an END run the

fault vector (M2-149). The bit details are in chapter 8, section 8.2.

2.5 Execution and time

• **Instruction cost:** ALU 30.1 ns, window `ld`/`wr` 60.6 ns, `sti` to SDRAM 97.8 ns, `ldi` from SDRAM 211 ns; SRAM costs the same as SDRAM (M2-102). Per copy in a loop, `nop`2, `movi`, `mov` and add cost 1 cycle, `nop` (`08`) and a passing `wait  11` 1.25, `ld`, `st`, `wr` and `pld` 61.2 ns, `sti` 3.25 cycles and `ldi` 199.5 ns (M2-378). A two-instruction loop measures 33.8 ns per instruction (M2-09). The engine clock is ≈ 33.25 MHz (inferred, M2-103).

• **Peripheral latency:** a 512-byte buffer-to-memory DMA completes within 1..2 us (M2-114); to `0x048` bit 20 a memory-to-buffer DMA takes ≈ 2.7 us and a buffer-to-memory DMA ≈ 1.1 us (M2-117). An encode takes ≈ 4.1 us for BCH8 and BCH16 (M2-118). A decode with zero syndromes takes ≈ 4.0 us, plus ≈ 1.6 us (BCH8) or ≈ 2.3 us (BCH16) for any non-zero syndrome (M2-133). A 512-byte NAND data read takes ≈ 22 us at the setup timing word (M2-119). Every NAND command, address and data byte takes (`FMC_CTRL0` bits 14:12 + bits 18:16 + 2) × 7.06 ns (M2-127, M2-131, M2-135).

• **Overlap:** peripheral commands launch work and `wait` synchronises with it, so a program can overlap operations. Serial and different-slot DMA/ECC overlap is exact (M2-80). A buffer-to-memory DMA programmed while another runs starts no second transfer: the running one continues with the newly written address and buffer select, and the second completion never comes (M2-78, M2-114, M2-394, M2-603).

2.6 Instruction completion versus peripheral data

END, a passing `wait` and correct data are separate observations. `wait` 0, 1 and 2 do not consume their flags: a repeated wait passes, an acknowledgement blocks the next wait, and a new trigger re-arms it (M2-66). A program that waits twice on one unacknowledged DMA flag therefore programs its next DMA while the previous one runs, and the earlier transfer is cut although the program reaches END (M2-71, M2-78). A DMA into a buffer under its own ECC operation loses bursts and sets `FMC_STATUS` bit 25 (M2-80). An `ECC_RESULT` read after WAIT4 and acknowledgement does not give the current sector's count (M2-85); per-decode results are held in the 8-slot ring at `0x80c` (M2-91; chapter 10).

Chapter 3 — The register window as the ARM CPU sees it

Summary: the offsets and names of the register window, the memory map with the evidence for each word, the rules that apply only to CPU accesses, the CPU-visible interrupt lines, how the original CPU driver starts and finishes a run, and the instruction and wait behaviour visible through the window.

3.1 Offsets and names

The register window is at `mem:0x38a00000`; addresses in this book are offsets from that base unless stated. The original CPU driver is observed in `bl_NandReadOnly` (reset `_resetFMSS` at `mem:0x502c`, start `_startFMSS` at `mem:0x51b8`, interrupt clear at `mem:0x4cbc`, interrupt wait at `mem:0x51f4`, pause handling at `mem:0x4d08`), in `osos` (reset at `mem:0x08069ba4`, start at `mem:0x080daf1c`, read setup at `mem:0x0808d050`) and in `fw_restore` (FIFO reads at `mem:0x0802a640`) (chapter 14, section 14.3). Register names follow chapter 1, section 1.2.

3.2 Memory map: the register window

Offset	Role	Evidence and limits
000	FMC_CTRL0: controller enable bit 0, CE bits 8:1, DMA bit 24, cycle timing bits 14:12 and 18:16	bit 0 is required for transfers; cycle = (bits 14:12 + bits 18:16 + 2) × 7.06 ns per byte; bits 18:16 ≤ 1 shift read data by one byte; bits 30:28, 9, 10, 25, 26 are stored with no measured effect; bits 15, 19..23, 27, 31 read 0; storage mask 0x77077fff (M2-127, M2-131, M2-135, M2-139, M2-191); bits 30:28 do not move the command, latch or read completions (M2-407); a DMA programmed with bit 24 clear is held and starts when bit 24 is set, with the DMA address of that moment (M2-184, M2-338)
004	FMCTRL1: command, address, data, DMA and reset trigger	observed command words; written alone, bit 0 starts an address operation, bit 1 a data read, bits 8 and 9 a DMA, and the other 28 bits start no operation (M2-411; chapter 9); bit 31 zeroes 00c..05c and sets 038 to 7, 0x800 changes none of them (M2-645)
008	NAND command byte	observed
00c, 010, 02c	address bytes and address-cycle count minus one	observed; 00c/010 store 32 bits, 02c 3 bits (M2-83)
014	data/parity buffer selection	inferred from transfer sequences; stores 16 bits, its bits 7:0 are 058 and bits 15:8 read at 05c (M2-83, M2-643); a DMA in writes data buffer 0 only with 1 and parity buffer 0 only with 0x10, no buffer with 0 or the other tested values (M2-668); a DMA in flight follows a rewrite of it from a word boundary on (M2-603, M2-672)
018	written 1 before data reads by the original program	stores 4 bits (M2-83, M2-191); no effect on page-read data, ECC results or a buffer loopback (M2-104, M2-405)
01c	ECC slot index	stores bit 0 (M2-83, M2-417); an encode uses the slot that bit selects, a second kick replaces the first, and a change during an encode gives a parity of neither slot (M2-417, M2-418, M2-419)
024	transfer count	reads 8 after an 8-byte READ ID (M2-336)
028	storage word	9 bits; does not count operations, pld/sti loops or FIFO reads (M2-156, M2-173, M2-199)
030	NAND transfer byte count minus one	stores 13 bits (M2-83); a DMA moves 512 bytes whatever it holds (M2-189)
034	DMA bus address	stores 32 bits (M2-83); a DMA uses it aligned down to a multiple of 4 (M2-664); a DMA in flight follows a rewrite of it at the same byte offsets from the next 32-byte boundary on (M2-603, M2-671)
Offset	Role	Evidence and limits
038	FMC_DMA_CTRL	reads 7 after reset and at power-on (M2-123, M2-430); 0x10..0x80 fault (M2-79); 0x40..0x1000 and 0x1fc7 read back unchanged, and any of bits 12:6 set, or bits 5:3 = 2, 4 or 7, makes the next DMA out fault after delivering only its first 32 bytes, while with bits 5:3 = 1 a DMA out writes each group of four words as four copies of its first word; bits 31:13 are not stored (M2-396, M2-669); bits 12:6 count up from 0 during a DMA and return to 0 at

Offset	Role	Evidence and limits
		its completion (M2-113, M2-117, M2-398); with bits 2:0 = 0..6 a DMA completes more slowly than with 7, and with 2, 4, 5 or 6 it leaves a residual count in bits 12:6 (2 reads back 0x42, 4 reads 0x84; M2-79, M2-609)
040	FIFO/metadata latch command	observed; 0x52 latches the chip ID into 060/064 (M2-124)
044	FMC interrupt routing	bits 0, 1, 2, 11, 12 route the command, address, transfer, DMA and status-latch flags onto VIC0 line 20 and into CS_IRQ bit 4 (M2-142, M2-200)
048	FMC_STATUS completion flags	bits 1/2/3/20/23 = command/address/data/DMA/status latch (M2-112); bits 19:12 read 0xc3 (bits 12, 13, 18, 19) after reset and are not cleared by writing 1, 0x41 after a short chip read, 0x3c (bits 14..17) after a 512-byte read; a data read of 1..128 bytes leaves 0x41, of 256 or 384 bytes 0x14, of 511 or 512 bytes 0x3c, and 32 or 128 FMC_FIFO writes leave 0x41 or 0x3c (M2-642); a DMA in, a status latch and the set-up word restore 0xc3 from 0x41 and from 0x3c (M2-203, M2-497); writing all ones clears a pending DMA flag and nothing else, writing 0 changes nothing (M2-412); bit 24 never set by a status latch (M2-163); bit 25 set by a DMA into a buffer under its own ECC operation (meaning inferred, M2-80); bit 26 set by an address operation after a latch without the 0x30020 release, not by a refused command (M2-181, M2-610); bit 27 set after reading a programmed sector, clear after an erased one, cleared by writing 0x0ff00ffe (M2-145); the completion flags are acknowledged by writing the bit
04c	latched NAND chip status	0x80 busy, 0xe0 ready, 0x70 with no CE (M2-87, M2-136); ignores writes (M2-412)
058	transfer word of the original programs: 014 bits 7:0	stores 8 bits (M2-83); a write replaces 014 bits 7:0 and a later 014 write replaces it (M2-643); it reads 0 after a buffer loopback, whose DMA out leaves 014 = 0x100 (M2-405); CPU writes before the start read back unchanged (M2-599); written before a DMA in that writes 014 = 0 it has no effect (M2-597)
05c	written 0x200 before the final DMA: 014 bits 15:8	stores bits 15:8, which are 014 bits 15:8 in place (M2-83, M2-643); no effect on page reads (M2-104); reads 0x100 after a buffer loopback, whose DMA out leaves 014 = 0x100 (M2-405); a CPU write of 0x200 before the start reads back unchanged (M2-599)
060..068	FIFO/metadata data words	observed; ID repeats after five bytes (M2-124)
080	CPU FIFO read port	consuming read (observed, fw_restore mem:0x0802a640); after FMCTRL1 0x1c2 and 0x340 CPU reads return the page data in 32-byte transfers (M2-436); sequencer reads return 0 after a READ ID and from DMA-filled buffers (M2-124, M2-207); writes never reach a data buffer: without 0x340 the buffer stays unchanged, and after 0x340 buffer 0 receives RAM from FMC_DMA_ADDR + 0x200 on, never a written value (M2-207, M2-341, M2-617)

Offset	Role	Evidence and limits
100..1fc	block of unknown function	sequencer-writable words only at 1b8..1d8; the other words keep nothing and read 0x12 from the sequencer; writes change no measured timing (M2-160, M2-210)
200..3fc	copy of 000..1fc	offset bit 9 not decoded below 400; the 100 block appears only at 100 and 300 (M2-155)
400..7fc	no register set	read 0 (M2-155); storage words read 0 after sequencer or CPU writes, without aliasing the primary words (M2-82); no FMC_CTRL0 bit enables them (M2-139); commands and DMAs through +0x400 do nothing (M2-183, M2-185); no tested enable write makes them hold a value (M2-393); a sequencer ld costs the same as at 00c or 60c (M2-596)
800	ECC identification word	reads 0x20070618 (M2-116)
804	ECC_CTRL: operation, BCH16 bit 2, metadata length bits 13:8	storage mask 0x3f04; no code other than BCH8/BCH16 (M2-98, M2-101, M2-191, M2-197); lengths above 32 encode the data alone (M2-662); 81c = 1 zeroes it (M2-645)
808, 814	no established role	808 reads 0x700; storage masks 0x1f81 and 0x2f (M2-116, M2-191); with 808 bits 12:8 = 0, 810 bit 5 is set after every completion (threshold role inferred, M2-434); 814 values change no BCH8 decode or encode result (M2-660)
80c	ECC_RESULT: 8-slot ring of per-decode results	each 80c read, by the sequencer or by the CPU, also through the copy at a0c, advances the read position; reads of 90c and b0c read 0 and do not (M2-90, M2-91)
810	ECC_STATUS	bit 0 done, bit 1 encode done, bit 2 decode done, bit 3 uncorrectable, bit 5 ≥ 7 corrected bits, bit 6 erased; bits 1, 2, 6 are write-1-to-clear; writing 1 clears only bit 0; bit 4 reads 1, bits 7..12, 16, 24 and 31 never set (M2-88, M2-94, M2-95, M2-97, M2-99, M2-100, M2-420)
818	corrected-bit running total << 11 | decode count	M2-89
81c	ECC reset	a write with bit 0 set clears 818, sets 810 to 0x10, resets the ring positions and zeroes 804; other bits do nothing (M2-91, M2-96, M2-421, M2-645)
820..	decode error locations, 13-bit fields, ascending by byte, higher bit first within a byte	M2-128, M2-132, M2-138, M2-714
840..85c	encode parity	word order reversed, big-endian words (M2-116); 844 keeps no written bits (M2-191)
900..9fc	none	read 0 after encodes and decodes; 940..950 drop writes (M2-171, M2-191)
a00..bfc	copy of 800..9fc	offset bit 9 not decoded in 800..bff (M2-155)
c00	FMSS_CTRL: start word, STOP, abort, clearing step	chapter 2, chapter 8
c04	FMSS_PROG: program bus address	observed; cleared by the clearing step (M2-54)

Offset	Role	Evidence and limits
c08	CS_STATUS: state code	0, 3, 4, a, b read by the CPU in the states of chapter 2; 1, 2 or 9 while a program runs, varying between identical runs; 5 read by the sequencer (M2-70, M2-384)
c0c	CS_IRQ: bit 0 END, bit 1 sig, bit 2 fault, bit 3 pause, bit 4 routed FMC flag	sig sets bit 1 and continues; bit 1 is write-1-to-clear (M2-34, M2-65, M2-370); bit 4 sticky (M2-200)
c10	initialization word	original prologue writes 1; stores 0xdf; non-zero makes an END set VIC1 raw bit 22 (M2-182, M2-191)
c14	counter/timer	0x7f at power-on and after reset (M2-281, M2-430); start-word bits 15:8 after a wait that passes before the first timeout step, counting down by one per step while a wait stalls, 0x7f after a wait that ends after one or more steps, also when the CPU releases it (M2-172, M2-177); 0 after a completed tmwait (M2-27); ignores writes (M2-176)
c18..c34	r0..r7; r6 at c30 is SEQ_RESULT	device (M2-02)
c38	FMSS_PC	program-relative byte offset; a CPU or program write jumps (M2-162, M2-278)
c3c	fault information	faulting offset + 8 in the handler, 0 after the run; 32 bits (M2-154, M2-179)
c40, c44	instruction word latches	device; cleared by the clearing step (M2-54)
c48, c54	unassigned storage	32 bits / mask 0x83f, kept by a start (M2-154); CPU writes release no wait (M2-143); any of C48 bits 1..7 makes a tmwait fault at the wait timeout instead of ending, bit 0 ends a count-1 tmwait one instant early, bits 8..31 change nothing (M2-179, M2-627)
c4c	fault vector	M2-11
c50	fault-handler state and completion bits	bits 0..5 mirror pending events 0..5 (M2-18, M2-28, M2-112, M2-209)
c58	fault policy	reads back AND 0xd6; bit 2 vector/continue; bits 1, 4, 6 and 7 stored without effect on the timeout path; no value diverts an END (M2-11, M2-70, M2-148, M2-149, M2-612)
c5c	vector-associated state	1 after a timeout vector, 0 after continuation (chapter 8)
c60, c64	buffer reset request / completion	observed handshake (M2-20); c60 = 1 while running holds the engine (M2-121)
c68	no established role	stores 3 bits (mask 7) (M2-191)
c6c	buffer configuration	0 normal path, 0x10 alternate path (observed)
d00..d7c	parameter RAM, 32 words	CPU read/write and sequencer reads (M2-03); cleared by the clearing step (M2-54)
d80..dfc	none	reads 0; CPU and program writes do not persist; no alias into d00..d7c (M2-04, M2-147)
e00..ffc	none	offset bit 9 is decoded in c00..fff: no word of c00..dff appears at +200 (M2-155)

The bit maps of FMC_STATUS, ECC_STATUS and ECC_CTRL are in chapters 9 and 10; the control block is in chapter 8.

3.3 CPU-only rules

• **Consuming reads:** 080 and the FIFO data words 060/064/068 are FIFO ports; a CPU read consumes data (observed). Reading them is part of an operation, not an observation of state.

• ECC_RESULT **reads:** a CPU read of 80c, also through its copy at a0c, advances the ring's read position like a sequencer read, while reads of 810/818 and writes of 80c do not (M2-91). A CPU snapshot of the window that includes 80c or a0c therefore moves the slot from which the next program's 80c reads start.

• C60 **while running:** a CPU write C60 = 1 during a run holds the engine at C08 = 1 beyond STOP and the buffer handshake; C60 = 0 releases it (M2-121, M2-126).

• **No CPU-written event:** CPU writes to C0C, C10, C48, C54 and C60..C6C release none of wait 6..10 and 12..15 (M2-120, M2-125, M2-143), nor do writes of C40, C44, C50, C14, C5C or C3C into a waiting wait 6..10 or 13..15 (M2-386).

• C58**:** CPU and sequencer writes both read back masked by 0xd6 (M2-70).

• C14**:** CPU writes (0x55) and program writes are ignored (M2-176; chapter 8, section 8.7).

• C38**:** a CPU write of the PC takes effect outside a pause: before the start the program starts at the written offset, while a program runs it jumps there (M2-278).

• C3C**,** C48**,** C54**:** a start keeps the values the CPU wrote (M2-154).

• **Storage masks:** CPU and sequencer writes give the same masks over the window (M2-191).

• FMC_FIFO **(**080**):** the CPU reads page data through it after FMCTRL1 0x1c2 (data read into the buffer) and 0x340, 32 bytes per transfer (M2-436); sequencer writes to it reach no data buffer, without 0x340 and after it (M2-207, M2-617).

• **CPU-driven NAND:** the programs' register sequences written by the CPU, with each wait replaced by a poll of its flag, read, erase and program pages and run decodes and encodes with the same results as the sequencer (M2-435, M2-436; chapter 12).

• C5C**:** its M2 role is not established; the M1 cleanup write to it has no M2 counterpart in the original driver.

• **GPRs:** values the CPU writes to r0..r7 before a start survive into the program (M2-02, M2-157).

• C08**:** the CPU reads the state codes of chapter 2; the sequencer reads 5 (M2-70).

• **Reset and power-on values:** the complete reset leaves FMC 00c..05c, ECC_CTRL and the parameter RAM 0 and FMC_DMA_CTRL 7 (M2-123); the FMC words and FMC_DMA_CTRL come from FMCTRL1 bit 31 and ECC_CTRL from 0x81c = 1, written by the CPU or the sequencer alike (M2-645). The power-on window, and the window left by the kernel boot or by the previous cleanup, differ from the one after the ordinary reset only in C00 (0x7f60 instead of the last start word) and C04 (0 instead of the program address) (M2-281, M2-430). Power-on values include 004 0x40000000, 020 0x7f167f, 038 7, 048 0xc3000, 1cc 0x8060, 1d0 0x3010d0, 800 0x20070618, 808 0x700, 810 0x10, C08 0, C14 0x7f, C50 0xff1800; the GPRs, the parameter RAM and the other FMC and ECC storage words read 0 (M2-430). The first program after power-on runs on its first start (M2-431).

3.4 CPU-visible interrupt lines

• The original CPU driver polls CS_IRQ & 0xd for END, fault and pause; it does not wait on a VIC line (observed, mem:0x51f4).

• NAND controller flags drive VIC0 line 20 when routed in FMC 044 (bits 0, 1, 2, 11, 12: command, address, transfer, DMA, status latch); the line follows a routed flag while it is pending, and CS_IRQ bit 4 records every routed flag of the run (M2-142, M2-200). An END with a routed flag still pending leaves USB working (M2-208).

• An END raises VIC1 raw bit 22 while C10 is non-zero, a fault-handler END included; after an END with C10 = 0 and after a run that stops without an END (held, paused, looping or faulted without a handler END) it is clear, and it is clear while a program runs (M2-182).

• sig raises no VIC0 or VIC1 line beyond the run's baseline (M2-370).

• wait2 conditions are VIC0 hardware lines. Line 5 is always asserted, line 8 is the timer interrupt, line 16 is DMAC0 and line 20 is the FMC interrupt (M2-142, M2-204); condition 19 passes in the DMA and ECC contexts

(M2-167). Conditions 32..255 never pass, VIC1 included (M2-74, M2-140, M2-211).

• A VIC software interrupt is visible in the raw VIC status but releases `wait2` on none of 59 lines (M2-140). Inferred: `wait2` samples the peripheral request lines before the VIC's software OR.

• No ECC interrupt is found: after encodes and decodes no VIC line rises for any tested enable candidate (M2-212, M2-217). The source behind line 5 and the ECC interrupt are open (chapter 17, section 17.3).

CPU interrupt numbers are distinct from `wait` event numbers.

3.5 Starting and finishing a run

The original CPU driver (`bl_NandReadOnly`; observed):

1. **Reset** (`mem:0x502c`): `C00` = 8 (STOP), poll `C08` = 0; if `C08` = 4, `C00` = 2 (abort) and poll for 0 or 3. Unless `c6c` = `0x10`: `c60` = 1, wait for `c64` = 1, `c60` = 0. Then `C00` = `0x80000`, `004` = `0x800`, poll `004` bit 30, `004` = `0x80000000`, `81c` = 1 (ECC reset). This is the M2 sequence; M1's `C00` = `0x30000` reset does not occur.

2. **Program:** `c04` = the program's bus address; clear `CS_IRQ` (`mem:0x4cbc`: write `0xd`, poll until bits 0, 2, 3 are clear).

3. **Start** (`mem:0x51b8`): `CS_IRQ` = `0xff`; `c6c` = 0, `c60` = 1, wait for `c64` ≠ 0, `c60` = 0 (or `c6c` = `0x10` on the alternate path); `c00` = `0xffb5` (bootloader) or `0xfff5` (restore OS).

4. **Wait** (`mem:0x51f4`): poll `CS_IRQ` bits 0, 2, 3. Bit 3 (pause): PC += 8, acknowledge bit 3, set `C00` bit 0 (`mem:0x4d08`). Bit 0 alone: success. Bit 2: failed run, followed by the reset of step 1.

The parameters are written after the reset, because the clearing step erases the parameter RAM (M2-54, M2-55). The recovery from states the original reset does not handle (pause inherited from an earlier run, blocked `evjmp`, active status latch, `C60` hold) is described in chapter 2, section 2.1.

3.6 Instruction and wait behaviour seen through the window

Instructions are eight bytes, little-endian: `arg:u16, reg:u8, opcode:u8, immediate:u32` (M2-01; chapter 5). The `wait` events used by the original programs, with their final measured sources (chapter 7):

wait event	source
0	command completion, `FMC_STATUS` bit 1 (observed, M2-19)
1	address completion, bit 2
2	DMA completion, bit 20
3	data transfer completion, bit 3
4	ECC completion: `810` bit 1 after an encode, bit 2 after a decode, write-1-to-clear (M2-97, M2-99)
5	status latch, `FMC_STATUS` bit 23 when the chip reports ready (M2-111, M2-130, M2-193); a latch written one or two instructions after an unwaited `70h` sets bit 3 instead and the wait faults (M2-593)
11	passes at once in every measured state and operand form, also on every iteration through NAND busy periods (M2-10, M2-107, M2-387)
12	masked NAND status: passes iff status & arg & mask ≠ 0 (M2-134); after a latch on a busy chip it releases only at ready (M2-137)
6..10, 13..15	no source found among NAND, ECC, DMA, status, CPU-write and VIC soft-line stimuli (M2-107, M2-111, M2-143, M2-206, M2-386)
16..255	never pass (M2-159)

`wait` 0, 1, 2 do not consume their flags; acknowledging blocks the next wait and a new trigger re-arms it (M2-66). After one status latch the controller keeps polling the chip status until ready; without a latch, or with `wait 12` arg = 0, the wait never passes (M2-130). A passing `wait 12` with mask `0x40` also passes on the no-CE echo `0x70` (M2-87, M2-134); it is not by itself evidence of a valid chip status byte.

Measured controls of individual instructions:

- pause (12): two forms stop with CS_IRQ = 8, C08 = a and PC at the instruction; CPU resume continues to the following instruction (M2-14, M2-25).
- nop (08): with R = 0..7 the following marker and END execute (M2-17, M2-625); with R = 8..0xff it holds at its own offset until the timeout fault of an unmet wait, also with a pending flag (M2-377, M2-625).
- Shifts by 31: SHL of 1 gives 0x80000000, SHR gives 1 (M2-07); every count ≥ 32 gives 0 (M2-146).
- pld (18) of 0xc30 reads r6 (M2-08); pld/pst use offset bits 11:0 and reach r0..r7 through 0xc18..0xc34 (M2-175).
- tmwait (15): with count = start-word bits 15:8 ≥ 1 it ends at the (count + 1)-th timeout instant after its entry, 492.747 us × count + 450 us after the start when entered at the start (126.1 ms for 0xff), whatever its fields, with C14 = 0; with count 0 it takes the timeout fault (M2-27, M2-373, M2-584); consecutive tmwaits each wait a full period (M2-150); C48 bits 7:0 change its end (M2-627).
- Undefined opcodes 1a..fe act as an unmet wait, also with a pending flag (M2-282, M2-583).
- end with non-zero R, arg or imm stops like the zero form (M2-164).
- jr jumps to r[arg] iff r[R] ≠ 0 (M2-110); a target selector with any of bits 8..15 set takes r0 (M2-673).

Opcodes 09, 0f, 10, 16 and the extended register operands are described with their measured forms in chapter 6; the M1 meaning of a retained mnemonic is not assumed beyond those forms.

Chapter 4 — The register window as the sequencer sees it

Summary: the address forms the sequencer uses, the segments and mirrors of its window, the register file with the roles of register numbers 8, 10 and above, parameter RAM, system memory and program fetch, and what the sequencer sees differently from the CPU.

4.1 Window offsets and target widths

Address form	Meaning	Example
CPU MMIO address	ARM physical register address	`mem:0x38a00c30` is r6/result
Sequencer window offset	offset used by direct `wr`/`st`/`ld`, or a register value used by `pld`/`pst`	`0xc30` names the same register
System-memory bus address	code and data address, used by `C04` or by `ldi`/`sti`	a buffer in SDRAM from `0x08000000`
Program offset	byte offset within the current program, used by branches, `jr`, the PC and the fault vector	the value in `C4C`

Branch, `jr` and vector targets are byte offsets within the current program, not absolute addresses; `C04` supplies the bus base. Their low 3 bits are ignored and they do not wrap between 2^15 and 2^24, so the PC has at least 25 bits (M2-105, M2-122, M2-725). STOP sets PC = 0 (M2-53), so a final PC of 0 does not describe the path a program took.

The primary FMC storage words have these widths (M2-83): `00c`, `010`, `034` 32 bits; `014` 16 bits (`0xffff`); `018` 4 bits; `01c` 1 bit; `02c` 3 bits; `030` 13 bits (`0x1fff`); `058` 8 bits; `05c` bits 15:8 (`0xff00`). `028` holds 9 bits (M2-173). `C58` stores the bits of `0xd6` (M2-70). The sequencer and the CPU see the same masks; the further measured masks are `000` `0x77077fff`, `804` `0x3f04`, `808` `0x1f81`, `814` `0x2f`, `844` and `940..950` 0, `C10` `0xdf`, `C54` `0x83f` and `C68` 7, with `C3C` and `C48` 32 bits (M2-154, M2-191).

`pld` and `pst` use offset bits 11:0 only: offsets with any of bits 12..31 set (bits 12, 13, 16 or 20; each of bits 17..30; `0x1d64`, `0x2d64`, `0x8d64`, `0xfd64`, `0x10d64`, `0x80000d64`) and the aliases `0x1c18`, `0x1c1c`, `0x1c28`, `0x10c28`, `0x1d68`, `0x10d70`, `0xfd6c` reach the word at offset & `0xfff` and change no other word. Through `0xc18..0xc34` `pld` reads r0..r7 (through its pointer register r1 it reads the pointer itself) and `pst` writes r0..r7. Unaligned `pld`/`pst` offsets (low bits 1..3) read 0 and drop the write (M2-08, M2-175).

4.2 Segments and mirrors, sequencer view

The segments are those of chapter 3, section 3.2: NAND controller `0x000..`, ECC engine `0x800..`, control block `0xc00..`, parameter RAM `0xd00..0xd7c`. Measured differences from the M1 layout:

- **Offset bit 9:** not decoded below `0x400` and in `0x800..0xbff` (`0x200..0x3fc` and `0xa00..0xbfc` are copies), decoded in `0xc00..0xfff`: none of the 38 holding words of `0xc00..0xdff` (`C10`, `C3C`, `C48`, `C4C`, `C54`, `C58`, `0xd00..0xd7c`) appears at +`0x200` (M2-155).
- `0x100..0x1fc`**:** a block of unknown function, visible again at `0x300` only (each of `0x1bc..0x1d8` copied to +`0x200`, none to +`0x400` or +`0x600`; M2-155). Its sequencer-writable words are `0x1b8` (1 bit), `0x1bc..0x1c8`, `0x1d4`, `0x1d8` (32 bits) and `0x1cc`, `0x1d0` (24 bits); the other words keep nothing and read `0x12` from the sequencer (M2-160). `0x3a8..0x3b4` read `0x12` from the sequencer and 0 from the CPU (M2-210). Details in chapter 9, section 9.12.
- `0x400..0x7fc`**:** reads 0 (M2-155). Its storage words read 0 from the sequencer after sequencer or CPU writes and do not alias the primary words (M2-82); no `FMC_CTRL0` bit makes them hold values (M2-139); a READ ID and DMAs programmed through +`0x400` do nothing (M2-183, M2-185). A sequencer `ld` of `0x40c` costs the same 2 cycles as one of `0x00c` or of the unused `0x60c`, so access time does not distinguish a decoded slot from undecoded space (M2-596). There is no second register set.

- `0x900..0x9fc`**:** reads 0 after encodes and decodes; `0x940..0x950` drop writes (M2-171, M2-191).
- `0xd80..0xdfc`**:** reads 0; CPU writes and program `wr`/`pst` writes do not persist, with no alias into `0xd00..0xd7c` (M2-04, M2-147).
- M1's second-channel decode does not apply; M1 results are comparison hypotheses only.

4.3 Register file r0..r7, PC and extended register roles

The sequencer has eight 32-bit GPRs at `0xc18..0xc34`; the CPU may preload them, and r6 at `0xc30` is the result register, with no separate result latch in the tested cases (M2-02). Register numbers above 7 follow the M1 roles (M1 chapter 4.3), except that source 10 in an ALU or MOV register form is the destination itself (M2-43..M2-48, M2-52, M2-65, M2-141). The selector matrices (258 destination, 256 branch, 255 indirect-address, 287 store-source, 256 JR, 256 argument-source and 64 source-10 cases) give the M1 roles except selector 10 (M2-43, M2-44, M2-46, M2-47, M2-48, M2-52).

Role	8	10	9 and 11..255
MOVI/MOV/ADD destination	PC: a local write jumps	all eight GPRs receive the value; with source 10 each GPR is its own source (M2-365)	GPRs unchanged
LD/PLD/LDI and immediate ALU destination	`ld`/`pld`: discarded, no jump (M2-616)	the loaded or computed value goes to r0..r5 and r7; the PC is not written (M2-616)	not measured
MOV destination with source 8 or 10	—	—	9 and 11..15: no GPR written, latch unchanged (M2-678, M2-679)
MOV / immediate ALU argument source	current instruction offset	destination's own value	r0
Register ALU argument source	current instruction offset	destination's own value, for all six ALUs	r0
ST register-field source	register-bus latch	register-bus latch	register-bus latch
BZ/BNZ tested register	never branches	never branches	never branches
JR target source	self-loop	r0	r0
LDI address source	at PC = `0x60` reads `e1a0f00e`	r0	r0
PLD address source	at PC = `0x60` reads 0	r0	r0

Source 10. With destination r1, r0 = 11001111 and r1 = 22002222, register-form AND and OR give 22002222, ADD gives 44004444 and SUB gives 0 (M2-44). Across all eight GPR destinations and eight operand pairs, all six ALUs use the destination itself; with r0 = 1 and r1 = 3, source-10 SHL gives 24 where source-0 SHL gives 6 (M2-52). Where two candidate sources yield the same value, a matching result does not distinguish them.

Register-bus latch. For every extended `st` source selector after a seeded r7, and for 13 predecessor contrasts at selectors 9, 14 and 200, the stored value is the previous register-bus value: MOVI, MOV, ADD, LD, PLD and LDI provide their write value, ST its read value, WR and NOP r0; WAIT11 preserves the previous value (M2-47). A MOV with the broadcast destination r10 does not update the latch: after `mov r10, r3` an extended `st` stores the value r0 held before the broadcast (M2-366). A `movi` or `mov` to a discarded extended destination (r12) does not update it either (M2-622), nor does a `mov` to r9 or r11..r15 with source selector 8 or 10 (M2-679). No constant-zero extended source is inferred.

Source field width. A register-form source selector with any of bits 8..15 set uses neither its low byte nor the r0 fallback of selectors 9 and 11..255 (M2-364, M2-614): for `mov`, `add`, `or` and `sub` it reads the value that the last source selector before it read (r0 after `movi`, the source register after `mov` or `add`, the instruction offset after `mov` with selector 8), kept over a `nop2` and over an `ld`, `st` or `wr` (none of them sets it: neither the loaded word, the stored register nor the immediate, M2-674..M2-677), and 0 when only `ld`, `st` or `wr` came before it since the reset, as also in `and`, `shl` and `shr` (M2-614, M2-618). This value is not the extended-`st` latch (M2-622). An address selector with bits 8..15 set uses r0, the fallback of the undecoded address selectors, for `ldi`, and `pld`/`pst` behave as an access through r0 (M2-615); a `jr` target selector with bits 8..15 set also takes r0 (M2-673). For `sti` a high-bit address selector uses the value the last source selector read: r0 after `movi`, r5 after `mov r2` with selector 5

(M2-680, M2-681). With destination r10, source 8 writes the instruction offset to every GPR (M2-365).

PC. A MOVI to r8 jumps locally; MOV r2, r8 returns the instruction offset (M2-38, M2-43). A `wr` or `st` to the window word `0xc38` also jumps to the written offset, and an `ld` of `0xc38` reads the instruction's own offset (M2-162). `jr` with target source 8 loops on itself (M2-46). The low bus word read by LDI at PC = `0x60` is not proof of a full ROM mirror (M2-48).

Low bus. `ldi` from `0x20000000` + off and from off return the same word for each of off = 0, 4, `0x60`, `0x100`, `0x1000`, `0x8000`, `0xfffc` (M2-432). Over `0x20000000..0x2000ffff` the word depends only on the 16-KiB quarter of the address, never the word at the addressed offset: `0xe1a0f00e` (`mov pc, lr`, a word at 18 ROM offsets in `0..0x3fff`) for `0x0000..0x3fff`, `0x1affffef` (the ROM word at `0x4d9c`, a bne loop) for `0x4000..0x7fff`, `0xe59f42b0` (the ROM word at `0x9b5c`) for `0x8000..0xbfff` and 0 for `0xc000..0xffff`, at every sampled offset (M2-682, M2-683). Inferred: each quarter returns a word that the ROM's own data path last held, plausibly the last word the CPU fetched from that quarter before leaving the ROM; the ROM contents themselves are not readable by the sequencer at these addresses (M2-683).

Reset. STOP and abort preserve the GPRs (M2-53, M2-422); the clearing step `C00` = `0x80000`, alone or with STOP or abort, zeroes them (M2-54, M2-422). At power-on they read 0 (M2-430).

An `ldi` from an unmapped bus address (0x50000000) hangs the whole system; a DMA from that address completes (M2-717, M2-718).

Open: extended `pst` register roles other than those above and fetch from an invalid target (chapter 17, items 2 and 7).

4.4 Parameter RAM

- **Size and place:** 32 words (128 bytes) at `0xd00..0xd7c`, written and read by the CPU and read by the sequencer (M2-03). `pld` and `pst` reach the same parameter/GPR window (M2-08, M2-175).
- **Upper window:** `0xd80..0xdfc` reads 0; CPU writes and program `wr`/`pst` writes do not persist, read back from both sides, and do not alias into `0xd00..0xd7c` (M2-04, M2-147).
- **Reset:** STOP and abort preserve all 32 words; separate buffer, FMC and ECC reset steps preserve them too; the clearing step `C00` = `0x80000` zeroes them, as does the complete reset (M2-53, M2-54, M2-123, M2-422). They read 0 at power-on (M2-430). Parameters written before a reset are therefore lost (M2-55).
- **Use:** the meaning of each word is defined by the program (chapter 12).

4.5 System memory and the program

- **Program fetch:** 8-byte instructions from the bus address in `C04` (M2-01).
- `ldi`/`sti`**:** a 32-bit bus address held in a GPR; words are little-endian and the address aligns down to a multiple of 4 (M2-05). A store followed by an unaligned reload returns the stored word.
- **Cost:** `sti` to SDRAM 97.8 ns, `ldi` from SDRAM 211 ns; SRAM costs the same as SDRAM (M2-102); per copy in a loop `ldi` costs 199.5 ns (M2-378).
- **DMA:** the RAM address aligns down to a multiple of 4 bytes in both directions (M2-664); a rewrite of `FMC_DMA_ADDR` during a DMA takes effect at a 32-byte boundary (M2-671).
- **Not established:** unaligned instruction fetch. Coherency between the CPU cache and the sequencer or DMA is not provided by any measured snooping.

4.6 What the sequencer sees differently from the CPU

- `C08`**:** the sequencer reads 5 in every tested control, while the CPU reads 0, 1, 2, 3, 4, 9, a or b (M2-70, M2-384).
- **Register-bus latch:** extended `st` source selectors store the previous register-bus value instead of a register (M2-47).
- **Extended address sources:** LDI at PC = `0x60` reads `e1a0f00e` and PLD reads 0 through extended address selectors 8 (M2-48).

• `0x400..0x7fc`**:** reads 0 (M2-82, M2-155).

• `0x100` **block:** words without storage read `0x12` from the sequencer; `0x3a8..0x3b4` read 0 from the CPU (M2-160, M2-210).

• **Indirect window:** `pld`/`pst` reach the window through a register, with offset bits 11:0 only; unaligned offsets read 0 and drop writes (M2-08, M2-175).

• **Low bus:** `ldi` from `0x0000xxxx` and `0x2000xxxx` returns the same words, one fixed boot-ROM word per 16-KiB quarter instead of the addressed word (M2-432, M2-682, M2-683).

Chapter 5 — Instruction encoding

Summary: The eight-byte instruction, the meaning of each field per opcode, the opcode summary table, the ALU operand rule, branch and jump targets, register write ordering, instruction cost and byte order. The notation follows M1. Details per opcode are in chapter 6; register numbers above 7 are in chapter 4. Evidence IDs (M2-nn) are defined in chapter 18.

5.1 Encoding

```
word0 = opcode << 24 | R << 16 | arg    (arg: u16)
word1 = imm                            (u32)

byte:   +0       +1        +2   +3      +4       +5        +6         +7
        arg[7:0] arg[15:8] R    opcode  imm[7:0] imm[15:8] imm[23:16] imm[31:24]
```

Both words are little-endian (M2-01). Every instruction is eight bytes and is fetched from `FMSS_PROG + pc`. A program is an array of instructions; branch and jump targets are byte offsets from the program start (section 5.6). For example, `movi r6, #0x600d600d` encodes `00 00 06 05 0d 60 0d 60`, and `wr [0xc58], #4` encodes `58 0c 00 01 04 00 00 00`, as in the M1 notation.

The fields mean:

- R names a register, an event number, an interrupt line number or a tested register, depending on the opcode.
- `arg` is a window byte offset (register-access instructions) or a register selector (register-operand instructions); `wait 12` uses it as a status operand.
- `imm` is a value, a mask or a branch target.

`rR` means register R, and `rA` the register selected by `arg`. `[REG]` is a window offset; `[rA]` is an address held in rA, whose address space depends on the instruction (`ldi`/`sti`: system memory; `pld`/`pst`: register window).

5.2 Operand map

What each field means for each opcode. **used**: the field carries the named operand. **use 0**: the ordinary encoding; other values are not measured on M2. **tested**: the listed nonzero values were run on a device and gave the same result as the zero form, unless the cell states a different outcome; other values are open. M1's exhaustive ignored-field results (M1 chapter 5.2) are not M2 measurements.

op	mnemonic	R	arg	imm
`00`	`end`	tested: 1, 0x80, 0xff (latched in `C40`)	tested: non-zero up to 0xffff (latched in `C40`)	tested: 0xff, 0x80000000, 0xffffffff (latched in `C44`)
`01`	`wr`	tested: 1, 3, 7, 0x80, 0xff	used: window offset	used: value
`02`	`st`	used: source register	used: window offset	tested: 1, 4, 0x10, 0x100, 0xff, 0x80000000, 0xffffffff
`03`	`ldi`	used: destination register	used: register holding the bus address (a selector with any of bits 8..15 set uses r0)	tested: 1, 4, 0x10, 0x100, 0xff, 0x80000000, 0xffffffff
`04`	`ld`	used: destination register	used: window offset	used: AND mask
`05`	`movi`	used: destination register	tested: 1, 0xa, 0x1234, 0xd70, 0x8000, 0xffff	used: value
`06`	`mov`	used: destination register	used: source register (a register-form source with any of bits 8..15 set reads the value the last source selector read)	tested: 1, 4, 0x10, 0x100, 0xff, 0x80000000, 0xffffffff

op	mnemonic	R	arg	imm
07	wait	used: event number	event-dependent: wait 12 status operand	event-dependent: wait 12 mask
08	nop	tested: 0..0xff; R = 0..7 continue, R = 8..0xff hold in the wait state (C08 = 4) until the timeout fault	tested: 1, 2, 0xd60, 0x8000, 0xffff	tested: 1, 0xd64, 0x80000000, 0xffffffff
09	wait2	used: interrupt line number N	tested: 0x40, 1, 0xd60, 0xffff	tested: 0x40, 0xd64, 0xffffffff
0a..0d, 13, 14	ALU	used: destination, also first source in the register form	used: source register (a register-form source with any of bits 8..15 set reads the value the last source selector read)	used: immediate; 0 selects the register form
0e, 17	bnz, bz	used: tested register	tested: 1, 0xff, 0xffff	used: byte target
0f	jr	used: condition register (jump iff r[R] ≠ 0)	used: register holding the target (a selector with any of bits 8..15 set uses r0)	tested: 1, 0xffffffff
10	sig	tested: 1, 3, 7, 0xff	tested: 1, 0xd60, 0xffff	tested: 1, 0xd64, 0x80000000, 0xffffffff
11	sti	used: source register	used: register holding the bus address (a selector with any of bits 8..15 set uses the value the last source selector read)	tested: 1, 4, 0x10, 0x100, 0xff, 0x80000000, 0xffffffff
12	pause	tested: 0..15	tested: 1, 0xd60, 0xd64	tested: 0xd64, 0xff00, 0xffffffff
15	tmwait	tested: 1, 0xff (and 24 earlier forms)	tested: 1, 0xffff	tested: 1 .. 0x10000
16	evjmp	tested: 1, 2, 3, 4, 7, 0xff	tested: 1, 2, 0xd60, 0xffff	tested: 0xd64, 0xffffffff, a program label (never a jump target)
18, 19	pld, pst	used: destination / source register	used: register holding the window offset (bits 11:0; a selector with any of bits 8..15 set acts as an access through r0)	tested: 1, 4, 0x10, 0x100, 0xff, 0x80000000, 0xffffffff
ff	nop2	tested: 3, 7, 0xff	tested: 1, 0xd60, 0xffff	tested: 0xd64, 0xffffffff

The "tested" values of 08, 10, 16 and ff include the three forms of the M1 comparison set (R/arg/imm = 0/0/0, 3/1/0xd64 and 7/0xd60/0xffffffff), all of which match the M1 expectation (M2-34, M2-65). The further 08 values are those of M2-377 and M2-625, the 10 values those of M2-370, the 15 values those of M2-27 and M2-373, the 16 values those of M2-391 and the all-ones ff word that of M2-367. The end values are those of M2-164, the wr, st, ldi, mov, movi, bnz, bz, sti, pld and pst values those of M2-165 and the wait2 pairs those of M2-187. The 12 forms are those of chapter 6.

Two field rules depart from "ignored": nop (08) with any R = 8..0xff does not continue but holds at its own offset in the wait state (C08 = 4) until the timeout fault, at the time of an unmet wait 3, while R = 0..7 continue (M2-377, M2-625, M2-685, M2-686); and a register-form source field is decoded with its bits 8..15: with any of them set (0x104, 0x108, 0x1ff alike) the source is neither the low-byte register nor the r0 of sources 9 and 11..255 (M2-40) but the value that the last source selector before it read: r0 after movi (source field 0), the source register after mov and add, the instruction offset after a mov with selector 8, kept over a nop2 and over an ld, st or wr, none of which sets it, and 0 when only ld, st or wr ran before it since the reset (M2-364, M2-614, M2-618, M2-674..M2-677). An address selector of ldi and a target selector of jr with any of bits 8..15 set use r0, as the undecoded selectors do (M2-615, M2-673), and pld/pst with such a selector behave as an access through r0 (the accessed word itself is not observed; M2-615). An sti address selector with any of bits 8..15 set uses the value the

last source selector read, r0 after `movi` and r5 after `mov` with selector 5 (M2-680, M2-681). A `mov` to r9 or r11..r15 discards its write also with source selector 8 or 10 (M2-678, M2-679). Field widths of R and arg beyond the tested values are not measured.

5.3 Opcodes

The M1 mnemonics are the reference names. `nop`, `wait2`, `jr`, `sig`, `pause`, `tmwait`, `evjmp` and `nop2` are names given in these documents after the measured behaviour; no original M2 program uses them. A name preserves the M1 comparison; the **status** column states how much of the behaviour is established on M2: **confirmed** (original programs and/or device), **tested forms** (device-confirmed in the measured forms and states only), **partial** (part of the behaviour measured, part open).

op	mnemonic	semantics	status
00	`end`	stop, raise CS_IRQ bit 0; r6 is the program-defined result	confirmed
01	`wr [REG], #imm`	window[arg] = imm, with the register's write side effects	confirmed
02	`st [REG], rR`	window[arg] = rR	confirmed
03	`ldi rR, [rA]`	rR = mem32[rA], system-memory bus address	confirmed
04	`ld rR, [REG], mask`	rR = window[arg] AND imm	confirmed
05	`movi rR, #imm`	rR = imm	confirmed
06	`mov rR, rA`	rR = rA	confirmed
07	`wait N`	block until event N is set, else time out and fault	confirmed for the measured events (chapter 7)
08	`nop`	no operation for R = 0..7 (zero form 1.25 cycles, the cost of a passing wait); R = 8..0xff hold as an unmet wait (C08 = 4) to the timeout fault (M2-625, M2-686)	tested forms
09	`wait2 N`	block until interrupt request line N is raised (sampled before the VIC software OR, inferred), else time out and fault	partial: lines 5, 8, 16, 19, 20 pass, 32..255 never; line 5 source and ECC line open
0a	`and`	ALU, section 5.5	confirmed
0b	`or`	ALU	confirmed
0c	`add`	ALU, wraps modulo 2^32	confirmed
0d	`sub`	ALU, wraps modulo 2^32	confirmed
0e	`bnz rR, target`	if rR ≠ 0: pc = imm	confirmed
0f	`jr rA`	if r[R] ≠ 0: pc = rA; no link value saved	confirmed
10	`sig`	set CS_IRQ bit 1 (W1C) and continue; no interrupt line; 1 cycle	tested forms
11	`sti [rA], rR`	mem32[rA] = rR, system-memory bus address	confirmed
12	`pause`	stop on the instruction with C08 = 0x0a and CS_IRQ bit 3; the CPU resumes it	tested forms
13	`shl`	ALU, logical left; count ≥ 32 gives 0	confirmed
14	`shr`	ALU, logical right; count ≥ 32 gives 0	confirmed

op	mnemonic	semantics	status
15	`tmwait`	timed wait: ends without a fault at the (count + 1)-th timeout instant after its entry (492.747 us × count + 450 us when entered at the start); count 0 faults; it ends at the instant at which the count reaches C48 bits 7:0 and faults if they exceed the count (M2-373, M2-584, M2-627, M2-688)	tested forms
16	`evjmp`	wait for an FMC/ECC event, then jump to the fault vector without a fault; idle times out and faults	tested forms
17	`bz rR, target`	if rR = 0: pc = imm	confirmed
18	`pld rR, [rA]`	rR = window[rA]	confirmed
19	`pst [rA], rR`	window[rA] = rR	confirmed
ff	`nop2`	no operation; 1 cycle; an all-ones (erased) word executes as nop2	tested forms
1a..fe	undefined	same outcome as an unmet `wait`: faults at the timeout, no other change; also with a pending flag; the operand fields of 1a have no effect (M2-282, M2-583, M2-684)	tested forms

5.4 Unassigned opcodes

Every opcode byte other than the 19 common instructions was run in three operand forms under C58 = 0 and C58 = 4, and seven control opcodes were run with command, address, DMA, ECC and status stimuli (M2-64, M2-65). The signatures of the control opcodes are consistent with the named set: `nop` 08, `jr` 0f, `sig` 10, `pause` 12, `tmwait` 15, `evjmp` 16 and `nop2` ff. Every value `1a..fe` gives exactly the outcome of 1a and behaves as an unmet `wait`: with the timeout enabled it faults at the same time as an unmet `wait 3` (M2-81), with the timeout disabled it stays in C08 = 4 without a fault (M2-56), and it changes no register, scratch or window word (M2-282). The same holds for every value `1a..fe` with a pending command, address, DMA, ECC or status flag (M2-583). No value in `1a..fe` acts as an instruction in the tested forms; there is no distinct illegal-instruction trap. An erased (all-ones) word is `ff` with all-ones fields and executes as nop2, also in runs of 8 and 64 words and with a command, DMA, ECC or status operation pending (M2-367). With 64 random R/arg/imm words 1a gives the zero-form outcome in every case: its operand fields have no effect (M2-684).

5.5 ALU operand rule

```
imm != 0: rR = r(arg) OP imm
imm == 0: rR = rR     OP r(arg)
```

- There is no distinct immediate-zero form; `mov` performs the copy (M2-06).
- ADD and SUB wrap modulo 2^32; SHL and SHR are logical. Counts below 32 shift normally and every count ≥ 32 gives 0, in the immediate and the register form (counts up to `0xffffffff`, values 1, `0x80000000`, `0xffffffff` and `0x80000001`): there is no modulo-32, 64 or 256 and no 8-bit-immediate aliasing of the count (M2-06, M2-07, M2-146).
- Source selector 10 reads the destination register in every MOV and ALU form, immediate and register, for all six ALUs and all eight GPR destinations (M2-52, M2-141); with the broadcast destination r10 each GPR is its own source (`add r10, sel10, #5` adds 5 to every GPR), and source 8 gives every GPR the instruction offset (M2-365).
- A register-form source field with any of bits 8..15 set reads the value that the last source selector read, in `mov`, `add`, `or` and `sub` (r0's value after `movi`), and 0 when no source has been read since the reset, also in `and`, `shl` and `shr` (M2-364, M2-614, M2-618). M1 uses the destination only in the MOV and immediate forms and r0 in the

register form; this is an M2 difference in selector decoding, not in the immediate-zero encoding.

• There is no compare instruction; comparisons are synthesized by subtraction and a conditional branch. The original programs have no unconditional jump: they branch on a register known to be non-zero. `jr` (`0f`) jumps through a register, conditional on r[R] (section 5.6).

5.6 Branch and jump targets

• `bnz`/`bz` targets (`imm`) and `jr` targets (r[arg]) are byte offsets from the program start (M2-105, M2-122).

• The low three bits are ignored: targets +1, +2 and +4 land on the aligned instruction, +8 on the next one.

• Targets are not reduced modulo 2^15..2^24: `bnz` and `jr` land exactly on targets up to 2^24 + 0x100, so the PC has at least 25 bits (M2-105, M2-725). M1's rule that a `jr` target above 0xfff does not jump (M1-F097) does not apply to M2 in these measurements (M2-122).

• A `jr` whose target lies outside the program fetches from the system memory at that offset; its outcome depends on that memory's contents (M2-144).

• `jr` jumps iff r[R] ≠ 0 (M2-110). Writing a local target into r8 with `movi`, `mov` or add also jumps (M2-43).

5.7 Register write ordering

There is no register-write hazard. A register written by the immediately preceding instruction is seen at distance 0 by add, `mov`, the `ldi` address, the `sti` data and the `bnz` condition (32 of 32 cases, M2-108). A `jr` with R = 0 and r0 = 0 does not jump, whatever the distance from the instruction that writes its target register (M2-106, M2-109); this is the conditional rule of section 5.6 (M2-110), not a hazard.

5.8 Instruction cost

Per-instruction times are loop differences over 1,000 and 21,000 iterations, with the empty loop subtracted; costs add linearly per copy (1..8 copies in a 20,000-iteration loop, residual ≤ 1.1 us, and 16 copies in a 30,000-iteration loop, M2-378). The cycle column assumes the inferred engine clock of 33.25 MHz (M2-103), which also matches the timeout step of 2^14 cycles. The program placement (SDRAM or internal SRAM) does not change the times (±0.5 ns, M2-102).

instruction	ns (device)	cycles at 33.25 MHz
`movi`, `mov`, add, `nop2` (`ff`), an all-ones word, `sig`, untaken `bnz`, `bnz` taken to the next instruction	30.1	1.00
`nop` (`08`), a passing `wait 11`, a passing `wait 0`	37.65	1.25
`ld` from GPR / PRAM / FMC / ECC, from a `+0x400` word or an unused offset; `wr`, `st` and `pld` of PRAM or FMSS words	60.6..61.2	2.0
`sti` to SDRAM	97.8	3.25
`ldi` from SDRAM	199.5..211	6.6..7.0
sub + taken `bnz`	67.6	2.25

Sources: M2-102; `sig` M2-370, the all-ones word M2-367, the branch forms and linearity M2-378, `wait 11` and `wait 0` M2-387, the `+0x400` and unused-offset `ld` M2-596. The quarter-cycle fractions (`nop` 08, `wait 11`, branch, `sti`) are measured, not explained (chapter 17, item 6). A simple counter loop gives a slope of about 33.8 ns per instruction (M2-09).

5.9 Byte order

• Instructions, system-memory words (`ldi`/`sti`) and parameter words are little-endian (M2-01, M2-05). The tested unaligned `ldi`/`sti` addresses round down to a word boundary (M2-05); branch targets ignore their low three bits (section 5.6).

• The BCH8 and BCH16 code streams are taken most significant bit first: data, then metadata, then parity (M2-77, M2-86).

• Encode parity is readable at `0x840..0x85c` with the word order reversed and big-endian words (M2-116). See chapter 10.

Chapter 6 — Opcode reference

Summary: One entry per named opcode, in opcode order, followed by one entry for the opcode values without an established meaning. The entries use the M1 mnemonics and operand notation and are not numbered. The summary table, the ALU operand rule, target encoding and instruction cost are in chapter 5. Evidence IDs (M2-nn, M1-Fnnn) are defined in chapter 18.

6.1 Conventions and measurement notes

• Fields are those of chapter 5.1: R (8 bits), `arg` (16 bits), `imm` (32 bits). rR is register R; rA the register selected by `arg`. `[REG]` is a register-window byte offset; `[rA]` uses the address held in rA. `ldi`/`sti` access system memory, `pld`/`pst` the register window.

• `nop`, `wait2`, `jr`, `sig`, `pause`, `tmwait`, `evjmp` and `nop2` are documentation mnemonics retained from M1, not names recovered from Apple code; no original M2 program uses these opcodes. A name does not import M1's ignored operands, event routing, timing or control-register side effects.

• **Original** means use in the original M2 programs (chapter 14); **device** means a qualified M2 capture; **inferred**, **open** and **emulated** keep the meanings of chapter 1. A field is not called ignored because the original programs leave it zero, and an opcode is not called illegal because a short observation times out.

• Times are device measurements; cycle counts assume the inferred 33.25 MHz engine clock (M2-102, M2-103). Program placement in SDRAM or SRAM does not change them.

• Register numbers 8..255 decode differently per operand role. The role table is in chapter 4; each entry below gives the roles that apply to it (M2-43, M2-46, M2-47, M2-48, M2-141).

• **Status** is **confirmed** (original programs and/or device, in all measured forms), **tested forms** (device-confirmed only in the listed forms and states), **partial** or **open**.

6.2 Opcodes

`00` end

• Stop the program.

• Syntax: end.

• Encoding: R = arg = imm = 0, as in the original programs. Non-zero R, arg and imm are ignored (R 1, 0x80, 0xff; arg up to 0xffff; imm 0xff, 0x80000000, 0xffffffff): the instruction stops like the zero form, without a fault (CS_IRQ = 1), and the following instruction does not run (r6 keeps its value). C40 latches R << 16 | arg and C44 the imm (M2-164).

• Operation: the program stops and CS_IRQ bit 0 is set; r6 (`0xc30`) holds the program-defined result (M2-02). An END reached after a fault, at the fault vector or at the next instruction, also shows the fault bit: CS_IRQ = 5 (M2-10, M2-11, M2-282). No C58 value diverts an END to the fault vector (M2-148, M2-149). An END with C10 non-zero sets VIC1 raw bit 22, also an END in a fault handler; with C10 = 0 it stays clear (M2-182).

• Original programs: used by every program.

• Status: confirmed.

• Evidence: M2-01, M2-02, M2-10, M2-11, M2-148, M2-149, M2-164, M2-182, M2-282, chapter 14.

`01` wr

• Write an immediate value to a register-window word.

• Syntax: `wr [REG], #imm`.

• Encoding: arg = window byte offset, imm = value; R = 0.

• Operation: window[arg] = imm. The addressed register supplies any side effects (a command byte to the NAND controller, a start code, an acknowledgement).

• Timing: `wr` to PRAM 60.6..61.2 ns, 2 cycles (M2-102, M2-378).

• R = 1, 3, 7, 0x80 and 0xff are ignored (M2-165). A `wr` to `0xc38` (the PC) jumps to the written offset without executing the fall-through instruction (M2-162).

• Original programs: used for every fixed register value.

• Status: confirmed.

• Evidence: original programs; device control, PRAM and C58 cases (M2-03, M2-70, M2-162, M2-165).

02 st

• Store a sequencer register to a register-window word.

• Syntax: `st [REG], rR`.

• Encoding: R = source register, arg = window byte offset; imm = 0.

• Operation: window[arg] = rR.

• Register numbers: with R = 8..255, `st` stores the register-bus latch, the last value that passed through the register file. `movi`, `mov`, `add`, `ld`, `pld` and `ldi` leave the value they wrote; `st` leaves the value it read; `wr` and `nop` leave r0; `wait 11` keeps the previous value. Measured for every extended selector after a seeded r7, with 13 predecessor contrasts at selectors 9, 14 and 200; no constant-zero extended source is inferred (M2-41, M2-47). A broadcast `mov r10, r3` does not update the latch: the following extended `st` stores the value r0 held before the broadcast, not the broadcast value (M2-366), and a write to a discarded extended destination (`movi r12`, `mov r12`) leaves the latch unchanged (M2-622). This latch differs from the value a high-bit source selector reads (M2-618).

• Non-zero imm (1, 4, 0x10, 0x100, 0xff, 0x80000000, 0xffffffff) is ignored (M2-165). A `st` to `0xc38` jumps to the stored offset (M2-162).

• Original programs: used (control words built in a register, list pointers).

• Timing: 61.2 ns, 2 cycles (M2-378).

• Status: confirmed.

• Evidence: M2-41, M2-47, M2-162, M2-165, M2-366, M2-378, M2-622; device control case storing a CPU-preloaded r3.

03 ldi

• Load a word from system memory.

• Syntax: `ldi rR, [rA]`.

• Encoding: R = destination, arg = register holding a bus address; imm = 0 (the tested non-zero imm values are ignored, M2-165).

• Operation: rR = mem32[rA], a little-endian 32-bit load. The tested unaligned addresses round down to the word boundary (M2-05). An address register written by the preceding instruction is used at distance 0 (M2-108).

• Register numbers: address selector 8 reads the low bus word: at instruction offset 0x60 it returns `e1a0f00e`; selectors 9..255 use r0 (M2-48). An `ldi` from `0x20000000` + off and one from off return the same word for off = 0, 4, 0x60, 0x100, 0x1000, 0x8000, 0xfffc (M2-432). Over `0x20000000..0x2000ffff` the loaded word depends only on the 16-KiB quarter of the address and is never the addressed ROM word: `0xe1a0f00e` (`mov pc, lr`, at 18 ROM offsets in the first quarter) for `0x0000..0x3fff`, the ROM word at `0x4d9c` (`0x1affffef`) for `0x4000..0x7fff`, the ROM word at `0x9b5c` (`0xe59f42b0`) for `0x8000..0xbfff`, 0 for `0xc000..0xffff` (M2-682, M2-683). Inferred: each quarter returns the word its ROM data path last held, plausibly the last word the CPU fetched from that quarter (M2-683). An address selector with any of bits 8..15 set uses r0, not its low byte (M2-615). Destination 10 writes the loaded word to all GPRs and not to the PC (M2-616); destination 8 and the other extended destinations are open. A load from the unmapped bus address 0x50000000 hangs the whole system, also when the CPU reads no FMSS register during the run; only a power cycle recovers it (M2-718,

M2-611).

• Timing: from SDRAM 199.5..211 ns, 6.6..7 cycles (M2-102, M2-378).

• Original programs: used to read per-request lists (chip enables, DMA addresses).

• Status: confirmed.

• Evidence: M2-05, M2-48, M2-102, M2-108, M2-165, M2-378, M2-432, M2-611, M2-615, M2-616, M2-682, M2-683.

04 ld

• Read a window register, keeping the bits selected by a mask.

• Syntax: `ld rR, [REG], mask`.

• Encoding: R = destination, arg = window byte offset, imm = AND mask (`0xffffffff` keeps the whole word).

• Operation: rR = window[arg] AND imm. Reading has the side effects of the selected register: FIFO reads are not ordinary storage reads (chapter 9). Read by the program, `C08` returns 5 in the tested states (M2-70, M2-384), and `0xc38` returns the instruction's own offset (M2-162).

• Register numbers: destination 10 writes the value to all GPRs and not to the PC; destination 8 discards it: no jump, no GPR changes and the next instruction runs (M2-616), unlike `movi`, `mov` and add into r8 (M2-43).

• Timing: from GPR, PRAM, FMC or ECC registers 60.6..61.2 ns, 2 cycles (M2-102, M2-378).

• Original programs: used for status tests and read-modify-write of control words.

• Status: confirmed.

• Evidence: M2-43, M2-70, M2-102, M2-162, M2-378, M2-384, M2-616; device ALU, PRAM and indirect cases.

05 movi

• Load an immediate.

• Syntax: `movi rR, #imm`.

• Encoding: R = destination, imm = value; arg = 0 (1, 0xa, 0x1234, 0xd70, 0x8000 and 0xffff are ignored, M2-165).

• Operation: rR = imm.

• Register numbers (all 256 tested, M2-43): R = 8 writes the PC and jumps to the local target `imm` (M2-38); R = 10 writes all eight GPRs r0..r7 (M2-39, M2-43); R = 9 and 11..255 change no GPR.

• Timing: 30.1 ns, 1 cycle (M2-102).

• Original programs: used (constants, markers, loop counters).

• Status: confirmed. Other state effects of the extended destinations and the width of an r8 target beyond the local tests are open.

• Evidence: M2-38, M2-39, M2-43, M2-102, M2-165.

06 mov

• Copy a register.

• Syntax: `mov rR, rA`.

• Encoding: R = destination, arg = source; imm = 0 (the tested non-zero imm values are ignored, M2-165). A source field with any of bits 8..15 set (0x103, 0x104, 0x108, 0x10a, 0x1ff, 0x803, 0x8003, 0xff03) reads neither the low-byte register nor r0 but the value the last source selector before it read: r0 after `movi`, r5 after `mov` from r5, the old r3 after `add r3,r3,#1`, the instruction offset after a `mov` with source 8, kept over a `nop2` and over an `ld`, `st` or `wr` (none of them sets it), and 0 when only `ld`, `st` or `wr` ran before it since the reset (M2-364, M2-614, M2-618, M2-674..M2-677).

• Operation: rR = rA. A source written by the preceding instruction is used at distance 0 (M2-108).

• Register numbers (all 256 tested in each role): destination 8 is the PC, 10 writes all eight GPRs, 9 and 11..255 change no GPR (M2-43); source 8 returns the current instruction offset (M2-38), source 10 the destination register's own value (M2-141), sources 9 and 11..255 r0 (M2-40). With the broadcast destination 10, source 8 writes the instruction offset to every GPR and source 10 makes each GPR its own source (M2-365); destinations 9 and 11..15 with source 8 or 10 write no GPR and leave the register-bus latch (M2-678, M2-679).

• Timing: 30.1 ns, 1 cycle (M2-378).

• Original programs: used.

• Status: confirmed.

• Evidence: M2-38, M2-40, M2-43, M2-108, M2-141, M2-165, M2-364, M2-365, M2-378, M2-614, M2-618, M2-674..M2-679.

07 wait

• Block until a hardware event is set.

• Syntax: `wait N`.

• Encoding: R = event number. arg and imm are used by `wait 12` as the status operand and mask (original programs use 0x40/0x40, and 0x20 for enhanced status); for the other events the originals leave them 0; `wait 11` gives the same outcome with (0x40, 0x40) and (0xffff, 0xffffffff) (M2-387), and for the other events whether they are ignored is open.

• Operation: the engine waits (C08 = 4) until event N is set. If the timeout is enabled and expires, a fault is raised (CS_IRQ bit 2) and execution continues at the fault vector or at the next instruction according to C58 bit 2; bits 1, 4, 6 and 7 change neither path (M2-11, M2-612). For a wait entered at the start of a run the timeout is 492.747 us × (count + 1) + 447.5 us for start words `count << 8 | 0xb5`, counts 8..256 (M2-81), 2^14 engine cycles per count (inferred, M2-103). The fault falls on instants fixed relative to the start of the run (447.5 + 492.747 k us for n = 0xb), not a fixed time after the wait entry: with count 0 a wait entered 4..407 us after the start faults at 446..448 us and one entered at 477 or 544 us at 940 us (M2-433). Counted from the wait's entry, a wait of count c ≥ 1 faults at the (c + 2)-th instant and a wait of count 0 at the first (M2-584); the instants are fixed to an event before the start of the run, not to the start itself (M2-588): the per-case reset sets the prescaler phase, which then runs during the CPU parameter writes (about 0.08 us earlier per extra write), and n = 4..6 fit the same phase within the 2-us resolution (M2-631); start-word bits 7:4 = n select a step of 2^3 cycles for n = 2, 2^5 for n = 3 and 2^(n+3) for n = 4..11, saturating at n = 11 (M2-115, M2-216). n = 0 faults at once (0..4 us for every tested count); n = 1 faults after about 2 engine cycles per count (2 us for count 0x0f, 4 us for 0x40, 8 us for 0x80, 14..16 us for 0xff; M2-115).

• Events (chapter 7):

  • 0, 1, 2: command sent, address sent, DMA done (FMC_STATUS bits 1, 2, 20). A wait does not consume its flag: repeated and triple waits pass; an acknowledgement blocks the next wait; a new trigger re-arms it (M2-66).

  • 3: NAND data transfer done (original programs).

  • 4: ECC done: ECC_STATUS (`0x810`) bit 1 after an encode, bit 2 after a decode, both write-1-to-clear; writing 1 clears only bit 0 (M2-67, M2-97, M2-99).

  • 5: passes after a status latch, when the chip reports ready (M2-130, M2-193); among events 5..10 and 12..15 only event 5 passed in seven prepared states (M2-111). The standard latch (`70h`, `wait 0`, acknowledge, latch) passes every time (M2-161), and with a waited and acknowledged `70h` every tested ordering passes (M2-130). With an unwaited `70h`, a latch straight after it passes, and a latch one instruction after it (an acknowledge, or one `movi r5,#0` filler) faults; 20 or 200 filler instructions before the latch pass (M2-68, M2-178); the two faulting orderings fault straight after a reset and after any of eight different preceding programs (M2-390). Measured in `nop2` units, a latch 0 or 3..20 instructions after the unwaited `70h` passes and one 1 or 2 instructions after it faults: it sets FMC_STATUS bit 3 instead of bit 23 (M2-593). After a passed latch and only the acknowledge of bit 23, a second latch without a new `70h` passes (M2-494). The latch rules are in chapter 7, section 7.5.

  • 11: passes at once in every tested state (idle with C58 = 0 or 4, timeout disabled, DMA or encode running, command or status latch pending, every flag acknowledged, C10 = 0, `0x044` all ones, the chip busy in a page

read, a reset or a block erase) and costs 37.7 ns, 1.25 cycles, as `nop` (`08`) and a passing `wait 0` (M2-10, M2-69, M2-107, M2-387).

• 12: passes iff status AND arg AND mask ≠ 0 (M2-134); against a busy chip it releases only when the chip signals ready (M2-130, M2-137); it passes with FMC_STATUS bit 23 cleared (M2-68).

• 8: never passes, also after a complete page read with DMA out (M2-206).

• 15: fails in both orders for all five clear-enable stimuli (M2-69).

• 16..255: never pass in eight states, with no modulo-16 alias (M2-159).

• 6..10, 13, 14, 15 time out after every further tested stimulus: unacknowledged DMA, data and spare reads, a FIFO READ ID, a metadata write, BCH8 slot-1 and BCH16 encodes, `0x81c` = 1, commands on CE1..CE3, a pending `sig`, an uncorrectable and an erased-sector decode completion, a command with `0x044` all ones, CPU writes of C40, C44, C50, C14, C5C and C3C, and all VIC0 or VIC1 software lines (M2-386).

• No event with arg = mask = 0 follows NAND ready/busy (M2-107). CPU writes to C0C, C10, C48, C54 and C60..C6C release none of events 6..10, 12..15 (M2-143).

• Original programs: `wait 0..4` and `wait 12` synchronize peripheral and status work (M2-19).

• Status: confirmed for the listed events; events 6..10, 13, 14 have no established source.

• Evidence: M2-10, M2-11, M2-19, M2-66..M2-69, M2-81, M2-97, M2-99, M2-107, M2-111, M2-115, M2-130, M2-134, M2-137, M2-143, M2-159, M2-161, M2-178, M2-193, M2-206, M2-216, M2-386, M2-387, M2-390, M2-433, M2-494, M2-584, M2-588, M2-593.

`08` nop

• No operation.

• Syntax: `nop`.

• Encoding: the zero form R = arg = imm = 0; R = 1..7, arg = 1, 2, 0xd60, 0x8000, 0xffff and imm = 1, 0xd64, 0x80000000, 0xffffffff continue as the zero form (for R = 1, 3, 7 and the arg/imm values the GPRs and the whole window equal those of the zero form). Every R = 8..0xff (also with arg 0xffff and imm 0xffffffff) does not continue: the instruction holds at its own offset until the timeout fault at the time of an unmet `wait 3` (4388..4390 us with start word `0x07b5`, CS_IRQ 5), also with a pending command, address, DMA, ECC or status flag for R = 8..15 (M2-377, M2-625). With the timeout disabled (start word 1) R = 8 and 0xff hold with C08 = 4, the value of an unmet `wait 3`: the hold is a wait state (M2-685, M2-686).

• Operation: execution continues with the following instruction. As a predecessor of an extended `st`, it leaves r0 in the register-bus latch (M2-47).

• Timing: 37.65 ns, 1.25 cycles (M2-102, M2-378), the cost of a passing wait.

• C14: a loop of `nop` with start word `0xffb5` leaves C14 = 0xff (start-word bits 15:8), as a passing `wait 11` does (M2-625). `08` acts as a wait whose condition holds for R < 8 and that no tested stimulus satisfies for R ≥ 8 (M2-625, M2-686).

• Original programs: not used.

• Status: tested forms: zero form (M2-17), the three M1-comparison forms in idle, DMA and ECC states (M2-34, M2-65) and every R = 0..0xff (M2-625). The condition waited for with R ≥ 8 is unknown (chapter 17, item 4).

• Evidence: M2-17, M2-34, M2-65, M2-102, M2-377, M2-378, M2-625, M2-685, M2-686.

`09` wait2

• Block until an interrupt request line of the CPU is raised.

• Syntax: `wait2 N`.

• Encoding: R = line number N; arg and imm are ignored: (0, 0), (0x40, 0x40), (0, 0x40), (1, 0xd64), (0xd60, 0xffffffff) and (0xffff, 0) give the same outcome (M2-187).

• Operation: `wait2 N` continues once interrupt request line N is raised; N numbers VIC0 lines as on M1 (M1-F174). Otherwise it waits for the timeout and faults like an unmet `wait`. A VIC software interrupt (SOFTINT)

is visible in the raw VIC status but releases none of 59 lines; inferred: `wait2` samples the peripheral request lines before the VIC's software OR (M2-140).

- Lines seen to release it:
  - 5: always (M2-74, M2-140).
  - 8: when the timer interrupt fires (M2-140).
  - 16: DMAC0 activity (M2-140, M2-144).
  - 19: the USB interrupt; passes depending on USB traffic (M2-74, M2-144).
  - 20: the NAND controller interrupt. `wait2 20` follows VIC0 line 20, which is raised while a flag routed by FMC `0x044` (bits 0, 1, 2, 11, 12) is pending, not the sticky CS_IRQ bit 4; with `0x044` all ones it passes 4–7 us after a DMA, a command or a status latch, with `0x044` = 0 never (M2-142, M2-200, M2-204). As on M1, the controller's sources share line 20 (M1-F175).
- No N in 0..255 tracks a NAND erase, program or read busy period (M2-167). Numbers 32..63 (VIC1) never pass, in every state, with a VIC1 software interrupt raised, and for 22, 53, 54, 55 although VIC1 raw bit 22 was set during some runs (M2-211). Numbers 64..255 never pass (M2-34, M2-74).
- Open: no ECC operation raises a line, so the ECC engine's line and enable are not found (M2-212, M2-217); the device behind line 5 is not identified.
- Original programs: not used.
- Status: partial.
- Evidence: M2-74, M2-140, M2-142, M2-144, M2-167, M2-187, M2-200, M2-204, M2-211, M2-212, M2-217.

`0a` and

- Bitwise AND.
- Syntax: `and rR, rA, #imm` / `and rR, rA`.
- Encoding: R = destination, arg = source; non-zero imm gives rR = rA AND imm, zero imm gives rR = rR AND rA (chapter 5.5).
- Register numbers: source 10 is the destination itself in both forms: with r0 = `11001111` and r1 = `22002222`, register-form `and r1` through source 10 gives `22002222` (r1 AND r1), where M1 would use r0 (M2-44, M2-141).
- A register-form source with any of bits 8..15 set reads 0 with no source read before it (M2-364, M2-614); after a source read, the value that the last source selector read is measured for `mov`, `add`, `or` and `sub` (M2-618).
- Timing: 1 cycle (ALU, M2-102).
- Original programs: used.
- Status: confirmed.
- Evidence: M2-06, M2-44, M2-52, M2-65, M2-141, M2-364, M2-614, M2-618.

`0b or`

- Bitwise OR.
- Syntax: `or rR, rA, #imm` / `or rR, rA`.
- Encoding: as and.
- Register numbers: register-form source 10 gives r1 OR r1 = `22002222` in the same test (M2-44, M2-141).
- A register-form source with any of bits 8..15 set reads the value that the last source selector read (r0 after `movi`; M2-364, M2-614, M2-618).
- Timing: 1 cycle (ALU, M2-102).
- Original programs: used.
- Status: confirmed.

• Evidence: M2-44, M2-52, M2-65, M2-141, M2-364, M2-614, M2-618.

0c add

• Add modulo 2^32.

• Syntax: `add rR, rA, #imm` / `add rR, rA`.

• Encoding: as and; non-zero imm gives rA + imm, zero imm gives rR + rA.

• Operation: wraps modulo 2^32 (M2-06). Operands written by the preceding instruction are used at distance 0 (M2-108).

• Register numbers: register-form source 10 gives r1 + r1 = 44004444 (M2-44, M2-141). Destination 8 jumps locally and destination 10 writes all eight GPRs, tested with immediate 1 (M2-43).

• A register-form source with any of bits 8..15 set reads the value that the last source selector read (r0 after `movi`; M2-364, M2-614, M2-618).

• Timing: 30.1 ns, 1 cycle (M2-102).

• Original programs: used.

• Status: confirmed.

• Evidence: M2-06, M2-43, M2-44, M2-102, M2-108, M2-141, M2-364, M2-614, M2-618.

0d sub

• Subtract modulo 2^32.

• Syntax: `sub rR, rA, #imm` / `sub rR, rA`.

• Encoding: as and; non-zero imm gives rA - imm, zero imm gives rR - rA.

• Operation: wraps modulo 2^32, including results below zero (M2-06).

• Register numbers: register-form source 10 gives r1 - r1 = 0 (M2-44, M2-141). Destination 10 with an immediate writes the result to all GPRs and not to the PC (M2-616).

• A register-form source with any of bits 8..15 set reads the value that the last source selector read (r0 after `movi`; M2-364, M2-614, M2-618).

• Timing: sub plus a taken bnz 67.6 ns, 2.25 cycles (M2-102).

• Original programs: used (equality tests by subtraction).

• Status: confirmed.

• Evidence: M2-06, M2-44, M2-102, M2-141, M2-364, M2-614, M2-616, M2-618.

0e bnz

• Branch if non-zero.

• Syntax: `bnz rR, target`.

• Encoding: R = tested register, imm = target byte offset from the program start; arg = 0.

• Operation: if rR ≠ 0, pc = imm; otherwise execution continues. The low three target bits are ignored (+1, +2, +4 land on the aligned instruction, +8 on the next); targets are not reduced modulo 2^15..2^24 and land exactly up to 2^24 + 0x100 (PC ≥ 25 bits) (M2-105, M2-725). A condition register written by the preceding instruction is used at distance 0 (M2-108).

• Register numbers: tested registers 8..255 never branch, for zero and non-zero controls (M2-46).

• Timing: sub plus a taken bnz 67.6 ns, 2.25 cycles (M2-102).

• Original programs: used (loops, status tests).

• arg values 1, 0xff and 0xffff are ignored; the branch is still taken (M2-165).

• Status: confirmed.

• Evidence: M2-46, M2-102, M2-105, M2-108, M2-165.

0f jr

• Jump through a register, conditionally.

• Syntax: `jr rA` (condition register in R).

• Encoding: R = condition register, arg = register holding the target; imm 0, 1 and 0xffffffff give the same result.

• Operation: if r[R] ≠ 0, pc = r[arg]; otherwise execution continues (R = 0..7, 16 of 16 cases, M2-110). With R = 0 and r0 = 0 it never jumps (M2-109). The target is a byte offset from the program start with the low three bits ignored and no wrap at 2^15..2^20, as for `bnz` (M2-122). A target outside the program fetches from the system memory at that offset, so the outcome depends on that memory (M2-144). All GPRs keep their values: no link value is saved (M2-34, M2-46).

• Register numbers: target selector 8 (the current instruction offset) loops on the instruction itself; selectors 9..255 and every selector with any of bits 8..15 set (0x100, 0x101, 0x105, 0x1ff, 0x8005) use r0 (M2-42, M2-46, M2-673). Extended R values are open.

• Original programs: not used.

• Status: confirmed for R = 0..7 and local targets.

• Evidence: M2-34, M2-42, M2-46, M2-109, M2-110, M2-122, M2-144, M2-673.

10 sig

• Signal the CPU and continue.

• Syntax: `sig`.

• Encoding: R = arg = imm = 0; R/arg/imm = 3/1/0xd64 and 7/0xd60/0xffffffff, R 1 and 0xff, arg 0xffff and imm 1, 0x80000000 and 0xffffffff give the same result: the fields are ignored (M2-34, M2-370).

• Operation: CS_IRQ (C0C) bit 1 is set and execution continues; at the following END, CS_IRQ = 3. Writing 2 to C0C clears the bit (write 1 to clear) and the next `sig` sets it again. No VIC0 or VIC1 raw line rises beyond the run's baseline (M2-370).

• Timing: 30.2 ns, 1 cycle (M2-370).

• Original programs: not used.

• Status: tested forms: the M1 expectation matches in the three forms in idle, DMA and ECC states (M2-34, M2-65), and in the forms of M2-370.

• Evidence: M2-34, M2-65, M2-370.

11 sti

• Store a word to system memory.

• Syntax: `sti [rA], rR`.

• Encoding: R = source register, arg = register holding the bus address; imm = 0 (the tested non-zero imm values are ignored, M2-165).

• Operation: mem32[rA] = rR, little-endian; the tested unaligned addresses round down to the word boundary (M2-05). A data register written by the preceding instruction is used at distance 0 (M2-108).

• Register numbers: an address selector with any of bits 8..15 set (0x105) uses the value the last source selector read: r0 straight after `movi` instructions, r5 after `mov r2` with selector 5 (M2-680, M2-681), the rule of the high-bit register sources of `mov` and the ALU (M2-618).

• Timing: to SDRAM 97.8 ns, 3.25 cycles (M2-102, M2-378).

• Original programs: used.

• Status: confirmed. Extended source selectors are not measured.

• Evidence: M2-05, M2-102, M2-108, M2-165, M2-378, M2-680, M2-681.

12 pause

• Stop and hand control to the CPU.

• Syntax: pause.

• Encoding: 20 forms pause: R = 0..15 with arg = imm = 0, and R = 2 with arg = 0xd60 and imm = 0, 0xff00 or 0xffffffff, or arg = 0xd64 and imm = 0xffffffff (chapter 16); the M1-comparison forms 3/1/0xd64 and 7/0xd60/0xffffffff also pause (M2-34). Labels of a masked comparison in these forms do not change the behaviour.

• Operation: the engine stops with the PC on the pause instruction, C08 = 0x0a and CS_IRQ bit 3 (IRQ 8); C00 reads `0x4ffb4`; r6 is unchanged (M2-14, M2-25). The engine stays paused until the CPU acts. The CPU resumes it by writing the PC of the following instruction, acknowledging CS_IRQ bit 3 (C0C = 8) and writing a start word (C00 = 0x4ffb5, 0xfff5 or 1); the program then runs to END. With the CPU writes at 100–101 us, END follows at 102–103 us (M2-25, M2-34). The acknowledgement is not required: PC = offset + 8 and a start word continue the program with or without it (11 cases), and without the PC write it stays paused (12 cases) (M2-188). With two start words written straight after the PC write the program continued in three of four runs and stayed paused in one (M2-512).

• Original programs: no program uses it; the original CPU handler (`bl_NandReadOnly` `mem:0x4d08`) advances PC by 8, acknowledges IRQ 8 and sets start bit 0.

• Reset from pause: the STOP poll of an ordinary reset did not return a paused engine to idle (M2-15). A paused run is recovered by resume to END (3–4 us), and an inherited pause by resume plus abort (M2-26, M2-75). See chapter 8.

• Status: tested forms.

• Evidence: M2-14, M2-25, M2-26, M2-34, M2-75, M2-188, M2-512.

13 shl

• Logical shift left.

• Syntax: `shl rR, rA, #imm` / `shl rR, rA`.

• Encoding: as and; non-zero imm is the count, zero imm shifts rR by the count in rA.

• Operation: logical. Counts below 32 shift normally; every count of 32 or more gives 0, in the immediate and the register form: counts 32, 33, 63, 64, 255, 256 and 0xffffffff on 1, 0x80000000 and 0xffffffff, and counts 32..255, 2^8..2^31, 0x101, 0x11f, 0x1001f, 0x8000001f and 0xfffffffe1 on 0x80000001. There is no modulo-32, 64 or 256 and no 8-bit-immediate aliasing of the count (M2-07, M2-146).

• Register numbers: register-form source 10 uses the destination as the count: with r0 = 1 and r1 = 3, `shl r1` through source 10 gives 24 (3 << 3), through source 0 it gives 6 (M2-52, M2-141). All eight GPR destinations and eight value pairs match.

• A register-form source with any of bits 8..15 set reads 0 with no source read before it (M2-364, M2-614); after a source read, the value that the last source selector read is measured for `mov`, `add`, `or` and `sub` (M2-618).

• Timing: 1 cycle (ALU, M2-102).

• Original programs: used.

• Status: confirmed.

• Evidence: M2-07, M2-52, M2-141, M2-146, M2-364, M2-614, M2-618.

14 shr

• Logical shift right.

• Syntax: `shr rR, rA, #imm` / `shr rR, rA`.

• Encoding: as `shl`.

• Operation: logical, including the high sign bit; counts below 32 shift normally and every count of 32 or more gives 0, with the counts of `shl` (M2-07, M2-146).

• Register numbers: register-form source 10 uses the destination as the count: with r0 = 1 and r1 = 3, source 10 gives 0 (3 >> 3), source 0 gives 1 (M2-52, M2-141).

• A register-form source with any of bits 8..15 set reads 0 with no source read before it (M2-364, M2-614); after a source read, the value that the last source selector read is measured for `mov`, add, `or` and `sub` (M2-618).

• Timing: 1 cycle (ALU, M2-102).

• Original programs: used.

• Status: confirmed.

• Evidence: M2-07, M2-52, M2-141, M2-146, M2-364, M2-614, M2-618.

15 `tmwait`

• Wait on the timer.

• Syntax: `tmwait`.

• Encoding: the zero form R = arg = imm = 0; the fields are ignored: 24 operand forms (M2-27) and R 1, 0xff, arg 1, 0xffff, imm 1..0x10000 give the zero-form time (M2-373).

• Operation: the engine waits for start-word count (bits 15:8) timeout instants, then continues without a fault: counted from its entry it ends at the (count + 1)-th instant, one instant before an unmet `wait` of the same count, also when entered after the first instant (M2-584). Entered at the start of a run it ends 492.747 us × count + 450 us after the start (within 3 us; 943, 3898, 31986 and 126103 us for counts 1, 7, 64 and 255), one 2^14-cycle step less than the `wait` timeout of the same start word (M2-27, M2-373). With count 0 it does not end: it takes the timeout fault at the first instant, as a count-0 `wait` (M2-584). C58 = 0 gives the same END, and program writes of C14 (0 .. 0xffffffff) before it do not change the time: 126101..126102 us at count 255 against 126099..126101 us without a write, and no change with 0x1000 at counts 0x10 and 1; single runs ended 24 us (0x1000, two captures) and 13 us (0x800) early without recurring (M2-373, M2-629). In ten further count-255 runs, five with a C14 = 0x800 write and five without, one run without a write ended 23 us early and the five with it at 126101..126102 us: the isolated early ends are run-to-run spread, not an effect of the write (M2-689, M2-690). During the wait C14 counts down by one per timeout step from start-word bits 15:8, and a completed `tmwait` leaves C14 = 0 (M2-13, M2-27, M2-172, M2-373). The forms also end in the idle, DMA and ECC states (M2-34, M2-65). Consecutive `tmwaits` each wait a full period; one started with C14 = 0 does not reload and end at once (M2-150).

• C48: with C48 = 0xffffffff or 0x55555555 a `tmwait` does not end after its period (3879..3900 us at count 7) but faults at the `wait` timeout (4388..4390 us) in every run; C48 = 0 and the tested presets of C10, C3C and C54 leave it unchanged (M2-179). At count 1, each single bit 1..7 and 0xaaaaaaaa make it fault at the `wait` timeout (1434..1436 us, C14 0x7f), bits 8..31 change nothing (941..943 us, C14 0), and bit 0 ends it one instant early (449 us, C14 = 1) (M2-627). At count 7, C48 = v = 1..7 ends it v instants early (3406 .. 449 us against 3896 us for C48 = 0, 493 us per unit) with C14 = v, and C48 = 8 makes it fault at the `wait` timeout (4390 us) (M2-687). A `tmwait` ends at the instant at which its C14 count reaches C48 bits 7:0; a value above the count is never reached and gives the timeout fault (M2-688).

• Original programs: not used.

• Status: tested forms.

• Evidence: M2-13, M2-27, M2-34, M2-65, M2-150, M2-172, M2-179, M2-373, M2-584, M2-627, M2-629, M2-687, M2-688, M2-689, M2-690.

16 `evjmp`

• Wait for an event, then jump to the fault vector without a fault.

• Syntax: `evjmp`.

• Encoding: R = arg = imm = 0; the fields are ignored: R/arg/imm = 3/1/0xd64 and 7/0xd60/0xffffffff (M2-34, M2-65), and R = 1, 2, 4, 0xff, arg = 1, 2, 0xffff, imm = 0xffffffff or a program label give the zero-form outcome

for every tested source; the target is always C4C, never imm (M2-391).

• Operation: with a command, address, data, DMA, ECC or status event the engine continues at the fault vector with CS_IRQ bit 2 clear (CS_IRQ = 1 at the vector's END) (M2-58). It catches an event that arrives during the wait (a command flag 100, 1000 or 5000 us into it) or is already pending (DMA), jumps to C4C for C58 = 0, 2 and 4 with CS_IRQ bit 2 clear and leaves the flag set (FMC_STATUS `0xc3002` / `0x1c3000` at the target). The handler sees the source's flag: FMC_STATUS bit 1 and C50 bit 0 for a command, bit 20 and C50 bit 2 for a DMA, ECC_STATUS `0x13` and C50 bit 4 for an encode, bit 23 and C50 bit 5 for a status latch; the departure comes 3..25 us after the start (M2-391). An acknowledged flag is no event, and a `70h` written through `+0x408` during the wait raises none. With no event it times out after the full period and continues at the next instruction with C58 = 0, through the fault vector with C58 = 4 (CS_IRQ bit 2 set, CS_IRQ = 5) (M2-58, M2-186).

• Timeout disabled (start word 1): `evjmp` without an event stays at C08 = 0x0b, with the PC on the `evjmp` and CS_IRQ = 0, and survives STOP (M2-57, M2-62). Abort (C00 = 2) gives C08 = 3; a forced reset (C00 = 0x80000) before another STOP gives C08 = 0, after which the full reset and the next independent program succeed (M2-62, M2-63). This recovery is repeatable (16 of 16, M2-75). Other CPU C00 orders on the blocked state: STOP leaves 0x0b (C00 `0x40000`), abort gives 3 (C00 `0x40002`), STOP then abort 3, abort then STOP 0x0b; `0x80000` alone or after STOP or abort gives C08 = 0, C00 `0x7f60` and PC 0, and STOP after it C08 = 0 with C00 = 0 (M2-392). See chapter 8.

• Original programs: not used.

• Status: tested forms.

• Evidence: M2-57, M2-58, M2-62, M2-63, M2-75, M2-186, M2-391, M2-392.

17 bz

• Branch if zero.

• Syntax: `bz rR, target`.

• Encoding: R = tested register, imm = target byte offset; arg = 0 (1, 0xff and 0xffff are ignored, M2-165).

• Operation: if rR = 0, pc = imm; otherwise execution continues.

• Register numbers: tested registers 8..255 never branch, even when r0 is zero (M2-46).

• Original programs: used, alone or paired with `bnz`.

• Status: confirmed. The target-bit rule is measured for `bnz` and `jr` (M2-105, M2-122); for `bz` it is not measured separately.

• Evidence: M2-46, M2-65, M2-165.

18 pld

• Load a window word through a register.

• Syntax: `pld rR, [rA]`.

• Encoding: R = destination, arg = register holding a window byte offset; imm = 0 (the tested non-zero imm values are ignored, M2-165).

• Operation: rR = window[rA & 0xfff]: only offset bits 11:0 are used. Offsets with any of bits 12..31 set (bits 12, 13, 16, 20, each of 17..30, and `0x1d64`, `0x2d64`, `0x8d64`, `0xfd64`, `0x10d64`, `0x80000d64`) and the aliases `0x1c18`, `0x1c1c`, `0x1c28`, `0x10c28`, `0x1d68`, `0x10d70`, `0xfd6c` reach the word at offset & 0xfff. Through `0xc18..0xc34` it reads r0..r7; through the pointer register r1 itself it reads the pointer. An unaligned offset (low bits 1..3) reads 0 (M2-08, M2-175).

• Register numbers: address selector 8 at instruction offset 0x60 reads 0; selectors 9..255 use r0 (M2-48); with a selector with any of bits 8..15 set the access behaves as one through r0 (M2-615). Destination 10 writes the loaded word to all GPRs and not to the PC; destination 8 discards it, with no jump (M2-616). The other extended destinations are open.

• Original programs: not used (none of the ten M2 microcode programs in `code/*/fmss_asm/` contains it).

• Status: confirmed. These M2 measurements supersede M1's "3G only" qualification for this opcode.

• Evidence: M2-08, M2-48, M2-165, M2-175, M2-615, M2-616.

19 pst

• Store a register to a window word through a register.

• Syntax: `pst [rA], rR`.

• Encoding: R = source register, arg = register holding a window byte offset; imm = 0 (the tested non-zero imm values are ignored, M2-165).

• Operation: window[rA & 0xfff] = rR, with the offset rule of `pld`: the store reaches the word at offset & 0xfff and changes no other word; through `0xc18..0xc34` it writes r0..r7 (M2-175); an unaligned offset (low bits 1..3) drops the write (M2-08, M2-175). Stores to `0xd80..0xdfc` do not persist (M2-147). With an address selector with any of bits 8..15 set the store behaves as one through r0: it changes no observed word (M2-615).

• Original programs: not used (none of the ten M2 microcode programs contains it).

• Status: confirmed.

• Evidence: M2-08, M2-147, M2-165, M2-175, M2-615.

1a..fe and other values without an established meaning

• Every opcode byte other than the 19 common instructions runs in three operand forms under C58 = 0 and C58 = 4, and seven control opcodes run with command, address, DMA, ECC and status stimuli; all execute without cleanup loss (M2-64, M2-65). The signatures of the control opcodes are consistent with the named set (`08`, `0f`, `10`, `12`, `15`, `16`, `ff`).

• Every value `1a..fe` is an undefined opcode that behaves exactly as an unmet `wait`. With the timeout enabled it faults (CS_IRQ bit 2) at the same time as an unmet `wait 3` (M2-81) and continues at the next instruction or at the fault vector according to C58 bit 2 (CS_IRQ = 5 at the END; M2-11, M2-282). With the timeout disabled it stays at C08 = 4 without a fault (M2-56). It changes no GPR, scratch, PRAM or window word in the three operand forms (M2-282).

• With a pending command, address, DMA, ECC or status flag every value `1a..fe` behaves the same: the next instruction is not reached and the timeout fault comes at the time of an unmet `wait 3` (M2-583).

• All 228 values `1b..fe` give exactly the outcome of 1a in every scanned state, operand form and C58 setting; only `ff` (`nop2`) differs. No value in `1a..fe` acts as a hidden instruction in the tested forms, and none is distinguishable from an unmet `wait`: there is no distinct illegal-instruction trap (M2-282). M1 undefined opcodes behave the same (M1-F052), with the M2 timeout length (M2-81).

• With 64 random R/arg/imm words 1a gives the zero-form outcome (an unmet wait to the timeout fault, GPRs unchanged) in every case: its operand fields have no effect (M2-684).

• Status: tested forms.

• Evidence: M2-11, M2-56, M2-64, M2-65, M2-81, M2-282, M2-583, M2-684.

ff nop2

• No operation (second encoding).

• Syntax: `nop2`.

• Encoding: R = arg = imm = 0; R/arg/imm = 3/1/0xd64 and 7/0xd60/0xffffffff give the same result. An all-ones (erased) word, op 0xff with R 0xff, arg 0xffff and imm 0xffffffff, executes as `nop2`: runs of 1, 8 and 64 such words continue to the END, as does one word with a command, address, DMA, ECC operation or status latch pending (M2-367).

• Operation: execution continues with the following instruction.

• Timing: 30.1 ns, 1 cycle, the same as `movi` (M2-102, M2-378); an all-ones word 30.2 ns (M2-367).

• Original programs: not used.

• Status: tested forms, in idle, DMA and ECC states (M2-34, M2-65), and the all-ones word (M2-367).

- Evidence: M2-34, M2-65, M2-102, M2-367, M2-378.

6.3 Test coverage

Matrix	Cases	Content
confirmation	531	the 19 common instructions: explicit results, all eight MOV destinations, source/destination aliasing, immediate-zero forms, wrap and shift edges, taken and untaken branches, RAM and window effects, idle and pending/cleared waits
M1 comparison	78	M1 hypotheses for `nop`, `nop2`, `wait2`, `jr`, `sig`, `pause`, `tmwait`, `evjmp`
opcode scan	1,422	every other opcode byte in three operand forms under C58 = 0 and C58 = 4
control-opcode states	105	seven control opcodes with command, address, DMA, ECC and status stimuli, three operand forms each

All 531 confirmation and 78 M1-comparison cases match; source selector 10 differs from the M1 prediction and follows the M2 rule (M2-65, M2-141). The scan and state matrices record changed registers, scratch bytes, instruction latches, control state and IRQs, with separate fall-through and vector markers; they are signatures, not assigned mnemonics. A later full execution of all families reproduces the earlier outcomes except 79 of 31,974 cases whose difference has an identified cause: USB and DMAC line activity, a shorter fault window turning TIMEOUT into FAULT, `jr` targets in SDRAM outside the program, and previously unrun CPU-write cases (M2-144). Cleared-ECC `wait` controls clear all ECC status flags with `ECC_STATUS = 0x1ff` (writing 1 clears only bit 0, M2-97). The families and acceptance criteria are in chapter 13.

Chapter 7 — Events, waits and the timeout

Summary: the hardware flags that `wait` (07) waits for, the interrupt request lines that `wait2` (09) waits for, the status latch behind `wait` events 5 and 12, the acknowledge rules, the timeout and its length, and the other event-like opcodes. Finding IDs (M2-nn) refer to chapter 18; M1-Fnnn are the M1 findings listed there.

Two separate number lists. As on M1, `wait` and `wait2` both take a number N in their R field, but the numbers name different things. "`wait` event N" is an entry of the event table in section 7.2; "`wait2` condition N" is an interrupt request line (section 7.7). `wait2` condition 5 has nothing to do with `wait` event 5.

7.1 The `wait` instruction

`wait N` (07, R = N) blocks the program until event N is set, then continues with the next instruction. An event that does not come ends in a timeout (section 7.9) when start-word bit 2 is set; with start word 1 (bit 2 clear) an unmet `wait 3` stays waiting with `CS_STATUS` (C08) = 4 and no fault (M2-56). The timeout raises `CS_IRQ` bit 2 and continues at the fault vector C4C when C58 bit 2 is set, or at the next instruction when it is clear (chapter 8, section 8.15; M2-11). An illegal opcode faults at the same time as an unmet `wait` (M2-81).

The arg and imm fields are not ignored for every event: `wait 12` uses them as the operands of its status test (section 7.6). For the other events the tested forms use arg = imm = 0.

How the original programs use the events (bootloader `bl_NandRead0nly` RESET/READID/READ and restore-OS WRITE/ERASE, chapter 11; M2-19): `wait` events 0..4 follow the command, address, DMA, data and ECC operations, each followed by its acknowledge write, and `wait` event 12 with arg = mask = 0x40 (also 0x20 for enhanced status) qualifies the NAND status.

7.1.1 Idle behaviour

With nothing pending, `wait 11` reaches END in about 4 us and the other fifteen events 0..15 fault after about 126.6 ms under the tested control state (M2-10). A passing `wait 11` costs 37.4..37.7 ns (1.25 cycles at 33.25 MHz), as `nop` (08) and as a passing `wait 0` whose flag is pending (M2-387). `CS_IRQ` then reads 5 once the fault handler reaches END: bit 2 (fault) and bit 0 (END) are separate flags. A bare `wait 12` with zero operands and no prepared status transaction faults; this is consistent with the operand rule of section 7.6.

7.2 Wait events

event	source (FMC_STATUS = FMC 048)	set when	acknowledge	C50 bit	details
0	FMC_STATUS bit 1	a command byte has been sent (after the bus cycle)	FMC_STATUS = 2	0 (from the third instruction after the command write)	M2-66, M2-112, M2-152
1	FMC_STATUS bit 2	the address cycles have been sent	FMC_STATUS = 4	1	M2-66, M2-112
2	FMC_STATUS bit 20	a DMA has finished	FMC_STATUS = 0x100000	2	M2-66, M2-78, M2-112
3	FMC_STATUS bit 3	a NAND data transfer (data read, parity read) has finished	FMC_STATUS = 8	3	M2-112
4	ECC_STATUS (0x810) bit 1 after an encode, bit 2 after a decode	the ECC operation has finished	write 2 (encode) or 4 (decode) to 0x810; 0x81c = 1 also re-arms after an encode	4	section 7.4, M2-97, M2-99
5	FMC_STATUS bit 23	the polled chip is ready after a status latch	FMC_STATUS = 0x1800000 clears bit 23	5 (set at once)	section 7.5, M2-130, M2-193
6..10	none found	never passed; event 8 also not after a complete page read with DMA out or a one-sector read (M2-206)	—	—	section 7.8, M2-386
11	none found	passes at once in every state tried	—	—	M2-10, M2-69, M2-107, M2-387
12	latched NAND status byte (04c)	the chip signals ready and status & arg & mask ≠ 0	—	—	section 7.6, M2-134, M2-137
13..15	none found	never passed	—	—	section 7.8, M2-69, M2-107, M2-111, M2-386
16..255	none	never pass in eight states; no modulo-16 alias	—	—	M2-159

• Events 0..3 and 5 pair with the same FMC_STATUS bits as on M1 (M1-F004); event 4 follows 0x810 bit 1 or bit 2, and a write of 1 (bit 0) does not clear it.

• wait event 11 passes in all five tested stimulus groups, in either order relative to an acknowledge, and immediately after a block-erase confirm while the chip is busy (M2-69, M2-107). It passes at once (END 3..13 us after the start) idle with C58 = 0 or 4, with start word 1 (timeout disabled), during an unacknowledged DMA, during a BCH8 encode, with a command flag or a status latch pending, after every flag is acknowledged, with C10 = 0 and with 0x044 all ones, and with the operands (0x40, 0x40) and (0xffff, 0xffffffff); it also passes on every iteration of loops that run through NAND array busy periods (after 00h/address/30h, after FFh and during a block erase) (M2-387). It is therefore not the chips' ready line, as on M1 (M1-F057). No tested state makes it wait, so the condition it tests is not identified (chapter 17, item 10).

• With command stimulus wait 0 passes with FMC_STATUS = 0x000c3002; with DMA stimulus wait 2 passes with 0x001c3000; with status stimulus wait 5 passes with 0x008c3000. Idle wait 0 faults. These are qualified state/control pairs.

• An operation and its wait complete asynchronously: a write that acknowledges an event before the operation completes does not clear it, and all five immediate-acknowledge controls (command, address, DMA, ECC, status) reach END (M2-33).

C50 bits 0..5 mirror the pending flags of events 0..5 (command, address, DMA, data/parity transfer, ECC, status latch): each is set while its flag is pending and clears with its acknowledgement; bits 11..12 read 1 while the ECC engine is idle and 0 while it works (M2-112, M2-209; chapter 8, section 8.14). Sequencer reads of FMC_STATUS or FMC_NSTAT before a wait do not consume event 5 (M2-178).

7.3 Consumption and acknowledgement

The flags behind events 0..2 are levels. A wait does not consume them, and an acknowledge write does (M2-66). Measured with C58 = 4, so that a timeout vectors instead of continuing; before/after snapshots are stored in scratch memory:

stimulus / event	repeated and triple wait	acknowledge between two waits	acknowledge, trigger again, wait
command / wait 0	all pass	second wait faults after clearing 048 bit 1	passes
address / wait 1	all pass	second wait faults after clearing 048 bit 2	passes
DMA / wait 2	all pass	second wait faults after clearing 048 bit 20	passes
ECC encode / wait 4	all pass	passes after 0x810 = 1 (bit 0 only); C50 completion bit remains	passes
status / wait 12, arg = mask = 0x40	all pass	passes after 048 = 0x1800000, with bit 23 and C50 bit 5 clear	faults at the second command's wait 0, before the second wait 12

- A flag left set lets the next wait on it pass at once. A second DMA programmed after such a stale wait 2 starts while the first runs and cuts it; with every wait 2 acknowledged the same program is byte exact (M2-78).
- The acknowledge that clears event 4 is the 0x810 bit of the operation (2 after an encode, 4 after a decode); writing 1 clears only bit 0 and leaves the event set, so an encode status 0x13 becomes 0x12 and wait 4 still passes (M2-67, M2-97).
- A status latch leaves the controller in status mode: the next command's wait 0 stalls until an FMCTRL1 write (M2-32, M2-181; section 7.5). This is the fault in the last row.
- wait 12 is not an alias of FMC_STATUS bit 23 (M2-68).
- With C58 = 0 a timeout continues at the next instruction; a program that then clears the timeout IRQ can report END although its wait failed. Such cases are read from their stored IRQ words and elapsed times.

7.4 Wait event 4 (ECC)

after	wait 4 follows	re-arms the next wait 4	does not re-arm
encode	0x810 bit 1 (W1C)	write 2, 0x1ff, or 0x81c = 1	write 1 (clears only bit 0)
decode	0x810 bit 2 (W1C)	write 4, 0x1ff	write 1, 2, 0x40

(M2-97, M2-99; 0x81c = 1 sets 0x810 to 0x10, M2-96.) The original programs acknowledge with 1 after wait 4, which leaves the completion bit set; an ECC_RESULT read after wait 4 and that acknowledge does not give the current sector's count or uncorrectable bit (M2-85; retained completion is the inferred cause). Twenty repeated-encode programs end with completion status 0x13 throughout, with no fresh completion established (M2-72). The other 0x810 bits are in chapter 10.

7.5 Wait event 5 and the status latch

A status latch is the 70h command followed by FMCTRL1 = 0xca. One latch is enough: the controller keeps polling the chip's status until the chip is ready (M2-130, M2-136). Each latch sets FMC_STATUS bit 23 exactly when the chip reports ready (0xe0); a busy status (0x80) leaves it clear (M2-193). A latch never sets FMC_STATUS bit 24 (0 of 2743 latch records, with 0x044 = 0, 1, 2, 0xffff or any single bit; M2-163).

• **Latch and** wait 5**.** The standard latch (70h, wait 0, acknowledge, latch) passes wait 5 every time: 1337 of 1337 in repeated series, 2644 of 2644 first latches (wait 5 or wait 12, 0x044 = 0, 1, 2 or 0xffff) and 410 of 410 in eleven forms (M2-161). A latch written straight after 70h, with no wait 0, also passes (41 of 41 in a series); it returns 0x70, later in the series 0xe0. A latch one instruction after a 70h whose wait 0 was not executed (an acknowledge or one movi r5,#0 filler instruction between) faults with bit 23 clear (M2-178). These are the two faulting orderings of an unwaited 70h; with 20 or 200 filler instructions before the latch wait 5 passes (9 of 11 orderings pass; M2-68). The two faulting orderings fault in 18 of 18 runs, straight after the per-case reset and after each of eight different preceding programs, with FMC_STATUS reading 0xc300a / 0xc3000 (bit 23 clear) before the wait 5: the fault depends on the ordering, not on an earlier program (M2-390). Measured in nop2 steps, a latch written straight after the 70h (no instruction between) passes with FMC_STATUS bits 1, 3 and 23 set and status 0x70, one written after one or two nop2 (30..60 ns), one nop, one movi or the acknowledge faults: bit 23 never rises in 24 samples of 188 ns and bit 3 is set (0xc300a) before the wait 5; with 3 to 20 nop2, or the acknowledge followed by 20 nop2, it passes with status 0xe0 (M2-593). Why that window sets bit 3 instead of bit 23 is unknown (chapter 17, item 11).

• **During a block erase,** wait 5 after one latch passes when the chip becomes ready, at the erase end (about 2.27 ms), with 04c = 0xe0; while the chip is busy 04c reads 0x80 from about 20 us on (M2-130, M2-136, M2-198).

• On an idle chip, with the 70h waited for (wait 0) and acknowledged before the latch, wait 5 passes for a single latch, two latches, the acknowledge just before the latch and a nop just before the latch; a latch without any 70h command also passes (M2-130). Without a chip enable the latch returns the command echo 0x70 (M2-87).

• **Next command after a latch.** After a status latch a new command is refused: its wait 0 times out, FMC_STATUS bit 1 stays clear and bit 26 is not set, until an FMCTRL1 write. A status clear, an ECC kick and FFh do not release it (M2-181). A second 70h + latch with no FMCTRL1 write after the first latch is refused at its wait 0 (342 of 342; M2-202). After the release FMCTRL1 = 0x30020 and the acknowledgement 0x1800000, the next 70h passes its wait 0 (11 of 11 forms; M2-181). An address operation in that position (after the latch and only the bit-23 acknowledge) does not complete either: its wait 1 times out, FMC_STATUS bit 2 stays clear and bit 26 sets (0x40c3000), while a refused command leaves bit 26 clear (M2-610).

• **Second latch after the release.** After a latch, its wait 5 and the release, a second 0xca latch without a new 70h passes wait 5 (41 of 41) with no operation between and after each of: set-up word, FMCTRL1 = 0x100e0, a 16-byte read, a DMA, address cycles, FMCTRL1 = 0x20, status clear, CE toggle and ECC kick (M2-237).

• **Second latch without the release.** After a latch, its wait 5 and only the acknowledge of bit 23, a second 0xca latch without a new 70h also passes wait 5 and returns 0xe0, with no operation between and after each of FMCTRL1 = 0x20, a status clear, a CE toggle, a BCH8 encode with its wait 4, the set-up word, FMCTRL1 = 0x100e0 and the release: a latch needs no re-arm (M2-494).

• **Operations after an unreleased latch.** After a latch (70h waited, 0xca, wait 12, acknowledge 0x1800000) without the 0x30020 release, a DMA in, a DMA out, a 512-byte data read, a 16-byte parity read, an 8-byte FIFO read, a BCH8 encode and a second latch complete as after the release; only a new command and an address operation are refused (M2-641).

• **Two chips.** A second chip's poll after an unreleased latch is refused at wait 0. With the release (0x30020 + acknowledge) between polls, polls of CE0 and CE1 during an erase pass in both orders (four polls each, status 0xe0) (M2-198).

• **Latch after** 90h**.** A latch issued after a 90h command sets no bit 23 (wait 5 times out) and leaves the first READ ID byte 0xad in FMC_NSTAT; the next 70h after the release is accepted (M2-239).

• Among wait events 5..10 and 12..15 in seven prepared states, only wait 5 after a status latch passed (M2-111). No wait event 0..15 with arg = mask = 0 follows the NAND ready/busy line (M2-107): the ready state

is observed only through the status latch.

• A status transaction survives the sequencer's END and blocks the reset's FMC phase until it is released (M2-29, M2-31; chapter 8, section 8.22).

• The latch returns the chip's status on FMC_DATA_SEL = 0x100 as on 1 (chapter 9, section 9.6; M2-169).

7.6 Wait event 12 (masked status)

wait 12 with operands arg and mask (the arg and imm fields) passes when the chip signals ready and **status & arg & mask ≠ 0** (M2-134, 18 pairs against 0xe0 and the no-CE echo 0x70). With arg = 0 or mask = 0 it never passes.

wait 12 case	outcome
one latch during an erase, 0x40/0x40	passes at the erase end (about 2.27 ms)
one latch during an erase, arg 0, mask 0x40	faults
no latch during an erase, 0x40/0x40	never passes
one latch, 0x80/0x80	passes only at the erase end, although 04c reads 0x80 from about 20 us; at release 04c = 0x80, 048 bit 23 clear
one latch, 0x20/0x20	passes at the erase end; 04c = 0xe0, 048 = 0x8c3000
one latch, 0x01/0x01	faults
wait 5, then 0x80/0x80	passes at the erase end
no chip enable, latch, 0x40/0x40	passes on the echo 0x70

(M2-130, M2-134, M2-136, M2-137.)

Inference (M2-137): wait 12 is evaluated when the chip signals ready, and the operand test is then applied to the status byte; the 0x80/0x80 case was released by that event before the ready status byte (0xe0, bit 23) arrived. wait 12 passes after 048 bit 23 is cleared (section 7.3). A mask of 0x40 cannot tell the ready byte 0xe0 from the echo 0x70, which also has bit 6 set (M2-87).

7.7 wait2 and the interrupt request lines

wait2 N (09, R = N) waits until interrupt request line N is asserted, or faults at the timeout. N is a VIC0 line number, as on M1 (M1-F174; M2-140, M2-142). The arg and imm fields are ignored: (0, 0), (0x40, 0x40), (0, 0x40), (1, 0xd64), (0xd60, 0xffffffff) and (0xffff, 0) give the same outcome (M2-187).

wait2 condition N	line	when it passes
5	VIC0 line 5	in every state tried; permanently asserted, source unidentified; high before and after restored pulses across gates0..108; no during-pulse samples, so their involvement is not excluded (M2-744)
8	VIC0 line 8, timer	when the CPU-enabled timer interrupt fires, also while the chip is busy (M2-140, M2-167)
16	VIC0 line 16, DMAC0	on DMAC0 activity
19	VIC0 line 19	in DMA and ECC states; its outcome depends on USB traffic (M2-74, M2-144)
20	VIC0 line 20, NAND controller	while a NAND controller flag enabled in FMC 0x044 is pending, 4–7 us after the operation (M2-142, M2-204)
32, 33	VIC1 lines 0, 1 (GPIO interrupt groups)	when that line is asserted by hardware; each condition follows only its own line (M2-723)

wait2 condition N	line	when it passes
34..63	VIC1	never in any tested state; soft interrupts never release (M2-211)
64..255	—	never (M2-74)

Clock pulses on initially enabled gates 1 and 2 leave VIC0 raw at 0x20 and VIC1 raw at zero before, during and after the measured 28-us window. Gate0 was already disabled. A loss of communication during gate3 supplies no interrupt sample or gate identity, so the source of line 5 remains unidentified (M2-729). The corrected clock-pulse harness restores clocks before reading peripherals; its before/after samples cannot establish the line's state during the disabled interval (M2-730). With that pulse, initially enabled gate3 also returns with exact restoration and VIC0 raw 0x20 before and after. The following worker FREE job stays queued while USB remains live; this supplies no gate identity or during-window line measurement (M2-742). The identical gate3 manifest also precedes loss of USB in a session without the EXECFIRMWARE entry marker; no RUN acceptance survives. This does not establish a clock identity or make entry history the cause (M2-746, M2-747).

- **Line 20 and 0x044.** 0x044 bits 0, 1, 2, 11 and 12 route the command, address, transfer, DMA and status-latch flags onto VIC0 line 20. The line follows a routed flag only while it is pending, and of the 32 single 0x044 bits only the operation's own bit raises it (M2-200). With the command flag pending, wait2 20 passes for 0x044 bit 0 alone of the 32 single bits (address bit 1, transfer 2, DMA 11, status 12). It follows the line, not the sticky CS_IRQ bit 4, and a write of 0x844 does not release it (M2-204). With 0x044 = 0 the line is never raised (M2-142). As on M1, the controller's sources share line 20 (M1-F175).

- **No NAND busy line.** No wait2 N in 0..255 tracks a NAND erase, program or read busy period; only the always-asserted line 5 and the CPU-device lines 8, 16 and 19 pass (M2-167).

- **No ECC line.** After BCH8 encodes and decodes no VIC line rises beyond the no-candidate baseline, for every tested ECC enable candidate (0x800, 0x808, 0x814, 0x818/0x844 single bits, 0x860..0x9fc, with 0x044 = 0 or all ones), and wait2 20 and wait2 43 never pass after an encode (M2-212, M2-217). The ECC engine's line and enable are not found.

- **VIC1.** Condition N follows VIC line N also above 31: wait2 32 passes when VIC1 line 0 is asserted by hardware and wait2 33 for line 1, each only for its own line (M2-723); line 20 + 32 = 52 never rose in any set-up tried (M2-724). Without a hardware line, conditions 32..63 never pass in any state (620 cases) or with a VIC1 software interrupt raised. wait2 22, 53, 54 and 55 time out also after the sequencer polled VIC1 raw status and when free-running in a 32 ms window (28 runs in each of four series), although the CPU sampled VIC1 raw bit 22 set during 10 to 14 of each series (M2-211). VIC1 raw bit 22 is set at the stop after a run that executed an END with C10 non-zero, and clear after an END with C10 = 0 and after a run that stops without an END (chapter 8, section 8.6; M2-182).

- A VIC SOFTINT raise is visible in the raw VIC status but releases wait2 N on none of 59 lines, held for 2 ms (M2-140). Inference: wait2 samples the peripheral request lines before the VIC's software OR, not the raw status word. This is why sampled raw VIC snapshots (initially line 20 and 0 raised) did not track wait2 releases.

- Of the 1,024 sampled conditions in the four tested contexts (idle, command, DMA, ECC), 1,010 fault (M2-74). The open points are in chapter 17, item 13.

7.8 Events that nothing releases

wait events 6..10 and 13..15 never passed:

- idle, and after a block-erase confirm with the chip busy (arg = mask = 0; M2-107);

- in seven prepared states: idle, ECC encode, erased decode, status latch and others (M2-111);

- event 15 in both orders for all five clear-enable stimuli (M2-69);

- event 8 after a complete CE0 page read with DMA out, after a one-sector CE0 read with C58 = 0 or 4 following wait 5 or a wait 12 ready poll, and idle (M2-206);

- with a CPU write 1 ms into the wait to C0C, C10, C48, C54, C60, C64, C68 or C6C (72/72 fault; M2-120, M2-125, M2-143);

• after an unacknowledged DMA out, an unacknowledged 512-byte data read and spare read of an open page, a FIFO READ ID (0x040 = 0x52), a metadata write, a DMA with 0x044 all ones, a BCH8 slot-1 and a BCH16 encode, 0x81c = 1, a 70h on CE1, CE2 or CE3, a pending sig (CS_IRQ bit 1), the completion of an uncorrectable and of an erased-sector BCH8 decode (0x810 = 0x1d / 0x55) and a 70h routed by a CPU-written 0x044 of all ones; with a CPU write 1 ms into the wait of C40 = 1, C44 = 1, C50 = 0xffffffff, C14 = 0, C5C = 1 or C3C = 0; and with all VIC0 or all VIC1 software lines raised from 1 to 2.5 ms (every case times out; M2-386).

Event numbers 16..255 never pass in eight states (2,640 cases), with no modulo-16 alias onto events 0..15 (M2-159). wait 12 likewise never passes without a status latch or with a zero operand (section 7.6). What, if anything, sets events 6..10 and 13..15 is open (chapter 17, item 9). Idle non-completion does not establish that an event does not exist.

7.9 Timeout length

The timeout counts steps of the engine clock (about 33.25 MHz, inferred from instruction timing; M2-103). Start-word bits 15:8 are the count, bits 7:4 = n select the step (chapter 8, section 8.2):

• **Step.** One step is 2^3 engine cycles for n = 2, 2^5 for n = 3 and 2^(n+3) for n = 4..11; n ≥ 11 saturates at 2^14 cycles (M2-115, M2-216). n = 0 faults at once (0..4 us for every tested count). n = 1 is not immediate: counts 0x0f, 0x40, 0x80 and 0xff fault at 2, 4, 8 and 14..16 us, 2 engine cycles per count within the 2-us resolution (M2-115). For n = 2..6 the fault time is linear in count + 1 with steps of 0.237, 0.964, 3.844, 7.697 and 15.395 us (five counts per n; M2-216), so 2^(n+3) holds from n = 4 on but not for n = 2 and 3.

• **Count.** With n = 11 (start words count << 8 | 0xb5, count + 1 = 8..256, including multi-bit counts) an unmet wait 3 and an illegal opcode entered at the start of the run both fault at **fault_at = 492.747 us × (count + 1) + 447.5 us**, largest residual 1.1 us (M2-81). 492.747 us equals 2^14 cycles at 33.25 MHz. Start word 0x20b5 faults at 16,708 us. The idle ≈ 126.6 ms of section 7.1.1 equals the formula at count 0xff.

Measured fault times:

n (bits 7:4)	step (cycles)	count 0x07	count 0xff	count 0x0f
0	at once	0 us	0xff05 ≈ 4 us	
1	2	0..2 us	0xff15 ≈ 16 us	0x0f15 ≈ 2 us
2	2^3	4 us	0xff25 ≈ 62 us	
3	2^5	8 us	0xff35 ≈ 246 us	
4	2^7	30 us	0xff45 ≈ 986 us	0x0f45 ≈ 62 us
5	2^8	62 us	0xff55 ≈ 1,978 us	
6	2^9	124 us	0xff65 ≈ 3,942 us	
7	2^10	262 us	0xff75 ≈ 7,900 us	0x0f75 ≈ 510 us
8	2^11	510 us	0xff85 ≈ 15,784 us	
9	2^12	1,062 us		
10	2^13	2,172 us		
11	2^14	4,390 us		
12..15	2^14	4,390 us	0xffc5 ≈ 126,594 us	

Timeout instants. The timeout fault falls on instants one step apart, whose phase is set before the run starts, not a fixed time after the wait is entered: 447.5 + 492.747 × k us for n = 0xb (period 2^14 cycles) and about 16.5 + 30.8 × k us for n = 7 (period 2^10 cycles). With count 0 a wait 3 entered 4..407 us after the start faults at 446..448 us and one entered at 477 or 544 us at 940/941 us (start word 0x00b5); with 0x0075 a wait entered 3..17 us after the start faults at 16..18 us and one entered at 23 or 29 us at 46/48 us (M2-433). A count-0 wait therefore faults at the first instant after its entry. Counted from the entry, a wait of count c ≥ 1 faults at the (c + 2)-th instant and a tmwait of count c ≥ 1 ends at the (c + 1)-th; entered after the first instant, a wait or tmwait of count 1..3 ends exactly one instant later than entered at the start, for n = 0xb, and the same counting holds for n = 7 and 9 (M2-81, M2-373, M2-584).

The first instant of a count-0 wait entered at the start is 446..448 us for n = 0xb..0xf, 202 us for n = 0xa, 78 us for n = 9, 16 us for n = 7 and 8, 4 us for n = 4 and 0..2 us for n = 5 and 6, the same in repeats (three runs after the per-case reset: 16/16/18 us for n = 7, 78/78/78 us for n = 9, 448/446/446 us for n = 0xb, 4/0/0 us for n = 4, 0 us for n = 5 and 6). For n = 7..0xb these are (2^(n+3) − φ mod 2^(n+3)) engine cycles after the start with one phase φ ≈ 1488 cycles; the same phase predicts 1.4 us for n = 4..6, within the 2-us resolution of the measured times (M2-588, M2-631). Extra CPU parameter writes before the start bring the first instant closer to the start by about 0.08 us per write (64 writes: 4..6 us; 90 writes: 8 us for n = 9), and without the per-case reset the instants fall elsewhere (n = 7: 26/10 us; n = 9: 98/96 us; n = 0xb: 96/100 us): the per-case reset sets the prescaler phase, and the prescaler runs while the parameters are written (M2-588, M2-631). No single step of the reset sets it alone: for n = 9 the full reset gives 76/78 us and the skipped reset 24/34 us, and of the steps run alone only `C00` = `0x80000` comes near the reset value (82/82 us), while FMCTRL1 bit 31 (30/32 us) and `0x81c` = 1 (28/32 us) stay near the no-reset value (M2-691, M2-692). The combination of steps that sets the phase and why a wait ends one instant after a `tmwait` of the same count are unknown (chapter 17, item 6). Short times include the measurement's polling quantization.

Only the engine timeout is a hardware event. A host observation window that ends earlier reports a TIMEOUT without the engine's fault, and a worker, reset or transport failure means the instruction under test did not execute; see chapter 15.

7.10 Other event-like opcodes

• `tmwait` **(**15**).** A `tmwait` entered at the start of a run ends **492.747 us × count + 450 us** after the start (within 3 us; count = start-word bits 15:8): 943, 3898, 31,986 and 126,103 us for counts 1, 7, 64 and 255, one 2^14-cycle step less than the `wait` timeout of the same start word (M2-373). Counted from its entry it ends at the (count + 1)-th timeout instant, so entered after the first instant it ends one instant later; a count-0 `tmwait` does not end but takes the timeout fault at the first instant, like a count-0 `wait` (M2-584). All 24 forms of the first survey end after 126,101–126,103 us with `C14` = 0 and `CS_IRQ` = 1, distinct from the ≈ 126.6 ms `wait` fault (M2-27). R, arg and imm are ignored, `C58` = 0 gives the same END, and `C14` written before it (0 .. `0xffffffff`, eight values) does not change the time (M2-373, M2-629); `C14` reads `0x7f` before it, counts down by one per step during it and reads 0 after it (M2-13, M2-172, M2-373). Consecutive `tmwaits` each wait a full period; one started with `C14` = 0 does not reload and end at once (M2-150). With `C48` = `0xffffffff` or `0x55555555` a `tmwait` does not end after its period (3898 us at count 7) but faults at the `wait` timeout (4388..4390 us), in every run; `C48` = 0 leaves it unchanged (M2-179). At count 1 each single `C48` bit 1..7 makes it fault, bit 0 ends it one instant early (449 us, `C14` = 1) and bits 8..31 change nothing (M2-627). At count 7, `C48` = v = 1..7 ends it v instants early (3406, 2914, 2421, 1928, 1435, 942 and 449 us against 3896 us for `C48` = 0) with `C14` = v, and `C48` = 8 makes it fault at the `wait` timeout (M2-687): a `tmwait` ends at the instant at which its `C14` count reaches `C48` bits 7:0, and a value above the count gives the timeout fault (M2-688; chapter 8, section 8.12). Of ten count-255 runs, five with a `C14` = `0x800` write, one without the write ended 23 us early and the others within 126099..126108 us: isolated early ends are run-to-run spread, not an effect of a `C14` write (M2-689, M2-690).

• `nop` **(**`08`**).** With R = 0..7 it continues; the zero form costs 1.25 cycles, the cost of a passing `wait`, and a loop of it leaves `C14` holding start-word bits 15:8 as a passing `wait  11` does; with R = 8..`0xff` it holds until the timeout fault at the time of an unmet `wait  3` (4388..4390 us with `0x07b5`), also with a pending command, address, DMA, ECC or status flag for R = 8..15 (M2-625). With the timeout disabled `C08` reads 4 during the hold, the value of an unmet `wait  3`: the hold is a wait state (M2-685, M2-686). What it waits for with R ≥ 8 is unknown (chapter 17, item 4).

• `evjmp` **(**16**).** `evjmp` waits for an event and then continues at the fault vector `C4C` without a fault: it vectors with `CS_IRQ` = 1 for command, address, data, DMA, ECC and status stimuli (M2-58). It catches an event that arrives during the wait (a command flag 100, 1000 or 5000 us into it) and one already pending (DMA), jumps to `C4C` for `C58` = 0, 2 and 4 and leaves the flag set (`FMC_STATUS` `0xc3002` / `0x1c3000` at the target). With no event it times out after the full period and then continues at the next instruction with `C58` = 0 or through the fault vector with `C58` = 4 (`CS_IRQ` bit 2 set); a departure leaves `CS_IRQ` bit 2 clear in the handler. An acknowledged flag is no event, and a `70h` written through `+0x408` 1 ms into the wait raises none (M2-186). R, arg and imm are ignored (R = 1, 2, 4, `0xff`; arg = 1, 2, `0xffff`; imm = `0xffffffff` or a program label): for a pending command, DMA, BCH8 encode, status latch, command + DMA and DMA + encode it departs to `C4C` 3..25 us after the start, never to imm, and the handler sees the source's flag: `FMC_STATUS` bit 1 and `C50` bit 0 (command), `FMC_STATUS` bit 20 and `C50` bit 2 (DMA), `ECC_STATUS` `0x13` and `C50` bit 4 (encode), `FMC_STATUS` bit 23 and `C50` bit 5 (status latch) (M2-391). With start word 1 (no timeout) it stays at `C08` = `0xb`, `CS_IRQ` = 0 and survives STOP (M2-57); the `C00`

sequences that release it are in chapter 8, section 8.22 (M2-62, M2-392).

• `sig` **(**`10`**) and** `nop2` **(**`ff`**)** keep their M1 names. In idle, DMA and ECC states `sig` sets `CS_IRQ` bit 1 and continues (`CS_IRQ` = 3 at END) and `nop2` continues (M2-34, M2-65). `sig` ignores R, arg and imm; writing 2 to `CS_IRQ` clears the bit and the next `sig` sets it again; it raises no VIC0 or VIC1 raw line beyond the run baseline and costs one cycle (M2-370; chapter 6).

• `pause` **(**12**)** hands control to the CPU (chapter 8, section 8.20).

Raw VIC words, enabled IRQs, sequencer IRQ flags and FMC/ECC status bits are different domains; a coincident bit in a snapshot does not map one onto another.

References: chapter 14 (outcomes), chapter 13 (experiments), chapter 15 (interpretation), chapter 18 (findings).

Related chapters: chapter 6, chapter 8, chapter 9, chapter 10, chapter 17.

Chapter 8 — Sequencer control registers

Summary: the registers of the sequencer's control block (0xc00 to 0xc5c), the buffer handshake words 0xc60 to 0xc6c and the parameter RAM at 0xd00: what each holds, who writes it and how it behaves; then the program lifecycle, pause, the reset and its effects, and the states that block a reset with the writes that release them. Offsets are relative to mem:0x38a00000. Source anchors in bl_NandReadOnly: reset mem:0x502c, start mem:0x51b8, IRQ wait mem:0x51f4, pause handling mem:0x4d08. Finding IDs (M2-nn) refer to chapter 18. The timeout length is in chapter 7, section 7.9.

8.1 Sequencer control block

"Device" marks behaviour measured on the device, "source" use by the original code, and "inferred" an interpretation.

offset	name	what it is	written by
0xc00	FMSS_CTRL	start word, stop, abort and the clearing reset	CPU
0xc04	FMSS_PROG	bus address of the program	CPU
0xc08	CS_STATUS	state code; not a simple busy Boolean	read only
0xc0c	CS_IRQ	flags: END, fault, pause	set by hardware, acknowledged by CPU and program
0xc10	FMSS_C10	initialization word, stores the bits of 0xdf; the prologue writes 1; non-zero makes END raise VIC1 line 22	program
0xc14	FMSS_C14	counter/timer; ignores writes	hardware
0xc18 to 0xc34	r0 to r7	the program's registers; r6 (0xc30) is SEQ_RESULT	CPU and program
0xc38	FMSS_PC	program-relative PC; a write jumps	program; CPU, also while running
0xc3c	FMSS_C3C	faulting offset + 8, read by the original fault handler; 32-bit storage	hardware; CPU and program
0xc40, 0xc44		instruction word latches	hardware
0xc48, 0xc54		storage words: C48 32 bits (any of bits 1..7 makes a tmwait fault, bit 0 ends it one instant early), C54 mask 0x83f	CPU and program
0xc4c	FMSS_C4C	fault vector	program prologue
0xc50	FMSS_C50	completion bits	hardware
0xc58	FMSS_C58	fault routing	program prologue
0xc5c	FMSS_C5C	vector-associated state	hardware
0xc60, 0xc64		buffer reset request / completion	CPU / hardware
0xc68		no role found; stores the bits of 7	CPU
0xc6c		buffer configuration	CPU
0xd00 to 0xd7c	PRAM	32 parameter words	CPU and program
0xd80 to 0xdfc		read 0, drop writes	—

Storage masks, the same for CPU and sequencer writes (M2-154, M2-191): C10 0xdf, C3C and C48 32 bits, C54 0x83f, C58 0xd6 (M2-70), C68 7. A start keeps CPU-written C3C, C48 and C54 values: after CPU writes of all ones the program reads 0xffffffff, 0xffffffff and 0x83f (M2-154). Offset bit 9 is decoded in 0xc00..0xfff: none of the 38 holding words of 0xc00..0xdff appears at +0x200 (M2-155). 0xd80..0xdfc read 0 and drop program wr/pst writes, with no alias into 0xd00..0xd7c (M2-147).

8.2 FMSS_CTRL (0xc00): start word and control

Values written by the original CPU driver (source):

value	effect
0xfff5, 0xffb5	start a program (0xffb5 in the bootloader)
8	STOP
2	abort; the original reset takes this branch when C08 = 4
0x80000	the clearing reset step, written before the FMC reset
bit 0	resume after a pause (the pause handler sets it)

Start-word fields (device):

bits	effect
0	start at instruction 0. The first start word after the ordinary reset, and the first after power-on, runs a program whose instruction 0 is not END; no priming start is needed (M2-151, M2-431). A plain start while a program runs is ignored (M2-158)
1, 3, 16	hold: the program stays at instruction 0, with C00 bit 18 set, until a start word without these bits (0xffb5 then runs it once from instruction 0); there is no single-step (M2-153, M2-157). Written while a program runs, bit 1 freezes it exactly (C08 = 3) and a later start resumes it; bit 3 or 16 restarts it from instruction 0 after the release (M2-158). Bit 3 does not read back (M2-153)
2	timeout enable: with start word 1 an unmet wait 3 or illegal opcode 1a waits with C08 = 4 and no fault (M2-56), and evjmp blocks in state 0xb (M2-57)
7:4	timeout step n: 2^3 engine cycles for n = 2, 2^5 for n = 3 and 2^(n+3) for n = 4..11, saturating at n = 11; immediate for n = 0, 2 engine cycles per count for n = 1 within the 2-us resolution (M2-115, M2-216). Bit 6 acts only as part of this field (M2-201). 0xfff5 and 0xffb5 have n = 0xf and 0xb, both in the saturated range
15:8	timeout count: a wait entered at the start faults after count + 1 steps plus a fixed offset (count + 1 = 8..256; M2-81), on a grid of timeout instants whose phase the per-case reset sets before the run starts (M2-433, M2-588, M2-631); counted from its entry a wait of count ≥ 1 faults at instant count + 2 and a tmwait ends at instant count + 1, and count 0 faults at the first instant (M2-373, M2-584); C14 holds the count after a quickly passing wait (M2-172; chapter 7, section 7.9)
17	no observed effect (M2-201); preloaded r2..r5 survive starts with and without it (0xfff5, 0x20001, 0x20000; M2-157)
19	inhibits the start like bits 1, 3 and 16, but C00 then reads 0x7f60, bit 18 clear (M2-153, M2-201)
18, 20..31	no observed effect (M2-201)

Worked example, the driver's 0xffb5:

```
0xffb5 = 1111 1111 1011 0101
         └─15:8──┘ └7:4┘3210
```

Bit 0 = 1 (start), bit 1 = 0 (no hold), bit 2 = 1 (timeout enabled), bit 3 = 0, bits 7:4 = 0xb (n = 11: step 2^14 cycles ≈ 492.7 us), bits 15:8 = 0xff (count 255). An unmet wait faults after 492.747 us × 256 + 447.5 us ≈ 126.6 ms (M2-81). 0xfff5 differs only in n = 0xf, which saturates to the same step; 0x07b5 (count 7) faults after ≈ 4.39 ms.

Raw readbacks (device; concrete observations, not a bit map):

state	C00 reads
running; at a pause	0x4ffb4 (M2-384)
running after a timeout fault (in the fault handler, or continuing with C58 = 0), start word 0x01b5	0x401b4 (M2-384)
held by start bit 1, 3 or 16; a wait held by start word 1	bit 18 set; 0x40000 (M2-153, M2-384)
start inhibited by bit 19	0x7f60 (M2-153)
after STOP, while a loop runs or in the blocked evjmp state	0x40000 (M2-392, M2-422)
after abort (C00 = 2)	0x40002 (M2-392, M2-422)
after the clearing step 0x80000, alone or after STOP or abort	0x7f60 (M2-392, M2-422)
after 0x80000 followed by STOP	0 (M2-392, M2-422)
held by C60 = 1	0x407b4
at power-on, and in the untouched window after a probe cleanup	0x7f60, against the last start word 0xffb4 after the ordinary reset (M2-281, M2-430)

While a program runs, waits, is held by start word 1 or stops at a pause, C00 reads the start word with bit 18 set and bit 0 clear (M2-384).

Other start words are not qualified.

8.3 FMSS_PROG (0xc04)

The bus address of the program, written by the CPU before the start (source; every successful probe). A paused program can be redirected by changing it (section 8.20). STOP preserves it; the clearing step clears it (M2-54, M2-422). It reads 0 at power-on and in the window left by a probe cleanup (M2-281, M2-430).

8.4 CS_STATUS (0xc08)

A state code (device, CPU reads unless noted):

value	state
0	idle; at power-on; after every END, also a fault-handler END; after STOP, and after the clearing step (M2-53, M2-62, M2-384, M2-422, M2-430)
1	running: a delay loop, a fault handler after a vectored timeout fault (C58 = 4; C5C = 1) or a program running on after a C58 = 0 fault, the value varying with 2 and 9 between identical runs (M2-384); in every run of an ld loop and one of three of an sti loop (M2-693); also the state held by C60 = 1 (section 8.22)
2	running, as 1; in one of three runs of a nop2 loop (M2-384, M2-693)
3	after abort (C00 = 2), from a running loop or the blocked evjmp state (M2-62, M2-392, M2-422); a running program frozen by start-word bit 1 (M2-158)
4	waiting: every unmet wait, a wait2, a tmwait, a nop with R ≥ 8 and a wait held by start word 1, read inside the state (M2-384, M2-685), or stalled on an illegal opcode; the original reset takes its abort branch for 4
5	the value the program itself reads (sequencer-side read) in every tested state, also with a DMA, ECC operation or command in progress (M2-70, M2-112, M2-384)
8	running: read in two of three runs of an sti loop (M2-693)

value	state
9	running, as 1: read in one run-on and one fault-handler run and in two of three runs of a nop2 loop, not a separate state (M2-384, M2-693)
0xa	stopped at pause (12) (M2-14, M2-384)
0xb	evjmp waiting for its event (M2-384); blocked with the timeout disabled, where it survives STOP (M2-57)

Read once 3 ms into a 400 ms loop of one instruction kind, three runs each, C08 gives 0x9, 0x9, 0x2 for nop2, 0x8, 0x1, 0x8 for sti and 0x1, 0x1, 0x1 for ld (M2-693). Sampled continuously by the CPU during a run it changes within one loop: 1, 2 and 9 in ALU and nop2 loops, also 7 in an SDRAM-access loop and 5 in a window-access loop, 4 in an unmet wait (M2-719). The running value varies between identical runs of one kind; the CPU takes one read per run at an instant whose phase within the loop it does not control, so the values cannot be assigned to an instruction or a pipeline step (chapter 17, item 8).

8.5 CS_IRQ (0xc0c)

bit	set when (device)
0	the program reached END
1	sig (10) executed; the program continues; cleared by writing 2 (M2-34, M2-65, M2-370)
2	a fault (timeout or illegal opcode)
3	the program is stopped at pause
4	an FMC flag routed by 0x044 was raised during the run; stays set after the flag's acknowledgement (M2-200)
5, 6	never set after BCH8 encodes and decodes, with 0x044 = 0 or all ones and every tested ECC enable candidate (M2-212)

- CS_IRQ = 5 is fault plus END: a fault handler that reaches END, not a clean program result. CS_IRQ = 4 (fault without END) remains when the engine is held by C60 = 1 after its timeout vector.
- The original code writes the flags to acknowledge them. The original pause handler acknowledges bit 3.
- sig sets bit 1 in every tested operand form in idle, DMA and ECC states (CS_IRQ = 3 at END; M2-34, M2-65, M2-370). Writing 2 clears the bit and the next sig sets it again; sig raises no VIC0 or VIC1 raw line beyond the run baseline (M2-370).
- A CPU write to C0C during a wait releases none of wait events 6..10, 12..15 (M2-120).

8.6 FMSS_C10 (0xc10)

The original program prologue writes 1. C10 stores the bits of 0xdf (M2-191). VIC1 raw bit 22 is set at the stop after a run that executed an END (a fault-handler END included) with C10 non-zero (1 or 0xdf), and clear after an END with C10 = 0 and after every run that stops without an END (held, paused, looping, or faulted with no handler END); it is clear while a program runs (M2-182). Presets of 0 or all ones change neither the wait timeout, a tmwait, a DMA nor the BCH8 parity (M2-179). STOP and abort preserve C10 = 1; the clearing step clears it (M2-54, M2-422). A CPU write during a wait releases no event (M2-120).

8.7 FMSS_C14 (0xc14): counter/timer

observation (device)	C14
at power-on; after STOP, after the clearing step, after the ordinary reset and in the window left by a probe cleanup	0x7f (M2-281, M2-430)
before the first wait of a run (also the sequencer's own read at the start)	0x7f (M2-177, M2-384)
after a wait that passes before the first timeout step (DMA, encode, wait 0..5 and 11, wait2 5, an evjmp departure)	start-word bits 15:8: 0xff for 0xffb5, 0xfe .. 0x7f with one of bits 8..15 cleared, 0x07 for 0x07b5, 0xef for 0xefb5 (M2-172)
about 1 ms after such a wait, through later passing waits and after the END (CPU read)	the loaded value, unchanged (0xef for 0xefb5; M2-172)
while a wait or a tmwait stalls	counts down by one per timeout step: 0xbd 33 ms into a second wait after a fault, 0xbc 34 ms into a second tmwait (start word 0xffb5) (M2-172)
at the end of a short observation of a running tmwait	0x35 (M2-13)
after a timeout fault, whatever the start word (ten start words, fault times following bits 15:8)	0x7f (M2-177)
after a wait 0 released by a CPU 70h write after one or more timeout steps (1, 2 or 4 steps for counts 3, 0x10, 0x40 with n = 0xb, and count 0x10 with n = 9; 1..120 ms into it with 0xffb5), also 1 ms later, and after a wait 5 that passed at an erase end 2.27 ms into it	0x7f (M2-177)
after a loop of nop (08, R = 0) with start word 0xffb5	0xff, start-word bits 15:8, as after a passing wait 11 (M2-625)
after a completed tmwait	0 (M2-27, M2-373, M2-584)
after a CPU write of 0x55 or program writes of 0x12, 0, 0xff, 0x33	unchanged (0x7f) (M2-176)

A wait loads C14 with start-word bits 15:8 and C14 counts down by one per timeout step while the wait stalls; a wait that passes before the first step leaves the loaded value, which C14 keeps until the next wait, one that ends after one or more steps (by its timeout or by its event, also a release 1.5 ms in) leaves 0x7f, and a run without a passing wait leaves the initial 0x7f (M2-172, M2-177). The sequencer reads 0x7f before a passing wait and the loaded value right after it (M2-172); that a stalled wait also loads at its entry is inferred from its count-down. C14 ignores CPU and program writes, which also leave the tmwait time unchanged (M2-176, M2-629). The value 0x7f after a stalled wait is the same for every tested count, step field and number of steps, and equals the power-on and reset value (M2-177, M2-430); whether it is a fixed constant or a value held elsewhere is not separable, since C14 ignores writes (chapter 17, item 8). Consecutive tmwaits each wait a full period; a tmwait started with C14 = 0 does not reload and end at once (M2-150).

8.8 r0 to r7 (0xc18 to 0xc34)

The eight GPRs; r6 (0xc30) is SEQ_RESULT (M2-02). Its value is defined by the program's ABI, not a universal hardware error code (chapter 12). STOP preserves the GPRs, the clearing step clears them (M2-53, M2-54).

8.9 FMSS_PC (0xc38)

The program-relative PC.

- At a pause it stays on the pause instruction; the original pause handler advances it by one eight-byte instruction (source; M2-14).
- STOP sets it to 0 (M2-53). The blocked evjmp state shows the offset of the evjmp (0x20, 0x38 in the measured programs; M2-57, M2-392), and the clearing step returns it to 0 (M2-392).
- It is register r8: a movi r8 with a local target lands there, and mov r2, r8 returns the instruction's offset (M2-38, M2-43).
- A program wr or st to 0xc38 jumps to the written offset; the fall-through instruction is not executed. An ld of 0xc38 reads the instruction's own offset (M2-162).

• CPU writes take effect outside a pause: written before the start, the program starts at the written offset; written while a loop runs, the program jumps there; written during an unmet `wait 3`, the `wait` still times out and the run continues at the written offset (M2-278).

8.10 FMSS_C3C (0xc3c)

In a fault handler C3C holds the faulting instruction's offset + 8, also for a nested fault (waits at 0x28, 0x40, 0x70 give 0x30/0x88, 0x48/0xa0, 0x78/0xd0); it reads 0 after the run (M2-179). It stores 32 bits from the CPU and the sequencer, and a start keeps a CPU-written value (M2-154). Presets of 0 or all ones change neither the `wait` timeout, a `tmwait`, a DMA nor the BCH8 parity (M2-179). The sequencer reads 0 at the start of a run (M2-384). The original fault handler reads it.

8.11 0xc40 and 0xc44: the held instruction

Instruction word latches: an end latches R << 16 | arg in C40 and its imm in C44 (M2-164). They show the encoded words of a held pause (12) and `tmwait` (15), C40 = 0x0e060000 / C44 = 0x20 in a local BNZ loop before the clearing step, and C40 = 0x07060000 (the `wait`) in the C60 hold. STOP and abort preserve them (they hold the last instruction); the clearing step clears them (M2-54, M2-422).

8.12 0xc48 and 0xc54

Storage words: C48 holds 32 bits and C54 the bits of 0x83f, from the CPU and the sequencer, and a start keeps CPU-written values (M2-154). Presets of 0 or all ones change neither the `wait` timeout (4388..4393 us at count 7), a DMA in, the BCH8 parity nor a DMA out. A `tmwait` ends at 3879..3900 us with C48 = 0 and with every preset of C10, C3C and C54, but with C48 = 0xffffffff or 0x55555555 it does not end after its period and faults at the `wait` timeout (4388..4390 us) instead, in every run (M2-179). With a count-1 `tmwait`, C48 = 0 and each single bit 8..31 let it end at 941..943 us with C14 = 0, C48 = 1 ends it one timeout instant earlier (449 us, C14 = 1), and each single bit 1..7 and 0xaaaaaaaa make it fault at the `wait` timeout (1434..1436 us) (M2-627). With a count-7 `tmwait`, C48 = v = 1..7 ends it v timeout instants early (3406 .. 449 us against 3896 us for C48 = 0) with C14 = v, and C48 = 8 makes it fault at the `wait` timeout (4390 us) (M2-687). C48 bits 7:0 are the compare value of `tmwait`: it ends at the instant at which its C14 count equals them, never when they exceed the count, and bits 31:8 do not act on it (M2-627, M2-688). C54 has no observed effect. CPU writes during a `wait` release none of `wait` events 6..10, 12..15 (M2-120, M2-143).

8.13 FMSS_C4C (0xc4c): the fault vector

The byte offset of the fault handler, written by the original programs. A fault with C58 bit 2 set continues there, and `evjmp` continues there after its event for every tested C58 value (0, 2, 4) (M2-58, M2-186). No END continues there: no C58 value diverts an END to C4C (M2-149). STOP preserves it; the clearing step clears it (M2-54).

8.14 FMSS_C50 (0xc50): completion bits

bit	set by (device)
0	command flag (`wait` event 0) pending
1	address flag (event 1) pending
2	DMA flag (event 2) pending
3	data or parity transfer flag (event 3) pending
4	ECC completion (event 4) pending
5	status-latch flag (event 5) pending
6, 8	never set, also after a command written through +0x408 (M2-183, M2-209)
11, 12	read 1 while the ECC engine is idle, 0 while it works (M2-112)

• Bits 0..5 mirror pending `wait` events 0..5: an operation's bit is set while its flag is pending and clears with the flag's acknowledgement (M2-112, M2-209).

• Bit 0 after a `70h` write: clear in the first two sequencer reads after the write, set from the third instruction after it, kept through a timeout fault and a write of 1, and cleared by the `FMC_STATUS` bit 1 acknowledgement. It is not set without a command or after an acknowledged one (M2-152).

• Idle value `0x00ff1800`, unchanged before and after the fault handler writes 1, in both the no-fault and the faulting handler controls (M2-18, M2-28). After a status latch it read `0x00ff1820`.

• The original fault handler reads bit 0 and writes 1 (source).

• Bit 4 stays set after `0x810` = 1, which does not acknowledge the ECC completion; bit 5 clears with `FMC_STATUS` = `0x1800000` (chapter 7, section 7.3).

8.15 FMSS_C58 (0xc58): fault routing

• A write reads back as value AND `0xd6`, from the CPU and from the sequencer: zero, all ones and every single bit, 68 cases (M2-70). Bit 0 is not stored: `C58` = 1, 3, 5, 7 read back 0, 2, 4, 6 (M2-148).

• Bit 2 set: a timeout continues at the fault vector; clear: at the next instruction, after about the same timeout (idle `wait 1` sweep; M2-11).

• No `C58` value diverts an END to `C4C`: `C58` = 0..7, `0x12` and `0x16`, with `C4C` written by the CPU (before or after `C58`) or by the program, and `C58` = 2 and 6 in every write order, end at the fall-through END with `C5C` = 0 (M2-149). An END with bit 0 written leaves `C5C` = 0 and runs no fault vector (M2-148); bit 0 does not select start-at-vector (M2-12).

• `evjmp` jumps to `C4C` after its event for `C58` = 0, 2 and 4; with no event it times out and then follows bit 2 like a `wait` (M2-186).

• Bits 1, 4, 6 and 7 are stored and change neither the timeout path nor the handler: with `C58` = `0x40`, `0x80` or `0xc0` a timeout continues at the next instruction, with `0x44` or `0x84` it vectors to `C4C`; with 0, 2, `0x10` or `0x12` an unmet `wait 3`, a count-0 `tmwait` and the undefined opcode 1a continue after the fault (`C5C` 0), with 4, 6, `0x14` or `0x16` they vector (`C5C` 1); bit 2 alone decides (M2-612).

• The original programs write it in their prologue.

8.16 FMSS_C5C (0xc5c)

Timeout-vector cases end with `C5C` = 1, ordinary continuation cases with 0; `C5C` reads 1 while the fault handler runs (M2-384). An END, whatever `C58` holds, leaves `C5C` = 0 (M2-148, M2-149), and the sequencer reads 0 at the start of a run (M2-384). Its full bit map is open (chapter 17, item 8).

8.17 Buffer handshake (0xc60 to 0xc6c)

• `C60` / `C64`**: buffer reset request / completion.** The original CPU handshake writes `C60` = 1, waits for `C64` = 1 and writes `C60` = 0 (source; M2-20).

• `C60` **= 1 during a run holds the engine** (M2-121). Written 1 ms into a `wait`, the timeout still vectors to the handler, but the engine then stays with `C00` = `0x407b4`, `C08` = 1, `C60` = `C64` = 1, `C40` = `0x07060000`, `CS_IRQ` = 4. STOP and the buffer handshake do not reset it, and the hold survives an emCORE reload. Writing `C60` = 0 releases it (M2-126; section 8.22). Restored to 0 at 2.5 ms, while the `wait` is still pending, the run faults at 4.24 ms and the ordinary reset succeeds.

• `C68`**.** No role found.

• `C6C`**: buffer configuration.** The original normal path writes 0, the alternate bypass `0x10` (source).

• CPU writes to `C60`..`C6C` during a `wait` release none of `wait` events 6..10, 12..15; every following reset succeeds once `C60` returns to 0 (M2-125, M2-143).

8.18 Parameter RAM (0xd00 to 0xd7c)

0xd00..0xd7c retain 32 CPU-written words (M2-03). 0xd80..0xdfc read 0 and drop CPU writes and program wr/pst writes, read back from both sides, with no alias into 0xd00..0xd7c (M2-04, M2-147). STOP preserves PRAM; the clearing step and the full reset zero it (M2-49, M2-54, M2-123). Details of the windows are in chapter 4.

8.19 Program lifecycle

The original CPU driver places the program in coherent bus-visible RAM, resets the engine, FMC and ECC, then initializes the parameter ABI and GPR inputs, clears the sequencer IRQ flags, sets C04, completes the buffer handshake and writes the start word (source; M2-20). The original programs initialize C10, C58, C4C and the peripheral flags themselves. Starting the program, selecting a NAND CE and starting a NAND or DMA transaction are three different operations.

Because the clearing reset step zeroes the GPRs and PRAM, parameters are written after the reset (M2-55; a reset after the parameters fails the native NAND tests in emulation, M2-50).

An END sets CS_IRQ bit 0 and leaves the program's r6 result. A fault sets bit 2 and vectors or continues; the handler still reaches END, which gives CS_IRQ = 5.

8.20 pause (12) and CPU resume

pause stops the program with C08 = 0xa, CS_IRQ bit 3, PC on the pause instruction and C00 = 0x4ffb4 (M2-14; the operand forms are in chapter 6).

- **Resume.** The original CPU handler advances PC by 8, acknowledges bit 3 and sets start bit 0. Both measured forms then reach the following marker and END (M2-25). The program continues after the CPU writes PC = offset + 8 and a start word, with or without the CS_IRQ bit 3 acknowledgement (11 cases); without a PC write it stays paused (12 cases) (M2-188). With two start words after the PC write the program continued in three of four runs and stayed paused in one (M2-512). A PC write takes effect outside a pause too (section 8.9; M2-278).
- **Redirect.** Writing C04 to an END-only program, PC = 0, acknowledging and resuming with the normal control fields ends the paused program; the ordinary reset then completes (M2-26).
- **Inherited pause.** A pause inherited from an earlier run needs resume followed by abort (M2-26).
- **STOP.** A reset that writes STOP and waits for C08 = 0 fails after a pause (M2-15); a model in which the pause persists through STOP reproduces the failure cascade (M2-16, inferred and emulated).

8.21 Reset

The original reset (mem:0x502c) writes STOP and waits for C08 = 0, taking its abort branch only for C08 = 4; it polls the C60/C64 buffer handshake and FMCTRL1 bit 30, each with a 100 us bound in the tested build, and writes C00 = 0x80000 before the FMC reset. Effects of the individual steps (device, M2-53, M2-54, M2-123):

step	effect
STOP (C00 = 8)	PC = 0, C08 = 0; preserves all eight GPRs, all 32 PRAM words, C10 = 1, C04, C40/C44, C4C
STOP, then C00 = 2, buffer C60 1 → 0, FMC 004 = 0x800 → 0x80000000, or ECC 0x81c = 1	preserve GPRs and PRAM
FMC 004 = 0x80000000 (alone, after 0x800 or as 0x80000800)	zeroes FMC 00c, 010, 014, 018, 01c, 02c, 030, 034, 058, 05c and sets FMC_DMA_CTRL to 7; ECC_CTRL kept (M2-645)
ECC 0x81c = 1	zeroes ECC_CTRL; the FMC words kept (M2-645)
FMC 004 = 0x800	changes none of these words (M2-645)
STOP, then C00 = 0x80000	clears GPRs, PRAM and the non-zero C04, C10, C40, C44, C4C; C14 stays 0x7f
C00 = 0x80000 alone, or in any order with STOP or abort, while a loop runs	clears r0..r7, PRAM, C04, C10, C4C, C40, C44; C00 reads 0x7f60 (0 when STOP follows) (M2-422)

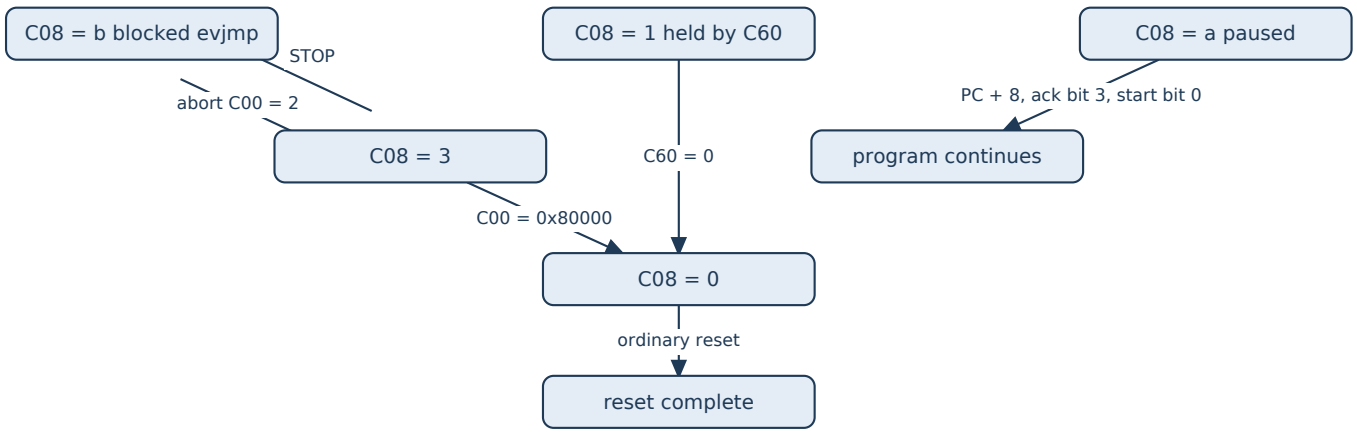
step	effect
STOP, abort, or STOP then abort while a loop runs	clear nothing: GPRs, PRAM, C04, C10, C4C kept, C40/C44 hold the last instruction; C08 = 0 after STOP, 3 after abort (M2-422)
complete reset	the same as the clearing step, plus FMC 00c..05c and ECC_CTRL read 0; FMC_DMA_CTRL reads 7 (M2-123)

No C00 sequence changes FMC 0x00c..0x038, 0x058, 0x05c or ECC_CTRL (M2-422): FMCTRL1 bit 31 zeroes the FMC words and sets FMC_DMA_CTRL to 7, and 0x81c = 1 zeroes ECC_CTRL, from CPU and sequencer writes alike; these two steps account for the zeroing of the complete reset (M2-123, M2-645); no measurement writes the C60/C64 handshake on its own.

Power-on. The window at power-on (first access after a power cycle) equals the window after the ordinary reset in every word of 0x000..0xffc except C00 (0x7f60 at power-on, the last start word 0xffb4 after the reset) and C04 (0 at power-on, the program address after the reset); it equals word for word the untouched window left by a probe cleanup (M2-281, M2-430). Power-on values include C14 = 0x7f, C08 = 0, C50 = 0x00ff1800, CS_IRQ = 0, FMC_DMA_CTRL = 7, FMC_STATUS = 0xc3000, 0x1cc = 0x8060, 0x1d0 = 0x3010d0, 0x808 = 0x700 and 0x810 = 0x10, with every GPR and PRAM word 0 (M2-430). The ordinary reset therefore reproduces the power-on state apart from C00 and C04, and the first program after power-on whose instruction 0 is not END runs on its first start, with no priming start (M2-431).

8.22 States that block the reset and their release

state	signature	release	cleanup method code	findings
paused	C08 = 0xa, CS_IRQ = 8	resume or redirect (section 8.20); inherited: resume and abort	1; 3	M2-25, M2-26
waiting with the timeout disabled	C08 = 4, no fault	ordinary reset	—	M2-56
blocked evjmp with the timeout disabled	C08 = 0xb, PC = the evjmp offset, CS_IRQ = 0; STOP leaves 0xb, reset phase 1 fails	abort C00 = 2 (→ C08 = 3), then C00 = 0x80000 before another STOP (→ 0), then the ordinary reset (phase 6, rc 0)	0x01000318	M2-57, M2-59, M2-60, M2-62, M2-63
status transaction active after END	reset phase 4 (FMC acknowledgement) fails with FMCTRL1 = 2	FMCTRL1 = 0x30020, FMC_STATUS = 0x1800000, another reset	4	M2-29, M2-31
held by C60 = 1	C08 = 1, C60 = C64 = 1, STOP ineffective	C60 = 0, wait for C08 0 or 3, abort and C00 = 0x80000 if needed, then the complete reset	0x20 (0x40 with the fallback)	M2-121, M2-126



• **Blocked** evjmp**.** A STOP written after the abort, without the clearing step, returns the engine to 0xb (M2-60). With the sequence of the table, both an inherited state and a fresh zero-field evjmp with C00 = 1 reach C08 = 0;

the full reset and the next independent marker (END, CS_IRQ = 1, r2 = 0x72657365) succeed. These recoveries take 107 us and 106 us, including the initial failed STOP poll (measured durations, not a timing specification).

• **Status transaction.** An active FMC status transaction survives the sequencer's END. A program that latches status, faults at its wait and reaches its handler's END (r6 = 0xfa17fa17, CS_IRQ = 5, C08 = 0; FMCTRL1 = 0xa, FMC_STATUS = 0x008c3000, C50 = 0x00ff1820) leaves a reset that fails at stage 4 with C08 = 0 after 102 us. Two-latch programs fail reset phase 4 with FMCTRL1 = 2 and recover with the release of the table; each following marker reaches END. Sequencer status alone does not identify which peripheral handshake failed (chapter 9).

• **Coverage.** In one full execution of all families (31,001 sampled records) no setup, reset or cleanup failed (M2-64): the blocked-evjmp release ran 16 times, the status release 1,342 times, pause resume 197 times and pause resume with abort once; the ordinary reset handled the rest (M2-75). 23 of those records skip the reset/preflight explicitly; they are not separately measured resets.

• C00 **orders on a blocked** evjmp**.** CPU writes 1 ms and 2 ms into a blocked start = 1 zero-field evjmp leave: STOP C08 = 0xb (C00 0x40000); abort C08 = 3 (C00 0x40002); STOP then abort 3; abort then STOP 0xb; 0x80000 alone or after STOP or abort C08 = 0, C00 0x7f60, PC 0; 0x80000 then STOP C08 = 0, C00 = 0. Every sequence containing the clearing step reaches C08 = 0, and the reset that follows recovers the engine in all eight (M2-62, M2-392).

References: chapter 14 (outcomes), chapter 13 (experiments), chapter 15 (interpretation), chapter 16 (device findings), chapter 18 (findings).

Related chapters: chapter 2, chapter 3, chapter 6, chapter 7, chapter 9, chapter 12.

Chapter 9 — NAND controller, buffers and DMA

Summary: The registers of the NAND controller (FMC) that the sequencer drives: the configuration word, the operation trigger, the command/address/data registers, DMA, the status latch, the FMC_STATUS flags, the interrupt enable, the FIFO, the +0x400 range, the bus timing and the 0x100 block. The sequencer itself is described in chapters 2 to 8; the ECC registers at 0x800.. are in chapter 10.

Register names follow M1 (M1 chapter 9). Meanings come from original-code use (chiefly bl_NandReadOnly READ at mem:0x6438 and READID at mem:0x7550; program sources code/bootloader/fmss_asm/READ.s, READID.s) and from device measurements cited by ledger ID (chapter 18).

9.1 Register map

Offset	M1 name	Fields and behaviour	Evidence
0x000	FMC_CTRL0	configuration word, fields in section 9.2; storage mask 0x77077fff	S, D: M2-127, M2-131, M2-135, M2-139, M2-191
0x004	FMCTRL1	operation trigger/reset; distinct from sequencer C00; codes in section 9.3	S
0x008	FMC_CMD	NAND command byte: ff reset, 90 ID, 00/30 read, 70 status; write programs also use 80/10 and 60/d0	S
0x00c / 0x010	FMC_ADDR0 / FMC_ADDR1	packed address bytes; a page at column zero uses row << 16 and row >> 16; storage mask 32 bits each	S; D: M2-83
0x014	FMC_DATA_SEL	data-buffer selection: source uses 0x10 for the FIFO, 0x102 / 0x201 for the alternating buffers; a status latch returns the chip status with 0x100, 0x200 or 1 selected (section 9.6); storage mask 0xffff; 0x058 is its bits 7:0 and 0x05c its bits 15:8, so a later 0x058 write replaces bits 7:0 (0x5541 then 0x058 = 0xd reads 0x550d, for sequencer and CPU writes alike); a DMA in flight follows a rewrite from the next bytes on, and of the tested values a DMA in writes data buffer 0 only with 1 and parity buffer 0 only with 0x10, a DMA out delivers data buffer 0 only with 0x100 (section 9.5)	S; D: M2-83, M2-169, M2-196, M2-603, M2-643, M2-668, M2-702
0x018	—	source writes 1 before data reads; storage mask 0xf (it reads the written value & 0xf); 1 before a data read changes neither page-read data nor ECC results, and 1, 2, 3 or 0xff leave a 512-byte buffer loopback exact; role unresolved	S; D: M2-83, M2-104, M2-405
0x01c	—	ECC slot selection: stores bit 0 only, and an encode uses the slot that bit selects (chapter 10, section 10.2)	S; D: M2-83, M2-417
0x024	—	reads 8 after an 8-byte READ ID transfer (the number of bytes transferred; one transfer size measured)	D: M2-336
0x028	—	9-bit storage word (0x5a5 written by the sequencer reads 0x1a5 from both sides); no counter: it does not advance over commands, chip reads, DMAs, ECC encodes, FMC_FIF0 reads or pld/sti/ldi loops, and does not drift between runs	D: M2-156, M2-173, M2-199
0x02c	FMC_ANUM	address cycles minus one; storage mask 0x7; a four-cycle erase uses 3	S; D: M2-83, M2-168

Offset	M1 name	Fields and behaviour	Evidence
0x030	FMC_DNUM	NAND transfer byte count minus one (0x1ff for a 512-byte sector); storage mask 0x1fff; a DMA moves 512 bytes whatever it holds	S; D: M2-83, M2-189
0x034	FMC_DMA_ADDR	DMA bus address; original READ writes it once per 2 KiB chunk; storage mask 32 bits; a DMA in flight follows a rewrite from the next bytes on (section 9.5)	S; D: M2-83, M2-603
0x038	FMC_DMA_CTRL	DMA control; original M2 READ never writes it; reads 7 after reset; bits 12:6 change during a DMA; bits 2:0 set its speed (burst length inferred); bits 5:3 stored, a DMA out completes only with bits 5:3 = 0 or 1; bits 31:13 not stored; details in section 9.5	S; D: M2-76, M2-79, M2-113, M2-117, M2-123, M2-396, M2-398, M2-609, M2-669, M2-699, M2-700, M2-701
0x040	—	FIFO/metadata command: 0x52 ID latch, 0xc2 corrected-metadata latch, 0xc1 metadata input. After 0x52 it reads 0x50, after 0xc1 it reads 0xc0	S; D
0x044	—	interrupt routing of the controller's flags: bits 0, 1, 2, 11, 12 = command, address, transfer, DMA, status latch (section 9.8)	D: M2-142, M2-200
0x048	FMC_STATUS	completion and state flags, acknowledged by writing masks (section 9.7)	S; D: M2-66, M2-112, M2-145, M2-203, M2-412
0x04c	FMC_NSTAT	latched NAND status byte (section 9.6); distinct from the sequencer IRQ; writes of 0 and all ones are ignored	S; D: M2-87, M2-136, M2-412
0x058	—	FMC_DATA_SEL bits 7:0: a write replaces them and a read returns them (the source writes 1, 0x10, 0x20, which select the data or parity buffer through FMC_DATA_SEL); storage mask 0xff; it reads 0 after a buffer loopback because the loopback's DMA out leaves FMC_DATA_SEL = 0x100, and a later FMC_DATA_SEL write replaces a value written before it (a DMA in with FMC_DATA_SEL = 0 written after 058 = 1, 2, 0x10 or 0x20 therefore changes neither data buffer)	S; D: M2-83, M2-405, M2-597, M2-599, M2-643
Offset	M1 name	Fields and behaviour	Evidence
0x05c	—	FMC_DATA_SEL bits 15:8 in place: a write sets them (its bits 7:0 are not stored) and a read returns FMC_DATA_SEL & 0xff00; the source writes 0x200 (data buffer 1 out) before the final DMA; it reads 0x100 after a buffer loopback (the loopback's last select), a CPU-written 0x200 reads back unchanged; 0x100..0x3ff leave the loopback exact, and 0x200	S; D: M2-83, M2-104, M2-405, M2-599, M2-643

Offset	M1 name	Fields and behaviour	Evidence
		changes neither page-read data nor ECC results	
0x060 / 0x064 / 0x068	FMC_META0..2	FIFO/metadata data words; READ ID bytes appear in 0x060/0x064 (section 9.9)	S; D: M2-124
0x080	FMC_FIFO	CPU FIFO read port (restore OS mem:0x0802a640); CPU reads after a data read deliver page data; sequencer reads return 0 with or without the 0x52 latch and from DMA-filled buffers; writes never reach a data buffer (section 9.9)	S; D: M2-124, M2-207, M2-341, M2-436, M2-617
0x100..0x1fc	—	block of unknown function; sequencer-writable words only at 0x1b8..0x1d8 (section 9.12)	D: M2-160, M2-210
0x200..0x3fc	—	copy of 0x000..0x1fc: offset bit 9 is not decoded below 0x400	D: M2-155
0x400..0x7fc	—	read 0; no second register set operates there (section 9.10)	D: M2-82, M2-139, M2-155, M2-183, M2-185, M2-393

Reset state: the per-case reset zeroes 0x00c..0x05c and leaves FMC_DMA_CTRL reading 7 (M2-123); the step that does so is FMCTRL1 bit 31 (0x80000000), which zeroes 0x00c, 0x010, 0x014, 0x018, 0x01c, 0x02c, 0x030, 0x034, 0x058 and 0x05c and sets 0x038 to 7, while FMCTRL1 0x800 changes none of them (M2-645). The power-on values are the same: 0x004 reads 0x40000000, 0x020 0x7f167f, 0x038 7, 0x048 0xc3000, and the other storage words 0 (M2-430).

9.2 FMC_CTRL0 (0x000): configuration word

Bits	Meaning	Evidence
0	data-path enable: with bit 0 clear, a data transfer does not complete and delivers no data	D: M2-135 (role inferred)
8:1	chip enables	S
9, 10	stored; toggling changes neither read time nor data	D: M2-135, M2-139
11	set in the measured base word; toggling changes neither read time nor data	D: M2-135
14:12	timing field, part of the cycle (section 9.11)	D: M2-127, M2-131
15	not implemented (reads 0)	D: M2-139
18:16	timing field, part of the cycle; also places the read latch: values ≤ 1 shift read data by one byte	D: M2-127, M2-131
19..23	not implemented (read 0)	D: M2-139
24	DMA enable: a DMA programmed while it is clear is held (no byte moves, `wait 2` times out, FMC_STATUS bit 20 clear, 0x03c static) while NAND-to-buffer transfers complete; setting it starts the held DMA with the FMC_DMA_ADDR value at that moment; set, DMA is exact (section 9.5)	S; D: M2-184, M2-251, M2-338
25, 26	stored; toggling changes neither read time nor data	D: M2-135, M2-139
27	not implemented (reads 0)	D: M2-139
30:28	stored; no measured effect on command, address, data in/out, status latch, page open or the completion times of a command, a latch, an address-to-data read and a command after ready	D: M2-127, M2-131, M2-135, M2-139, M2-407
31	not implemented (reads 0)	D: M2-139

The storage mask is 0x77077fff, the same for CPU and sequencer writes (M2-191). No FMC_CTRL0 bit makes the +0x400 words hold values (M2-139).

9.3 FMCTRL1 (0x004): operation codes

Words used by the original programs (S):

Code	Operation
0x1	address cycles
0xe2	data read
0x42	first-sector data read
0x12	parity/metadata into ECC
0x100 / 0x180	DMA out; 0x180 is the final-transfer variant
0xca	status polling (latch)
0x20	status release (CPU/boot-read code)
0x10020	status release (READ)
0x30020	status release (WRITE/ERASE)

Code	Operation
0x033338e0	initial setup

Single bits, each written alone with FMC_DNUM 15, FMC_DATA_SEL 1 and no CE selected (D, M2-411):

Bit	Operation started	Flags
0	address operation	C50 bit 1 at once, FMC_STATUS bit 2
1	data read	C50 bit 3; FMC_STATUS 0xc3000 → 0x8041008 (bits 3 and 27 set, 13 and 19 cleared)
8, 9	DMA	FMC_DMA_CTRL counting at once, then FMC_STATUS bit 20 and C50 bit 2
other 28 bits	none alone	no change of 0x048, C50, 0x038, 0x04c, C08, 0x810

What the source words do beyond these single bits (0xca, 0x20/0x30020, 0x340, the set-up word as combinations) is known only as whole words (0x340: section 9.9).

Replacing an operation. An FMCTRL1 operation written before the previous one completes suppresses the previous operation's completion flag but not its data (M2-332). 0x100 (DMA out of buffer 0) replaced at once by 0xe2 (chip read into a buffer) still delivers 497..509 of the 512 bytes to RAM (a run of 4..16 bytes ending at byte 23 reads 0), and after 1, 4 or 16 delay loops all 512, but FMC_STATUS bit 20 never sets and `wait 2` times out; after 64 loops, with the DMA complete, both operations complete. The replacing read completes (bit 3, `wait 3` passes) only when FMC_DATA_SEL is switched to 2 before the 0xe2 write. A read replaced at once by a DMA loses the read's flag while the DMA completes. The serial order (DMA, `wait 2`, acknowledge, read, `wait 3`) is exact.

9.4 Command, address and data transfer

A command is one write of FMC_CMD; it sets FMC_STATUS bit 1. An address transfer sends FMC_ANUM + 1 bytes from FMC_ADDR0/FMC_ADDR1 after FMCTRL1 = 1 and sets bit 2. A data or parity read moves FMC_DNUM + 1 bytes into the selected buffer and sets bit 3 (M2-112). Data-in (a 512-byte transfer after 80h + address) uses the same per-byte cycle as data-out (M2-131).

Programs on CE0 and CE1 (single raw and ECC pages, four and eight pages per run, one page on each CE in one run) and erases on CE0 and CE1 read back exactly: erased pages are all 0xff, and the status after each is 0xe0 (M2-194). A two-plane program (80h, row of block 3000, data, 11h, ready, 81h, row of block 3001, data, 10h, ready) reads back exactly on both blocks, and a two-plane erase (60h, row, 60h, row, D0h, ready) leaves both pages all 0xff, with status 0xe0 after each (M2-339). A four-cycle erase (FMC_ANUM = 3) erases the block on CE0 and CE1 with FAIL clear, in the same time as the three-cycle erase (M2-168).

The original read program alternates two sector-buffer selections and two ECC slots (M2-21). It can issue a page on one CE while waiting for another and transferring a previous page. These are distinct levels of concurrency: CE scheduling, NAND-to-buffer transfer, ECC processing, and buffer-to-system-memory DMA. The driver groups DMA pointers in 2 KiB chunks although ECC is sector-based; a chunk is a software transfer grouping, not another physical NAND page.

Metadata and parity travel through selections separate from the 512-byte data buffer. FIFO/latch commands expose some of these bytes to the CPU/sequencer. The source selectors do not establish arbitrary buffer sizes. 0x058 is FMC_DATA_SEL bits 7:0 (M2-643); a DMA in with FMC_DATA_SEL = 0 changes neither data buffer (M2-597, M2-668); slot selection and arbitration are in chapter 10, section 10.2 (M2-417, M2-418, M2-419).

9.5 DMA

Transfers. Serial DMA is byte exact: memory → data buffer → memory on both slots with FMC_DMA_CTRL 0 and 7, and 4 KiB NAND page reads with FMC_DMA_CTRL 0, 1, 2, 4, 7 and 8 (M2-76). The original M2 READ sets FMC_DMA_ADDR once per 2 KiB chunk, never writes FMC_DMA_CTRL, and writes 0x05c = 0x200 before its final DMA (S).

Length. A DMA moves 512 bytes whatever FMC_DNUM holds (0..0x1fff tested), in, out and in both directions, on both slots (M2-189).

Alignment. The RAM address is aligned down to a multiple of 4 bytes in both directions: an address 1, 2 or 3 above a word boundary reads or writes from that boundary, offsets 4, 8, 16 and 32 are used exactly, and nothing outside the 512 bytes changes (M2-664).

Targets. A DMA in completes for every FMC_DATA_SEL value tested, but writes data buffer 0 only with 1 and parity buffer 0 only with 0x10; with 0, 3, 4, 8, 0x40, 0x80, 0x11, 0x100, 0x400, 0x4000 or 0x8000 no data or parity buffer changes (M2-668). A DMA out (FMCTRL1 0x100) also completes for every value tested and delivers data buffer 0 only with 0x100; with 0, 1, 0x300, 0x400 and 0x1100 it writes 512 zero bytes (M2-702).

Parity buffer. A 512-byte DMA into parity buffer 0 (FMC_DATA_SEL = 0x10) leaves the last 64 bytes of the source (offsets 448..511) in the buffer: a DMA out returns those 64 bytes repeated over 512, and programming 128 bytes from the buffer writes them twice. An inbound DMA with FMC_DATA_SEL = 0x1000 completes but leaves the buffer 0xff (M2-232).

FMC_CTRL0 **bit 24.** A DMA programmed with bit 24 clear is held: its wait 2 times out, FMC_STATUS bit 20 stays clear, 0x03c stays at the start address and no byte moves in either direction or slot, also 2000 delay loops later, while the NAND-to-buffer transfers of a page read complete (M2-184, M2-338). Setting bit 24 starts the held DMA: 0x03c advances by 0x200, bit 20 sets, the next wait 2 passes and all 512 bytes arrive. It runs with the FMC_DMA_ADDR value at that moment: rewritten while the DMA is held, the output lands at the new address (an inbound DMA reads the new source) and the old area stays untouched (M2-338). With bit 24 set the loopback and the page-read DMAs are exact (M2-184).

form	RAM after the buffer-to-RAM DMA	evidence
bit 24 clear until the END	no byte; DMA pointer static	M2-184
bit 24 clear, set again 200 delay loops before the next operation	all 512 bytes	M2-338
bit 24 clear during the DMA, set again after its timed-out wait 2 and followed a few instructions later by the next operation or the END	the first 32 bytes of each DMA (the rest untouched in the loopback; in a page read 32 data bytes then zeros in sectors 0..4)	M2-251

In the last form the held DMA starts at the bit-24 write and is cut by the next controller operation or the END (cut inferred, M2-251). An outbound DMA programmed right after the held inbound one has started moves nothing, while its wait 2 passes (M2-251).

Rates. A 512-byte buffer-to-memory DMA completes within 1..2 us (M2-114). Measured from the trigger to FMC_STATUS bit 20, DMA in (memory to buffer) takes ≈ 2.7 us and DMA out (buffer to memory) ≈ 1.1 us (M2-117).

FMC_DMA_CTRL **(**0x038**).**

Condition	Read value	Evidence
after reset / at every case start	7	M2-79, M2-123
during DMA in	0xa07 (bits 12:6 = 40, mid-count)	M2-113, M2-398
during DMA out	0x1e07	M2-113
during a transfer (written 7)	bits 12:6 count up from 0 in steps of 8 (0x200; single samples show values in between) to 112..120 (0x1c07..0x1e07), then return to 0 (0x038 = 7): inbound 13..16 samples of 188 ns, outbound 5..6, the same for FMC_DNUM 3, 15, 63, 255 and 511 (a DMA always moves 512 bytes); bits 5:3 stay 0	M2-117, M2-398
bits 2:0 written 0..7 before a DMA	the DMA completes exactly, more slowly the smaller the value (bits 12:6 still changing after 48 samples for 0, 16..17 samples for 6 and 7); afterwards bits 12:6 read (v + 1) × ceil(128 / (v + 1)) − 128: 0 for v = 0, 1, 3, 7 and 1, 2, 4, 5 for v = 2, 4, 5, 6 (0x42, 0x84, 0x105, 0x146)	M2-609
written 2 / 4	0x42 / 0x84 after a DMA (the residual count of the row above), as on M1	M2-79, M2-609

Condition	Read value	Evidence
written `0x10..0x80`	DMA faults	M2-79
written `0x40..0x1000` (alone and with bits 2:0 = 7), `0x1fc7`, 0	reads back unchanged; with any of bits 12:6 set the next DMA never completes (`wait 2` faults); with 0 the loopback is exact	M2-396
bits 12:6 non-zero, or bits 5:3 = 2, 4 or 7, before a DMA out	the DMA does not complete (bit 20 clear, `0x038` keeps the value) and its target receives only the first 32 bytes: buffer words 0 and 1 and six zero words (words 0..3 and four zero words for bits 12:6 = 120)	M2-669
bits 5:3 = 3, 5 or 6 (`0x1f`, `0x2f`, `0x37`) before a DMA out	the DMA does not complete (`wait 2` faults, `0x038` reads the written value after the write and after the wait) and writes 32 bytes, source words 0 1 0 1 2 3 4 5, nothing after	M2-699, M2-700
bits 5:3 = 1 (`0xf`)	the DMA out completes and writes each group of four words as four copies of the group's first word	M2-669
`0xf` before a DMA in	the DMA completes and stores buffer word 4g + j (j = 0..3) from source word 4g + 5j; where 4g + 5j passes the 128-word source, from the RAM after it	M2-699, M2-701
bits 31:13 written (`0x2007`, `0x80000007`, `0xffffe007`)	reads 7: not stored; the DMA is exact	M2-669
after a DMA superseded by the next	`0x187` mostly; also `0x147`, `0x107`, `0xc7`	M2-79
after a plain DMA started on a stale flag	`0x207`	M2-78

The values after superseded DMAs leave bits 12:6 non-zero, as during a running DMA. The residual counts fit bits 12:6 counting the 4-byte words of the 512-byte transfer (128, modulo 128) in bursts of (bits 2:0) + 1 words; this reading is inferred (M2-609). Of the eight values of bits 5:3 only 0 and 1 let a DMA out complete; 2..7 stop it after 32 bytes (M2-669, M2-700). With bits 5:3 = 1 the out direction repeats each burst's first word while the in direction reads the source words 4g + 5j; read as an address pattern, the transfer runs as four-beat bursts 16 bytes apart whose beats step 20 bytes (inferred from the word order, M2-701). While a DMA runs bits 12:6 read non-zero values that change from sample to sample and are 0 again after it (M2-722). Why a non-zero preset of bits 12:6 stops a DMA after 32 bytes is open (chapter 17, item 16).

Completion flag. DMA completion is `FMC_STATUS` bit 20 (`wait` event 2). `WAIT2` does not consume it; it stays set until acknowledged by writing its mask, after which the next `WAIT2` blocks until a new completion (M2-66). If the flag is left unacknowledged, the next DMA's `WAIT2` passes at once and a further DMA is programmed while the previous one runs (M2-78):

- the running DMA is cut: 32..64 bytes (32/36 in the ECC-sequence program) reach the output, and with a plain DMA the previous output stops after 256 bytes;
- a DMA started while another runs never completes; reset recovers the controller (M2-114).

A DMA superseded by a second DMA programmed 0 or 16 `nop2` after its `FMCTRL1` write (M2-394; `FMC_DNUM` does not change the outcome, as it does not change a DMA, M2-189):

order	outcome
in, then in	both complete; buffer 1 holds the second input except its first 8..64 bytes, which hold first-input bytes at the same offsets (or 24..32 bytes matching neither input in untagged fills); buffer 0 holds the first input except 0, 8, 12 or 32 leading bytes, which hold first-input bytes 84..95 or 64..95
order	outcome
out (buffer 0 to address P), then out (buffer 1 to address Q)	the second `wait 2` never passes (it faults at 126.6 ms); buffer-1 bytes appear at their own offsets, first at P for 16..52 bytes, then at Q from that offset (56..120) to byte 511, its earlier bytes at

order	outcome
	neither address; P holds buffer-0 bytes before that run (8..72 bytes, with 4..16 bytes matching neither in untagged fills), after 16 nop2 also after it up to offset 196..236, and is untouched beyond

With both inputs tagged per word, every output word of the four overlaps is a word of one of the two inputs, the initial content or zero (M2-697, M2-698; word ranges, init = not written):

order, gap	first output	second output
out, out, 0	P: 0..1 buffer 0, 2..5 buffer-0 words 0..3, 6..8 buffer 0, 9 buffer-1 word 120, 10..13 buffer 1, 14..127 init	Q: 0..13 init, 14..127 buffer 1
out, out, 16	P: 0..7 buffer 0, 8 buffer-1 word 120, 9..13 buffer 1, 14..57 buffer 0, 58..63 buffer-0 words 0..5, 64..127 init	Q: 0..13 init, 14..127 buffer 1
in, in, 0	buffer 0: all first input	buffer 1: 0..7 first input, 8..127 second input
in, in, 16	buffer 0: 0..7 first-input words 16..23, 8..127 first input	buffer 1: 0..15 first input, 16..127 second input

Words are at their own offsets unless a source word is named. The bytes that matched neither input in untagged fills are repeated words of the same inputs from other offsets, among them buffer-1 word 120 at the select switch.

A DMA in flight follows FMC_DMA_ADDR and FMC_DATA_SEL as they are rewritten, at the same offsets: a new address written at once after the FMCTRL1 write takes effect from byte 32 (from byte 256 after 16 nop2), a new buffer select from byte 8 (232); rewriting FMC_DNUM or writing FMCTRL1 0x100 again changes nothing, and 0x038 = 7 written at once changes bytes 8..23 of the output and leaves 0x187 at completion. A RAM-to-buffer DMA follows the same registers: a new source written at once fills the buffer from byte 32, and a new buffer select written at once moves the whole transfer into the other buffer. A second DMA programmed over a running one is therefore a rewrite of the running one's registers: no second transfer starts, there is one completion, and the byte maps above follow from the rewritten select and address (M2-603; that the transfer moves in bursts and reads the registers live is inferred). The split points shift with the gap and a preceding BCH8 or BCH16 encode and differ in some identical runs (M2-394). For a single register rewrite g nop2 after the FMCTRL1 0x100 write of a buffer-to-RAM DMA:

g	FMC_DMA_ADDR rewrite takes effect at byte	FMC_DATA_SEL = 0x200 rewrite takes effect at byte
0	32	8
2	32	32
4	64	44
8	128	120
16	256	256 or 228
32	480	476 or 492
64	none (the DMA has completed)	none

The address switch falls on a multiple of 32 bytes and is the same in both runs per gap (M2-671); the select switch falls on a word boundary, never later than the address switch, and in 7 of 14 runs the word at the switch holds 0xd5fa1f44 whatever the offset (M2-672). With both buffers tagged per word this word is buffer-1 word 120 (byte offset 480): 16 nop2 after the start the select switch falls at word 63, 57 and 58 in three runs, the word at the switch is buffer-1 word 120 in the first two and every later word comes from buffer 1 at its own offset; 32 nop2 after the start the switch falls at word 114 in all three runs with no fixed word (M2-694, M2-695, M2-696). That buffer reads lead the RAM writes by up to one 32-byte burst is inferred.

With every WAIT acknowledged, the same programs are byte exact on both slots with FMC_DMA_CTRL 7 or 0 (M2-76, M2-78).

DMA and ECC overlap. Serial and different-slot DMA/ECC overlap is exact for BCH8 and BCH16 (M2-80). A DMA on the buffer whose own ECC encode is still running is not interlocked, in either direction:

DMA, 0, 8 or 32 nop2 after the encode kick	buffer and output	parity	evidence
into the buffer	the DMA completes, but 68..164 old bytes stay in word-aligned runs of 4..32 bytes between runs of new data	matches neither the old nor the new data at gaps 0 and 8, the old data at gap 32	M2-80, M2-399
out of the buffer	344..424 of the 512 bytes delivered: runs of 12..32 bytes about every 96 bytes differ; the buffer is unchanged	that of the buffer data	M2-401

Both set FMC_STATUS bit 25 (0x020c3000). After 128 nop2, or with the DMA after WAIT4, the transfer is exact, the parity is that of the data the encode read and bit 25 stays clear; no other case sets bit 25 (M2-71, M2-80, M2-399, M2-401). Bit 25 is a buffer-collision flag (meaning inferred).

9.6 Status latch and FMC_NSTAT (0x04c)

FMC_CMD = 70h followed by FMCTRL1 = 0xca starts the status latch. FMC_NSTAT then holds the chip status byte:

Value	Meaning	Evidence
0x80	chip busy (erase, program or read)	M2-87
0xe0	chip ready	M2-87
0x70	no CE selected: echo of the command byte	M2-87
0	no latch issued since reset	M2-136

Live polling. One latch makes FMC_NSTAT follow the chip: through a block erase it reads 0 at the first ≈ 20 us sample, 0x80 from the next, and 0xe0 at ready (≈ 2.27 ms). Without a latch it stays 0 for 3.2 ms; a latch after the erase reads 0xe0 (M2-136). Measured with status polls of ≈ 21.6 us, a program stays busy ≈ 325 us (15 polls), an erase more than 1 ms and a read one poll (M2-87).

Buffer selection. With FMC_DATA_SEL = 0x100 the latch returns the real status as with 1: 0x80 while the chip is busy, 0xe0 when ready; an idle CE0 latch reads 0xe0 and a latch with no CE selected 0x70 with both values (M2-169). A poll on 0x100 reads 0x80 while a page programs and 0xe0 once it is ready (M2-195). Multi-page program runs write every page with the status poll on FMC_DATA_SEL 0x200 (four and eight pages, two CEs), 0x100 and 1 (eight pages each) (M2-196).

0x70 has bit 6 set, so a bit-6 mask cannot tell the no-CE echo from a ready chip (M2-87). A latch issued after 90h leaves the first READ ID byte 0xad in FMC_NSTAT and sets no FMC_STATUS bit 23 (M2-239). How WAIT5 and WAIT12 evaluate the latch is in chapter 7.

Latch state after END. An active status-latch transaction survives sequencer END and prevents the next command's WAIT0 completing (M2-31, M2-32): in both status-read orderings and in the variant without a read, the program stalls at the second command's WAIT0 (PC 0xa0, 0xa0, 0x70) after the first WAIT5 completes. In the two-latch controls the following reset fails in phase 4 with FMCTRL1 = 2. Reading FMC_NSTAT before WAIT5 does not cause this. The command after a latch is refused until an FMCTRL1 write; status clear, ECC kick and FFh do not release it (M2-181, M2-202; chapter 7, section 7.5). A second latch (0xca without a new 70h) needs no re-arm: after the first latch's WAIT5 and only the acknowledge of bit 23 it passes WAIT5 with 0xe0, with nothing between and after FMCTRL1 0x20, a status clear, a CE toggle, a BCH8 encode, the set-up word, 0x100e0 or the release (M2-494). An address operation in that position does not complete: its wait 1 times out and FMC_STATUS bit 26 sets (M2-610). A DMA in or out, a 512-byte data read, a 16-byte parity read, an 8-byte FIFO read, a BCH8 encode and a second latch complete after an unreleased latch as after the release; only a command and an address operation are refused (M2-641). With a 70h that is not waited for, a latch written straight after it passes, one written one or two instructions later (one or two nop2, one nop, one movi or the bit-1 acknowledge) sets bit 3 instead of bit 23 and its WAIT5 faults, and one written three or more instructions later passes (M2-593). An early FMC_STATUS read captures 0xc3000, a later one 0x8c3000, so completion is ordered in time. The original CPU/boot-read code releases the latch with FMCTRL1 = 0x20, READ with 0x10020, WRITE/ERASE with 0x30020 (S). A 0x30020 release plus a status acknowledgement permits reset and a following marker in all three controls (M2-31). This qualifies those transitions, not every FMCTRL1 bit or every event-consumption rule.

9.7 FMC_STATUS flags

Bit	Mask	Set by	wait event	Evidence
1	0x2	command sent	0	M2-112
2	0x4	address sent	1	M2-112
3	0x8	data or parity read done; also set at the end of data-in	3	M2-112
19:12	0xff000	a field of state bits: 0xc3 (bits 12, 13, 18, 19) after reset and at power-on, surviving a write of 1; 0x41 (bits 12, 18) after a chip read of 1..128 bytes (also after a 512-byte read) and after parity reads of 13..64 bytes; 0x14 (bits 14, 16) after 256 and 384 bytes; 0x3c (bits 14..17) after 511 and 512 bytes; 32 writes to FMC_FIFO leave 0x41, 128 writes 0x3c; a DMA in (data or parity buffer) and a status latch restore 0xc3 from 0x41 and from 0x3c, the set-up word also (keeping bit 27); command, address, metadata encode and DMA out do not; for L = 160, 192 and 224 the value is 0x41 and for 255, 257, 320, 383, 385, 448, 480 and 508 0x14, so a data read of 1..224 bytes leaves 0x41, of 255..508 bytes 0x14 and of 511 or 512 bytes 0x3c, the changes falling between 224 and 255 and between 508 and 511 bytes	—; sampled during a read it goes 0xc3 → 0x41 → 0x14 (with a transient 0) → 0x3c as the read advances (M2-720)	M2-112, M2-203, M2-430, M2-497, M2-642, M2-706, M2-707
20	0x100000	DMA done	2	M2-112
23	0x800000	status byte latched, when the chip reports ready	5	M2-112, M2-193
24	0x1000000	never set by a status latch (any 0x044 value)	—	M2-163
25	0x2000000	DMA into or out of a buffer under its own ECC encode (collision flag, meaning inferred)	—	M2-80, M2-399, M2-401
26	0x4000000	set when an address operation does not complete after a status latch with no 0x30020 release (0x40c3000); not set by a command refused after a status latch	—	M2-181, M2-610
Bit	Mask	Set by	wait event	Evidence
27	0x8000000	set after the transfer of a programmed sector,	—	M2-145

Bit	Mask	Set by	wait event	Evidence
		clear after an erased one; cleared by writing 0x0ff00ffe		

Bit 27. After reading a programmed page bit 27 is set, after reading an erased page clear: a raw read ends with 0x8041000 against 0x41000, the ECC path with 0x803c000 against 0x3c000 (the raw read ends with the 128-byte spare read, which sets bits 12/18 again; the ECC path ends with a 512-byte read, which clears all four, M2-203). Per sector it is set after the metadata/parity or data transfer of every written sector (all-zero and random pages, 16 of 16) and of no erased sector (0 of 8), also for a single 512-byte, a parity-only or an ECC-decode read (M2-145).

Flags are acknowledged by writing their mask (S): writing all ones clears a pending DMA flag (bit 20) and leaves bits 12, 13, 18, 19 set, writing 0 changes nothing, and writing all ones when idle sets no bit (M2-412). WAIT0, WAIT1 and WAIT2 do not consume bits 1, 2 and 20; acknowledging makes the next wait block, and a new operation re-arms it (M2-66). Masked WAIT12 (arg = mask = 0x40) passes even after bit 23 is cleared by writing 0x1800000 (M2-68). Sequencer C50 bits 0..5 repeat the same completions plus ECC (chapter 8).

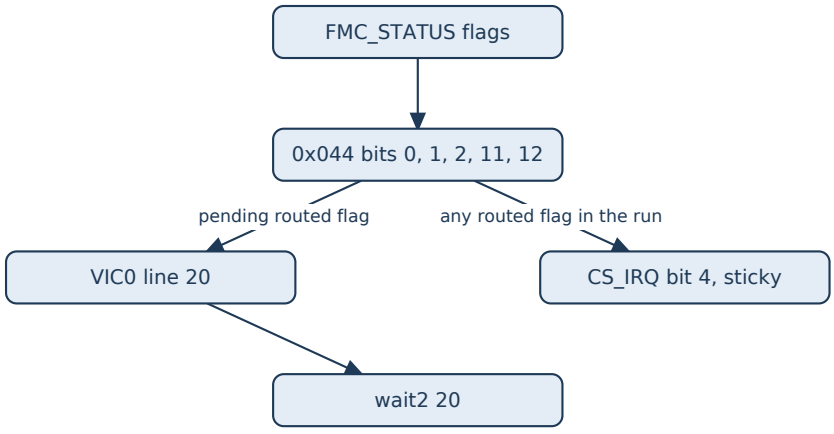
Observed values: idle 0xc3000 (bits 12, 13, 18, 19); after a status latch 0x8c3000; during a 512-byte read 0x8041000 → 0x8014000 (about half way) → 0x803c008 (done); during data-in 0xc3000 until 0xc3008 at the end (M2-112, M2-203).

9.8 Interrupt

0x044 routes the controller's flags onto VIC0 line 20, as on M1 (M1 chapter 9):

0x044 bit	flag
0	command (FMC_STATUS bit 1)
1	address (bit 2)
2	data or parity transfer (bit 3)
11	DMA (bit 20)
12	status latch (bit 23)

Line 20 follows a routed flag only while it is pending, and of the 32 single 0x044 bits only the operation's own bit raises it. CS_IRQ bit 4 is set by every routed flag raised during the run and stays set after the flag's acknowledgement; after an address operation it also reflects the preceding 90h command (M2-200). wait2 20 follows line 20, not CS_IRQ bit 4 (M2-204). With 0x044 = 0xffffffff, VIC0 raw bit 20 is raised after a DMA, a command and a status latch, and wait2 20 passes 4–7 us after the same operations; with 0x044 = 0 line 20 is never raised (M2-142). An END, also a fault-handler END, with an enabled flag still pending leaves USB working under the probe's cleanup (M2-208). ECC operations raise no line (M2-212, chapter 10, section 10.15).



9.9 FIFO and READ ID

READ ID through FIFO latch 0x52 places the ID bytes in 0x060/0x064: 0x060 = 0xb614d5ad, 0x064 = 0x14d5ad44 (AD D5 14 B6 44, then the ID repeats from AD); 0x068 = 0. The CPU port 0x080 reads 0 with or without the latch (M2-124).

Sequencer reads of FMC_FIFO (0x080) return 0 from DMA-filled data buffers 0 and 1 and parity buffer 0 (130 reads each, with and without FMCTRL1 = 0x340) (M2-207). Without an FMCTRL1 write, 32 or 128 writes to it leave the buffer unchanged and hold no written value (M2-207). FMCTRL1 = 0x340 with FMC_DATA_SEL 1 (its wait 2 passing and acknowledged) writes 512 zero bytes to FMC_DMA_ADDR (S). A transfer from RAM S + 0x200 on into data buffer 0, at the same offsets, follows with no further FMCTRL1 write: a page programmed from buffer 0 holds RAM S + 0x200..S + 0x37f in its first 384 bytes and the earlier buffer content in the rest when the data-out set-up follows at once (0x038 bits 12:6 read 96 after it, bit 20 does not set again), and RAM S + 0x200..S + 0x3ff in all 512 bytes when 32 or 128 port writes come between (bit 20 sets again); no written value appears. With a DMA out programmed right after the acknowledge, buffer words 8..15 and 0..7 hold S + 0x220.. and S + 0x240.. and the DMA's own target is not written. With FMC_DATA_SEL 0 the 0x340 also zeroes S and buffer 0 keeps its content (M2-617). The second transfer runs to its end with no further write: after 2000 delay loops all 128 words of a page programmed from buffer 0 are RAM S + 0x200 + 4k (M2-703, M2-704). An FMC_DATA_SEL write stops it where it is: written 0, 8 or 16 nop2 after the acknowledge it leaves 0, 16 or 32 RAM words in buffer 0 and the earlier content in the rest, with nothing moved in the 2000 delay loops after it (M2-703, M2-705); the data-out set-up's FMC_DATA_SEL write is such a write. No DMA out of data buffer 0 after 0x340 returns the buffer, with or without port writes in between: a plain read-back leaves its target holding what was there (all 512 bytes, or all but 16 bytes, which read 0) (M2-341), and read-backs after polling 0x038, after rewriting it to 7 or repeated each leave 0x038 at 0x207 or 0x407 and deliver, at their own address or 0x200 above it, the RAM bytes that follow the 0x340 output, zeros or part of the original buffer data, never a written word (M2-613; that they continue the transfer started by 0x340 is inferred). With FMC_DATA_SEL 0 the same sequence reads back the buffer, in some runs with 16 bytes reading 0 (M2-341). Reads of the port do not advance FMC 0x028 (M2-173).

CPU reads of the port deliver NAND page data in 32-byte transfers: per chunk FMC_DNUM = 31, FMC_DATA_SEL = 1, FMCTRL1 = 0x1c2, a poll of FMC_STATUS bit 3, then FMC_DATA_SEL = 0x100, FMCTRL1 = 0x340 and eight reads of 0x080; the bytes equal those of a DMA raw read of the same page (M2-436). These reads follow a chip transfer, while the sequencer reads above were of DMA-filled buffers.

The qualified CPU FIFO path clears FMC_CTRL0 bit 24 (DMA mode) before these control words. A caller switching from DMA must select FIFO mode too: the generic CPU path that retained bit 24 produced failing FIFO data and corrupted earlier DMA readbacks. Unintended memory transfers are the inferred cause, consistent with the 0x340 effects above (M2-728). The corrected per-read mode selection passes native regression checks; those checks do not constitute a new device readback measurement (M2-730).

After an 8-byte READ ID transfer, FMC 0x024 reads 8 (M2-336).

Register snapshots deliberately omit consuming FIFO/data ports; an all-zero placeholder there is not a read value.

9.10 The +0x400 range

No register set operates at +0x400. 0x400..0x7fc read 0 (M2-155). Sequencer and CPU writes of complementary patterns to 0x0c, 0x10, 0x14, 0x18, 0x1c, 0x2c, 0x30, 0x34, 0x58, 0x5c at +0x400, in both orders, read back 0 from the sequencer, and the primary words keep their own values: no aliasing (M2-82). +0x40c, +0x410 and +0x434 read 0 with every FMC_CTRL0 bit set; no FMC_CTRL0 bit enables the set (M2-139). A sequencer ld of +0x40c costs the same 2 cycles as one of the decoded 0x00c and of the unused 0x60c, so access time does not tell a decoded slot from undecoded space (M2-596).

- **Commands.** A 90h or 70h written through +0x408 changes no +0x400 word and raises no first-set event 0 or C50 bit 6 (C50 0x00ff1800 in eight samples right after the write). A full READ ID through +0x400 (set-up word +0x404, CE in +0x400, 90h to +0x408, address, DNUM 7, FMCTRL1 0xe2 or 0x1c2) times out at wait 6, 7 and 9, leaves +0x424 and every +0x400 word 0 and sets no C50 bit 6 or 8, while the same READ ID through the primary set passes wait 0, 1 and 3 (M2-183, M2-336).

- **Enables.** No candidate enable write of the 17 tested (C10 = 0, 2, 3, 0xff; 0x018 = 1, 2, 0xff; 0x058 = 1, 0xff; 0x05c = 0x200, 0xffff; 0x01c = 2, 0xff; C48 = 1; C54 = 1; 0x400 = 0x1077803; 0x438 = 7) makes a +0x400 word hold a value (M2-393).

- **DMA.** A DMA programmed through +0x404 raises no first-set `wait 2`, and `wait 8` never passes. A +0x400 RAM → buffer → RAM loopback (slot 0 and 1) leaves the output untouched with both `wait 8` timed out while the primary buffer keeps its content, and a +0x400 parity DMA leaves parity buffer 0 of the first set unchanged (M2-185).

The M1 channel-1 pin mapping is unqualified on S5L8720, and four attached CEs do not imply two independent controller channels.

9.11 Bus timing

Every NAND bus cycle — command, address, data in and data out — takes

```
cycle = (FMC_CTRL0[14:12] + FMC_CTRL0[18:16] + 2) × ≈ 7.06 ns
```

per byte (114 ns at 7/7) (M2-127, M2-131, M2-135). A status loop shows only its 70h command's cycle. `FMC_CTRL0[18:16]` ≤ 1 shifts a programmed page by exactly one byte (a leading 0x00, then the page from byte 0, the last byte lost); on an erased page the first byte reads wrong. Inference: the latch samples one cycle before the chip drives the byte (M2-131). `FMC_CTRL0[30:28]` has no measured effect on any operation, and moves none of the completions of a command, a latch, an address-to-data read and a command after ready in 188-ns samples (M2-139, M2-407).

Operation	Time	Evidence
512-byte NAND data read, setup timing word	≈ 22 us (≈ 43 ns per byte)	M2-119
23-byte parity read	≈ 2.3 us	M2-119
512-byte data-in, `FMC_CTRL0[14:12]` or `[18:16]` 0 → 7	33.0 … 58.4 us	M2-131
page open (00h, address, 30h, status ready)	≈ 44 us (16 opens 707 ± 1 us; 701 / 706 us at `[14:12]` = 0 / 7), mostly array read	M2-139
DMA in / out, trigger to bit 20	≈ 2.7 us / ≈ 1.1 us	M2-117
chip busy: program / erase / read	≈ 325 us / ≈ 2.27 ms / one poll	M2-87, M2-136

9.12 The 0x100 block

0x100..0x1fc is a block of unknown function, visible again at 0x300 and nowhere else: each of 0x1bc..0x1d8 is copied to +0x200, none to +0x400 or +0x600 (M2-155). The sequencer-writable words are exactly (M2-160):

words	width	value read before the write
0x1b8	1 bit	0
0x1bc..0x1c8, 0x1d4, 0x1d8	32 bits	0xffffffff
0x1cc, 0x1d0	24 bits	0x8060, 0x3010d0

The values are those each case read before the program's write (each case restores the word); they are also the values at power-on and after every reset (M2-160, M2-430). The other words keep nothing and read 0x12 from the sequencer (M2-160); 0x3a8..0x3b4 read 0x12 from the sequencer and 0 from the CPU (M2-210). The CPU reads, at power-on and after every reset (M2-160):

words	CPU read
0x100..0x1b8	0
0x1bc..0x1d8	the values above (0xffffffff, 0x8060, 0x3010d0)
0x1dc	0xffffffff
0x1e0..0x1fc	0x12

The copy at 0x300..0x3fc reads the same (M2-160). Writing 0x1b8..0x1d8 with 0, 1 or 0xffffff changes neither the time of a DMA, a DMA with BCH8 encode, a 4-byte data read, a 16-page open with a 512-byte read or an unmet `wait`, nor the data or FMC 0x048 (M2-210).

Chapter 10 — ECC engine and codeword layout

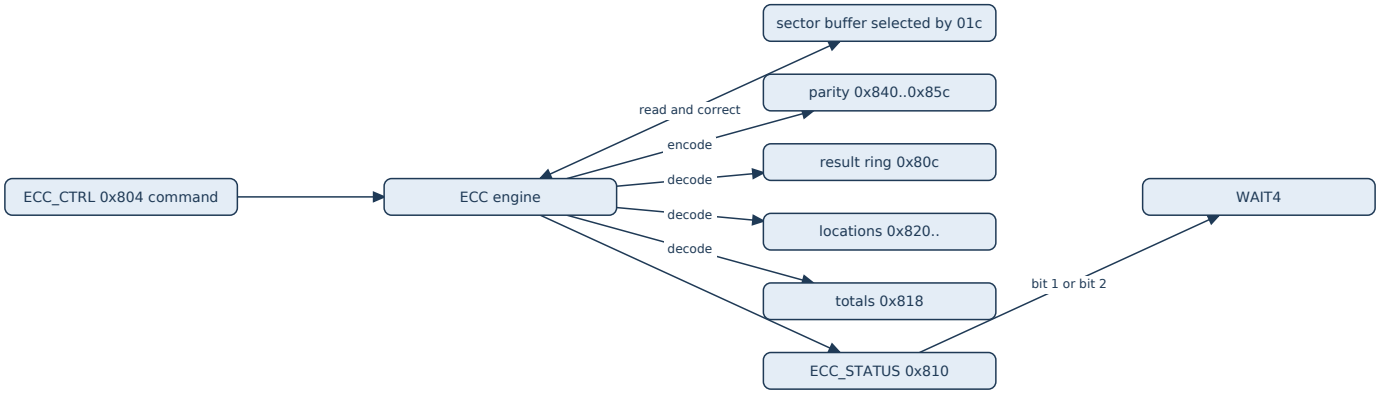
Summary: the ECC engine at 0x800. Its registers, the two BCH codes it computes (BCH8 and BCH16 over GF(2^13)), the stream stored on the chip, encode, decode and correction, the error-location words, the result ring and status bits, timing, WAIT4 and its acknowledgement, the data-path hazard and the interrupt state.

10.1 How error correction works

The engine works on one 512-byte data sector at a time, with BCH8 or BCH16 selected per operation. An encode computes the parity of a sector (data and, for sector 0, metadata); a decode checks a sector against its stored parity, corrects it in place when it can, and records a result, a count and the bit locations of the corrections. Completion is signalled on ECC_STATUS (0x810), which wait 4 (WAIT4) follows.

The original programs use the 512-byte sector with BCH8 and BCH16 only, and no ECC_CTRL word selects another code (M2-197). Source anchors: bl_NandReadOnly::_m2SetEccMode at mem:0x4fcc, READ at mem:0x6438, initialization at mem:0x55d8; restore WRITE at mem:0x080379e8; program sources code/bootloader/fmss_asm/READ.s and code/root-restore/fmss_asm/WRITE.s.

The engine does not depend on the sequencer: the same register writes issued by the CPU, each WAIT replaced by a poll of its flag, decode the boot-ROM format (three sectors of 26 parity and 512 data bytes, BCH16, 0x808 = 0x80) and the main BCH8 format with the same results as the original readers, and CPU-kicked encodes give the host BCH8 parity (slots 0 and 1, with metadata 0xa02) and BCH16 parity (slot 0, with metadata 0xa06) (M2-435, M2-436).



10.2 Registers

Register	Meaning	Evidence
0x01c (FMC)	selects the ECC buffer/slot by bit 0, the only stored bit (slots)	source; M2-21, M2-417
0x800	identification word, reads 0x20070618	M2-116
0x804	ECC_CTRL: command and mode (section 10.3); reads back without the command bits; storage mask 0x3f04	M2-98, M2-101, M2-116, M2-191
0x808	reads 0x700 (also at power-on); storage mask 0x1f81; bits 12:8 = 0 make 0x810 bit 5 set after every completion (bit-5 threshold, inferred: section 10.10)	M2-116, M2-191, M2-430, M2-434
0x80c	ECC_RESULT: 8-slot ring of decode results, advanced per read by the sequencer or the CPU, also through the copy at 0xa0c (section 10.9)	M2-90, M2-91
0x810	ECC_STATUS: completion, uncorrectable, count-threshold and erased bits (section 10.10)	M2-88, M2-94, M2-95, M2-97, M2-99, M2-100
0x814	storage mask 0x2f; values 1, 2, 4, 8, 0x20, 0x2f read back and change neither a BCH8 decode (status, totals, data, time) nor BCH8 encode parity; 0x2f changes neither BCH16 encode parity nor a BCH16 decode (0x810, 0x80c, 0x818, data)	M2-191, M2-660, M2-708, M2-709, M2-710
0x818	(running total of corrected bits << 11) | number of decodes (section 10.11)	M2-89, M2-132
0x81c	a write with bit 0 set = ECC reset: clears 0x818, sets 0x810 to 0x10, puts the ring's read and write positions at slot 0; values without bit 0 do nothing	M2-91, M2-96, M2-421
0x820..	error locations of the last decode, 13-bit fields (section 10.8)	M2-128, M2-132, M2-138, M2-714
0x840..0x85c	parity of the last encode (section 10.6); 0x844 holds no written bits	M2-116, M2-191
0x900..0x9fc	read 0 after encodes and decodes; 0x940..0x950 drop writes	M2-171, M2-191

A per-case reset zeroes ECC_CTRL (M2-123); the step that does so is the ECC reset 0x81c = 1, which zeroes ECC_CTRL and no FMC word (M2-645). The storage masks are the same for CPU and sequencer writes (M2-191). The M1 ECC area has no M2 counterpart: 0x900..0x9fc (0x990 included) read 0 after BCH8 and BCH16 encodes of zero, ones and ramp data and after decodes, and 0 or 0xffffffff written to 0x940..0x950 before an encode or decode changes neither the parity nor the decode (data, 0x810, 0x818) (M2-72, M2-171). Offset bit 9 is not decoded in 0x800..0xbff: 0xa00..0xbfc is a copy of 0x800..0x9fc (M2-155). At power-on 0x800 reads 0x20070618, 0x808 0x700, 0x810 0x10 and the other ECC storage words 0, as after the reset (M2-430). M1's offsets (ECC_KICK 0x80c, ECC_RESULT 0x810, ECC_MODE 0x814, ECC_DONE 0x840) name different registers on M2.

Slots. FMC 0x01c stores bit 0 only (2, 4, 8, 0x100, 0x80000000 read 0; 3, 7, 0xff read 1), and an encode uses the data buffer of the slot that bit selects: the parity at 0x840.. is that of the slot-0 data for even values and of the slot-1 data for odd values; WAIT4 passes in every case (M2-417). Two encode kicks back to back (0x01c = 0, kick, 0x01c = 1, kick, then WAIT4) give one result, the slot-1 parity with 0x810 = 0x13: the second kick replaces the first (M2-418). Changing 0x01c from 0 to 1 while a slot-0 BCH8 encode runs gives a parity that matches neither slot's data (M2-419).

10.3 ECC_CTRL (0x804)

Bits	Effect (device)
0	decode
1	encode; with bit 0 also set the operation acts as a decode, and 0x003 / 0x007 encode nothing
2	BCH16 (clear = BCH8)
7:3	no effect on encode parity
13:8	metadata length in bytes: 0, 3, 10, 12 measured; 13, 16, 20 and 32 cover the 12 written metadata bytes followed by 0xff; 33, 64, 128 and 255 give the parity of the data alone, as 0
15:14	ignored; not stored
23:16	no effect on encode parity
31:24	no effect on encoding

Evidence: M2-98, M2-101, M2-197, M2-662. Reading 0x804 returns the written word without the command bits (M2-116); the stored bits are 0x3f04 (M2-191).

No ECC_CTRL word selects a code other than BCH8 and BCH16. Single bits 3..7 and 16..31, every pair of them and all of them together, with BCH8 and BCH16 and with metadata lengths 0x80 and 0xff, give BCH8 or BCH16 parity as bit 2 selects, or none: 438 encodes classified, never the M1 RS-8/12/16 parity (M2-197); with lengths 0x80 and 0xff that parity covers the data alone (M2-662).

The original driver writes 0xa01 / 0xa05 to decode the first sector with ten metadata bytes and 0xa02 / 0xa06 to encode it (BCH8 / BCH16). Subsequent sectors use 1 / 5 or 2 / 6, without metadata. The original driver selects BCH16 for eight-sector pages with a 218-byte spare and uses 26 parity bytes per sector; BCH8 uses 13 (M2-22). These selection rules belong to that driver and format, not to a hardware prohibition on other combinations.

M1's bit-15 Reed-Solomon selector falls in M2's ignored length bits 15:14 and selects no code (M2-197).

10.4 Stream on the chip

For the 4 KiB page workload:

Portion	BCH8 bytes	BCH16 bytes
First sector metadata	10	10
Parity per sector	13	26
Data per sector	512	512
Total parity, eight sectors	104	208
Metadata + parity	114	218

The stream is metadata, parity, data for sector 0, then parity and data for each remaining sector: meta[10] | par[n] | data[512], then seven par[n] | data[512] (n = 13 or 26). A 128-byte BCH8 spare leaves 14 spare-tail bytes after the 114 metadata/parity bytes. The CPU metadata output uses a 12-byte container; only ten bytes are the page metadata. M1's three metadata bytes per sector and its parity-buffer address and byte-order rules are not part of the M2 layout.

10.5 The codes

10.5.1 BCH8

BCH8 is M1's BCH8 code (M1 10.3.2; M2-77):

- field GF(2^13), x^13 + x^4 + x^3 + x + 1 (0x201b);
- generator 0x115f914e07b0c138741c5c4fb23, degree 104, roots alpha^1..alpha^16;
- message = 512 data bytes then the covered metadata bytes, most significant bit first;
- parity = message × x^104 mod g, 13 bytes, most significant byte first.

Sector 0 covers all ten page metadata bytes (M1: three slot bytes per sector). Sectors 1..7 cover only their data. Every non-erased raw sector examined (352 of 352, including pages written by the original firmware on CE0 and CE1) is a code word of this code (M2-77).

10.5.2 BCH16

BCH16 is a narrow-sense binary BCH code with t = 16 over GF(2^13)/0x201b: generator degree 208 (roots alpha^1..alpha^32); message = data then metadata, most significant bit first; 26 parity bytes, most significant byte first. All 14 engine-encoded sectors match, and encoding is repeatable (M2-86). With 0x814 = 0x2f a BCH16 encode gives the same parity and 0x810 (0x13) as with 0, and the parity equals the host BCH16 code (M2-708, M2-710).

10.6 Encode and the parity registers

An encode (ECC_CTRL bit 1) computes the parity of the selected sector. After an encode, 0x840..0x85c hold the parity, equal to the parity written to NAND (M2-116):

Code	Register order	Bytes
BCH8	low byte of 0x84c, then 0x848, 0x844, 0x840, each word big-endian	13
BCH16	0x858 (2 bytes), then down to 0x840	26

The word order is reversed and each word is big-endian. After an encode ECC_STATUS reads 0x13 and WAIT4 passes on bit 1 (section 10.12).

Twenty programs with three encode commands each (BCH8/BCH16 × two slots × five between-kick actions) all end; every recorded encode stage reads 0x80c = 0 and 0x810 = 0x13, and reads of 0x900..0x910 are zero. Three commands plus three passing waits there do not prove three independently completed encodes (M2-72).

An encode kicked by CPU writes of ECC_CTRL, with no sequencer program running, gives the same parity as the host BCH8 and BCH16 codes; pages programmed with it, with software-encoded parity or from raw images read back exactly through every reader (chapter 12; M2-712).

10.7 Decode and correction

A decode (ECC_CTRL bit 0) checks the sector against its parity and corrects data, metadata and parity bits in place.

Code	Corrected	Beyond the radius	Evidence
BCH8	1..8 bit errors in data, metadata, parity or mixtures (including bit 0, bit 4095, an 8-bit burst, 4 + 4 mixes), exactly	9, 10 and 16 data errors and 5 data + 4 parity: data returned **unchanged**	M2-84
BCH16	up to 16 bit errors in data, metadata and parity	17, 20, 24 and 9 + 8 parity errors: data returned unchanged	M2-92

No miscorrection occurs beyond the radius: 13,440 random BCH8 sectors with 9..64 errors and 11,520 random BCH16 sectors with 17..64 errors are all uncorrectable, with the data unchanged (M2-174). All-zero and all-0xff data with valid parity, and clean sectors, read exact. The raw pages carrying host-built code words with known errors were verified by raw read-back before decoding, and were read through the original READ sequence.

A decode of a sector without valid parity reports uncorrectable: 2,080 such decodes all returned 0x80c = 1. They had no valid parity written for the input, so they test neither the correction radius nor miscorrection (M2-73).

An all-zero sector-0 chunk (metadata, parity and data 0) decodes as a clean code word with BCH8 and BCH16 (0x810 bit 3 clear, 0x80c = 0): zero parity is the valid parity of zero data and is not treated specially; 0xff data with zero parity is uncorrectable (M2-214).

An erased sector sets ECC_STATUS bit 6 (0x55), and its ring entry is 0 (M2-95). The NAND controller's FMC_STATUS bit 27 also separates the two: it is set after the transfer of a programmed sector and clear after an erased one (chapter 9, section 9.7; M2-145).

A BCH16 decode of a host code word with 12 data errors gives 0x810 0x35, 0x80c 0, 0x818 0x6001 (12 corrections, one sector) and the clean data, the same with 0x814 = 0x2f as with 0 (M2-709, M2-710).

A decode kicked by the CPU alone corrects as the software BCH decoder does: with 1, 2, 4, 7 and 8 injected data errors, 3 errors in metadata or parity and one 9-error sector per page, both return the same data, metadata, correction counts and location sets, and both leave the 9-error sector uncorrectable and as stored (M2-711, M2-713).

10.8 Error locations (0x820..)

After a decode, 0x820.. hold the corrected bit locations as consecutive 13-bit fields from bit 0 of 0x820, in ascending byte order, and within one byte the higher bit first: a sector with errors at bits 2 and 3 of byte 512 reports 4099 before 4098 (M2-128, M2-714). Bits above the last location read 0. BCH8 uses up to 8 fields; BCH16 up to 16 (0x820..0x839). The 0x818 total field gives the number of corrections; in the location captures 0x818 bits 15:11 gave the count.

The location space is the message in order, data, metadata, parity, each byte numbered least significant bit first (M2-132, M2-138):

Part	Location
data byte b, bit i (i = 0 is the LSB)	b × 8 + i, 0..4095
sector-0 metadata byte b, bit i	4096 + b × 8 + i, 4096..4175
sector-0 parity byte b, bit i	4176 + b × 8 + i
parity byte b, bit i, sectors without metadata	4096 + b × 8 + i

Metadata and parity corrections appear as fields. Uncorrectable and clean sectors leave every field 0. All 36 locations of 1..8-error sectors, 15 of 15 numbering cases and 4 of 4 sector-0 parity cases matched (M2-128, M2-132, M2-138); the location sets of CPU-kicked decodes equal those of the software decoder (M2-713).

10.9 The result ring (0x80c)

0x80c holds the last eight decode results:

Entry	Meaning
n << 16	n bits corrected (the original driver extracts bits 20:16)
0	no error, or an erased sector
1	uncorrectable (bit 0; the original driver tests it)

Each read returns the slot under a read pointer and advances it: read i returns slot[i mod 8]. Reads of 0x810 / 0x818 and writes of 0x80c do not move the pointer. ECC reset (0x81c = 1) puts the read and write positions at slot 0, after which one read per decode returns that decode's result. The original CPU driver writes 0x81c = 1 in _resetFMSS, and its READ reads 0x80c exactly once per decode (M2-90, M2-91). Reads by the sequencer and by the CPU advance the pointer alike, also through the copy at 0xa0c; 0x90c and 0xb0c read 0 and do not move it (M2-91).

Extra reads shift later results. In READ-sequence captures with additional result reads, the read after WAIT4 and acknowledge did not report the current sector: it reads 0 for corrected and for most uncorrectable sectors, bit 0 is never set, and the few non-zero values appear only in the last sector of a page (M2-85). The ring behaviour

accounts for this (M2-90); M2-85 records retained completion as the earlier inferred cause.

10.10 ECC_STATUS (0x810)

Bit	Meaning (device)	Evidence
0	done; writing 1 clears it, without re-arming WAIT4	M2-67, M2-97
1	encode complete; the WAIT4 level after an encode; write 1 to clear	M2-97
2	decode complete; the WAIT4 level after a decode; write 1 to clear	M2-99
3	uncorrectable	M2-88
4	reads 1 when idle, after ECC reset, after an encode and after acknowledge 0x1ff; writing 1 changes nothing; meaning not established	M2-88, M2-96, M2-420
5	at least 7 corrected bits (BCH8: 6 → clear, 7 → set; the same threshold for BCH16); set after every encode and clean decode when 0x808 bits 12:8 are 0	M2-94, M2-100, M2-434
6	erased sector; write 1 to clear	M2-95, M2-99

Bits 7..12, 16, 24 and 31 are never set, neither by writing 1 to them nor by a BCH8 encode (M2-420). The count from which bit 5 is set equals 0x808 bits 12:8, which reset to 7: with them 0 bit 5 is set after every BCH8 encode and clean decode (0x33 / 0x35), with one of bits 8..12 set it stays clear (role inferred, M2-434). Bits 3 and 5 accumulate across decodes unless 0x1ff is written. The source writes 1 after WAIT4 and 0x1ff for broad clearing.

Value	State
0x10	idle, at power-on and after ECC reset (0x81c with bit 0 set)
0x13	encode complete
0x12	encode complete, after acknowledge 1
0x15	decode complete, 0..6 corrected bits
0x14	decode complete, after acknowledge 1
0x35	decode complete, 7 or more corrected bits
0x1d	decode complete, uncorrectable
0x55	decode of an erased sector

10.11 Corrected totals (0x818)

0x818 = (running total of corrected bits << 11) | number of decodes. The total (bits 31:11) counts every corrected bit, including metadata and parity bits, and skips uncorrectable sectors. ECC reset (0x81c = 1) clears it (M2-89, M2-96, M2-132).

10.12 WAIT4 and acknowledgement

wait 4 follows ECC_STATUS bit 1 after an encode and bit 2 after a decode (M2-97, M2-99). Writing 1 clears only bit 0, so WAIT4 still passes after an acknowledge of 1: a repeated WAIT4 after 0x810 = 1 passes, and the encode completion 0x13 becomes 0x12 (M2-67). Bits 1 and 2 are write-1-to-clear. In C50, bit 4 is the ECC flag and bits 11..12 indicate ECC idle (M2-112).

A write of 0x81c with bit 0 set after a completed encode changes 0x810 from 0x13 to 0x10, and a following WAIT4 faults; values without bit 0 (0, 2, 4, 0x80, 0x100) change nothing and WAIT4 still passes (M2-421).

The original-program WAIT4 control after an encode reaches END with ECC_STATUS = 0x13, bit 2 clear; that is the encode level (bit 1).

10.13 Timing

Operation	Time after the kick (device)	Evidence
encode, BCH8 or BCH16	≈ 4.1 us	M2-118
decode, zero syndromes	≈ 4.0 us	M2-133
decode, erased sector	≈ 4.1 us	M2-118
decode, any non-zero syndrome, BCH8	≈ 4.0 + 1.6 us: ≈ 5.8–6.0 us for 1..8 errors, ≈ 5.8 us uncorrectable	M2-118, M2-129, M2-133
decode, any non-zero syndrome, BCH16	≈ 4.0 + 2.3 us	M2-133

The extra time is independent of the error count and of whether the decode succeeds.

10.14 Data-path hazard

Serial and different-slot DMA/ECC overlap is byte exact for BCH8 and BCH16 (M2-80). A DMA on a buffer under its own encode (0, 8 or 32 nop2 after the kick) is not interlocked and sets FMC_STATUS bit 25 (0x048 = 0x020c3000 against 0x000c3000):

- into the buffer: 68..164 old bytes stay in 4..32-byte runs, and the parity matches neither the old nor the new data at gaps 0 and 8, the old data at gap 32 (M2-80, M2-399);
- out of the buffer: runs of 12..32 bytes about every 96 bytes of the output differ (344..424 of 512 bytes delivered), while the parity is that of the buffer data and the buffer is unchanged (M2-401).

After 128 nop2, or with the DMA after WAIT4, the transfer is exact and bit 25 stays clear; bit 25 appears only in these same-buffer cases (M2-71, M2-80, M2-399, M2-401; the meaning of the flag is inferred). Serial DMA itself is exact (chapter 9, section 9.5).

10.15 Interrupt

No ECC interrupt is found on M2. After BCH8 encodes and decodes, with 0x044 = 0 or all ones and 0x800, 0x808, 0x814, 0x818, 0x844 and 0x860..0x9fc written all ones, single bits of 0x808, 0x814, 0x818 and 0x844, ECC_CTRL bits 3..7 and 16..31 or 0x810 bits 9..31 written, CS_IRQ bits 5 and 6 stay clear while 0x810 shows every completion, no VIC raw line rises beyond the no-candidate baseline and wait2 20 and wait2 43 never pass, while wait 4 passes (M2-212). VIC1 line 11 (M1 ECC source 43) and every other VIC line stay at the baseline after BCH8 encode and decode completions for every tested ECC word (0x800, 0x808, 0x814, 0x860..0x9fc; values 0xffffffff, 1, 0xffff, 0xffff0000, 0x55555555, 0xaaaaaaaa; 0x044 = 0 or all ones; 751 cases) (M2-217). The NAND controller sources raise VIC0 line 20 (M2-142, M2-200). M1's 0x844 enable has no M2 counterpart: 0x844 is an encode-parity word (section 10.6).

10.16 Limits of the evidence and the model

Early zero ECC register snapshots and late register/ECC records that never executed are not measurements and not successful decodes (M2-23).

The M2 simulator runs original programs and the actual ARM physical driver against both layouts. Its BCH8 parity is the physical code of section 10.5.1; BCH16 parity is a hash stand-in. It checks data movement, metadata placement, mode selection, erased handling and error propagation. It does not model correction counts, miscorrection rates or Reed–Solomon. It distinguishes encode and decode readback and keeps the measured completion level after acknowledge 1; its internal done bit is not a physical register-bit claim. These limits are stated in the simulator contract.

Open: the meaning of ECC_STATUS bit 4, its untested bits 13..15, 17..23 and 25..30, the bits of `0x80c` outside the result fields, the role of `0x814` (stored, with no effect on BCH8 or BCH16 encode and decode, M2-660, M2-710) and of `0x808` beyond the inferred bit-5 threshold, and an ECC interrupt source outside the tested words (chapter 17, items 13 and 23).

References: ledger, evidence, device findings.

Chapter 11 — Original M2 programs and operation sequences

Summary: The original sequencer programs (RESET, READID, READ, ERASE, WRITE and the boot-page programs), how READ walks a page, how a run ends, and the completion and acknowledgement rules that every NAND transaction on M2 follows. The controller registers are in chapter 9, the ECC engine in chapter 10, the parameter words in chapter 12.

11.1 The original programs

The original firmware supplies sequencer programs; it does not implement the whole NAND transaction by ARM MMIO polling. The CPU supplies a program-specific parameter ABI, starts FMSS, handles completion, pause and fault, and interprets r6 and the DMA output. The logical VFL/FTL layers sit above this physical interface.

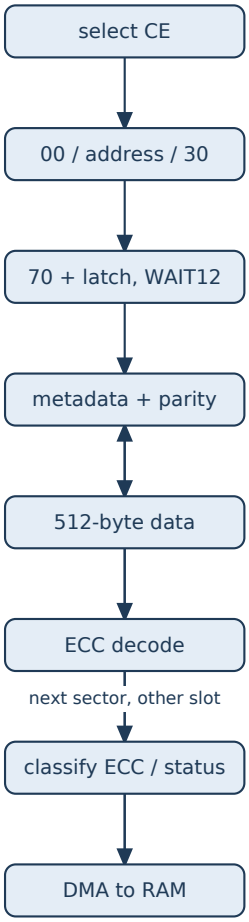
Program	Source anchor	Operation
RESET	`bl_NandReadOnly mem:0x76c0`	select CEs, send `ff`, wait command complete, deselect; the CPU adds a delay
READID	`bl_NandReadOnly mem:0x7550`; restore `mem:0x080377a0`	send 90, column 0, read five bytes, latch the first four into an ID word per CE
READ	`bl_NandReadOnly mem:0x6438`; restore `mem:0x08036688`	multi-CE issue/ready/transfer pipeline, sector ECC, metadata and DMA
ERASE	restore `mem:0x08036398`	address the target, confirm erase, poll status and report failure
WRITE	restore `mem:0x080379e8`	DMA data into slots, encode metadata/parity, transfer the page, program confirm/status
BLREAD / BLWRITE	restore manifests in `code/root-restore/fmss_asm/`	ROM-format boot-page programs with a distinct parameter/layout contract

The cross-version survey contains 73 program instances and eight distinct programs. Two WRITE generations have different result conventions, so an r6 field cannot be generalized across them. Exact source coordinates and control flow are in the program sources `code/bootloader/fmss_asm/*.s` and `code/root-restore/fmss_asm/*.s` at the anchors above. Apple's original ERASE and WRITE programs, unmodified, with their parameter blocks erase and program a block and the page reads back exactly, data and metadata (M2-726).

WRITE reads its DMA list pages × chunks + 1 times: one prefetch before the transfer loop (`WRITE.s` 0x050) and one read per chunk (0x3f0) before the chunk-count test, so the last entry read lies beyond the list and is not used for a transfer (source, M2-213).

11.2 Reading a page (program READ)

READ selects a CE, issues `00`/address/`30`, then polls NAND status with `70` and `WAIT12`. It transfers first-sector metadata/parity, then data, runs ECC, classifies the ECC and status results, and DMAs corrected data to system RAM. It repeats for the remaining sectors with alternating slots (M2-21). Separate counters and CE masks track issued, waiting and transferring pages. Parameters such as row pointers, CE pointers and DMA-list pointers advance during execution.



11.3 End and fault results

END means the microcode finished; the page and sector results still carry the outcome. Fault handlers read `c3c`, capture the selected CE state, deselect the chips, set a program-defined error result, acknowledge `c50` and END. An IRQ fault followed by END and a successful END are different results.

Reaching END in a program or erase path does not by itself certify page contents, codeword validity or every driver transfer. In particular, a program that leaves a DMA completion unacknowledged reaches END with its DMA output cut (section 11.4).

11.4 Completion and acknowledgement rules

Measured M2 rules that a NAND transaction follows (flags and waits in chapter 7 and chapter 9):

Step	Rule	Evidence
command, address, DMA	`WAIT0`/`WAIT1`/`WAIT2` do not consume `FMC_STATUS` bits 1/2/20; a repeated or triple wait passes; acknowledging makes the next wait block; a new operation re-arms it	M2-66
ECC	`WAIT4` follows `0x810` bit 1 after an encode and bit 2 after a decode (write-1-to-clear); writing 1 to `0x810` clears only bit 0, and `WAIT4` still passes	M2-67, M2-97, M2-99
Step	Rule	Evidence
status	one `70h + 0xca` latch polls until the chip is ready; `WAIT5` passes for the standard latch (`70h`, `wait 0`, acknowledge, latch) every time, and for every tested ordering in which the `70h` was waited for and acknowledged; a latch one or two instructions after an	M2-68, M2-130, M2-134, M2-137, M2-161, M2-169, M2-178, M2-196, M2-593

Step	Rule	Evidence
	unwaited 70h (one or two nop2, one nop, one movi or the bit-1 acknowledge between) sets FMC_STATUS bit 3 instead of bit 23 and its WAIT5 faults, while a latch straight after it or three or more instructions later passes; WAIT12 passes iff status & arg & mask ≠ 0 and releases only when the chip is ready; masked WAIT12 passes after FMC_STATUS bit 23 is cleared; the latch works with FMC_DATA_SEL 0x100, 0x200 and 1	
ready/busy	no WAIT 0..15 (arg = mask = 0) and no wait2 N follows the NAND ready/busy line	M2-107, M2-167
next command after status	an active status latch survives END and blocks the next command's WAIT0 until an FMCTRL1 write (FMCTRL1 = 0x20 / 0x10020 / 0x30020 in the originals); status clear, ECC kick and FFh do not release it	M2-31, M2-32, M2-181, M2-202
second latch	a second 0xca latch without a new 70h needs no re-arm: after the first latch's WAIT5 and only the bit-23 acknowledge it passes WAIT5; an address operation in that position does not complete (its wait 1 times out, FMC_STATUS bit 26 sets)	M2-494, M2-610
polls of two chips	with the release (0x30020 + acknowledge) between polls, polls of CE0 and CE1 during an erase pass in both orders; without it the second chip's 70h is refused	M2-198
DMA length	a DMA moves 512 bytes whatever FMC_DNUM holds	M2-189
one DMA at a time	a DMA programmed while the previous one runs rewrites the running transfer's address and buffer select, which it follows from that point on; no second transfer starts and the second wait 2 never passes	M2-78, M2-114, M2-394, M2-603
one operation at a time	an FMCTRL1 operation written before the previous one completes suppresses the previous completion flag, though its data move	M2-332
DMA enable	a DMA programmed with FMC_CTRL0 bit 24 clear is held until the bit is set	M2-338
DMA and ECC	a DMA into or out of a buffer whose own ECC encode is running is not interlocked and corrupts 4..32-byte runs; serial and different-slot overlap is exact	M2-80, M2-399, M2-401

The ECC acknowledgement and masked WAIT12 therefore behave differently from a common one-shot flag.

11.5 Reset, READ ID, program and erase as stimuli

READID is an independent peripheral baseline: it returned Hynix IDs on four CEs, and the FIFO READ ID returns AD D5 14 B6 44 (M2-124). NAND reset, status, address, DMA and ECC actions create controlled event sources for sequencer experiments; chip latency does not measure instruction latency.

Serial program and erase sequences run on the device: programs on CE0 and CE1 (single raw and ECC pages, four and eight pages per run, one page on each CE in one run) and erases on CE0 and CE1 read back exactly, erased pages all 0xff and status 0xe0 after each (M2-194); multi-page program runs poll status on FMC_DATA_SEL 0x200, 0x100 or 1 (M2-196); a four-cycle erase (FMC_ANUM = 3) erases the block like the three-cycle one (M2-168). A two-plane program (80h, row, data, 11h, ready, 81h, row of the second block, data, 10h, ready) of page 0 of blocks 3000 and 3001 reads back exactly on both blocks, and a two-plane erase (60h, row, 60h, row, D0h, ready) leaves both pages all 0xff, with status 0xe0 after each (M2-339).

The same register sequences run from the CPU alone, with no sequencer program and each `WAIT` replaced by a bounded poll of its flag: raw page reads by DMA and by `FMC_FIFO` return the bytes of the sequencer raw read, CPU-kicked decodes of the boot-ROM (BCH16) and main (BCH8) formats equal the original readers' output, and a CPU erase and a CPU program with host BCH8 parity read back exactly, also through the sequencer READ (M2-435, M2-436). Programs from engine-encoded, software-encoded and raw images in both formats read back exactly through every reader, and engine and software decoders agree on injected errors (chapter 12, section 12.7; M2-711..M2-716).

The experiment set contains serial M2 page read/write/erase sequences and selected overlap comparisons. It does not reproduce every original multi-CE pipeline or boot-page workload. Assembling the programs is not physical validation of those operations; the gaps are listed in chapter 13.

11.6 nano4g and nano5g

The nano4g and nano5g can share FMSS program logic and reset/start helpers while retaining different CPU-side clocks, pins, power, IRQ integration and parameter contracts. Shared program bytes alone do not transfer physical register or timing findings from one SoC to the other.

Chapter 12 — CPU interface and software ABI

Summary: The parameter words the original CPU driver passes to READ and READID, the lifetime of parameters across the reset, cache coherency and pause handling, the separate layers of result codes, the reset-verification and recovery records of the probe, and controller access by the CPU without a program.

12.1 Parameter words (READ / READID)

Hardware registers provide the execution interface; parameter RAM contents are defined by each microcode program. The table is the original bootloader READ/READID ABI from `bl_NandReadOnly` read setup `mem:0x4d9c`, mode setup `mem:0x4fcc` and ID setup `mem:0x5ab0` (chapter 14, section 14.3). These are not hardwired meanings of the parameter words.

Offset	Original meaning
`d08`	READID destination address
`d0c`	row-list pointer; READID instead uses the CE count
`d10`	CE-mask-list pointer
`d14`	per-sector status output pointer, or zero
`d18`	page count
`d1c`	metadata output pointer, 12-byte CPU record per page
`d20`	DMA pointer list, one pointer per 2 KiB chunk
`d24`	ECC histogram pointer, or zero
`d28`	chunks per page = sectors per page / 4
`d2c` / `d30`	subsequent-sector / first-sector ECC command
`d34` / `d38`	first-sector metadata+parity / parity-only transfer length minus one
`d3c` / `d40`	program scratch: page summary / chunk counter
`d4c`	NAND timing word (chapter 9, section 9.11)
`d7c`	program scratch: last ECC result

The eight GPRs can also be CPU-preloaded: the boot-page programs use a register-based ABI instead. Neither a NAND command byte nor a USB monitor command is an FMSS opcode.

12.2 Parameter lifetime and reset

The original program advances several pointers in place, so parameters are rewritten for each invocation. `C00` = `0x80000`, alone or with STOP or abort, clears all 32 PRAM words and all eight GPRs, while STOP and abort alone keep them (M2-54, M2-422); no `C00` sequence changes FMC `0x00c..0x05c` or `ECC_CTRL`, which the per-case reset zeroes (M2-123, M2-422): `FMCTRL1` bit 31 zeroes the FMC words and `0x81c` = 1 zeroes `ECC_CTRL` (M2-645). A reset after setup therefore destroys the inputs: parameters are written after the reset (M2-55). The emCORE adapter resets before writing parameters (M2-50). A start keeps the CPU-written GPRs and the CPU-written `C3C`, `C48` and `C54` (M2-154, M2-157), and the first start word after the ordinary reset runs the program (M2-151).

At power-on the window equals the window after the ordinary reset except `C00` (`0x7f60` at power-on, the last start word after a reset) and `C04` (0 at power-on, the program address after a reset); the GPRs and PRAM read 0, `C14` reads `0x7f`, `FMC_DMA_CTRL` 7 and `FMC_STATUS` `0xc3000` (M2-430). The first program after power-on runs on its first start, so no priming start is needed after a power cycle either (M2-431).

A CPU read of `ECC_RESULT` (`0x80c`, also through its copy at `0xa0c`) advances the result ring's read position like a sequencer read (M2-91): a CPU snapshot of the ECC window between decodes shifts the slot from which the

program's own 0x80c reads start; writing 0x81c = 1 resets the ring.

12.3 Completion and coherency

Program and input memory are cleaned from the CPU cache before the sequencer or DMA reads them; DMA-written output is invalidated before the CPU consumes it. The actual allocation and bus address are used: an arbitrary uncached alias is not interchangeable with the validated address path. END, fault and pause are separate completions.

The original pause handler skips the pausing instruction with PC += 8, acknowledges the pause and resumes. Both measured pause forms give the pause signature (M2-14), and an explicit CPU resume reaches the following marker and END (M2-25). The PC write and the start word continue the program with or without the acknowledgement; without the PC write it stays paused (M2-188). With the PC write followed by two start words the program continued in three of four runs and stayed paused in one (M2-512). A CPU PC write outside a pause moves a starting or running program to the written offset (M2-278); see chapter 8.

A CPU write of C60 = 1 during a run holds the engine (C08 = 1) beyond STOP and the reset handshake (M2-121); C60 = 0 releases it, and an engine still running then finishes normally (M2-126).

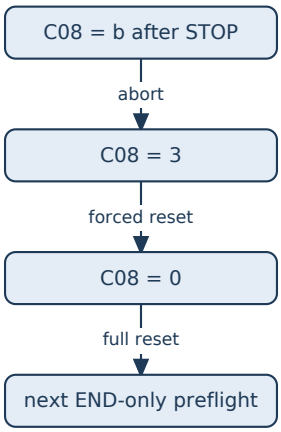
12.4 Result layers

r6 is visible at c30. A microcode result, an ECC per-sector result, a FIL return code and an emCORE worker return are separate layers. Worker 0xfffffffe is the probe's bounded setup/start/cleanup failure, not a hardware opcode or NAND status; it can accompany a valid preceding sample or an entirely unexecuted candidate.

A successful worker return, setup and cleanup do not establish CPU-visible DMA data: programs that leave a DMA completion unacknowledged reach END with every wait passing and a cut payload (M2-71, M2-78). Results therefore carry full payload comparisons and pre-cleanup observations. CPU writes scheduled within a case run on the device without a per-case USB exchange. Compressed host summaries change storage only, not firmware execution or the native result layout.

12.5 Reset verification and recovery

Issuing STOP does not prove a reset. A timeout-disabled evjmp started with C00 = 1 stays in C08 = b (PC 0x20, IRQ 0) after STOP, and later candidates fail the initial reset (M2-57, M2-59). For zero-field evjmp the measured sequence is b → 3 after abort → 0 after a forced sequencer reset, followed by a successful full reset and independent execution (M2-62, M2-63). Of the other C00 orders on a blocked state b, STOP leaves b (C00 0x40000), abort then STOP returns to b, and every order containing C00 = 0x80000 reaches C08 = 0 (M2-392). An intermediate C08 = 0 or 3 does not mark completion: when recovering a captured state-b failure the forced reset is still issued (M2-60). See chapter 8.



With this recovery, a full execution of all families (31,001 sampled records) has zero setup, reset, cleanup or USB failures (M2-64). Its recorded recoveries are forced state-b recovery 16 times, status release 1342 times, pause resume 197 times and one pause resume plus abort (M2-75).

12.6 Recovery diagnostic records

The probe retains a reset failure and attempts bounded abort and forced sequencer reset before checking the full reset and the next END-only preflight. Recovery method bits are software diagnostics, not hardware status bits:

Field	Meaning
firmware capability bit 27	reset while held is supported
method low byte `0x08` / `0x10`	abort / forced reset attempted
method `0x20`	`C60` hold recovery: `C60` = 0, wait for `C08` 0/3, then the complete ordinary reset repeated
method `0x40` (with `0x20`)	the `C60` recovery needed the abort and sequencer-reset fallback (`C00` = 2, then `0x80000`)
method bit 24	validity: status bytes 15:8 and 23:16 are present
status byte 15:8	`recovery_status.after_abort` (host name)
status byte 23:16	`recovery_status.after_forced_reset` (host name)

Records with method 8 alone contain no intermediate status measurement.

12.7 Controller access by the CPU alone

The CPU can drive the NAND controller and the ECC engine without a sequencer program: the programs' register sequences written by the CPU, with each `wait` replaced by a bounded poll of the flag the wait tests, give the same results as the sequencer path (M2-435). Raw page reads by DMA and through `FMC_FIFO` return the bytes of the sequencer raw read; CPU-kicked decodes in the boot-ROM format (three sectors of 26 parity and 512 data bytes, BCH16, `0x808` = `0x80`) and in the main BCH8 format (metadata, parity, data) return the raw data and metadata; a CPU erase leaves the pages all `0xff`, and a CPU program with host BCH8 parity reads back exactly by the raw paths, a CPU decode and the sequencer READ (M2-435).

step	CPU access
FIFO read of 32 bytes	select FIFO with `FMC_CTRL0` bit 24 clear; `FMC_DNUM` = 31, `FMC_DATA_SEL` = 1, `FMCTRL1` = `0x1c2`, poll `FMC_STATUS` bit 3, then `FMC_DATA_SEL` = `0x100`, `FMCTRL1` = `0x340` and eight reads of `0x080` (M2-436, M2-728)
encode	the kicked encode gives the host BCH8 parity in slots 0 and 1 (metadata command `0xa02`) and the host BCH16 parity (`0xa06`) (M2-436)

The reads of `0x080` return the transferred page data only in this sequence; sequencer reads of `0x080` from DMA-filled buffers return 0 (M2-207).

Reset and ready-status success do not establish payload correctness. A generic CPU sequence that kept DMA mode enabled for FIFO access completed its status polls and resets while all 12 round-trip controls failed. Select the requested data path for every read, preserving CE and timing fields; a sequence can switch DMA → FIFO → DMA without another controller reset. The corrected implementation is validated in the compiled-kernel model (M2-727, M2-728, M2-730); the earlier device-qualified operations below remain separate evidence.

The CPU path covers the whole page life cycle in both formats, with the engine or software BCH, and agrees with the sequencer path in every comparison:

operation	result	evidence
raw read, DMA and FIFO	boot-ROM reserved-region pages (CE0 pages 0, 1, 2 and 100, three BCH16 sectors each) and main-format BCH8 pages (rows 128, 129, 256000, 383872 and a sequencer-written page) equal the sequencer raw read	M2-711
decode, engine and software BCH	data, metadata and sector status equal those of the sequencer readers in both formats	M2-711

operation	result	evidence
erase and program	engine-encoded, software-encoded and raw images in the main and boot formats read back exactly through raw DMA/FIFO, the engine decoder, the software decoder and the sequencer readers; the CPU-kicked encode equals host BCH8 and BCH16	M2-712
injected errors	with 1, 2, 4, 7 and 8 data errors, 3 errors in metadata/parity and one 9-error sector per page, engine and software decode return the same data, metadata, correction counts and location sets; the 9-error sector is uncorrectable in both and left as stored	M2-713
error locations	the `0x820` fields list a byte's higher bit first (chapter 10, section 10.8)	M2-714
closing state	every job that touched the controller (89) leaves `C08` = 0 and `FMC_CTRL0` = 0 after its closing reset	M2-716

Job times, each including the reset before and after it (M2-715):

job	time
raw page read, DMA / FIFO	586 us / 795 us
engine-decoded page, main / boot format	648 us / 270 us
erase	2274 us
program	899..1831 us
software BCH8 decode of a main page	1956 us clean, 18.4 ms with 33 corrections
software BCH16 decode of a boot page	1370 us

Beyond the drivers' own sequences, CPU-issued command sequences read back exactly for the random column read and write, partial-page programming, overwriting a programmed page without an erase, out-of-order page programming within a block, copyback inside a block, and the two-plane program and erase (M2-731..M2-740). In the FIFO input path the CPU must clear `FMC_CTRL0` bit 24 as the drivers do; a sequence that left it set lost earlier DMA buffer content (M2-728, M2-730). Reads and programs remain exact down to a data cycle of 28.24 ns (timing fields 0/2) on the measured unit, a qualification specific to that unit and its operating conditions (M2-741).

Chapter 13 — M2 experiment coverage

Generated by `emCORE/tools/nano4g/coverage.py`.

All 142 original names, READ ID and 64 follow-up families have runnable entries (207 total). Status counts: 9 applicability, 64 followup, 25 partial, 109 ported.

Every family runs resident by default, including `m2_wedge` and `m2_regbus`. The host uploads manifests, polls progress and collects result pages; a full result arena resumes the same uploaded manifest. `--isolate-diagnostics` explicitly collects those two diagnostic families per case; `--single-probes` opts every family out of batching. The sequence-only `m2_cpu_direct` campaign groups up to 128 jobs per manifest (1,850 jobs in 15 worker RUNs on p48, M2-727). Its batch API also supports map jobs, but the campaign contains no map phase.

`ported` describes the current M2 implementation, not device qualification. `partial` retains explicit missing cases; `applicability` does not reproduce the original operation until the M2 mapping is established. A model run can stop at its first unsupported case. Physical counts below describe retained summary records, not every native submission or successful semantic check. See device findings for capture-specific execution qualification and state contamination.

Model column is the historical report `emCORE/build/m2-registry-after-capture.json`; it is not a fresh model run for the selected physical capture. Physical audit: `validation/m2-p16-audit.json`. Source: `experiments/m2-all-p16.json`.

Original questions: M1 chapter 13. M2 sources: `emCORE/tools/nano4g/` and shared `emCORE/tools/fmss/experiments/`.

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
`state`	ported	M2-specific replacement	emulated	1 sampled; 0 not executed; 0 case errors
`restore`	partial	native M2 reset; power-on defaults from the cold-device capture (`m2_cold_boot`)	emulated	2 sampled; 0 not executed; 0 case errors
`regmap`	ported	shared ISA; M2 reset/start/snapshots	emulated	1 sampled; 0 not executed; 0 case errors
`clock`	ported	shared ISA; M2 reset/start/snapshots	emulated	2 sampled; 0 not executed; 0 case errors
`pram`	ported	shared ISA; M2 reset/start/snapshots	emulated	1 sampled; 0 not executed; 0 case errors
`alu`	ported	shared ISA; M2 reset/start/snapshots	emulated	9 sampled; 0 not executed; 0 case errors
`memory`	ported	shared ISA; M2 reset/start/snapshots	emulated	1 sampled; 0 not executed; 0 case errors
`indirect`	ported	raw opcode discovery; M1 meanings are hypotheses	emulated	4 sampled; 0 not executed; 0 case errors
`resume`	ported	shared ISA; M2 reset/start/snapshots	emulated	7 sampled; 0 not executed; 0 case errors
`control`	ported	shared ISA; M2 reset/start/snapshots	emulated	1 sampled; 0 not executed; 0 case errors
`wait`	ported	shared ISA; M2 reset/start/snapshots	emulated	16 sampled; 0 not executed; 0 case errors
`c58`	ported	shared ISA; M2 reset/start/snapshots	emulated	37 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
op15	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x15 at 0x28	24 sampled; 0 not executed; 0 case errors
op12	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x12 at 0x30	8 sampled; 0 not executed; 0 case errors
c58ill	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x9 at 0x28	5 sampled; 0 not executed; 0 case errors
startseq	ported	raw opcode discovery; M1 meanings are hypotheses	emulated	14 sampled; 0 not executed; 0 case errors
startseq2	ported	raw opcode discovery; M1 meanings are hypotheses	emulated	18 sampled; 0 not executed; 0 case errors
startseq3	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: M2 opcode 0x8 at 0x0	16 sampled; 0 not executed; 0 case errors
oneshot	ported	shared ISA; M2 reset/start/snapshots	emulated	4 sampled; 0 not executed; 0 case errors
c50	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x9 at 0x30	4 sampled; 0 not executed; 0 case errors
holdout	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: M2 opcode 0x15 at 0x20	7 sampled; 0 not executed; 0 case errors
op12wait	ported	M2-specific replacement	unmodelled: M2 opcode 0x12 at 0x58	160 sampled; 0 not executed; 0 case errors
ev11	ported	M2-specific replacement	emulated	8 sampled; 0 not executed; 0 case errors
ev5	ported	M2-specific replacement	unmodelled: M2 WAIT event 5	4 sampled; 0 not executed; 0 case errors
c50x	partial	timeout/opcode/unaligned scratch; arbitrary faulting bus addresses omitted	unmodelled: M2 opcode 0x1a at 0x20	3 sampled; 0 not executed; 0 case errors
pending	ported	raw opcode discovery; M1 meanings are hypotheses	emulated	25 sampled; 0 not executed; 0 case errors
c14	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x15 at 0x28	7 sampled; 0 not executed; 0 case errors
op12ack	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x12 at 0x28	8 sampled; 0 not executed; 0 case errors
op10	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x10 at 0x28	1 sampled; 0 not executed; 0 case errors
unknown	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: M2 opcode 0x8 at 0x60	36 sampled; 0 not executed; 0 case errors
opsweep	ported	M2-specific replacement	unmodelled: M2 opcode 0x8 at 0x60	717 sampled; 0 not executed; 0 case errors
optime	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: M2 opcode 0x8 at 0x28	16 sampled; 0 not executed; 0 case errors
ctrlbits	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 start/control word 0x1	37 sampled; 0 not executed; 0 case errors
fmcmap	ported	raw opcode discovery; M1 meanings are hypotheses	emulated	1 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
evsweep	ported	M2-specific replacement	unmodelled: M2 WAIT event 5	128 sampled; 0 not executed; 0 case errors
rwmask	ported	M2-specific replacement	emulated	699 sampled; 0 not executed; 0 case errors
op0f	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0xf at 0x60	6 sampled; 0 not executed; 0 case errors
ctrlhold	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 start/control word 0x3	13 sampled; 0 not executed; 0 case errors
evclear	ported	M2-specific replacement	emulated	12 sampled; 0 not executed; 0 case errors
timeregs	ported	shared ISA; M2 reset/start/snapshots	emulated	15 sampled; 0 not executed; 0 case errors
ev5rule	ported	M2-specific replacement	unmodelled: M2 WAIT event 5	16 sampled; 0 not executed; 0 case errors
c08	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x9 at 0x20	7 sampled; 0 not executed; 0 case errors
chan1	applicability	candidate +400 set only; second-channel GPIO/event mapping unresolved	emulated	8 sampled; 0 not executed; 0 case errors
readev	ported	M2-specific replacement	unmodelled: M2 WAIT event 5	128 sampled; 0 not executed; 0 case errors
regnum	ported	M2-specific replacement	unmodelled: extended register operand	19 sampled; 0 not executed; 0 case errors
branch	ported	shared ISA; M2 reset/start/snapshots	unmodelled: unaligned instruction fetch 0x54	7 sampled; 0 not executed; 0 case errors
waitf	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 WAIT event 27	10 sampled; 0 not executed; 0 case errors
opcost	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: M2 opcode 0x8 at 0x40	30 sampled; 0 not executed; 0 case errors
op12rel	ported	M2-specific replacement	unmodelled: M2 opcode 0x12 at 0x20	55 sampled; 0 not executed; 0 case errors
pause	ported	shared ISA; M2 reset/start/snapshots	emulated	7 sampled; 0 not executed; 0 case errors
fmc28	ported	raw opcode discovery; M1 meanings are hypotheses	emulated	11 sampled; 0 not executed; 0 case errors
blk100	ported	M2-specific replacement	emulated	128 sampled; 0 not executed; 0 case errors
irqline	ported	M2-specific replacement	unmodelled: M2 opcode 0x12 at 0x20	8 sampled; 0 not executed; 0 case errors
irqen	ported	M2-specific replacement	emulated	264 sampled; 0 not executed; 0 case errors
ev5t	partial	repeated M2 latch comparison; delay/command-ack timing matrix remains	unmodelled: M2 WAIT event 5	6 sampled; 0 not executed; 0 case errors
r8	partial	M1 register-role comparison with local PC targets; direct LDI r8 low-address bus access remains unqualified	unmodelled: extended register operand	11 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
ev8	applicability	candidate +400 set only; second-channel GPIO/event mapping unresolved	emulated	8 sampled; 0 not executed; 0 case errors
evcons	partial	single waits in pending states; ordered repeated-wait combinations remain	unmodelled: M2 WAIT event 5	80 sampled; 0 not executed; 0 case errors
dmaloop	ported	M2-specific replacement	emulated	8 sampled; 0 not executed; 0 case errors
op0916	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x9 at 0x38	55 sampled; 0 not executed; 0 case errors
op16	ported	M2-specific replacement	unmodelled: M2 opcode 0x16 at 0x58	14 sampled; 0 not executed; 0 case errors
opstate	ported	M2-specific replacement	unmodelled: M2 opcode 0x8 at 0x58	49 sampled; 0 not executed; 0 case errors
op16b	partial	C58/pending states; delayed flag and vector target variants remain	unmodelled: M2 opcode 0x16 at 0x38	20 sampled; 0 not executed; 0 case errors
op09	ported	M2-specific replacement	unmodelled: M2 opcode 0x9 at 0x58	320 sampled; 0 not executed; 0 case errors
op09b	ported	M2-specific replacement	unmodelled: M2 opcode 0x9 at 0x58	288 sampled; 0 not executed; 0 case errors
op09c	partial	condition ranges ported; CE deselect/CE1 comparison remains	unmodelled: M2 opcode 0x9 at 0x58	128 sampled; 0 not executed; 0 case errors
chan1pins	applicability	candidate +400 set only; second-channel GPIO/event mapping unresolved	emulated	8 sampled; 0 not executed; 0 case errors
pcwrite	ported	shared ISA; M2 reset/start/snapshots	emulated	3 sampled; 0 not executed; 0 case errors
op12cont	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 opcode 0x12 at 0x30	7 sampled; 0 not executed; 0 case errors
ctrlfx	partial	timeout/opcode/event-jump cases; enabled command IRQ case remains	unmodelled: M2 start/control word 0x1	48 sampled; 0 not executed; 0 case errors
op09d	applicability	candidate +400 set only; second-channel GPIO/event mapping unresolved	emulated	8 sampled; 0 not executed; 0 case errors
fwidth	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: register offset field width 0x1d70	18 sampled; 0 not executed; 0 case errors
irqsrc	ported	M2-specific replacement	emulated	136 sampled; 0 not executed; 0 case errors
eccpar	ported	M2-specific replacement	emulated	12 sampled; 0 not executed; 0 case errors
bufsize	ported	M2-specific replacement	unmodelled: DMA buffer length other than 512	10 sampled; 0 not executed; 0 case errors
eccstr	ported	M2-specific replacement	unmodelled: requires physical BCH parity/correction; model has synthetic layout-only parity	68 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
`fields0`	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: indirect operand alignment/immediate	21 sampled; 0 not executed; 0 case errors
`tmofield`	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 start/control word 0x5	12 sampled; 0 not executed; 0 case errors
`eccstr2`	ported	M2-specific replacement	unmodelled: requires physical BCH parity/correction; model has synthetic layout-only parity	134 sampled; 0 not executed; 0 case errors
`ev5en`	ported	M2-specific replacement	unmodelled: M2 WAIT event 5	24 sampled; 0 not executed; 0 case errors
`tmobits`	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 start/control word 0xff15	15 sampled; 0 not executed; 0 case errors
`eccmode`	ported	M2-specific replacement	unmodelled: ECC_CTRL candidate 0x302	36 sampled; 0 not executed; 0 case errors
`eccchunk`	partial	captured on-flash M2 chunks; parity-buffer DMA and host BCH construction remain	emulated	4 sampled; 0 not executed; 0 case errors
`rawpage`	ported	M2-specific replacement	emulated	2 sampled; 0 not executed; 0 case errors
`rawwrite`	ported	M2-specific replacement	emulated	8 sampled; 0 not executed; 0 case errors
`writeseq`	partial	serialized M2 pages; overlapped pipeline and two-plane cases remain	emulated	11 sampled; 0 not executed; 0 case errors
`op09nand`	ported	M2-specific replacement	unmodelled: M2 opcode 0x9 at 0xb8	1024 sampled; 0 not executed; 0 case errors
`eraseseq`	partial	single-plane/CE/address-cycle cases; two-plane erase remains	emulated	20 sampled; 0 not executed; 0 case errors
`eccchunk2`	partial	captured-codeword corruption; mixed-error and direct parity-buffer cases remain	unmodelled: requires physical BCH parity/correction; model has synthetic layout-only parity	48 sampled; 0 not executed; 0 case errors
`eccmode2`	ported	M2-specific replacement	unmodelled: ECC_CTRL candidate 0xa0a	132 sampled; 0 not executed; 0 case errors
`tmobits2`	ported	shared ISA; M2 reset/start/snapshots	unmodelled: M2 start/control word 0xff55	5 sampled; 0 not executed; 0 case errors
`ev5ready`	ported	M2-specific replacement	unmodelled: M2 WAIT event 5	18 sampled; 0 not executed; 0 case errors
`pageorder`	ported	M2-specific replacement	emulated	33 sampled; 0 not executed; 0 case errors
`op09w33`	ported	M2-specific replacement	unmodelled: M2 opcode 0x9 at 0x58	48 sampled; 0 not executed; 0 case errors
`ev5arm`	partial	M2 rearm candidates; short-read and data-select variants remain	unmodelled: M2 WAIT event 5	16 sampled; 0 not executed; 0 case errors
`writeseq2`	partial	sequential/repeat programming; overlap/last-DMA comparison remains	emulated	12 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
eccmode3	ported	M2-specific replacement	unmodelled: ECC_CTRL candidate 0x302	40 sampled; 0 not executed; 0 case errors
ev5two	ported	M2-specific replacement	unmodelled: M2 WAIT event 5	6 sampled; 0 not executed; 0 case errors
b15ecc	applicability	M2 alternate-code selector unknown; candidate ECC_CTRL sweep only	unmodelled: ECC_CTRL candidate 0xa0a	14 sampled; 0 not executed; 0 case errors
eccmode4	ported	M2-specific replacement	emulated	24 sampled; 0 not executed; 0 case errors
finale	ported	M2-specific replacement	unmodelled: ECC_CTRL candidate 0xa0a	387 sampled; 0 not executed; 0 case errors
startdiff	ported	M2-specific replacement	unmodelled: M2 opcode 0x1a at 0x20	24 sampled; 0 not executed; 0 case errors
startbits	ported	M2-specific replacement	unmodelled: M2 opcode 0x1a at 0x20	420 sampled; 0 not executed; 0 case errors
rsdecode	applicability	M2 alternate-code selector unknown; candidate ECC_CTRL sweep only	unmodelled: ECC_CTRL candidate 0xa0a	14 sampled; 0 not executed; 0 case errors
vic20	ported	M2-specific replacement	emulated	144 sampled; 0 not executed; 0 case errors
fmc28b	ported	M2-specific replacement	emulated	10 sampled; 0 not executed; 0 case errors
seqfirst	ported	M2-specific replacement	emulated	1 sampled; 0 not executed; 0 case errors
w2state	partial	basic M2 states ported; c old-boot/enable/control-word sequence remains	unmodelled: M2 opcode 0x9 at 0x58	1792 sampled; 0 not executed; 0 case errors
c50b8	ported	M2-specific replacement	emulated	14 sampled; 0 not executed; 0 case errors
c58b1	ported	raw opcode discovery; M1 meanings are hypotheses	emulated	28 sampled; 0 not executed; 0 case errors
evsweep2	partial	forward state sweeps; reverse ordering remains	unmodelled: M2 WAIT event 16	3072 sampled; 0 not executed; 0 case errors
c50fault	ported	M2-specific replacement	unmodelled: M2 opcode 0x1a at 0x28	8 sampled; 0 not executed; 0 case errors
ctrlwords	partial	word/DMA/ECC matrix; nested fault/vector-offset cases remain	emulated	24 sampled; 0 not executed; 0 case errors
c08self	partial	M2 operation state captures; pause/continue/vector cases remain	emulated	5 sampled; 0 not executed; 0 case errors
latchrep	ported	M2-specific replacement	unmodelled: M2 WAIT event 5	2560 sampled; 0 not executed; 0 case errors
irq044	ported	M2-specific replacement	emulated	792 sampled; 0 not executed; 0 case errors
fmcbits	partial	known-page control/DMA variants; READID variants remain	emulated	9 sampled; 0 not executed; 0 case errors
fmcstat	ported	M2-specific replacement	emulated	32 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
`eccregs`	ported	M2-specific replacement	emulated	15 sampled; 0 not executed; 0 case errors
`seqrestart`	partial	M2 resets/acks; within-program three-encode comparison remains	emulated	12 sampled; 0 not executed; 0 case errors
`done5`	ported	M2-specific replacement	unmodelled: ECC_CTRL candidate 0xa0a	58 sampled; 0 not executed; 0 case errors
`progtime`	ported	M2-specific replacement	emulated	387 sampled; 0 not executed; 0 case errors
`startregion`	partial	region reads and scratch writes; reserved-block writes not selected	unmodelled: M2 start/control word 0xffd5	56 sampled; 0 not executed; 0 case errors
`bufport`	partial	candidate port reads; CPU/sequencer FIFO write experiment remains	emulated	24 sampled; 0 not executed; 0 case errors
`c58b2`	ported	raw opcode discovery; M1 meanings are hypotheses	emulated	29 sampled; 0 not executed; 0 case errors
`stprev`	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: extended register operand	39 sampled; 0 not executed; 0 case errors
`tmwait2`	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: M2 opcode 0x15 at 0x28	7 sampled; 0 not executed; 0 case errors
`done5b`	ported	M2-specific replacement	unmodelled: ECC_CTRL candidate 0x200a02	24 sampled; 0 not executed; 0 case errors
`rsresult`	applicability	M2 alternate-code selector unknown; candidate ECC_CTRL sweep only	unmodelled: ECC_CTRL candidate 0xa0a	14 sampled; 0 not executed; 0 case errors
`rschip`	applicability	M2 alternate-code selector unknown; candidate ECC_CTRL sweep only	unmodelled: ECC_CTRL candidate 0xa0a	14 sampled; 0 not executed; 0 case errors
`miscorr`	ported	M2-specific replacement	unmodelled: requires physical BCH parity/correction; model has synthetic layout-only parity	2080 sampled; 0 not executed; 0 case errors
`fullpage`	ported	M2-specific replacement	emulated	42 sampled; 0 not executed; 0 case errors
`dmactl`	partial	serialized read variants; original overlapped read remains	emulated	42 sampled; 0 not executed; 0 case errors
`fmcstat2`	ported	M2-specific replacement	emulated	8 sampled; 0 not executed; 0 case errors
`ch1buf`	applicability	candidate +400 set only; second-channel GPIO/event mapping unresolved	emulated	8 sampled; 0 not executed; 0 case errors
`evsweep3`	partial	M2 metadata/ECC/program/DMA; candidate second-channel/parity DMA remains	unmodelled: M2 WAIT event 16	2560 sampled; 0 not executed; 0 case errors
`blk100b`	ported	M2-specific replacement	emulated	99 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
straps	ported	M2-specific replacement	emulated	1 sampled; 0 not executed; 0 case errors
irqop	ported	M2-specific replacement	unmodelled: M2 WAIT event 15	48 sampled; 0 not executed; 0 case errors
w2vic	partial	raw VIC comparison; enable-source combinations remain	unmodelled: M2 opcode 0x9 at 0x58	1024 sampled; 0 not executed; 0 case errors
regsweep	ported	raw opcode discovery; M1 meanings are hypotheses	unmodelled: extended register operand	744 sampled; 0 not executed; 0 case errors
access	ported	M2-specific replacement	emulated	3680 sampled; 0 not executed; 0 case errors
readid	ported	M2 FIFO sequence	emulated	4 sampled; 0 not executed; 0 case errors
m2_resetcheck	followup	Start=1 WAIT/illegal/evjmp cleanup, independent END after each reset	not run	7 sampled; 0 not executed; 0 case errors
m2_status_recovery	followup	Status-latch failure, read-order controls and following execution	not run	7 sampled; 0 not executed; 0 case errors
m2_recovery	followup	Pause cleanup, subsequent execution and explicit CPU resume controls	not run	9 sampled; 0 not executed; 0 case errors
m2_shifts	followup	Full-width shift counts and immediate-zero encoding	not run	168 sampled; 0 not executed; 0 case errors
m2_pram_alias	followup	Parameter boundary, indirect high-bit alias and alignment	not run	100 sampled; 0 not executed; 0 case errors
m2_c58_bits	followup	CPU versus sequencer single-bit write/read masks	not run	68 sampled; 0 not executed; 0 case errors
m2_waitcons	followup	Repeated/ordered waits, acknowledge and rearm within one program	not run	80 sampled; 0 not executed; 0 case errors
m2_status_order	followup	Command acknowledgement and delay versus status-latch ordering	not run	33 sampled; 0 not executed; 0 case errors
m2_ce_conditions	followup	No CE and each CE selection with WAIT/opcode09 operand sweeps	not run	432 sampled; 0 not executed; 0 case errors
m2_delayed_vector	followup	Delayed CPU stimuli and alternate vector destinations	not run	36 sampled; 0 not executed; 0 case errors
m2_nested_fault	followup	Fault offsets and a second fault inside the handler	not run	12 sampled; 0 not executed; 0 case errors
m2_pause_state	followup	State across raw signal/pause/event opcodes and CPU resume candidates	not run	12 sampled; 0 not executed; 0 case errors
Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
m2_ecc_sequence	followup	Three encodes per program across	not run	20 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
		reset/ack/slot changes, BCH8 and BCH16		
m2_fifo_access	followup	CPU and sequencer writes to the candidate FIFO, then DMA readback	not run	12 sampled; 0 not executed; 0 case errors
m2_timeline	followup	Sampled register timelines (≈ 188 ns) during DMA, ECC, NAND reads, status latch, command, address	not run	not audited
m2_wait_cpu	followup	WAIT 6..10, 12..15 with one CPU-written control word 1 ms into the wait	not run	not audited
m2_status_wait	followup	WAIT5 and masked WAIT12 against a busy chip; WAIT5 latch orderings	not run	not audited
m2_ecc_decode_regs	followup	0x820..0x85c after decodes with 1..8 errors	not run	not audited
m2_reset_retention	followup	Storage words written in one case, read in the next before-map	not run	not audited
m2_jr_cond	followup	JR as conditional jump: R field 0..7 with the condition register zero or one	not run	not audited
m2_jr_width2	followup	JR target width and alignment with r0 = 1	not run	not audited
m2_wait_sources	followup	WAIT 5..10, 12..15 after encode, decode (erased/uncorrectable), meta latch, unacknowledged DMA, status latch	not run	not audited
m2_fmc_flags	followup	FMC_STATUS/C50/810/04c/C08 right after each controller operation, before any ack	not run	not audited
m2_dma_cut	followup	Second DMA started 0..256 nops after the first: bytes delivered (DMA rate) and 038	not run	not audited
m2_timeout_low	followup	Start-word bits 7:4 with count 0x07: WAIT3 fault time	not run	not audited
m2_ecc_regs	followup	Sequencer view of the ECC block 0x800..0x9fc after known encodes and decodes	not run	not audited
m2_jr_hazard	followup	JR target-register hazard: 0..6 nops between the target write and JR	not run	not audited
m2_jr_width	followup	JR target width and alignment with the hazard avoided	not run	not audited
Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
m2_alu_hazard	followup	Register-write hazards: movi/add/ld/ldi	not run	not audited

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
		producers, add/mov/ldi/sti/bnz consumers at distance 0, 1, 2, 4		
m2_jump_width	followup	JR and BNZ target width (bits 0..20) and alignment via landing pads	not run	not audited
m2_wait_rb	followup	WAIT 0..15 issued right after a block erase starts; status byte shows busy/ready at release	not run	not audited
m2_wait4_decode	followup	Decode-side WAIT4: which 810 write re-arms it after an erased-sector decode	not run	not audited
m2_bch16_threshold	followup	810 bit-5 threshold for BCH16 (4..8, 12 errors)	not run	not audited
m2_ecc_modes2	followup	ECC_CTRL bits 24..31, encode+decode words, metadata length 13/16/32	not run	not audited
m2_opcost	followup	Per-instruction time from 1000 vs 21000-iteration loops, SDRAM and SRAM program	not run	not audited
m2_fmc_misc	followup	FMC 018 before data reads and 05c before the final DMA on a page read	not run	not audited
m2_ecc_threshold	followup	BCH8 810 bit-5 threshold (1..8 errors), erased-page status, 818 under 0x81c	not run	not audited
m2_wait4_ack	followup	Which 0x810 write or 0x81c re-arms WAIT4 after an encode	not run	not audited
m2_ecc_modes	followup	ECC_CTRL encode words: metadata length and every other single bit, parity by raw read	not run	not audited
m2_ecc_ring	followup	0x80c result ring: length, read-pointer advance, reset; 0x810 bit-5 threshold with 1..8 errors	not run	not audited
m2_bch16_correct	followup	BCH16 decode of host code words with 0..24 errors (seven sectors per page)	not run	not audited
m2_ecc_status_timing	followup	BCH8 decode: when ECC_RESULT/810/818 report the corrected count and failure, four ack variants	not run	not audited
m2_bch16_encode	followup	Engine BCH16 parity for known data (BCH8 control), recovered by raw read	not run	not audited
Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
m2_status_busy	followup	Latched status byte during	not run	not audited

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
		erase/program/read busy, idle and no-CE		
m2_timeout_window	followup	Multi-bit start-word timeout counts versus the single-bit linear fit	not run	not audited
m2_bank_regs	followup	Primary versus +400 storage words: independent, mirrored or inert, CPU and sequencer writers	not run	not audited
m2_bch8_correct	followup	Host BCH8 code words with 0..16 known bit errors: correction radius and reported counts	not run	not audited
m2_dma_ack	followup	p16 payload loss: exact p16 serial program versus acknowledged WAIT2, with and without ECC	not run	not audited
m2_dma_ecc_overlap	followup	Serial controls versus concurrent DMA/ECC on same/different slots; every explicit WAIT now acknowledged (p16 ran the unacknowledged generator)	not run	24 sampled (p16 generator); 0 not executed; 0 case errors
m2_dma_visibility	followup	P16 partial payloads: plain DMA/BCH8/BCH16, CPU-timed holds and pre-cleanup checksum	not run	not audited
m2_opcode_confirm	followup	Expected-result checks of all 19 common ISA opcodes	not run	531 sampled; 0 not executed; 0 case errors
m2_opcode_m1	followup	Retest all eight M1 device-discovered opcode meanings as explicit hypotheses	not run	78 sampled; 0 not executed; 0 case errors
m2_opcode_scan	followup	All other opcode bytes, three operand forms, continue/vector policies	not run	1422 sampled; 0 not executed; 0 case errors
m2_opcode_states	followup	Revisit seven ambiguous control opcodes with five pending stimuli	not run	105 sampled; 0 not executed; 0 case errors
m2_wedge	followup	Register/PC/load transitions; per-case collection only with --isolate-diagnostics	not run	59 sampled; 0 not executed; 0 case errors
m2_regdest	followup	M1 destination roles: all 256 selectors, MOVI/MOV/ADD, local PC targets	not run	258 sampled; 0 not executed; 0 case errors
m2_regsrc	followup	M1 argument sources: all 256 selectors, MOV and every ALU/immediate-zero form	not run	256 sampled; 0 not executed; 0 case errors
m2_regbranch	followup	M1 branch-register roles: all 256 selectors, BZ/BNZ, zero/nonzero controls	not run	256 sampled; 0 not executed; 0 case errors

Experiment	Migration	M2 scope / gap	Historical model attempt	Physical retained records
m2_regaddr	followup	M1 LDI/PLD address roles with mapped fallback pointers; PC bus role separate	not run	255 sampled; 0 not executed; 0 case errors
m2_regst	followup	M1 last-register-bus-value rule: all extended selectors plus predecessor contrasts	not run	287 sampled; 0 not executed; 0 case errors
m2_regjr	followup	M1 JR source roles: all 256 selectors, mapped labels and PC self-loop	not run	256 sampled; 0 not executed; 0 case errors
m2_reg10	followup	P11 source 10 difference: all eight GPR destinations, distinct small shift counts, zero/large controls	not run	64 sampled; 0 not executed; 0 case errors
m2_resetpram	followup	Isolate which CPU reset writes clear PRAM/GPRs; paired bounded-loop controls	not run	8 sampled; 0 not executed; 0 case errors
m2_regbus	followup	Isolated PC-as-address applicability; run after collected register comparisons	not run	3 sampled; 0 not executed; 0 case errors

13.1 Additional matrices

Run `--followups`, or include them via the device runner's `--all`. Each case records errors independently. Model outcomes are in `emCORE/build/m2-followup-model.json`.

Family	Extends M1 experiments	Question
m2_resetcheck	ctrlfx	Start=1 WAIT/illegal/evjmp cleanup, independent END after each reset
m2_status_recovery	ev5	Status-latch failure, read-order controls and following execution
m2_recovery	op12,startseq	Pause cleanup, subsequent execution and explicit CPU resume controls
m2_shifts	alu	Full-width shift counts and immediate-zero encoding
m2_pram_alias	pram,indirect	Parameter boundary, indirect high-bit alias and alignment
m2_c58_bits	c58	CPU versus sequencer single-bit write/read masks
m2_waitcons	evcons	Repeated/ordered waits, acknowledge and rearm within one program
m2_status_order	ev5t	Command acknowledgement and delay versus status-latch ordering
m2_ce_conditions	op09c	No CE and each CE selection with WAIT/opcode09 operand sweeps
m2_delayed_vector	op16b	Delayed CPU stimuli and alternate vector destinations
m2_nested_fault	ctrlwords,c50x	Fault offsets and a second fault inside the handler

Family	Extends M1 experiments	Question
m2_pause_state	c08self	State across raw signal/pause/event opcodes and CPU resume candidates
m2_ecc_sequence	seqrestart	Three encodes per program across reset/ack/slot changes, BCH8 and BCH16
m2_fifo_access	bufport	CPU and sequencer writes to the candidate FIFO, then DMA readback
m2_timeline	dmactl,eccregs,fmcstat	Sampled register timelines (≈ 188 ns) during DMA, ECC, NAND reads, status latch, command, address
m2_wait_cpu	evsweep3,c58	WAIT 6..10, 12..15 with one CPU-written control word 1 ms into the wait
m2_status_wait	ev5,ev5ready,latchrep	WAIT5 and masked WAIT12 against a busy chip; WAIT5 latch orderings
m2_ecc_decode_regs	eccregs	0x820..0x85c after decodes with 1..8 errors
m2_reset_retention	restore,ctrlfx	Storage words written in one case, read in the next before-map
m2_jr_cond	regjr,branch	JR as conditional jump: R field 0..7 with the condition register zero or one
m2_jr_width2	regjr,fwidth	JR target width and alignment with r0 = 1
m2_wait_sources	evsweep,evsweep2,evsweep3	WAIT 5..10, 12..15 after encode, decode (erased/uncorrectable), meta latch, unacknowledged DMA, status latch
m2_fmc_flags	fmcstat,fmcstat2,c50	FMC_STATUS/C50/810/04c/C08 right after each controller operation, before any ack
m2_dma_cut	dmactl,dmaloop	Second DMA started 0..256 nops after the first: bytes delivered (DMA rate) and 038
m2_timeout_low	startbits,tmobits2	Start-word bits 7:4 with count 0x07: WAIT3 fault time
m2_ecc_regs	eccregs,eccpar	Sequencer view of the ECC block 0x800..0x9fc after known encodes and decodes
m2_jr_hazard	regjr,r8	JR target-register hazard: 0..6 nops between the target write and JR
m2_jr_width	regjr,fwidth	JR target width and alignment with the hazard avoided
m2_alu_hazard	alu,regsrc	Register-write hazards: movi/add/ld/ldi producers, add/mov/ldi/sti/bnz consumers at distance 0, 1, 2, 4
m2_jump_width	branch,regjr,fwidth	JR and BNZ target width (bits 0..20) and alignment via landing pads
m2_wait_rb	evsweep,ev5ready	WAIT 0..15 issued right after a block erase starts; status byte shows busy/ready at release
m2_wait4_decode	evcons,done5b	Decode-side WAIT4: which 810 write re-arms it after an erased-sector decode
m2_bch16_threshold	eccstr2	810 bit-5 threshold for BCH16 (4..8, 12 errors)
m2_ecc_modes2	eccmode4,b15ecc	ECC_CTRL bits 24..31, encode+decode words, metadata length 13/16/32
m2_opcost	opcost,optime,clock	Per-instruction time from 1000 vs 21000-iteration loops, SDRAM and SRAM program

Family	Extends M1 experiments	Question
m2_fmc_misc	fmcbits	FMC 018 before data reads and 05c before the final DMA on a page read
m2_ecc_threshold	eccregs,done5	BCH8 810 bit-5 threshold (1..8 errors), erased-page status, 818 under 0x81c
m2_wait4_ack	evcons,done5b	Which 0x810 write or 0x81c re-arms WAIT4 after an encode
m2_ecc_modes	eccmode,eccmode2,eccmode3,b15ecc	ECC_CTRL encode words: metadata length and every other single bit, parity by raw read
m2_ecc_ring	eccregs,rsresult	0x80c result ring: length, read-pointer advance, reset; 0x810 bit-5 threshold with 1..8 errors
m2_bch16_correct	eccstr,eccstr2	BCH16 decode of host code words with 0..24 errors (seven sectors per page)
m2_ecc_status_timing	eccregs,done5,rsresult	BCH8 decode: when ECC_RESULT/810/818 report the corrected count and failure, four ack variants
m2_bch16_encode	eccpar,eccstr2	Engine BCH16 parity for known data (BCH8 control), recovered by raw read
m2_status_busy	ev5,latchrep,ev5ready	Latched status byte during erase/program/read busy, idle and no-CE
m2_timeout_window	startbits,tmobits	Multi-bit start-word timeout counts versus the single-bit linear fit
m2_bank_regs	chan1,chan1pins	Primary versus +400 storage words: independent, mirrored or inert, CPU and sequencer writers
m2_bch8_correct	eccstr,eccstr2,miscorr,rawwrite	Host BCH8 code words with 0..16 known bit errors: correction radius and reported counts
m2_dma_ack	dmaloop,dmactl	p16 payload loss: exact p16 serial program versus acknowledged WAIT2, with and without ECC
m2_dma_ecc_overlap	dmactl,writeseq2	Serial controls versus concurrent DMA/ECC on same/different slots
m2_dma_visibility	dmaloop,dmactl	P16 partial payloads: plain DMA/BCH8/BCH16, CPU-timed holds and pre-cleanup checksum
m2_opcode_confirm	alu,branch,regmap,memory,indirect,wait	Expected-result checks of all 19 common ISA opcodes
m2_opcode_m1	op0f,op10,op12,op15,op16,w2vic	Retest all eight M1 device-discovered opcode meanings as explicit hypotheses
m2_opcode_scan	unknown,opsweep,fields0	All other opcode bytes, three operand forms, continue/vector policies
m2_opcode_states	opstate,op12wait,op16	Revisit seven ambiguous control opcodes with five pending stimuli
m2_wedge	regnum	Register/PC/load transitions; per-case collection only with --isolate-diagnostics
m2_regdest	regnum,r8,regsweep	M1 destination roles: all 256 selectors, MOVI/MOV/ADD, local PC targets
m2_regsrc	regsweep	M1 argument sources: all 256 selectors, MOV and every ALU/immediate-zero form
m2_regbranch	regsweep	M1 branch-register roles: all 256 selectors, BZ/BNZ, zero/nonzero controls

Family	Extends M1 experiments	Question
`m2_regaddr`	regsweep	M1 LDI/PLD address roles with mapped fallback pointers; PC bus role separate
`m2_regst`	stprev,regsweep	M1 last-register-bus-value rule: all extended selectors plus predecessor contrasts
`m2_regjr`	r8,regsweep	M1 JR source roles: all 256 selectors, mapped labels and PC self-loop
`m2_reg10`	regsweep	P11 source 10 difference: all eight GPR destinations, distinct small shift counts, zero/large controls
`m2_resetpram`	restore,ctrlfx	Isolate which CPU reset writes clear PRAM/GPRs; paired bounded-loop controls
`m2_regbus`	r8,regsweep	Isolated PC-as-address applicability; run after collected register comparisons
`m2_r5_latch_clear`	stprev,regsweep	`ld`, `st` or `wr` between a latch predecessor and a high-bit source selector: kept or cleared
`m2_r5_ext_dest`	regnum,regsweep	`mov` to r9, r11..r15 with source selector 8 or 10: GPR dump and extended-`st` latch
`m2_r5_sti_high`	regsweep	`sti` with address selector 0x105, every GPR the address of its own marker, with and without a preceding `mov`
`m2_r5_rom_ldi`	memory,r8	`ldi` of 0x20000000 + k × 0x1000 (k = 0..15) against the boot-ROM image
`m2_r5_op1a_operands`	unknown,fields0	Undefined opcode 0x1a with 64 random R/arg/imm words against the zero form
`m2_r5_nop_stall_c08`	opsweep,c08self	`nop` R = 8 / 0xff with the timeout disabled: C08 inside the hold against an unmet `wait 3`
`m2_r5_c48_count7`	op15	Count-7 `tmwait` with C48 = 0..8: end instant, C14 and fault
`m2_r5_tmwait_repeat`	op15	Count-255 `tmwait` with and without a C14 = 0x800 write, five runs each
`m2_r5_prescaler_step`	startbits,restore	Count-0 `wait 3` first fault instant after one CPU reset step versus full reset and none
`m2_r5_c08_long`	c08self	C08 3 ms into a 400 ms loop of `nop2`, `ld` or `sti`, three runs each
`m2_r5_select_tag`	dmactl	FMC_DATA_SEL rewrite 16 / 32 `nop2` into a DMA out with both buffers tagged per word
`m2_r5_overlap_tags`	dmactl,dmaloop	Out-then-out and in-then-in DMA overlaps (gap 0 / 16) with tagged inputs
`m2_r5_dma_ctrl_sel`	dmactl	DMA out after 038 = 0x1f, 0x2f, 0x37; DMA in after 0xf; control 7
`m2_r5_out_select`	bufport,dmactl	DMA out with FMC_DATA_SEL = 0, 1, 0x300, 0x400, 0x1100 (control 0x100), four tagged buffers
`m2_r5_x340_window`	bufport,writeseq2	FMCTRL1 0x340 second transfer: delay alone or an 014 write after 0/8/16 `nop2`, then a page program and raw read
`m2_r5_status_lengths`	fmcstat	FMC_STATUS bits 19:12 after data reads of 160..508 bytes
`m2_r5_bch16_814`	eccregs,eccstr2	BCH16 buffer encode and 12-error decode with 0x814 = 0x2f against 0

Family	Extends M1 experiments	Question
`m2_cpu_stack`	—	CPU-only NAND read/program/erase (no sequencer program), hardware and software ECC, boot and main formats, against the sequencer readers

Chapter 14 — Evidence: outcomes

This chapter records M2 outcomes, following the nano3g evidence chapter. Interpretation belongs in chapter 15, experiment scope in chapter 13, and outstanding questions in chapter 17.

14.1 Physical capture: p06

Source: `experiments/m2-all-p06.json`, 589,189,971 bytes, SHA-256 `4c804e08dd2e326219266d9eed55f71b489776f51a59b7971202c4920240f8c2`. The running image hash was not recorded. The capture contains 159 experiment entries, 140 retained sampled records, 22,972 setup/preflight non-executions and 6,083 explicit errors. These counts describe retained records, not unique native submissions: some legacy reports omit setup probes or overwrite labels.

The machine-readable audit retains per-experiment counts, exact JSON pointers and selected raw observations. The following table includes every returned entry. A host TIMEOUT is an observation-window outcome; it does not identify an illegal opcode. A record without sampled execution supplies no instruction or register measurement.

Experiment	Sampled outcomes	Not executed	Case errors
`state`	END: 1	0	0
`restore`	END: 2	0	0
`regmap`	END: 1	0	0
`clock`	END: 2	0	0
`pram`	END: 1	0	0
`alu`	END: 9	0	0
`memory`	END: 1	0	0
`indirect`	END: 4	0	0
`resume`	END: 6	0	0
`control`	END: 1	0	0
`wait`	FAULT: 15, END: 1	0	0
`c58`	FAULT: 37	0	0
`op15`	TIMEOUT: 24	0	0
`op12`	TIMEOUT: 1	7	0
`c58ill`	none	5	0
`startseq`	TIMEOUT: 1	6	0
`startseq2`	END: 9	0	0
`startseq3`	END: 8	0	0
`oneshot`	END: 2, TIMEOUT: 2	0	0
`c50`	TIMEOUT: 3, END: 1	0	0
`holdout`	TIMEOUT: 3, END: 4	0	0
`op12wait`	TIMEOUT: 1	159	0
`ev11`	none	4	4
`ev5`	none	2	2
`c50x`	none	3	0
`pending`	none	20	0

Experiment	Sampled outcomes	Not executed	Case errors
c14	none	7	0
op12ack	none	8	0
op10	none	1	0
unknown	none	36	0
opsweep	none	717	0
optime	none	10	0
ctrlbits	none	37	0
fmcmap	none	1	0
evsweep	none	112	16
rwmask	none	233	466
op0f	none	6	0
ctrlhold	none	13	0
evclear	none	12	0
timeregs	none	15	0
ev5rule	none	16	0
c08	none	7	0
chan1	none	8	0
readev	none	128	0
regnum	none	19	0
branch	none	7	0
waitf	none	10	0
opcost	none	30	0
op12rel	none	55	0
pause	none	7	0
fmc28	none	11	0
blk100	none	128	0
irqline	none	8	0
irqen	none	264	0
ev5t	none	6	0
r8	none	11	0
ev8	none	8	0
evcons	none	80	0
dmaloop	none	8	0
op0916	none	55	0
op16	none	14	0
opstate	none	49	0
op16b	none	20	0
op09	none	320	0
op09b	none	192	96
op09c	none	128	0

Experiment	Sampled outcomes	Not executed	Case errors
chan1pins	none	8	0
pcwrite	none	3	0
op12cont	none	7	0
ctrlfx	none	48	0
op09d	none	8	0
fwidth	none	18	0
irqsrc	none	136	0
eccpar	none	12	0
bufsize	none	10	0
eccstr	none	68	0
fields0	none	21	0
tmofield	none	12	0
eccstr2	none	134	0
ev5en	none	24	0
tmobits	none	15	0
eccmode	none	36	0
eccchunk	none	2	2
rawpage	none	2	0
rawwrite	none	5	3
writeseq	none	6	5
op09nand	none	512	512
eraseseq	none	4	16
eccchunk2	none	16	32
eccmode2	none	132	0
tmobits2	none	5	0
ev5ready	none	16	2
pageorder	none	14	19
op09w33	none	24	24
ev5arm	none	16	0
writeseq2	none	9	3
eccmode3	none	40	0
ev5two	none	0	6
b15ecc	none	14	0
eccmode4	none	24	0
finale	none	387	0
startdiff	none	24	0
startbits	none	396	0
rsdecode	none	14	0
vic20	none	144	0
fmc28b	none	10	0

Experiment	Sampled outcomes	Not executed	Case errors
seqfirst	none	1	0
w2state	none	1792	0
c50b8	none	14	0
c58b1	none	19	0
evsweep2	none	2048	1024
c50fault	none	8	0
ctrlwords	none	24	0
c08self	none	5	0
latchrep	none	2560	0
irq044	none	792	0
fmcbits	none	7	2
fmcstat	none	32	0
eccregs	none	15	0
seqrestart	none	12	0
done5	none	58	0
progtime	none	0	387
startregion	none	52	4
bufport	none	24	0
c58b2	none	21	0
stprev	none	39	0
tmwait2	none	5	0
done5b	none	24	0
rsresult	none	14	0
rschip	none	14	0
miscorr	none	2080	0
fullpage	none	42	0
dmactl	none	40	2
fmcstat2	none	8	0
ch1buf	none	8	0
evsweep3	none	2048	512
blk100b	none	99	0
straps	none	0	0
irqop	none	48	0
w2vic	none	1024	0
regsweep	none	743	0
access	none	736	2944
readid	none	4	0
m2_shifts	none	168	0
m2_pram_alias	none	100	0
m2_c58_bits	none	68	0

Experiment	Sampled outcomes	Not executed	Case errors
m2_waitcons	none	80	0
m2_status_order	none	33	0
m2_ce_conditions	none	432	0
m2_delayed_vector	none	36	0
m2_nested_fault	none	12	0
m2_pause_state	none	12	0
m2_ecc_sequence	none	20	0
m2_fifo_access	none	12	0
m2_dma_ecc_overlap	none	24	0
m2_opcode_confirm	none	531	0
m2_opcode_m1	none	78	0
m2_opcode_scan	none	1422	0
m2_opcode_states	none	105	0

14.2 Selected observed values

Case	Returned observation
regmap	r0..r7 at c18..c34 match; result equals r6
clock	2,000 / 20,000 two-instruction iterations: 138 / 1,355 us
pram	CPU writes retained d00..d7c; tested d80..dfc writes not retained
alu	logical right shift; wrap at 32 bits; tested counts 32/33 give zero; LD masks by AND
memory	little-endian RAM words; tested unaligned word accesses align down
indirect	op18 reads GPR/PRAM; op19 writes PRAM; 1d64 reads the d64 value
wait	WAIT11 passes in approximately 4 us; other idle 0..15 events fault at approximately 126.6 ms
c58	bit 2 distinguishes vector/continue; tested readback fits mask d6
op15	24 forms time out within their observation windows; C08=4, C14=35
op12 first case	C40/C44=12020d60/ffffffff; PC=30; C08=a; IRQ=8; worker=-2
op12wait first case	C40/C44=12000000/0; PC=58; C08=a; IRQ=8; worker=-2
startseq3, nop (08) first	following marker and END reached for zero-field encoding
holdout, shifts	SHL 1 by 31 gives 80000000; SHR by 31 gives 1
holdout, indirect r6	r2 and r6 both 6666aaaa
holdout, RAM	store then unaligned reload gives 13572468

All numbers in register/encoding columns are hexadecimal. The full qualification and chronology are retained in device findings. None of the 2,136 new confirmation/discovery/state-opcode cases produced a usable instruction measurement in p06.

14.3 Original-code observations

Source and address	Observation	Detailed reference
`bl_NandRead0nly`, mem:0x502c	reset writes C00=8, handles C08=4, resets controller and ECC	CPU map
`bl_NandRead0nly`, mem:0x51b8	start includes c60/c64 handshake and writes the start word	CPU map
`bl_NandRead0nly`, mem:0x4d08	pause IRQ handler advances PC by 8, acknowledges IRQ8, sets CTRL bit 0	control registers
`bl_NandRead0nly`, mem:0x4cbc / mem:0x51f4	interrupt clear / interrupt wait	CPU map
`fw_restore`, mem:0x0802a640	repeated reads of controller +080 consume FIFO data	NAND controller
Original READ/WRITE/ERASE programs	WAIT0..4 synchronize peripheral work; WAIT12 tests masked status	NAND operations

Source use establishes that these sequences occur in the original firmware; it does not prove every bit's meaning or all possible operand forms.

14.4 Emulated checks

The p09 revision tests actual ARM probe/worker code, packet-level USB transfers, cache visibility, persistent peripheral state and explicit CPU bus-stall injection. Current validation records counts and hashes. The native resident sequence completes nine pause controls, seven status controls and 531 common-opcode checks. Status-before-ISA ordering now works: clearing FMC/NSTAT on reset and deferring completions fixes the earlier inherited-state WAIT9 model gap.

p09 corrects an earlier inferred mechanism: the physical status programs stop at the second WAIT0, not the first WAIT5. The model now reproduces that location, FMC reset phase 4, release and following execution. Buffer-blocked and ignored release scenarios remain fault injection. The invalid-PC/CPU-stall test is also explicitly hypothetical; no regnum result identifies the physical wedging case.

Manifest validation checks requests without executing unknown instructions. BCH parity is synthetic and timing is uncalibrated; complete FMSS/VIC routing is still open. Historical p06/p07 failure-prefix replay remains in prior validation. See the simulator documentation for commands.

14.5 Physical capture: p07

The p07 findings give provenance and qualification. Source SHA-256: `751e38b88144d160fecb9ba2c1de5b0076e98ba2fcf547c8f4ac334bc5130653`. All 29,279 native records are retained: 360 sampled, 25,494 not executed and 3,425 dependency errors. Nine recovery comparisons match. All 24 `tmwait` forms reach END at 126,101–126,103 us, C14=0. ev5's first case faults and reaches END, then reset fails with C08=0; subsequent candidates have no execution.

Experiment	Sampled outcomes	Not executed	Case errors
`m2_recovery`	END: 7, TIMEOUT: 2	0	0
`state`	END: 1	0	0
`restore`	END: 2	0	0
`regmap`	END: 1	0	0
`clock`	END: 2	0	0
`pram`	END: 1	0	0
`alu`	END: 9	0	0
`memory`	END: 1	0	0
`indirect`	END: 4	0	0

Experiment	Sampled outcomes	Not executed	Case errors
resume	END: 7	0	0
control	END: 1	0	0
wait	FAULT: 15, END: 1	0	0
c58	FAULT: 37	0	0
op15	END: 24	0	0
op12	TIMEOUT: 8	0	0
c58ill	FAULT: 5	0	0
startseq	END: 14	0	0
startseq2	END: 18	0	0
startseq3	END: 16	0	0
oneshot	END: 2, FAULT: 2	0	0
c50	FAULT: 3, END: 1	0	0
holdout	FAULT: 2, END: 4, TIMEOUT: 1	0	0
op12wait	TIMEOUT: 80, FAULT: 71, END: 9	0	0
ev11	END: 4, FAULT: 4	0	0
ev5	FAULT: 1	3	0
c50x	none	3	0
pending	none	25	0
c14	none	7	0
op12ack	none	8	0
op10	none	1	0
unknown	none	36	0
opsweep	none	717	0
optime	none	10	0
ctrlbits	none	37	0
fmcmap	none	1	0
evsweep	none	128	0
rwmask	none	233	466
op0f	none	6	0
ctrlhold	none	13	0
evclear	none	12	0
timeregs	none	15	0
ev5rule	none	16	0
c08	none	7	0
chan1	none	8	0
readev	none	128	0
regnum	none	19	0
branch	none	7	0
waitf	none	10	0
opcost	none	30	0

Experiment	Sampled outcomes	Not executed	Case errors
op12rel	none	55	0
pause	none	7	0
fmc28	none	11	0
blk100	none	128	0
irqline	none	8	0
irqen	none	264	0
ev5t	none	6	0
r8	none	11	0
ev8	none	8	0
evcons	none	80	0
dmaloop	none	8	0
op0916	none	55	0
op16	none	14	0
opstate	none	49	0
op16b	none	20	0
op09	none	320	0
op09b	none	288	0
op09c	none	128	0
chan1pins	none	8	0
pcwrite	none	3	0
op12cont	none	7	0
ctrlfx	none	48	0
op09d	none	8	0
fwidth	none	18	0
irqsrc	none	136	0
eccpar	none	12	0
bufsize	none	10	0
eccstr	none	68	0
fields0	none	21	0
tmofield	none	12	0
eccstr2	none	134	0
ev5en	none	24	0
tmobits	none	15	0
eccmode	none	36	0
eccchunk	none	4	0
rawpage	none	2	0
rawwrite	none	8	0
writeseq	none	11	0
op09nand	none	1024	0
eraseseq	none	20	0

Experiment	Sampled outcomes	Not executed	Case errors
eccchunk2	none	33	15
eccmode2	none	132	0
tmobits2	none	5	0
ev5ready	none	18	0
pageorder	none	33	0
op09w33	none	48	0
ev5arm	none	16	0
writeseq2	none	12	0
eccmode3	none	40	0
ev5two	none	6	0
b15ecc	none	14	0
eccmode4	none	24	0
finale	none	387	0
startdiff	none	24	0
startbits	none	420	0
rsdecode	none	14	0
vic20	none	144	0
fmc28b	none	10	0
seqfirst	none	1	0
w2state	none	1792	0
c50b8	none	14	0
c58b1	none	28	0
evsweep2	none	3072	0
c50fault	none	8	0
ctrlwords	none	24	0
c08self	none	5	0
latchrep	none	2560	0
irq044	none	792	0
fmcbits	none	9	0
fmcstat	none	32	0
eccregs	none	15	0
seqrestart	none	12	0
done5	none	58	0
progtime	none	387	0
startregion	none	56	0
bufport	none	24	0
c58b2	none	29	0
stprev	none	39	0
tmwait2	none	7	0
done5b	none	24	0

Experiment	Sampled outcomes	Not executed	Case errors
rsresult	none	14	0
rschip	none	14	0
miscorr	none	2080	0
fullpage	none	42	0
dmactl	none	42	0
fmcstat2	none	8	0
ch1buf	none	8	0
evsweep3	none	2560	0
blk100b	none	99	0
straps	none	1	0
irqop	none	48	0
w2vic	none	1024	0
regsweep	none	744	0
access	none	736	2944
readid	none	4	0
m2_shifts	none	168	0
m2_pram_alias	none	100	0
m2_c58_bits	none	68	0
m2_waitcons	none	80	0
m2_status_order	none	33	0
m2_ce_conditions	none	432	0
m2_delayed_vector	none	36	0
m2_nested_fault	none	12	0
m2_pause_state	none	12	0
m2_ecc_sequence	none	20	0
m2_fifo_access	none	12	0
m2_dma_ecc_overlap	none	24	0
m2_opcode_confirm	none	531	0
m2_opcode_m1	none	78	0
m2_opcode_scan	none	1422	0
m2_opcode_states	none	105	0

14.6 Physical capture: p09

The p09 audit retains all 4,657 sampled records across 52 returned families. Source SHA-256: d4d4215df32a96381c58c6fff8c53f29215762e68d25b83ed278c40900ef4316. There are no retained setup/worker failures. All three faulting status controls recover and permit following execution; the first reset failure is phase 4. 531 common-opcode cases yield 526 matches and five wrong immediate-ack FAULT expectations, corrected to observed END. 78 M1-comparison expectations match. Broad opcode scans now have execution, with meanings still qualified per form. Communication disappears during regnum, with no returned case; 33 subsequent timeouts are transport failures. See device findings.

14.7 p10: M1 register-role comparisons on M2

All 59 `m2_wedge` requests reach END with IRQ=1; all 38 scored controls match. Capture SHA-256 `aa918f288abbf3777afb073fadcda131a487544f785ac757ddca0e2d26a21fb5`. Qualification: section 16.8.

14.8 p11: full selector comparisons and source-10 difference

`m2-all-p11.json`: 1,590 sampled records, 1,589 END plus the expected JR r8 self-loop TIMEOUT; of 1,569 scored cases, 1,568 match M1. The one difference, `m2_regsrc/argument-source/010`, gives AND/OR=22002222, ADD=44004444, SUB=0 (self-source, not M1's r0 rule). LDI/PLD at PC=0x60 return `e1a0f00e`/0. Qualification: section 16.9.

14.9 p12: source 10 and isolated reset phases

72 requests: 64 END/matches and eight expected observation TIMEOUTs with IRQ=0. Capture 693,753 bytes, SHA-256 `d6013102ed545f12fcc584ccc0f1144a9a50307cdcb0fe9d8c776cafb8093953`. All 1,664 MOV/ALU result slots match self-source behaviour (245 distinguish it from M1's r0 rule). STOP clears PC but preserves GPRs/PRAM; STOP then C00=80000 clears GPRs, PRAM, C04/C10/C40/C44/C4C. Qualification: section 16.10.

14.10 P13: full attempt, state-b reset failure

6,204 sampled records (one cleanup failure), 21,831 initial-reset failures and 2,959 dependency errors over 171 families. First failure ctrlfx/0002: start=1, evjmp at PC 0x20, C08=b, IRQ0; all later initial failures share that signature. All 531 common opcode and 78 M1 comparison checks before it match. Qualification: section 16.11.

14.11 P14: no candidates execute; forced reset was skipped

Seven initial phase 1 failures, C08=b/PC20/IRQ0, method 8 only. Qualification: section 16.12.

14.12 P15: effective reset and independent execution

`experiments/m2-all-p15.json`: seven sampled cases, no errors; four marker assessments match; WAIT3, zero-field illegal1a and evjmp started with C00=1 stay in their 2 ms observation TIMEOUTs. STOP leaves C08=b/PC20/IRQ0, abort reports 3, forced reset before another STOP reports 0, the ordinary reset finishes phase 6/rc0 and the next marker reaches END/IRQ1/r2=72657365 (method 01000318). Qualification: section 16.13.

14.13 P16: full campaign with effective recovery

Source: `experiments/m2-all-p16.json`, 407,282,777 bytes, SHA-256 `56c8e1c81c4deee4994ed28b0e97412818ea8e802f64085b0f22a191e81e7534`. All 172 families use resident batches; 31,001 native records are sampled. There are no setup/reset/cleanup/transport failures. Preflight is 1 in 30,978 cases and explicitly skipped (2) in 23. `attempt_finished=true`; `complete=false` reflects the one known M1 source-10 hypothesis mismatch, not incomplete execution.

Scoring class	Records
match	2296
mismatch against retained M1 hypothesis	1
discovery observation	1527
unscored native observation	27177

The 531 core and 78 M1 opcode-comparison cases all match. All 157 previously deferred event/ECC/DMA cases now execute. The detailed findings are in chapters 7–10 and 16, with the full audit and family summaries. No image hash or raw journal was supplied; bit 27 is present. The compressed copy retains the exact original bytes after decompression (23023680 bytes on disk).

14.14 P16 offline re-analysis

`tools/fmss_p16_offline.py` → `validation/m2-p16-offline.json` (analysis).

Record set	Outcome
`dmaloop` (8)	all outputs byte exact
`dmactl` page reads reaching END (24)	all 4 KiB pages byte exact; 0x10..0x80 controls fault
`m2_dma_ecc_overlap` (24, p16 generator)	first output exact for 32..64 bytes, second differs from byte 0; 0x038 mostly 0x187
raw sectors (`rawpage`, `rawwrite`, `fullpage`)	352/352 match M1 BCH8 with message data‖meta10 (sector 0) or data
erased raw sectors	8, all 0xff

14.15 P17: DMA acknowledgement and overlap

Source `experiments/m2-dma-p17.json.gz`, 48,343 bytes; 34 cases, all resets verified (analysis, `validation/m2-p17-audit.json`).

Family / cases	Outcome
`m2_dma_ack` acknowledged (6)	all byte exact (0x038 = 7 or 0)
`m2_dma_ack` exact p16 program (2)	first output exact for 32 / 36 bytes
`m2_dma_ack` plain stale (2)	first output exact for 256 bytes; 0x038 = 0x207
`m2_dma_ecc_overlap` serial + different-slot (16)	all byte exact; 048 = 0xc3000
`m2_dma_ecc_overlap` same-slot (8)	96..140 old bytes in 4..32-byte runs; 048 = 0x020c3000

14.16 P18: timeout, +400 words, BCH8 correction

Source `experiments/m2-dma-p18.json.gz`, 32,635 bytes; 34 cases, no case errors (results, `validation/m2-p18-analysis.json`).

Family	Outcome
`m2_timeout_window` (14)	fault_at = 492.747 × (count + 1) + 447.5 us, residual ≤ 1.1 us; WAIT3 = illegal opcode
`m2_bank_regs` (6)	every +400 word reads 0; primary values identical in both orders; masks in section 9.1
`m2_bch8_correct` (14)	raw writes exact; ≤ 8 errors corrected (25 sectors); > 8 left unchanged (4); ECC_RESULT not per-sector

14.17 P19: ECC status, BCH16, status during busy

Source `experiments/m2-p19.json.gz`, 47,756 bytes; 19 cases, all END (results, `validation/m2-p19-analysis.json`).

Family	Outcome
`m2_bch16_encode` (7)	BCH16 14/14 narrow-sense t = 16; BCH8 control 7/7; repeat identical
`m2_status_busy` (5)	04c 0x80 busy / 0xe0 ready; program 15 busy polls, erase ≥ 48, read 1; no CE 0x70
`m2_ecc_status_timing` (7)	810 bit 3 uncorrectable, bit 5 at 8; 818 corrected total << 11 | decodes; 80c ring of 8 advanced by reads

14.18 P20: result ring and BCH16 correction

Source `experiments/m2-p20.json.gz`, 26,727 bytes; 16 cases, all END (results, `validation/m2-p20-analysis.json`).

Family	Outcome
`m2_ecc_ring` (9)	period-8 ring; 80c reads advance it, 810/818 reads and 80c writes do not; 0x81c aligns to slot 0
`m2_bch16_correct` (7)	≤ 16 errors corrected (10 sectors), > 16 unchanged (4); 80c per sector correct after 0x81c

14.19 P21: ECC status bits, WAIT4 release, ECC_CTRL

Source `experiments/m2-p21.json.gz`, 44,789 bytes; 56 cases (results, `validation/m2-p21-analysis.json`).

Family	Outcome
`m2_ecc_threshold` (4)	810 0x15 for 1..6, 0x35 for 7..8, 0x55 erased; 81c clears 818
`m2_wait4_ack` (13)	only 810 writes containing bit 1, and 81c, re-arm WAIT4
`m2_ecc_modes` (39)	bits 15:8 metadata length, bit 2 BCH16; bits 3..7, 16..23 no parity effect

14.20 P22: decode WAIT4, BCH16 bit 5, ECC_CTRL, timing, FMC 018/05c

Source `experiments/m2-p22.json.gz`, 152,023 bytes; 89 cases (results, `validation/m2-p22-analysis.json`).

Family	Outcome
`m2_wait4_decode` (6)	only writes with bit 2 (4, 0x1ff) re-arm WAIT4 after a decode
`m2_bch16_threshold` (3)	810 bit 5 from 7 errors
`m2_ecc_modes2` (27)	bits 24..31 no effect; 0x3/0x7 decode; metadata 13/16/32 pad with 0xff
`m2_opcost` (48)	ALU 30.1 ns, window access 60.6 ns, ldi SDRAM 211 ns; SRAM = SDRAM
`m2_fmc_misc` (5)	018 / 05c: no data or ECC effect

14.21 P23: jump targets and WAIT vs ready/busy

Source `experiments/m2-p23.json.gz`, 11,281 bytes; 58 cases (results, `validation/m2-p23-analysis.json`).

Family	Outcome
`m2_jump_width` BNZ (21)	low 3 bits ignored; pads 2^4..2^14 hit; beyond program: SDRAM executed, PC 0x40200 once
`m2_jump_width` JR (21)	all fell through (target register written by the preceding instruction; cause r[R] = 0, not a hazard: M2-108, M2-110)
`m2_wait_rb` (16)	only WAIT 11 passes, immediately while busy

14.22 P24: register hazards and JR

Source `experiments/m2-p24.json.gz`, 8,218 bytes; 37 cases, all END (results).

Family	Outcome
`m2_alu_hazard` (8)	new value at every distance 0..4 (32/32)

Family	Outcome
m2_jr_hazard (8), m2_jr_width (21)	JR fell through in all 29 (r0 = 0)

14.23 P25 bundle

Source experiments/m2-p25.json.gz, 47,361 bytes; 165 cases, all resets verified (p25, validation/m2-p25-analysis.json).

Family	Outcome
m2_jr_cond (16)	jump iff r[R] ≠ 0
m2_jr_width2 (21)	consistent with exact jumps after an 8-byte pad shift (generator error)
m2_wait_sources (71)	only status latch → WAIT 5
m2_fmc_flags (15)	flag map (section 9.7, section 8.14)
m2_dma_cut (18)	gap ≥ 1.9 us exact; shorter: TIMEOUT, 8..48 bytes
m2_timeout_low (16)	count 7: n = 0, 1 fault within 0..2 us; step 2^(n+3) cycles for n = 4..11, saturating at n = 11; n = 2, 3 (4, 8 us) do not fit 2^(n+3) (steps 2^3, 2^5: M2-115, M2-216)
m2_ecc_regs (8)	parity at 0x840..0x85c; 0x800 = 0x20070618

14.24 P26 bundle (partial)

Source experiments/m2-p26.json.gz, 18,245 bytes. Valid: m2_timeline (19), m2_wait_cpu C0C/C10/C48/C54 (36). C60 = 1 by CPU held the engine; 66 later cases not executed (p26).

14.25 P43 bundle

Source experiments/m2-p43.json.gz, 183,161 bytes, SHA-256 46e9e6fade291332008c5384fb5716ca1cc0fadaef6c874612ed63936ec08f31; 18 families (17 m2_r5_*, generator emCORE/tools/nano4g/round5_experiments.py, and m2_cpu_stack, emCORE/tools/nano4g/cpu_stack_experiments.py), 0 case errors, every requested reset verified (the eight no-reset cases of m2_r5_prescaler_step skip the reset by design) (p43, validation/m2-p43-round5-analysis.json, validation/m2-p43-cpu-stack-analysis.json).

Family	Outcome
m2_r5_latch_clear (8)	high-bit source keeps the predecessor's value across ld/st/wr (M2-674..M2-677)
m2_r5_ext_dest (12)	no GPR written, latch unchanged (M2-678, M2-679)
m2_r5_sti_high (3)	r0 after movi, r5 after mov with selector 5: the last source value (M2-680, M2-681)
m2_r5_rom_ldi (16)	one ROM word per 16-KiB quarter, never the addressed word (M2-682, M2-683)
m2_r5_op1a_operands (65)	zero-form outcome for all 64 operand words (M2-684)
m2_r5_nop_stall_c08 (4)	C08 = 4 for R = 8, 0xff and unmet wait 3 (M2-685, M2-686)
m2_r5_c48_count7 (9)	ends 493 us earlier per C48 unit with C14 = C48; C48 = 8 faults (M2-687, M2-688)
m2_r5_tmwait_repeat (10)	126079..126108 us; one 23-us early end, without a C14 write (M2-689, M2-690)
m2_r5_prescaler_step (10)	reset 76/78 us, none 24/34, C00 82/82, FMCTRL1 30/32, 0x81c 28/32 (M2-691, M2-692)
m2_r5_c08_long (9)	nop2 9/9/2, ld 1/1/1, sti 8/1/8 (M2-693)

Family	Outcome
`m2_r5_select_tag` (6)	switch at word 63/57/58 (gap 16, buffer-1 word 120 at the switch twice), 114/114/114 (gap 32) (M2-694..M2-696)
`m2_r5_overlap_tags` (4)	every word an input word, initial content or zero (M2-697, M2-698)
`m2_r5_dma_ctrl_sel` (5)	038 bits 5:3 = 3/5/6 stall after 32 bytes; DMA in 0xf: word 4g + j from 4g + 5j (M2-699..M2-701)
`m2_r5_out_select` (6)	only 0x100 delivers buffer 0, all others 512 zero bytes (M2-702)
`m2_r5_x340_window` (9)	delay alone completes; 014 write after 0/8/16 nop2 leaves 0/16/32 RAM words (M2-703..M2-705)
`m2_r5_status_lengths` (11)	0x41 for 160..224, 0x14 for 255..508 (M2-706, M2-707)
`m2_r5_bch16_814` (4)	encode and decode identical with 0x814 = 0x2f (M2-708..M2-710)
`m2_cpu_stack` (93 sequencer-harness records, 107 CPU jobs)	211 checks: 208 match, 1 observation, 2 mismatches (both the location-order check: the engine lists the bits of one byte higher first, M2-714); controller idle after 89/89 jobs (M2-711..M2-716)

14.26 Reboot sessions: unmapped bus accesses

Sources `experiments/m2-reboot-ldi.json.gz` (2,466 bytes, SHA-256 `44e95cc6b5738f2ea96f9e386a04fa91fbd502efb36e1400dae756852690e753`) and `experiments/m2-reboot-dma.json.gz` (2,615 bytes, SHA-256 `f047be85b0cfb10b2049bfbf0018fc9efb3b164386e68c7c6da0eefa89ce2f9e`), each run alone with `--single-probes` after a power cycle (generator `emCORE/tools/nano4g/cold_experiments.py`) (docs/147, `validation/m2-reboot-analysis.json`).

Family	Outcome
`m2_reboot_ldi_hole` (4)	both controls END; `ldi/hole/stall-safe` lost USB (no record), the ordinary hole then timed out on a dead device (M2-718)
`m2_reboot_dma_hole` (4)	both controls END; `dma/hole` and `dma/hole/stall-safe` END after 9 us / at the 50-ms window, 034 = 0x50000000, device alive (M2-717)

14.27 P48 bundle (round 6)

Source `experiments/m2-p48.json.gz`, 11,778,983 bytes; the attempt did not finish (USB lost in `m2_r6_gate5`). Analysed families: `m2_r6_c08` (15), `m2_r6_read_status` (12), `m2_r6_latch` (12), `m2_r6_dma` (30), `m2_r6_vic1` (33), `m2_r6_pc` (36), `m2_r6_apple` (8) (generator `emCORE/tools/nano4g/round6_experiments.py`; `validation/m2-p48-round6-analysis.json`, `validation/m2-p48-round6-reviewed.json`). The CPU family and four saved gate cases are reviewed in `validation/m2-p48-cpu-gate-review.json` and docs/149. The capture SHA-256 is `190c5f1404c0a2b7d9506d54ece388e3c1747f220ac17a30651f84cadf6ccf5e`; the device image hash was not recorded.

Family	Outcome
`m2_r6_c08`	CPU-sampled C08 1/2/9, 7 (SDRAM), 5 (window), 4 (wait) (M2-719)
`m2_r6_read_status`	bits 19:12 0xc3 → 0x41 → 0x14 → 0x3c during reads (M2-720)
`m2_r6_latch`	bits 3/23 change in every order within sample resolution (M2-721)
`m2_r6_dma`	exact transfers; 038 bits 12:6 live during the DMA (M2-722)
`m2_r6_vic1`	`wait2` 32/33 released by hardware VIC1 lines 0/1 only; line 52 never rose (M2-723, M2-724)
`m2_r6_pc`	`bnz/jr` exact to 2^24 + 0x100, no alias (M2-725)
`m2_r6_apple`	original ERASE/WRITE erase and program blocks 3000/3001 exactly (M2-726)

Family	Outcome
`m2_cpu_direct`	1,850 jobs in 15 worker RUNs, all resets verified, 58.06 s execution; all 12 required round trips fail their erased/DMA/FIFO payload checks (M2-727, M2-728)
`m2_r6_gate5`	baseline and gate0..2 return; gate0 already off, gate1/2 initially on; VIC0 raw 0x20 and VIC1 raw 0 throughout each 28-us pulse, clocks restored; gate3 loses USB with no result (M2-729)

The CPU executor left DMA enabled during FIFO reads. The inferred stray-DMA corruption prevents using the 44 FIFO-containing cases to establish optional NAND responses. Separate DMA-only observations retain six mismatching cache/copyback/multi-plane hypotheses and 48 mismatching timing comparisons; passing a reference model does not supersede these device outcomes (M2-728).

14.27.1 Harness correction and model evidence

The corrected reader selects the path per read and restores CE/timing control. The clock pulse loads its state into registers before gating and restores clocks before any stack, SDRAM or peripheral read. GATE_MARK|2 fields r0/r1/r3/r4/r5 are `0xffffffff`, meaning unavailable; its timestamp is measured after restoration. Neither a stopped timer nor an interrupt state during the window is inferred from those fields (M2-730).

Payload SHA-256 `e9a79a1d12a274a9526b48938dbb477546e50ebceffd5167e0241838c62e286f` (89,600 bytes) passes 75 regressions and native preflight of all 1,850 CPU templates plus 161 gate cases. The CPU logs retain every descriptor/status marker, use at most 429 of 512 entries and drop no details. Four compact-map controls also pass. Reports: `validation/m2-p48-fixed-validation.json` and `validation/m2-p48-fixed-preflight.json`. CPU preflight explicitly uses `nand-command-reference-v1`, including synthetic parameter/features/alternate-ID responses; it does not establish optional Hynix support or physical timing. There is no post-fix device capture in this evidence set (M2-730).

14.28 P49 bundle

Source `experiments/m2-p49.json.gz`: `m2_cpu_direct` (1,850 CPU jobs in resident CPU batches, 0 required-control failures, every reset verified; `validation/m2-p49-cpu_direct-analysis.json`, M2-731..M2-741) and `m2_r6_gate5` (five completed cases: baseline and gates0..3; the following `BATCH_FREE` remains QUEUED, jobs done/started 62/63; M2-742) (docs/149).

All five gate records contain exact PWRCON restoration and successful reset/cleanup. The finite probe TIMEOUT is distinct from the host timeout on FREE. USB status polling remains live; gate4 is not submitted. Before/after VIC0 raw is `0x20`, VIC1 raw zero; no interrupt sample is taken while clocks are disabled. `validation/m2-p49-gate-worker-review.json` retains the evidence. The physical cause of the dispatch stall is not established. A native fault injection (timer IRQ8 masked, non-idle thread current) reproduces queued FREE on the old image and completes on the new direct worker dispatch path, which also adds live timeout diagnostics (M2-743; docs/150).

14.29 P50: clock-gate scan

Source `experiments/m2-p50.json.gz` (964,738 bytes), SHA256 `25570c99ced692b925d0a6b2318e9cd7c6a89ecb9dc7f087a79039224cc5ec11`: `m2_r6_gate5`, 110 cases (baseline and gates0..108), all END, worker rc0, initial reset and cleanup successful, exact restoration. Of the gate entries, 67 were enabled and 42 already off. All have VIC0 raw0x20 / VIC1 raw0 before and after restoration; during-pulse interrupt fields are unavailable, not measurements. No gate identity follows (M2-744).

USB is lost submitting the following RUN; the retained status is allocation job331 DONE, not the failed RUN. Gate109 is the next intended case by order, but acceptance, execution and restoration are unknown (M2-745). The image hash is absent. The subsequently supplied raw journal verifies all 110 result records and confirms the gate109 plan (see the following section). The five shared p49/p50 cases change from TIMEOUT to END because the observation floor rises from 3,000 to 160,000 µs; p50's loop ends in 13,525–13,548 µs. The queued FREE stall after gate3 does not recur. See `validation/m2-p50-gate-review.json`, `validation/m2-p50-round6-analysis.json` and docs/151.

P51 (`experiments/m2-p51.json.gz`, 36,531 bytes, SHA256 `c386c0f055942e1e9af17657c52c211cdcb03705cda16f977d0c978faefb2a4d`, with its raw journal): baseline and gates 0..2 return END with exact restoration; USB is lost submitting the next RUN, whose last journal `batch-plan` names gate 3, a case that completed in p50 (M2-745).

14.30 P51 and the p50/p51 journals

P51 (`experiments/m2-p51.json.gz` and `.jsonl.gz`) returns four cases: baseline and gates0..2, all END with successful reset/cleanup and exact clock restoration. It loses USB submitting the next RUN. The journal records the gate3 plan and allocation13 DONE; no acceptance or RUNNING status follows (M2-746). The p50 journal additionally confirms gate108 FREE330 DONE and the gate109 plan preceding its loss (M2-745).

All 110 p50 and four p51 journal case records equal their summary records; all raw request/output hashes verify. The five shared manifests are identical, including gate3. P50 reports the EXECFIRMWARE entry marker; p51 does not. That is the only capability difference. No image hash or CPU cache/MMU state was recorded, so the entry-history difference does not establish causality (M2-747). `validation/m2-p50-p51-journal-review.json` retains the evidence.

The next 53-case family collects a pre-start checkpoint over USB and records running-code identity; native packet/cache tests and preflight pass, without modelling physical gate effects (M2-748). See docs/152.

Chapter 15 — Interpreting M2 evidence

Use the instruction-level capture before the human-readable legacy summary. Some report labels and comments came from M1 hypotheses. An experiment called “illegal opcode”, “one-shot”, “conditional wait” or “timer” does not establish that interpretation on M2.

15.1 Required distinctions

Evidence	Valid conclusion	Invalid shortcut
Recorded manifest	the program was planned/submitted	every instruction ran
Worker completed with rc -2	bounded native path failed; inspect capture	all output is invalid, or all output is a hardware result
Zero samples, pre_ok=0	candidate execution not established	opcode ended instantly; register reads are all zero
Sampled TIMEOUT	no requested completion during the host interval	illegal instruction or infinite hardware stall
Fault followed by END	fault path completed	clean success
END with expected sentinel	that path reached sentinel/end	all fields/operands have confirmed semantics
`late_at_us=0`	scheduled write was not recorded as delivered	candidate write had no effect
Two operands alias	those operands agree in that context	all high bits are ignored
Source uses one register word	that word is used for that operation	each bit is independently understood
Model passes	model/code contract holds	physical silicon is validated

p06 illustrates all of these distinctions. The first `pause (12)` sample is useful even though cleanup fails. Later opcode sweeps are inconclusive even though USB and the worker keep delivering records. Failure continuation is valuable only if the analysis preserves this difference.

15.2 Order and state contamination

Treat experiments as a stateful sequence. A pause or timeout can leave state that reset does not clear. p06 recovers between `op12` and `startseq2`, then loses sampled execution permanently after the first `op12wait` case. A simulator that boots fresh for each family cannot discover this by construction; include persistent-session/native-batch tests as well as isolated instruction tests.

Full-map zeros are especially ambiguous: the capture may never have been filled, the snapshot window may have been skipped, or a consuming port may be represented by a placeholder. Do not turn such values into an access-mask table.

15.3 M1 comparison discipline

The M1 book supplies the questions and candidate programs. M2 has its own CPU reset/start handshake, ECC/status layout and source-defined WAIT12 status path. The tested indirect instructions `pld` (18) and `pst` (19) exist on M2 despite old “3G only” descriptions. Conversely, M1's JR/SIG/WAIT2/EVJMP/NOP2, timer scaling, one-shot behavior, mirrored channel map, power-on defaults and RS selection were unconfirmed on M2 at p06. Their later M2 measurements and differences from M1 are in chapter 19; the power-on values are in chapter 8, section 8.21, and what is still open is in chapter 17.

Distinguish a physical observation from the smallest explanatory model. In the p06 simulator update, pause IRQ/state/latches are measured; STOP preserving that pause is an inferred failure scenario consistent with the native return sequence. Neither establishes all reset paths or an automatic recovery algorithm.

15.4 P16 examples of interpretation boundaries

P16 is a complete execution attempt with one known comparison mismatch, not an incomplete run. Its source-10 mismatch deliberately retains M1's prediction; 64 dedicated M2 source-10 controls match the measured alternative.

C58=0 programs can acknowledge their own timeout IRQ and then return END. Stored before/after IRQ words distinguish that from a wait that passed. Likewise, all 24 p16 DMA/ECC programs reach END but fail payload comparison; their guarding WAIT2 tested an unacknowledged flag (section 9.5). Neither an END count nor RESULT 80c=0 proves complete DMA or a valid codeword, and a passing WAIT proves an event only if its flag was clear before the stimulus. WAIT2's initial VIC snapshot can miss a later transient; snapshot disagreement alone does not establish a different interrupt mapping.

15.5 P43 examples of interpretation boundaries

The two `m2_cpu_stack` mismatches compare the 0x820 location list with the injected locations in ascending order; the sets are equal and the engine lists the bits of one byte higher first (M2-714). A check that fixes an order the hardware does not follow fails without any wrong data.

The boot-ROM words that `ldi` returns are identified in the ROM image, but which earlier ROM access left each one is inferred (M2-683). The 20-byte beat step of a DMA in with 038 = 0xf is inferred from the word order alone (M2-701). One C08 read per run samples a loop at an uncontrolled phase, so differing values between runs of one instruction kind are not a state difference (M2-693). The BCH16 0x814 comparison first counted the 0x814 readback word as a decode difference; the decode registers and data are identical (M2-709, M2-710).

Chapter 16 — Physical M2 findings through P43

Sections 16.1 to 16.4 are the historical p06 observations; the later rounds follow in order from section 16.5 (p07) to section 16.23 (P25), with section 16.24 (P43). Results of the later rounds are in chapter 18.

Audited source: `experiments/m2-all-p06.json`, 589,189,971 bytes, SHA-256 `4c804e08dd2e326219266d9eed55f71b489776f51a59b7971202c4920240f8c2`. The capture started at `20261101T071245.434221Z`, reports S5L8720, probe v14, bulk USB and resident batches. The image hash was **not recorded**. Do not attribute it to the current local binary by filename or matching probe version.

16.1 What the completed campaign contains

All 159 experiment entries returned a result. `attempt_finished=true` means the host reached the end of the campaign; `complete=false` records errors. The summary retains these case records:

Classification	Records	Meaning
Sampled execution	140	`nsamples>0` and `polls>0`; includes END, fault and host timeout
Setup/preflight did not reach sampled execution	22,972	`pre_ok=0`, no samples; all have worker rc `0xfffffffe`
Explicit case errors	6,083	2,673 mutation-capability refusals; 3,410 dependency errors

These are **retained summary records**, not a count of unique native submissions. Legacy report dictionaries discard some setup probes and overwrite repeated labels. For example, `startseq2` submitted 18 cases but retains nine final captures; `startseq3` submitted 16 and retains eight. No p06 JSONL was supplied with this file. The full native order cannot be reconstructed from the summary alone. The coverage table gives retained counts per entry.

The reported capability word is `0x7a0000`: bulk, resident v1/v2 and discovery are present; mutation bit 18 is absent. The write refusals are harness decisions, not evidence that M2 rejects writes or that a NAND operation failed. This does not change the user's authorization to run those experiments.

16.2 Execution chronology and pause (12)

`state` through op15 reproduce the earlier architectural observations. op12 case `[0xd60] & ffffffff == r2 (1234)` is the first sampled case with a cleanup error:

Field	Observed value
Instruction latches C40/C44	`12020d60 / ffffffff`
PC (c38)	`0x30`, the pause (12) instruction
C08 / CS_IRQ	`0x0a / 0x08`
C00	`0x0004ffb4`
Observation interval	50,001 us, host TIMEOUT
Worker return	`0xfffffffe` (-2)

The seven subsequent op12 records and five `c58ill` records did not execute their requested instructions. Most retained `startseq` cases also did not execute; its explicit no-reset/no-preflight case was sampled and timed out. The retained `startseq2` and `startseq3` cases then reach END. This establishes recovery in this sequence, not the exact cause: omitted setup captures matter.

Later, `op12wait/idle/op12/n0/arg0/mask0/order0` reproduces the pause signature with a different encoding (`C40=12000000`, `C44=0`) at PC `0x58`, `C08=0xa`, IRQ=8, and a 20,001 us observation interval. Cleanup again returns -2. Its remaining 159 cases, and **every later experiment's retained probe captures**, have no sampled execution.

Interpretation: the two forms support a pause/CPU-handoff operation, matching the original M2 CPU driver's handling of IRQ bit 3. They do not establish all ignored operand bits or demonstrate successful resume. In particular, the old test label suggesting a masked comparison is a hypothesis, not the conclusion. The current reset helper handles C08=4 specially but has no equivalent branch for C08=0xa; bounded polling can return -2 in that state. This is a source-based explanation of the observed sequence, not an additional silicon measurement.

16.3 Positive architectural observations

Paths below are under `results/<experiment>/data` in p06. Exact JSON pointers and selected raw captures are in `validation/m2-p06-audit.json`.

Experiment/case	Observation	Scope
`regmap; control`	r0..r7 at c18..c34; r6 aliases result; CPU-preloaded r3 survives and can be stored	tested register values
`clock`	2,000/20,000 two-instruction loops take 138/1,355 us	slope ≈33.8 ns/instruction in this configuration, not a clock proof
`pram`	CPU writes persist d00..d7c; tested sequencer reads agree; d80..dfc writes do not persist	32 observed parameter words; no claim about other aliases
`alu`	logical SHR, arithmetic wraps; tested counts 32/33 produce zero; immediate-zero ADD uses register form; LD masks by AND	nine concrete cases
`memory`	little-endian words; tested unaligned loads/stores align down	system-memory operations, not instruction target alignment
`indirect`	18 reads GPR/PRAM window; 19 writes PRAM; 1d64 reads like d64	this alias only; not all high bits
`resume`	C58 bit 0 does not start at the vector; CPU reads it back clear	tested start configurations
`wait`	WAIT11 passes in ~4 us; other idle events 0..15 fault in ~126.6 ms	supplied idle state and operands
`c58`	bit 2 distinguishes vector/continue after timeout; tested readback fits `value & 0xd6`	low-value sweep plus 40/80/ff/100/ffff
`op15`	all 24 forms wait through the observation window; C08=4; C14 changes 7f→35	release condition and full timer semantics remain unknown
`startseq3/no-op (0x08) first`	instruction 08 is followed by marker and END	supports no-op-like behavior for this one encoding only
`holdout/shl 1 by 31 / shr by 31`	r2=80000000, r3=1, END	adds boundary count 31
`holdout/pld 0xc30 reads r6`	r2 and r6 both 6666aaaa	indirect result-register read
`holdout/sti then ldi at scratch+0x200`	aligned store, unaligned reload gives 13572468	another RAM alignment check
`holdout/loop of 100`	marker and END after ~8 us	branch/counter combination

16.4 Results that cannot answer their labels

- `oneshot`'s two fault candidates and `c50`'s three fault candidates were observed for only 50 ms. The idle WAIT timeout is ~126.6 ms here. No fault within that shorter interval does not establish one-shot suppression or the value that a fault handler would eventually read.
- `c50/program jumps to the handler (no fault)` does execute the handler and reads `00ff1800` before and after writing 1. It is a **no-fault** control.

- `startseq3` markers finish in 3–4 us, before proposed CPU writes at 2 ms. `late_at_us=0`: these rows do not measure the effects of those later writes.
- opsweep's 717 captures all lack execution. Its legacy “stopped fast” labels describe zero-initialized output, not 717 opcode behaviors. Likewise zero nanoseconds from `opcost`/`optime` is not an instruction timing result.
- All 531 confirmation, 78 M1-hypothesis, 1,422 discovery and 105 state-opcode cases are explicitly **inconclusive**. No new opcode aliases or illegal ranges are established by those sweeps.
- Later register maps, ECC results and READ ID zeros are not hardware values. Preserve the earlier successful READ ID capture; p06 does not contradict it.

The audit is reproducible without a device:

```
python3 tools/fmss_capture_audit.py experiments/m2-all-p06.json \
  --output validation/m2-p06-audit.json
```

16.5 p07: pause recovery and the next failure

Source: `experiments/m2-all-p07.json`, 641,793,213 bytes, SHA-256 `751e38b88144d160fecb9ba2c1de5b0076e98ba2fcf547c8f4ac334bc5130653`. Capture start: `20261101T161039.840011Z`. All 160 entries use native-case records, resident batches and bulk USB. The minimum observation window is 160,000 us. The capability word is `0xfe0000`, including mutation and recovery. The running image hash is again absent; no adjacent JSONL was supplied. Do not identify the physical image with a local build merely because their capabilities agree.

Classification	Native case records
Sampled	360
Initial setup/preflight did not execute candidate	25,494
Dependency errors	3,425
Total retained	29,279

All sampled cases except the first ev5 case return worker rc=0. These counts are not successful semantic confirmations: sampled cases include faults and observation timeouts. The audit retains recovery diagnostics and exact JSON pointers. The full campaign reaches its end, but the broad opcode families again have no sampled execution.

16.5.1 `pause` cleanup and CPU resume work in the measured controls

All nine `m2_recovery` comparisons match. The two measured operand forms retain IRQ8/C08=0x0a and r6=0 while paused. Trusted-END cleanup returns rc=0 in 3–4 us (method 1), and the following controls reach END with r2=0x7265636f. CPU writes PC to the following instruction, acknowledges IRQ8 and writes CTRL=0x4ffb5 at 100–101 us; both forms then reach END at 102–103 us with r6=0x600d600d. These are physical confirmations of those sequences, not merely accepted register writes.

The first baseline starts with an inherited pause at PC=0x58. Its recovery uses method 3 (trusted-END attempt plus abort fallback), returns to idle in 1,006 us, and the baseline executes. This demonstrates the combined fallback sequence in that starting state; the capture does not isolate abort as the sole cause. The later direct pause controls use method 1 successfully. Thus recovery depends on starting state; neither path is established as a universal reset recipe.

op12 now samples all eight comparison-labelled cases, all pausing with IRQ8. `op12wait` samples all 160 cases: 80 `pause` cases retain IRQ8 and time out, while 80 `wait` cases yield 9 END and 71 faults. Across these cases, 20 distinct pause encodings are observed: R=0..15 with arg=imm=0, plus R=2 with arg=0xd60 and imm=0/0xff00/0xffffffff or arg=0xd64 and imm=0xffffffff. The comparison labels do not turn `pause` into a masked comparison. Nor do state labels such as “erase” prove that the intended peripheral operation actually completed.

16.5.2 `tmwait`, faults and event controls

All 24 `tmwait` (15, experiment op15) forms reach END in 126,101–126,103 us, with IRQ=1 and C14=0. This resolves the earlier 50 ms observation timeout: the tested instruction does eventually continue. It supports the timer-wait name; the exact reload value, divisor and all operand effects still need tests.

The three faulting c50 controls now reach their handlers at about 126.6 ms. They read 0x00ff1800 before and after writing 1 to C50, as does the no-fault handler control. M1's event/acknowledgement interpretation is not established by these unchanged values. oneshot now records two END and two fault outcomes; full one-shot behavior still needs the longer dedicated sequences.

The op12wait controls pass wait 0 after command stimulus (FMC_STATUS=0x000c3002), wait 2 after DMA stimulus (0x001c3000), wait 4 after ECC stimulus (ECC_STATUS=0x13), and wait 5 after status stimulus (FMC_STATUS=0x008c3000). wait 11 passes in all five tested stimulus groups; the other selectors fault. The post-run ECC_STATUS=0x13 does not have bit 2 set: do not promote the current model's simple bit-2 predicate to a measured hardware equation. These are qualified state/control pairs, not full event definitions. Idle wait 0 faults. The paired pause cases retain IRQ8. The ev11 matrix passes its four wait 11 forms and faults its four wait 12 forms. Pending, acknowledged and consumed event behavior is still only partially measured.

16.5.3 First failure: status-latch experiment, then reset

/results/ev5/data/0000~1latch_matrix issues a status command, starts the latch with FMCTRL1=0xca and waits on event 5. It faults after 126,595 us and reaches its fault-handler marker: r6=0xfa17fa17, IRQ=5, C08=0. Its final pre-cleanup snapshot has FMCTRL1=0x0000000a, FMC_STATUS=0x008c3000 and C50=0x00ff1820.

Cleanup then fails: stage 4, method 0, reset rc=-3 after 102 us, C08 still 0. The next case fails at initial reset (stage 1), before preflight; every later candidate capture is unexecuted. This is different from p06's paused C08=0x0a failure. A sequencer reaching END does not guarantee its peripherals can reset.

The original reset polls both the c60/c64 buffer handshake and FMCTRL1 bit 30, each with a 100 us bound in this build. Current diagnostics do not record which poll failed or its final peripheral registers. The 102 us/idle signature therefore does **not** identify one of those polls uniquely. Capture the reset phase and c60/c64/c6c/FMCTRL1 before claiming a cause or a working recovery.

The late 531 confirmation, 78 M1-comparison, 1,422 discovery and 105 state-opcode checks remain inconclusive. Their zero-filled outputs supply no new instruction semantics. The next measurement should run pure ISA controls before peripheral stimuli, and isolate this status-latch/reset transition with phase diagnostics.

16.6 Reproduction after p07 (historical model hypothesis)

The first model reproduced FAULT/END and reset failure by making the NSTAT read consume WAIT5, with either reset handshake blocked. p09's sampled PCs reject that causal explanation: the first WAIT5 passed. The updated model retains the persistent transaction, blocks the next command, and defaults to the physically observed FMC reset phase. Ignored-release and buffer-failure scenarios remain explicit fault injection. Earlier model matches did not prove that mechanism.

16.7 p09: status recovery, opcode controls and communication loss

Source experiments/m2-all-p09.json, SHA-256 d4d4215df32a96381c58c6fff8c53f29215762e68d25b83ed278c40900ef4316; image hash not recorded. There are 4,657 sampled records in 52 returned families, then regnum loses USB communication. No regnum output is retained. The 33 subsequent USB timeouts supply no further architectural measurements.

All three status-order variants fault at the second command's WAIT0 and recover by method 4, with subsequent markers reaching END. Reset failure phase 4, FMCTRL1=2, identifies FMC acknowledgement. NSTAT read ordering does not consume the first WAIT5 as previously hypothesized.

The 531 common-opcode cases yield 526 matches and five incorrect negative expectations: immediate acknowledgement before WAIT still reaches END for each peripheral stimulus. Expectations and asynchronous event modeling are corrected. All 78 M1-comparison expectations match for the tested encodings/states. The 1,422 discovery, 717 legacy sweep and 105 state-opcode samples are observations, not automatically decoded semantics.

The old regnum writes r8=0x5eed0008. If M2 r8 is PC, it fetches outside mapped memory. That hypothesis reproduces a worker/USB stall under explicit CPU-bus fault injection. Physical attribution remains unknown. The corrected probes use local targets; the p10 m2_wedge run isolated each request with durable results (now selected explicitly with --isolate-diagnostics). Full provenance, model limits and next command are in p09 analysis.

16.8 p10: M1 register-role comparisons on M2

All 59 `m2_wedge` requests execute and reach END, with IRQ=1, preflight and cleanup success, and all 38 scored controls matching. Capture SHA-256: `aa918f288abbf3777afb073fadcda131a487544f785ac757ddca0e2d26a21fb5`. The file is named `m2-all-p10.json` but contains only that isolation family. No image hash or raw journal was supplied. These are confirmations of selected M1 behaviors on M2, not new meanings discovered for r8/r10. See the qualification table and p10 analysis. At the end of p10, remaining role predictions still needed physical comparison; p11 below supplies that evidence.

16.9 p11: full selector comparisons and source-10 difference

`m2-all-p11.json`: 1,590 sampled records, all worker/preflight/cleanup returns successful; 1,589 END plus the expected JR r8 self-loop TIMEOUT. There is no communication loss. Seven whole resident families plus three isolated bus controls completed. Of 1,569 scored cases, 1,568 match M1 and one differs.

`m2_regsrc/argument-source/010` saves four differing register-form outputs: AND/OR=22002222, ADD=44004444, SUB=0. With r0=11001111 and destination r1=22002222, this is self-source behavior rather than M1's r0 rule. SHL/SHR large counts give zero under both interpretations. All eight broadcast GPRs, local MOVI/MOV/ADD PC destinations, BZ/BNZ/JR selectors and ST predecessor contrasts now have M2 measurements. LDI/PLD at PC=0x60 return `e1a0f00e`/0.

The simulator also needed PRAM clearing across reset to match the untouched words in successive probes. Its exact placement at CTRL=80000 is inferred, not a phase-isolated hardware observation. P11 analysis records provenance, limits, native replay and the 64-case next experiment.

16.10 p12: source 10 and isolated reset phases

All 72 requests return in two resident runs: 64 END/matches and eight expected observation TIMEOUTs with IRQ=0. Every preflight/worker/cleanup succeeds. Capture is 693,753 bytes, SHA256 `d6013102ed545f12fcc584ccc0f1144a9a50307cdcb0fe9d8c776cafb8093953`.

All 1,664 MOV/ALU result slots match self-source behavior; 245 distinguish it from M1's r0 register-form rule, including 42 SHL and 21 SHR results. All eight GPR destinations and eight value pairs are covered; r0 is a deliberate control.

STOP clears PC but preserves GPRs/PRAM. STOP then C00=80000 clears GPRs, PRAM, C04/C10/C40/C44/C4C. Separate buffer/FMC/ECC reset sequences preserve GPR/PRAM. This confirms the reset-before-parameters requirement in the tested states. Raw C00/C14 observations are retained without a full bit/timer-model claim. Analysis, replay and next command.

16.11 P13: full attempt, state-b reset failure

The 171-family capture retains 6,204 sampled records (one cleanup failure), 21,831 initial-reset failures and 2,959 dependency errors. USB remains alive. The first failure is ctrlfx/0002: start=1, evjmp at PC0x20, C08=b, IRQ0; STOP does not clear it, and the ordinary reset only attempts abort for C08=4. All later initial failures share that same state signature. The four proposed event/DMA/ECC families never execute. Earlier p11/p12 results remain valid. All 531 common opcode and 78 M1 comparison checks before this failure match. Capture hash, native reproduction and reset correction.

16.12 P14: no candidates execute; forced reset was skipped

The seven-case resident reset check returns seven initial phase 1 failures, C08=b/PC20/IRQ0, method 8 only. Abort passed the 0/3 check, so the bit-26 kernel skipped forced reset before its next STOP. Both possible intermediate values reproduce all seven failure signatures through the old native ARM binary; this does not distinguish which intermediate value hardware returned. The corrected kernel always resets while held, captures those intermediate values and requires the ordinary reset plus the next preflight to succeed. Audit, qualification and next run.

16.13 P15: effective reset and independent execution

`experiments/m2-all-p15.json` contains seven sampled cases in one resident batch, with no errors. All seven pass END-only preflight and both reset stages; four marker assessments match. WAIT3, zero-field illegal1a and evjmp started with C00=1 remain in their expected 2 ms observation TIMEOUTs.

The inherited state on entry and the newly triggered evjmp both recover: STOP leaves C08=b/PC20/IRQ0, abort reports 3, forced reset before another STOP reports 0, and the ordinary reset finishes phase 6/rc0. The next marker reaches END/IRQ1/r2=72657365. Method 01000318 retains both intermediate statuses. This resolves the p13/p14 failure for the tested sequence. It adds no event consumption, general opcode-field or physical BCH-parity result. Audit, replay and next combined run.

16.14 P16: all families execute; data-path questions remain

All 172 families return 31,001 sampled records with zero reset/cleanup errors. All 531 core and 78 M1 opcode comparisons match; source 10 repeats its already qualified M2 difference. No new opcode is identified solely by the discovery signatures. All 68 C58 CPU/sequencer masks match d6; all five C08 self-reads give 5.

Repeated waits establish non-consuming command/address/DMA flags. Acknowledging them makes the second wait fault; retriggering restores progress. ECC WAIT4 survives the tested ack 1, and masked status WAIT12 survives clearing bit 23. WAIT5 remains sensitive to command/latch ordering; WAIT15 fails in all tested clear-enable sequences. The timeout control words change duration substantially.

All 24 overlap/serial programs end, but none has a fully correct first 512-byte output. Same-slot cases uniquely set 048 bit 25. All 20 repeated-encode programs end with 13 completion observations; fresh completion is not yet proved. All 2,080 `miscorr` records report RESULT 1 without valid input parity, so they do not establish a BCH correction radius. See p16 analysis.

16.15 P16 offline: serial DMA and BCH8 settled from retained records

Serial DMA is byte exact (`dmaloop` 8/8, `dmactl` 24/24 page reads). The overlap payload loss follows from unacknowledged WAIT2 flags in the p16 generator, not from the data path (inferred; `m2_dma_ack` tests it). The physical BCH8 code equals M1's, with sector 0 covering ten metadata bytes: 352/352 raw sectors match. See p16 offline analysis.

16.16 P17: acknowledgement confirmed; same-slot overlap corrupts

Acknowledging the WAITs makes the p16 program exact; serial and different-slot DMA/ECC overlap is exact for BCH8 and BCH16. Writing a buffer during its own ECC operation drops word-aligned bursts and sets FMC_STATUS bit 25. See p17.

16.17 P18: timeout rule, +400 words, BCH8 radius

The start-word timeout is linear in the full count field. The +400 storage words read zero without aliasing. BCH8 corrects up to eight bit errors anywhere in data, metadata and parity and leaves data unchanged beyond that in the four tested cases; ECC_RESULT does not report the current sector with the original read sequence. See p18.

16.18 P19: BCH16, status byte and ECC status interface

BCH16 is the t = 16 narrow-sense code of the same field. The latched status byte follows busy/ready; 0x70 is the no-chip echo. Per-sector decode status is 0x810 bits 3/5 and the 0x818 running total; 0x80c is a read-advanced ring. See p19.

16.19 P20: result ring and BCH16 radius

0x80c is an eight-slot ring advanced by its own reads and realigned by 0x81c; BCH16 corrects up to 16 errors. See p20.

16.20 P21: ECC status bits and WAIT4

810: bit 1 encode level (WAIT4), bit 2 decode, bit 3 uncorrectable, bit 5 ≥ 7 corrected, bit 6 erased; ack 1 clears only bit 0. ECC_CTRL bits 15:8 are the metadata length. See p21.

16.21 P22: WAIT4 sources, engine clock

WAIT4 follows 810 bit 1 (encode) or bit 2 (decode). Instruction timing and the timeout step agree on a 33.25 MHz engine clock (inferred, M2-103). See p22.

16.22 P23: branch targets, JR hazard, no ready/busy WAIT

BNZ targets are byte offsets with the low 3 bits ignored; every JR fell through with its target register written by the preceding instruction, later explained by the JR condition r[R] ≠ 0, not by a register hazard (M2-108, M2-110); no WAIT follows NAND ready/busy (M2-105, M2-107). See p23.

16.23 P25: JR, flags, DMA timing, prescaler, parity registers

See p25.

16.24 P43: round 5 and the CPU-only NAND path

All 18 families executed with every requested reset verified. Round 5 settles the high-bit selector and latch rules (M2-674..M2-681), the boot-ROM view of `ldi` (M2-682, M2-683), opcode 0x1a and the `nop` hold (M2-684..M2-686), the C48 compare and `tmwait` repeatability (M2-687..M2-690), the reset-step origin of the timeout phase (M2-691, M2-692), the DMA select, overlap, 038 and 0x340 questions (M2-694..M2-705), FMC_STATUS bits 19:12 by length (M2-706, M2-707) and 0x814 under BCH16 (M2-708..M2-710). C08 while running remains a single sample per run (M2-693). The CPU alone reads, programs and erases both page formats with hardware or software ECC, equal to the sequencer readers (M2-711..M2-716); 208 of 211 checks match, 1 is an observation, and the 2 mismatches are the location-order expectation (M2-714). See p43.

16.25 Reboot sessions: unmapped bus accesses

A DMA from 0x50000000 completes and leaves the device responsive (M2-717); an `ldi` from the same address hangs the whole system, also in stall-safe mode, and only a power cycle recovers it (M2-718). Both are the opposite of M1, which settles M1-F059 (chapter 19 now has no unresolved finding). See docs/147.

16.26 P48: round 6

The CPU sampler shows C08, FMC_STATUS bits 19:12 and FMC_DMA_CTRL bits 12:6 changing while the operation runs (M2-719..M2-722). `wait2` 32 and 33 follow the hardware VIC1 lines 0 and 1 (M2-723). The PC is at least 25 bits wide (M2-725). Apple's original ERASE and WRITE programs run correctly on the device (M2-726). The clock-gate family lost USB after four cases and the CPU-direct family was not analysed. See docs/148.

16.27 P49: CPU-issued command sequences

With the corrected FIFO set-up (M2-730) every required round trip reads back exactly; the qualified command variants are listed in M2-731..M2-740, and the fastest qualified data cycle is 28.24 ns (M2-731..M2-741). See docs/149.

The gate family returns baseline and gates0..3 with successful cleanup, then FREE remains queued while USB status polling works (M2-742). Direct worker dispatch handles injected timer starvation in the native model, and the host now retains scheduler diagnostics on queued timeouts; the physical cause remains unresolved (M2-743). See docs/150.

16.28 P50: clock gates and VIC0 line 5

Baseline and gates0..108 return END with exact clock restoration and successful reset/cleanup (110 cases; 67 gates initially enabled, 42 already disabled). VIC0 line5 is high before and after restoration, but no during-pulse samples exist, so these results exclude no gate as its source (M2-744). The queued FREE stall after gate3 does not recur. USB is lost submitting the next RUN; its acceptance and execution are unknown. Gate109 is the intended next case, and no gate109..159 result exists (M2-745). See docs/151.

16.29 P51: gate3 request and entry history

Four cases complete through gate2, then USB is lost submitting the gate3 RUN. The journal proves that manifest is byte-identical to p50's successful gate3 request; the lost RUN has no observed acceptance or execution (M2-746). P50 reports EXECFIRMWARE entry and p51 does not. Image identity and CPU state were not captured, so this is an entry-history association, not a cause (M2-747). The new checkpoint build records those missing distinctions (M2-748; docs/152).

Chapter 17 — Open questions

Summary: every point that the other chapters mark as open, not measured, unexplained or only inferred, in the form of the M1 book's chapter 17. Each entry gives what is known so far, the finding IDs of chapter 18 and the chapter with the details. Nothing here is a new finding, and each open point is listed once. The comparison with M1, finding by finding, is chapter 19. Points that were open in the first M2 survey and are now settled are listed in section 17.8.

17.1 Instruction set and sequencer behaviour

#	open point	known so far	details
2	Exact PC width above 2^24; instruction fetch from an invalid or unaligned target	branch and JR targets are byte offsets with the low 3 bits ignored and land exactly up to 2^24 + 0x100, so PC ≥ 25 bits (M2-105, M2-122, M2-725). Not measurable with this harness: a jump beyond 2^20 that does not wrap executes unbounded memory, and a fetch from an unmapped address is inferred to hang the bus, which stops USB before any result reaches the host (M2-611)	chapter 4, chapter 5
4	The condition that `nop (08)` waits for with R = 8..0xff	the hold is a wait state: C08 reads 4 in it, as in an unmet `wait 3`, and it ends only at the timeout fault (M2-625, M2-685, M2-686); R = 0..7 continue at the cost of a passing wait (M2-378); no command, address, DMA, ECC or status flag releases R = 8..15 (M2-625). The condition is unknown because no stimulus the harness can generate releases it, as for the waits of item 9	chapter 6, chapter 7
6	The origin of the 0.25-cycle fractions; the reset steps that together set the prescaler phase; the mechanism that places a wait's fault at the (count + 2)-th and a `tmwait`'s end at the (count + 1)-th instant after the entry	engine clock ≈ 33.25 MHz (inferred, M2-103); costs add linearly per copy: 1.00 cycle for `nop2`, `movi`, `mov`, add, an untaken and a next-taken `bnz`, 1.25 for `nop (08)`, a passing `wait 11` and a passing `wait 0`, 2.03 for `ld`, `st`, `wr` and `pld`, 3.25 for `sti`, 2.25 for `sub` + a backward taken `bnz` (M2-102, M2-378, M2-387); timeout instants are spaced one step apart and fixed to a prescaler phase that the per-case reset sets (M2-433, M2-588, M2-631); of its single steps only C00 = `0x80000` brings the first instant near the reset value, and none reproduces it (M2-691, M2-692); bits 7:4 = n give 2^(n+3) cycles per step for n = 4..11 (M2-115, M2-216). The prescaler and the per-instruction fractions are not readable: the harness sees them only through fault and end instants at 2 us resolution and through loop durations, which show their sums, not their origin	chapter 5, chapter 7
#	open point	known so far	details
7	Final instruction and stall mechanism of the `regnum` case after which USB communication was lost; whether the reserved-field loss was the `ldi` from 0x11111110	no output retained (M2-35); an r8 write that sends the PC outside mapped memory reproduces a worker/USB stall only under injected CPU-bus faults (M2-36, emulated); the stall ends communication before any result reaches the host, so the executing	chapter 16

#	open point	known so far	details
		instruction cannot be named; a second loss came while the reserved-field forms of item 1 ran: no record reached the host and the device LCD showed partly drawn characters; which case ran is not recorded; run one per probe with valid addresses every form ends normally, and the only form that leaves mapped memory with the register values of the lost run is `ldi` with address selector 0x105, which uses r0 = 0x11111111 (M2-611, M2-615; bus hang inferred)	
8	Full bit maps of C00 and C5C; what the C08 values 1, 2, 8 and 9 read while a program runs encode; what the value `0x7f` of C14 is (a constant or a value held elsewhere)	start-word bits 1, 3, 16, 19 and the step field measured, the others without effect (M2-153, M2-157, M2-158, M2-201); C58 stores mask d6, bit 2 selects timeout vectoring, bits 1, 4, 6 and 7 are stored without effect (M2-11, M2-70, M2-148, M2-149, M2-612); C3C = faulting offset + 8; C48 acts on `tmwait` as a compare value (M2-179, M2-627, M2-688); read inside the state, C08 is 4 in every unmet wait, `wait2`, `tmwait` and `nop` hold, 0xa in `pause`, 0xb in a blocked `evjmp`, 3 frozen or aborted, 1 held by `C60` (M2-14, M2-56, M2-57, M2-62, M2-70, M2-121, M2-158, M2-384, M2-685); 3 ms into a loop of one instruction kind it reads `nop2` 0x9/0x9/0x2, `sti` 0x8/0x1/0x8 and `ld` 0x1/0x1/0x1 in three runs (M2-693); C00 bit 18 is set in every running, waiting, paused or held state; C14 reads `0x7f` after a wait released after one or more steps, after a timeout fault and at power-on (M2-172, M2-177, M2-430). Harness limit: the CPU reads C08 once per run at an instant whose phase within the running loop it does not control, and the value depends on that phase, so the running values cannot be assigned to instructions or pipeline steps; sampled by the CPU while a program runs C08 reads 1, 2 and 9 in ALU and `nop2` loops, 7 in an SDRAM-access loop, 5 in a window-access loop and 4 in an unmet wait, changing within one loop (M2-719)	chapter 8

17.2 Events and the status latch

#	open point	known so far	details
9	Sources of WAIT 6..10 and 13..15	no tested stimulus releases any of them: not NAND ready/busy (M2-107), not NAND, ECC, DMA, status or metadata states (M2-111), not CPU writes to C0C, C10, C48, C54, C60..C6C (M2-120, M2-125, M2-143), not unacknowledged DMA, data or spare reads, a FIFO READ ID, a metadata write, BCH8 slot-1 and BCH16 encodes, 0x81c = 1, commands on CE1..CE3, a pending `sig`, uncorrectable and erased-sector decode completions, a command routed by a CPU-written 0x044 = all ones, CPU writes of C40, C44, C50, C14, C5C or C3C, or VIC0/VIC1 soft lines (M2-386); WAIT8 not after a page read with DMA out (M2-206); WAIT15 fails after every clear-enable stimulus in both orders (M2-69); event numbers 16..255 never pass (M2-159)	chapter 7
10	The condition `wait 11` tests	it passes at once in every tested state (idle with C58 0 or 4, timeout disabled, DMA in flight, encode running, command or status latch pending, every flag acknowledged, C10 = 0, 0x044 all ones) and on every iteration through NAND array busy periods (after 30h, FFh and an erase start), in every operand form, at 1.25 cycles, the cost of a passing `wait 0` (M2-10, M2-69, M2-107, M2-387); no tested state makes it wait	chapter 7
#	open point	known so far	details
11	Why a latch written one or two instructions after an unwaited `70h` sets `FMC_STATUS` bit 3 instead of bit 23	a latch needs no re-arm: after its `wait 5` and only the bit-23 acknowledge a second latch passes with no operation between and after each of seven operations (M2-494), and after the `0x30020` release after any or no operation (M2-237); after a latch without the release a new command is refused until an `FMCTRL1` write, with bit 26 clear (M2-181, M2-202), and an address operation does not complete and sets bit 26 (M2-610), while a DMA in or out, a data, parity or FIFO read, an encode and a second latch complete as after the release (M2-641); status clear, ECC kick and FFh do not release (M2-181); with an unwaited `70h`, a latch with 0, 3..20 `nop`2, 20 or 200 instructions between passes, one after one or two `nop`2, one `nop`, one `movi` or the acknowledge faults with bit 3 set and bit 23 never set, after a	chapter 7

#	open point	known so far	details
		reset and after any of eight predecessor runs (M2-68, M2-178, M2-390, M2-593); with the 70h waited for and acknowledged every tested idle ordering passes (M2-130); the standard latch never fails (M2-161)	

17.3 Interrupts

#	open point	known so far	details
13	The device behind VIC0 line 5 (gates0..108 have before/after samples only, with 42 already disabled; none excluded by M2-744; no gate109..159 result, M2-745); VIC1 lines other than 0 and 1 asserted by hardware (line 52 never rose, M2-724); an ECC interrupt source and enable outside the tested words	wait2 follows VIC0 hardware lines 5 (always), 8 (timer), 16 (DMAC0), 19; soft interrupts never release it (M2-74, M2-140, M2-167); FMC flags routed by 0x044 bits 0, 1, 2, 11, 12 drive line 20 and CS_IRQ bit 4 (M2-142, M2-200, M2-204); wait2 32 and 33 follow hardware VIC1 lines 0 and 1, each only its own (M2-723); without a hardware line conditions 32..255 never pass (M2-74, M2-211); END sets VIC1 raw bit 22 while C10 ≠ 0 (M2-182); no ECC encode or decode raises a VIC line or CS_IRQ bit 5/6 for any tested enable (0x800..0x9fc, single bits, ECC_CTRL and 0x810 extra bits; M2-212, M2-217); sig raises no VIC line (M2-370); 28-us pulses on enabled gates 1/2 leave VIC0 raw 0x20 before/during/after; gate0 was already off, and no record returns from gate3 (M2-729). With the corrected pulse, gate3 also returns with exact restoration and VIC0 raw 0x20 before/after; the following FREE stays queued while USB works (M2-742). The pulse cannot identify a transient line change while gated (M2-730); the scheduler starvation cause is unresolved (M2-743). An identical gate3 request later precedes USB loss without the EXECFIRMWARE entry marker; no RUN acceptance or image hash separates the causes (M2-746, M2-747). The checkpoint harness supplies pre-start USB evidence and CPU/code identity, with native validation only (M2-748)	chapter 7, chapter 9, chapter 10

17.4 NAND controller registers

#	open point	known so far	details
15	Why the buffer-select switch of a running DMA falls at a different word in identical runs (57..63 at 16 nop2) while the address switch does not, and why the word at the switch is buffer-1 word 120	an address rewrite takes effect at a multiple of 32 bytes, equal in repeats; a select rewrite at a word boundary no later (M2-671, M2-672); at 16 nop2 the switch falls at word 63, 57 and 58 and the word at it is buffer-1 word 120 in two of three runs, at 32 nop2 at word 114 in all three (M2-694..M2-696); every byte of an overlapped DMA is an input byte, the initial content or zero (M2-697, M2-698). Not measurable with this harness: the program sees only the RAM image after the transfer, one per run, not the buffer read and RAM write order inside it	chapter 9
16	What FMC_DMA_CTRL (038) bits 5:3 select and why a non-zero preset of bits 12:6 stops a DMA out after 32 bytes; whether bits 2:0 set the burst length and bits 12:6 count words (inferred)	with bits 2:0 = v = 0..7 a DMA completes, more slowly for smaller v, and leaves bits 12:6 = (v + 1) × ceil(128 / (v + 1)) − 128 (M2-79, M2-609); any non-zero preset of bits 12:6 stops a DMA out after 32 bytes (M2-669); of bits 5:3 only 0 and 1 let a DMA out complete, 2..7 stop it after 32 bytes holding source words 0 1 0 1 2 3 4 5 (M2-669, M2-700); with 0xf a DMA out repeats each burst's first word and a DMA in stores buffer word 4g + j from source word 4g + 5j (M2-669, M2-701; four-beat bursts with 20-byte beat steps inferred). Not measurable with this harness: it sees the completed buffer or RAM and the 038 readback, not the bus beats, so the field meanings stay inferred from word orders; bits 12:6 read non-zero while a DMA runs and 0 again after it (M2-722)	chapter 9
18	Roles of 018 and the 0x100 block; the role of FMC_CTRL0 bit 0	018 stores 4 bits and changes no loopback or page read (M2-104, M2-405); 0x1b8..0x1d8 writes change no measured timing or data (M2-160, M2-210); bit 0 is required for a data transfer to complete, its data-path-enable meaning inferred (M2-135). Unknown because no measured transfer, timing or flag depends on these words beyond what is listed	chapter 9
#	open point	known so far	details
19	Combined bits of the 004 operation words beyond their measured effects; what FMC_STATUS bits 19:12 encode; what else sets bit 26	single FMCTRL1 bits: bit 0 starts an address operation, bit 1 a data read, bits 8 and 9 a DMA, the other 28 nothing alone (M2-411); bits 19:12 read 0xc3 after reset, a DMA in, a status latch and the set-up word, 0x41 after a read of 1..224 bytes or a	chapter 9

#	open point	known so far	details
		parity read, 0x14 after 255..508 bytes and 0x3c after 511 or 512 (M2-203, M2-497, M2-642, M2-706, M2-707); bit 26 sets when an address operation does not complete after an unreleased latch (M2-181, M2-610, M2-641). The three values change at length boundaries only, so the bits read as a state code rather than a count, and no other register shows what that state is; combined operation words beyond the original ones start operations that the harness cannot read out bit by bit; sampled during a read the field goes 0xc3 → 0x41 → 0x14 → 0x3c as the read advances (M2-720)	
20	Coherency of the CPU cache with sequencer and DMA accesses	a DMA from the unmapped address 0x50000000 completes and an `ldi` from it hangs the whole system (M2-717, M2-718); `ldi`/`sti` addresses (M2-05) and DMA RAM addresses (M2-664) align down to a multiple of 4; no measured snooping provides coherency	chapter 4, chapter 9
21	The M1 channel-1 pin mapping on S5L8720; a second controller channel elsewhere than +0x400	no register set at +0x400: its words read 0 after each of 17 candidate enable writes, and commands, READ ID and DMAs through it do nothing (M2-82, M2-183, M2-185, M2-393); four attached CEs do not imply two channels; a sequencer `ld` of 0x40c costs the same 2 cycles as one of 0x00c or of the unused 0x60c, so access time does not show whether +0x400 is decoded (M2-596)	chapter 9

17.5 ECC engine

#	open point	known so far	details
23	810 bits 13..15, 17..23 and 25..30 and the 80c bits outside the error count; the role of 814	810 bits 0..3, 5, 6 assigned (M2-88, M2-94, M2-95, M2-97, M2-99, M2-100); bit 4 reads 1, bits 7..12, 16, 24 and 31 are never set (M2-420); only bit 0 of an 81c write acts (M2-91, M2-96, M2-421, M2-645); 808 reads 0x700 (M2-116), 810 bit 5 threshold role inferred (M2-434); 814 stores mask 0x2f (M2-191) and leaves BCH8 and BCH16 encode parity, decode registers and decoded data unchanged (M2-660, M2-708..M2-710). Unknown because no tested encode, decode, error pattern or format sets the unassigned bits or depends on 814	chapter 10

17.6 Original programs and the CPU interface

The CPU-only read/program/erase results of M2-711..716 remain qualified for their measured paths. The generic sequence path subsequently exposed a DMA/FIFO mode-selection defect despite successful resets and ready statuses (M2-727, M2-728). Its correction is validated in the native model, without a new device readback result. The command reference profile exercises cache, copyback, multi-plane and discovery templates, but does not establish optional NAND support on the installed chip (M2-730).

#	open point	known so far	details

17.7 Reset stages

No point remains open here: FMCTRL1 bit 31 zeroes FMC `00c..05c` and sets FMC_DMA_CTRL to 7, and `0x81c` bit 0 zeroes ECC_CTRL, which accounts for what the complete reset does to these words (M2-123, M2-645; chapter 8).

17.8 Settled since the first M2 survey

These points were open in the first M2 survey or in an earlier version of the items above; the reference chapters give the results. Former item numbers are not reused.

point	findings
PC width (former item 2, part): at least 25 bits	M2-725
`wait2` and VIC1 hardware lines (former item 13, part): condition 32 + k follows VIC1 line k for k = 0, 1	M2-723
original WRITE/ERASE on hardware (former item 24)	M2-726
unmapped bus accesses (former item 20, first part): a DMA from 0x50000000 completes, an `ldi` from it hangs the system	M2-717, M2-718
Source selector 10 = destination register in every MOV/ALU form	M2-44, M2-52, M2-141
BNZ/BZ and JR target encoding; JR condition (r[R] ≠ 0)	M2-46, M2-105, M2-106, M2-109, M2-110, M2-122
No register-write hazards	M2-108
Undefined opcodes `1a..fe`: same outcome as an unmet wait, no separate trap	M2-56, M2-81, M2-282
`sig` (`10`) and `nop2` (`ff`) in the three tested forms, idle, DMA and ECC	M2-34, M2-65
Timeout length rule and start-word bits 7:4	M2-81, M2-103, M2-115
Instruction costs and engine clock	M2-09, M2-102, M2-103
STOP and reset-before-parameters	M2-53, M2-54, M2-55
Recovery from a held state and from C60 written while running	M2-61, M2-62, M2-63, M2-121, M2-126
Repeated waits, acknowledgement and rearm	M2-66
WAIT4 release: encode 810 bit 1, decode 810 bit 2, W1C	M2-67, M2-97, M2-99
No WAIT follows NAND ready/busy	M2-107
04c is chip status (0x80 busy, 0xe0 ready, 0x70 no-CE echo); status latch polls until ready	M2-87, M2-130, M2-136
WAIT12 condition: status & arg & mask ≠ 0; release on a busy chip only at ready	M2-68, M2-134, M2-137
FMC and C50 operation flags	M2-112
Serial DMA exact; cause of overlap payload loss	M2-71, M2-76, M2-78, M2-114
NAND read, write, command and address cycle timing	M2-119, M2-127, M2-131, M2-135
FMC_CTRL0 stored and inert bits	M2-135, M2-139
FIFO READ ID	M2-124

point	findings
BCH8 code and correction radius 8	M2-73, M2-77, M2-84
BCH16 code and correction radius 16	M2-86, M2-92
Per-decode results: 80c ring, read advance, 81c reset	M2-85, M2-90, M2-91
810 status bits (done, uncorrectable, ≥ 7 errors, erased) and 818 total	M2-88, M2-89, M2-93, M2-94, M2-95, M2-96, M2-100
ECC completion after repeated encodes	M2-67, M2-72, M2-97
ECC_CTRL single bits	M2-98, M2-101
Encode parity registers 0x840..0x85c	M2-116
Decode error locations: data, metadata, parity order and numbering	M2-128, M2-132, M2-138
DMA, ECC and decode timelines	M2-117, M2-118, M2-129, M2-133
Shift counts: every count ≥ 32 gives 0, no count aliasing	M2-146
Timeout step for n = 0 (fault at once), n = 2 and n = 3 (2^3, 2^5 cycles)	M2-115, M2-216
end and the ignored fields of `wr`, `st`, `ldi`, `mov`, `movi`, `sti`, `pld`, `pst` in the tested values	M2-164, M2-165
`pld`/`pst` offset bits 11:0 and unaligned offsets	M2-175
PC writes through `0xc38` by the program and the CPU	M2-162, M2-278
Start-word bits 1, 3, 16, 19 and the bits without effect; no priming start after a reset	M2-151, M2-153, M2-157, M2-158, M2-201
`C58` bit 0 and END diversion: no value diverts an END	M2-148, M2-149
`C3C` in a fault handler, also nested; `C3C`/`C48`/`C54` widths and retention over a start	M2-154, M2-179
`C14` ignores writes	M2-176
`C50` bits 0..5 mirror pending events	M2-152, M2-209
`wait` event numbers 16..255 and `wait 8`	M2-159, M2-206
WAIT5 orderings: faults only for a latch one or two instructions after an unwaited `70h`; every idle ordering with a waited `70h` passes; the standard latch never fails	M2-68, M2-130, M2-161, M2-178, M2-593
Command refused after a latch until an `FMCTRL1` write; two-chip polls; latch after `90h`	M2-181, M2-198, M2-202, M2-239
Status latch on `FMC_DATA_SEL` `0x100` and `0x200`	M2-169, M2-195, M2-196
`evjmp` events arriving during the wait, pending events and `C58`	M2-186
`wait2` ignores arg/imm; no NAND busy line; VIC1 conditions never pass	M2-167, M2-187, M2-211
FMC interrupt routing: `0x044` bits, VIC0 line 20, `CS_IRQ` bit 4	M2-142, M2-200, M2-204, M2-208
END sets VIC1 raw bit 22 while `C10` ≠ 0	M2-182
No register set at `+0x400`: reads 0, commands and DMAs through it do nothing	M2-82, M2-139, M2-155, M2-183, M2-185
Window masks, offset bit 9 decoding, the `0x100` block and `0xd80..0xdfc`	M2-147, M2-154, M2-155, M2-160, M2-191, M2-210
FMC `0x028` is a 9-bit storage word, not a counter	M2-156, M2-173, M2-199
`FMC_STATUS` bits 12/13/18/19, 24 and 27	M2-145, M2-163, M2-203
DMA length independent of `FMC_DNUM`; parity buffer 64-byte wrap	M2-189, M2-232
`FMC_CTRL0` bit 24 clear: no DMA completes and, left clear until END, no byte reaches RAM	M2-184, M2-251

point	findings
Four-cycle erase; programs and erases on CE0 and CE1	M2-168, M2-194
ECC_CTRL combinations: no code other than BCH8 and BCH16	M2-197
M1 ECC area 0x900..0x9fc reads 0 on M2	M2-171
No miscorrection beyond the radius; all-zero chunk decodes clean	M2-174, M2-214
Original WRITE DMA-list prefetch	M2-213
pld/pst reach: offset bits 11:0 only for every bit 12..31; r0..r7 through 0xc18..0xc34 (former item 5)	M2-175
evjmp (16) operand forms and sources; no +0x400 flag; every C00 order on a blocked start = 1 evjmp (former item 12)	M2-62, M2-186, M2-391, M2-392
FMC_CTRL0 bit 24: a DMA programmed with it clear is held and starts when it is set again, with the address registers at that moment (former item 14)	M2-184, M2-251, M2-338
A DMA writing or reading a buffer under its own encode: kept old bytes, lost output runs, parity and FMC_STATUS bit 25 (former item 17)	M2-80, M2-399, M2-401
ECC slot selector 01c: bit 0 selects the slot; back-to-back kicks; a slot change during an encode (former item 22)	M2-417, M2-418, M2-419
Power-on window: equal to the window after the ordinary reset except C00 and C04; C14 0x7f; the first program runs on its first start (former item 25)	M2-430, M2-431
sig routing and cost; tmwait time from the start-word count; erased words as nop2	M2-367, M2-370, M2-373
Instruction costs per copy; C08 after a fault and in a handler; C14 load and count rule; C48 and tmwait	M2-172, M2-177, M2-179, M2-378, M2-384
WAIT5 fault orderings independent of the preceding program	M2-390
FMCTRL1 single bits; 048 and 04c writes; FMC_STATUS bits 19:12	M2-203, M2-411, M2-412
FMCTRL1 DMA word replaced by a read	M2-332
FMC_DMA_CTRL (038) readback and fault bits	M2-396, M2-398
810 never-set bits, 81c bit 0, CPU reads of 80c advance the ring	M2-91, M2-420, M2-421
Two-plane program and erase; CPU-only NAND operation	M2-339, M2-435, M2-436
Undefined opcodes 1a..fe with a pending command, address, DMA, ECC or status flag: unmet wait (former part of item 3)	M2-583
tmwait entered later in a run; count-0 tmwait; C48 and tmwait in every run (former parts of item 4)	M2-179, M2-584
Timeouts of waits entered late in a run, counted from the entry; the step for start-word bits 7:4 = 1 (former parts of item 6)	M2-115, M2-584, M2-588
C08 read inside running and waiting states; C58 bits 6 and 7; C14 kept after the END (former parts of item 8)	M2-172, M2-384, M2-612
A second status latch needs no re-arm; an address operation after an unreleased latch fails with bit 26; the timing window of a latch after an unwaited 70h (former parts of item 11)	M2-494, M2-593, M2-610
Register rewrites during a running DMA: address and buffer select followed, FMC_DNUM and a repeated FMCTRL1 write ignored (former part of item 15)	M2-394, M2-603
FMC_DMA_CTRL bits 12:6 during and after a DMA; readback of 2 and 4 as 0x42/0x84; no value of bits 2:0 stops a DMA (former parts of item 16)	M2-398, M2-609
A DMA in with FMC_DATA_SEL = 0 writes neither data buffer; CPU writes of 058/05c read back (former part of item 18)	M2-597, M2-599
The set-up word restores FMC_STATUS bits 19:12 after a short read (former part of item 19)	M2-203, M2-497

point	findings
Ring-advancing copy of `0x80c` at `0xa0c`; `0x90c`/`0xb0c`	M2-91
High source- and address-selector bits of `and`, `or`, `sub`, `shl`, `shr`, `mov`, `ldi`, `pld`, `pst`; loads and immediate ALU results with destination r10 or r8 (former parts of item 1)	M2-364, M2-614, M2-615, M2-616
`nop` (`08`) operand map: R = 0..7 continue, R = 8..0xff stall, no pending flag releases them; the `nop` stall time (former part of item 4)	M2-377, M2-625
`C48` single bits and `tmwait`; `C14` writes and `tmwait` (former parts of item 4)	M2-373, M2-627, M2-629
Prescaler phase set by the per-case reset; first instants for n = 4..6 (former parts of item 6)	M2-588, M2-631
`C58` bits 1 and 4; `C14` = `0x7f` after a released wait for every count, step field and step number (former parts of item 8)	M2-177, M2-612
Operations that an unreleased status latch refuses: a command and an address operation only (former part of item 11)	M2-641
Switch offsets of address and select rewrites during a DMA (former part of item 15)	M2-671, M2-672
FMC_DMA_CTRL presets of bits 12:6 and 5:3 before a DMA; bits 31:13 not stored (former part of item 16)	M2-669
`058` and `05c` as bytes of FMC_DATA_SEL; DMA-in targets of every tested FMC_DATA_SEL value; data buffer 0 after FMCTRL1 = `0x340` and FIFO writes (former parts of item 18)	M2-617, M2-643, M2-668
The `0x550d` readback of FMC_DATA_SEL; FMC_STATUS bits 19:12 against the transfer length (former parts of item 19)	M2-642, M2-643
DMA RAM address alignment (former part of item 20)	M2-664
Role of 814 in BCH8 decode and encode; parity with metadata length `0xff` (former parts of item 23)	M2-660, M2-662
Reset steps that zero FMC `00c`..`05c` and ECC_CTRL: FMCTRL1 bit 31 and `0x81c` bit 0 (former item 26)	M2-123, M2-645
High-bit source selectors read the last source value; discarded extended destinations and the `st` latch; `jr` with high target-selector bits (former parts of item 1)	M2-618, M2-622, M2-673
Extended destinations 9 and 11..15 with extended sources 8 and 10: no GPR written, latch unchanged; `sti` with high address-selector bits: the last source value; `ld`, `st` and `wr` leave the value a high-bit source reads (former item 1)	M2-674, M2-675, M2-676, M2-677, M2-678, M2-679, M2-680, M2-681
Boot ROM through `ldi` at `0x2000xxxx`: one ROM word per 16-KiB quarter, never the addressed word (former part of item 2)	M2-682, M2-683
Operand fields of the undefined opcode `0x1a`: no effect in 64 random words (former item 3)	M2-684
`nop` hold for R ≥ 8 is a wait state; C48 compare rule at count 7; the isolated early `tmwait` ends are run-to-run spread (former parts of item 4)	M2-685, M2-686, M2-687, M2-688, M2-689, M2-690
No single reset step sets the prescaler phase alone (former part of item 6)	M2-691, M2-692
Word at a buffer-select switch: buffer-1 word 120; overlapped DMAs contain only input bytes, initial content or zero (former parts of item 15)	M2-694, M2-695, M2-696, M2-697, M2-698
FMC_DMA_CTRL bits 5:3 = 3, 5, 6 and the DMA in with `0xf` (former part of item 16)	M2-699, M2-700, M2-701
DMA-out FMC_DATA_SEL values; the transfer after FMCTRL1 = `0x340` completes by time alone and stops at an FMC_DATA_SEL write (former parts of item 18)	M2-702, M2-703, M2-704, M2-705

point	findings
FMC_STATUS bits 19:12 for read lengths 160..508 (former part of item 19)	M2-706, M2-707
814 and BCH16 encode and decode (former part of item 23)	M2-708, M2-709, M2-710
CPU-only NAND read, program and erase without a sequencer program, with engine and software ECC, both formats; decode location order within a byte	M2-711, M2-712, M2-713, M2-714, M2-715, M2-716

References: chapter 14 (outcomes), chapter 13 (what was done), chapter 15 (interpretation).

Chapter 18 — M2 finding and documentation ledger

Rows are grouped by the device run that first established them (p06 to p51 and the power-on capture), with the retained earlier physical captures, original M2 source analyses and the harness/model. A later row that refines or supersedes an earlier one is named in the earlier row. **D** = device observation, **S** = original-code use, **I** = interpretation, **M** = model validation. Each statement is limited to the described evidence, not all possible encodings.

ID	Finding	Class	Evidence / reference
M2-01	Eight-byte little-endian arg/R/op/imm format	S,D	original programs; p06 instruction latches; architecture chapter
M2-02	Eight GPRs at c18..c34, r6=result	D	p06 regmap/control
M2-03	d00..d7c retains 32 CPU-written words	D	p06 pram
M2-04	Tested d80..dfc writes do not persist	D	p06 pram; all 32 words: M2-147
M2-05	Tested RAM words are LE and align down	D	p06 memory and holdout
M2-06	Immediate-zero ALU selects register form; arithmetic wraps	D,S	p06 alu; original program dataflow
M2-07	Logical shifts, tested counts 1/31/32/33	D	p06 alu/holdout; counts ≥ 32: M2-146
M2-08	18/19 access window indirectly; 1d64 aliases d64	D	p06 indirect; holdout reads r6; offset rule M2-175
M2-09	Measured loop slope ≈33.8 ns/instruction	D	p06 clock; clock frequency remains inferred
M2-10	Idle WAIT11 passes; other tested 0..15 fault ~126.6 ms; WAIT11 in other states and operand forms: M2-387	D	p06 wait; operands/state qualified
M2-11	C58 bit 2 chooses vector/continue for measured timeout	D	p06 c58
M2-12	Tested C58 readback fits d6; bit 0 not start-at-vector	D	p06 c58/resume
M2-13	`tmwait` (15) stalls through 24 short observations (C08 = 4, C14 = 0x35 at the end of the observation)	D	p06 op15; completion M2-27, consecutive tmwaits M2-150
M2-14	Two `pause` (12) forms pause with IRQ8/C08=a/PC retained	D,I	p06 op12/op12wait; pause interpretation also matches CPU source
M2-15	Native cleanup fails after those pause samples	D	worker -2; later records lack execution
M2-16	STOP-persistent pause reproduces native failure cascade	I,M	test_p06 through compiled ARM batch/reset
M2-17	Zero-field `nop` (`08`) permits following marker and END	D	p06 startseq3; other operands: M2-377, M2-625

ID	Finding	Class	Evidence / reference
M2-18	c50 no-fault handler control reads 0x00ff1800 before/after ack	D	p06 c50 control; fault candidates timed out too early
M2-19	Original WAIT0..4 / WAIT12 synchronize peripheral/status work	S,I	bootloader READID/READ and restore WRITE/ERASE
M2-20	M2 reset/start includes c60/c64/c6c handshake	S	original CPU driver; real ARM tests
M2-21	Two sector/ECC selections used in original pipeline	S,I	READ; selector universality/arbitration unmeasured
M2-22	Original BCH8/16 formats use 13/26 parity bytes per 512 B	S	mode setup and microcode; BCH8 algebra M2-77, BCH16 algebra M2-86
M2-23	The late register/ECC/opcode zero records of the first survey capture are not measurements	D	p06 classified audit, zero samples and worker -2
M2-24	The first survey capture uses bulk/resident mode but lacks mutation capability	D	p06 metadata/capability word 7a0000 and refusals

18.1 P06 documentation topic review (historical)

The M1 reference was used as a topic checklist, not copied as M2 truth. The finding-by-finding comparison with M1 is chapter 19.

Topic	Current M2 destination	Review outcome
Overview, terms, evidence	README; 01; 15	M1 evidence vocabulary, numbering and notation retained; M2 execution qualification explicit
Execution architecture	02; 08	M2 handshake and observed pause separated from M1 reset/priming
CPU/sequencer memory maps	03; 04; 09; 12	known windows, GPR/PRAM, excluded FIFO reads and unresolved aliases
Instruction encoding and each opcode	05; 06	all 27 M1 mnemonics retained as matching or explicitly provisional M2 names; per-opcode syntax, encoding and evidence
Events, timers, waits	07	source mapping versus idle observations; three timeout layers
Control registers	08	state/IRQ/PC/vector details; no imported full bit map
NAND controller and DMA	09	original command words/slots; candidate channel map unresolved
ECC	10	M2 layout/control; synthetic-model limit and missing BCH algebra
Original NAND programs	11	source addresses, operation order and remaining pipeline gaps
Parameter/result ABI	12	software-defined parameters; worker versus hardware results
Every experiment entry	13	159 captured entries classified; 160 current entries, migration gaps retained
Evidence/model	14; 16	capture hash, provenance, 89 current tests and persistent-state regression
Findings/open questions	17; 18	targeted recovery first, then one full campaign

Historical numbered workspace chapters remain dated records. The active reference is this separate `docs/nano4g/FMSS_M2/` directory; docs/92 records the p06 update without rewriting older measured totals or image hashes.

18.2 p07 additions

ID	Finding	Class	Evidence
M2-25	Nine pause-recovery controls match; explicit CPU resume reaches following marker/END for both measured forms	D	p07 m2_recovery
M2-26	Trusted-END cleanup succeeds in direct controls; inherited pause needs combined abort fallback	D	p07 diagnostics, methods 1 and 3
M2-27	All 24 tmwait forms complete in 126,101–126,103 us with C14=0	D	p07 op15
M2-28	Faulting C50 controls read 0x00ff1800 before and after writing 1	D	p07 c50
M2-29	First ev5 case reaches fault-handler END, then reset fails while C08=0	D	p07 ev5/0000, stage 4 cleanup rc=-3
M2-30	Later probes fail initial reset; all broad opcode matrices remain inconclusive	D	p07 native-case audit

The earlier p06 ledger entries remain dated observations; p07 supersedes their unresolved resume/short-window questions only for the explicitly measured cases.

18.3 p09 additions and corrections

ID	Finding	Class	Evidence / reference
M2-31	Three two-latch controls fail reset phase 4, FMCTRL1=2, and method 4 restores following execution	D	p09 m2_status_recovery
M2-32	Status ordering variants stall at second WAIT0; first WAIT5 completes; NSTAT-consumption hypothesis withdrawn	D	sampled PCs a0/a0/70, p09 status controls
M2-33	Five immediate-ack controls END, invalidating their old FAULT expectations	D	p09 m2_opcode_confirm
M2-34	78 M1-comparison expectations match only their tested forms/states	D	p09 m2_opcode_m1
M2-35	regnum has no returned result; later USB timeouts are not opcode evidence	D	p09 and supplied console
M2-36	Old r8 immediate escapes program if M1 PC semantics apply; injected CPU bus stall prevents timeout/USB progress	I / emulated	test_p09.py; no physical case attribution

18.4 p10: qualified comparisons with M1

ID	Finding	Class	Evidence / reference
M2-37	All 59 isolation requests END, IRQ1, cleanup succeeds	D	p10 m2_wedge
M2-38	MOVI r8 local target lands; MOV r2,r8 returns the instruction offset at two sites	D	p10 007,020,021; M1 semantics already known
M2-39	MOVI r10 leaves seven visible GPRs at 0x78; later marker overwrites r6	D	p10 009; all-eight broadcast confirmed by M2-43
M2-40	Selected argument-source, address-source and branch forms match M1 roles	D	p10 001..006,018,019; selector table M2-43, M2-47, M2-48
M2-41	Extended ST zero follows MOVI r6,0; this alone does not prove constant zero or the full M1 latch rule	D / I	p10 005,006; M1 chapter 4.3; latch rule M2-47

18.5 p11: M2 selector table comparison

ID	Finding	Class	Evidence / reference
M2-42	All 1,590 requests return; expected JR r8 self-loop timeout; no worker/reset/transport failure	D	p11 audit
M2-43	MOVI/MOV/ADD confirm local PC destination, all-eight-GPR broadcast, other extended GPR discard	D	p11 m2_regdest, 258 cases; p41 m2_r3_reserved_fields 8 (single probes; loads and ALU immediate: M2-616); extended sources: M2-678, M2-679
M2-44	Source 10 register-form AND/OR/ADD/SUB use destination r1 itself; differ from M1 r0 expectation	D	p11 argument-source/010, four stored words
M2-45	Source-10 register shifts unresolved: both candidate counts give zero	D / I	p11 d28/d30; resolved by M2-52
M2-46	Extended BZ/BNZ never branch in zero/nonzero controls; JR uses r0 except self-loop 8	D	p11 branch/jump matrices
M2-47	Extended ST latch matches M1 for all selectors and 13 predecessor contrasts at 9/14/200 (ordinary destinations; after a broadcast `mov r10` the latch keeps the value before the broadcast: M2-366; discarded extended destinations leave it: M2-622)	D	p11 m2_regst
M2-48	LDI/PLD address fallback matches; PC = 0x60 reads 0xe1a0f00e/0	D	p11 m2_regaddr/m2_regbus; no full ROM-map claim; p41 m2_r3_reserved_fields 3 (single probes; high-bit address selectors: M2-615)
M2-49	PRAM clears between probes; assignment to CTRL=80000 in model is inferred	D / I	p11 untouched PRAM words, native reset replay
M2-50	Reset-after-parameters adapter fails native NAND tests when reset clears PRAM; prepare-before-parameters corrects it	emulated / source	emCORE nano4g adapter; read/boot and program/erase controller checks; reset-phase clearing measured in M2-54, M2-55

18.6 p12: resolved source-10 and reset questions

ID	Finding	Class	Evidence / reference
M2-51	All 72 requests return; 64 scored matches, eight expected TIMEOUTs, all preflight/worker/cleanup succeeds	D	p12 audit
M2-52	Source 10 uses destination itself for all six ALUs in tested GPR forms, including small-count shifts	D	p12 64 cases / 1,664 outputs; 245 distinguish M1 r0 rule
M2-53	STOP clears PC while preserving seeded GPR/PRAM values	D	p12 running versus STOP controls
M2-54	STOP then C00=80000 clears GPR/PRAM/C04/C10/C40/C44/C4C; separate buffer/FMC/ECC steps preserve GPR/PRAM; 0x80000 alone suffices and no C00 sequence touches FMC/ECC words: M2-422	D	p12 reset-phase contrasts
M2-55	Reset-before-parameters is required by the measured clearing effect; driver hardware I/O not exercised by these runs	D / software implication	p12 plus p11 adapter correction

18.7 p13: evjmp and the reset failure

ID	Finding	Class	Evidence / reference
M2-56	Start=1 WAIT3/illegal1a stay at C08=4 without fault; cleanup succeeds	D	p13 ctrlfx/0000..0001
M2-57	Start=1 evjmp gives C08=b, PC20, IRQ0; STOP fails phase 1	D	p13 ctrlfx/0002
M2-58	Zero-field evjmp vectors for six primary FMC/ECC stimuli; idle faults	D	p13 op16, seven controls

At p13, abort/forced-reset recovery was implemented but physically unqualified; explicit simulator hypotheses are not device evidence. Later unsampled p13 records add no opcode findings. See p13 analysis.

18.8 P14: status checks versus clearing a blocked instruction

ID	Finding	Class	Evidence / reference
M2-59	All seven cases fail initial reset; method 8, C08=b, PC20, IRQ0, phase 1	D	p14 m2_resetcheck
M2-60	Abort passed a 0/3 check; forced reset skipped before STOP returned to b; exact intermediate value unrecorded	D + source inference	p14 diagnostics and bit 26 control flow

P14 did not measure reset while held; p15 below supplies that observation. Native tests retain clearing, holding and ignored-write alternatives.

18.9 P15: reset while held and independent execution

ID	Finding	Class	Evidence / reference
M2-61	All seven candidates execute; four markers match, three expected TIMEOUTs; all preflight/worker/reset/cleanup succeeds	D	p15 m2_resetcheck
M2-62	Inherited and fresh start1 zero-field evjmp: STOP leaves b/PC20/IRQ0; abort gives 3; forced reset before STOP gives 0; full reset phase 6 rc0; other C00 orders: M2-392	D	p15 baseline initial and evjmp16/start1 cleanup; method 01000318
M2-63	Next independent marker after each WAIT3/illegal1a/evjmp timeout reaches END with r2 = 0x72657365	D	p15 after-reset cases

P15 qualifies this sequence, not every possible control word, operand or reset ordering. Default-model native replay covers all seven records; timing and complete C00 readback are not included in that equivalence.

18.10 P16: full execution and qualified hardware findings

ID	Finding	Class	Evidence / reference
M2-64	172 families/31001 sampled records; zero setup/reset/cleanup errors	D	p16 full audit
M2-65	531 core +78 M1 opcode comparisons match; source 10 repeats known M2 self-source difference; the 1,422-case opcode scan and the 105-case state matrix of seven control opcodes execute, and only 08, 10, 15, ff continue without a fault and 12 pauses (1a..fe: M2-282)	D	p16 confirmation/source matrices, m2_opcode_scan, m2_opcode_states
M2-66	Repeat/triple waits preserve command/address/DMA flags; ack blocks next wait; rearm restores	D	m2_waitcons C58=4 controls and intermediate captures
M2-67	WAIT4 survives 810=1; encode completion 13 becomes 12 in serial controls	D	m2_waitcons and m2_dma_ecc_overlap
M2-68	Masked WAIT12 survives clearing 048 bit 23; WAIT5 passes in 9 of 11 orderings of an unwaited 70h and the latch (including 20 and 200 instructions between them) and faults in two: an acknowledge, or one instruction (`movi`), between the 70h and the latch (also after a fresh reset: M2-390; gap of 0..20 instructions: M2-593)	D	m2_waitcons/status_order
M2-69	Event 15 fails in both orders for all five clear-enable stimuli; event 11 passes	D	m2_waitcons
M2-70	C58 write/read masks to d6 in 68 CPU/sequencer cases; C08 self-read 5 in five controls	D	m2_c58_bits/c08self
M2-71	All 24 serial/overlap cases end, but every first 512-byte output mismatches; bit 25 unique to eight same-slot cases (cause: M2-78)	D	full payload comparisons and stored 048 snapshots
M2-72	20 repeated-encode paths end with 13 completion and zero candidate 900..910 reads; fresh completion not established	D / limit	m2_ecc_sequence
M2-73	All 2080 miscorr results are 1; no correction-radius claim	D / limit	miscorr; radius M2-84, M2-92; no miscorrection M2-174
M2-74	WAIT2 passes 5/8/16/19 in qualified contexts; high 64..255 fail; exact interrupt mapping unresolved here	D	w2vic and wider sweeps; mapping M2-140, M2-142, M2-167
M2-75	Forced state-b recovery 16 times, status release 1342, pause resume 197 plus one resume/abort; all cleanup succeeds	D	p16 diagnostic methods

Default-model native replay checks 237 selected event/register/control outcomes. It does not reproduce the two status-latch timing contrasts. These limitations are separate from successful ARM worker/probe execution.

18.11 P16 offline re-analysis

ID	Finding	Class	Evidence / reference
M2-76	Serial DMA is byte exact: memory→buffer→memory both slots, and 4 KiB NAND page reads with FMC_DMA_CTRL 0,1,2,4,7,8	D	p16 dmaloop 8/8, dmactl 24/24
M2-77	BCH8 = M1 code (0x201b, g=0 x115f914e07b0c138741c5c4fb 23, MSB first); sector 0 message is data then 10 metadata bytes	D	352/352 p16 raw sectors, tools/fmss_p16_offline.py
M2-78	The overlap payload loss of the first full campaign follows from unacknowledged WAIT2 flags in its generator; a DMA programmed during the previous one cuts it	D	p16 overlap records; p17 m2_dma_ack: acknowledged 6/6 exact, unacknowledged reproduces the cut
M2-79	FMC_DMA_CTRL: 2→0x42, 4→0x84 readback; 0x10..0x80 fault reads; reads 7 at every case start (restored after 342 cases that left another value); 0x187-type values after overlapped DMAs	D	p16 dmactl before/after maps, overlap after maps
M2-80	Serial and different-slot DMA/ECC overlap is exact (BCH8/16); a DMA into a buffer under its own ECC operation keeps old bytes in 4..32-byte runs (counts: M2-399) and sets FMC_STATUS bit 25 (both directions and the parity: M2-399, M2-401)	D (flag meaning I)	p17 m2_dma_ecc_overlap 16 exact, 8 same-slot

The model now computes physical BCH8 parity (BCH16 remains a stand-in) and by default refuses a DMA started while an earlier completion is unacknowledged; `dma_hazard='p16'` is an inferred profile that reproduces the measured truncation.

18.12 P18

ID	Finding	Class	Evidence / reference
M2-81	Timeout fault at 492.747 us × (count + 1) + 447.5 us for start word count<<8 | 0xb5, counts 8..256 including multi-bit; WAIT3 and illegal opcode equal	D	p18 m2_timeout_window 14/14, residual ≤ 1.1 us
M2-82	+400 storage words read 0 from the sequencer after sequencer or CPU writes; no aliasing onto primary words	D (absence vs unclocked open)	p18 m2_bank_regs 6/6
M2-83	Primary storage masks: 0c/10/34 32 bits, 14 0xffff, 18 0xf, 1c 1, 2c 7, 30 0x1fff, 58 0xff, 5c 0xff00 (058 and 05c are bits 7:0 and 15:8 of 014: M2-643)	D	p18 m2_bank_regs
M2-84	BCH8 decode corrects 1..8 bit errors in data, metadata and parity; 9, 10, 16 errors left data unchanged (4 cases)	D	p18 m2_bch8_correct, raw writes verified
M2-85	ECC_RESULT read after WAIT4 + ack does not give the current sector's count or uncorrectable bit	D (cause I: retained completion)	p18 m2_bch8_correct

18.13 P19

ID	Finding	Class	Evidence / reference
M2-86	BCH16 = narrow-sense binary BCH t = 16 over GF(2^13)/0x201b, data then metadata MSB first, 26 parity bytes	D	p19 m2_bch16_encode 14/14
M2-87	04c = chip status: 0x80 busy, 0xe0 ready; 0x70 (command echo) with no CE	D	p19 m2_status_busy
M2-88	0x810 after decode: bit 3 uncorrectable, bit 5 set at 8 errors, bit 0 done	D	p19 m2_ecc_status_timing; bit 5 from 7: M2-94
M2-89	0x818 = corrected-bit running total << 11 | decode count	D	p19 m2_ecc_status_timing
M2-90	0x80c returns slot[read index mod 8] of per-decode results; reads advance it	D (pointer rule I)	p19 m2_ecc_status_timing 4 × 48 reads; ring rule M2-91

18.14 P20

ID	Finding	Class	Evidence / reference
M2-91	0x80c: 8-slot ring of decode results; each 80c read, by the sequencer or by the CPU, also through the copy at 0xa0c, advances the read pointer by one slot; reads of 0x90c and 0xb0c (which read 0), 810/818 reads and 80c writes do not; 0x81c = 1 resets read and write positions to slot 0	D	p20 m2_ecc_ring 7 variants; p37 m2_open_ecc_status_bits ring/cpu-pre0..2, cpu-pre2-reset; p38, p39 m2_r3_ring_alias 7
M2-92	BCH16 decode corrects up to 16 bit errors in data, metadata and parity; > 16 (4 cases) leaves data unchanged	D	p20 m2_bch16_correct
M2-93	810 bit 5 set from 8 errors (BCH8 and BCH16); clear at ≤ 5 (BCH8) and ≤ 3 (BCH16)	D	p19, p20; threshold 7 for both: M2-94, M2-100

18.15 P21

ID	Finding	Class	Evidence / reference
M2-94	810 bit 5 set from 7 corrected bits (BCH8)	D	p21 m2_ecc_threshold
M2-95	810 bit 6 marks an erased sector (0x55); 80c = 0 for it	D	p21 m2_ecc_threshold
M2-96	0x81c = 1 clears 818 and sets 810 to 0x10	D	p21 m2_ecc_threshold, m2_wait4_ack
M2-97	After an encode WAIT4 follows 810 bit 1 (W1C); writing 1 clears only bit 0	D	p21 m2_wait4_ack 13 cases
M2-98	ECC_CTRL bits 15:8 = metadata length (0, 3, 10, 12); bit 2 BCH16; bits 3..7, 16..23 no parity effect	D	p21 m2_ecc_modes 19 words

18.16 P22

ID	Finding	Class	Evidence / reference
M2-99	After a decode WAIT4 follows 810 bit 2 (W1C); bit 6 W1C	D	p22 m2_wait4_decode
M2-100	810 bit 5 set from 7 corrected bits for BCH16	D	p22 m2_bch16_threshold
M2-101	ECC_CTRL bits 24..31 no encode effect; decode+encode acts as decode; metadata length up to 32, bytes > 12 read 0xff	D	p22 m2_ecc_modes2
M2-102	Instruction cost: ALU 30.1 ns, nop (08) 37.65 ns, window ld/wr 60.6 ns, sti SDRAM 97.8 ns, ldi SDRAM 211 ns, sub + taken bnz 67.6 ns; SRAM = SDRAM	D	p22 m2_opcost; p38, p39 m2_r3_bank_decode 6 (window ld 60.6 ns)
M2-103	Engine clock ≈ 33.25 MHz; timeout = 2^14 cycles per start-word count for bits 7:4 ≥ 11 (other steps M2-115, M2-216)	I	M2-81 + M2-102
M2-104	FMC 018 = 1 before data reads and 05c = 0x200 before the final DMA do not change page-read data or ECC results	D	p22 m2_fmc_misc

18.17 P23

ID	Finding	Class	Evidence / reference
M2-105	BNZ targets: byte offsets, 8-byte granular (low 3 bits ignored), no wrap at 2^15..2^20; PC ≥ 19 bits (refined by M2-725: ≥ 25 bits)	D	p23 m2_jump_width BNZ
M2-106	JR does not jump when its target register is written by the immediately preceding movi	D	p23 m2_jump_width JR 21/21; p16 m2_regjr contrast; cause r[R] = 0 (M2-110), no hazard (M2-108)
M2-107	No WAIT 0..15 (arg = mask = 0) follows NAND ready/busy; WAIT 11 passes at once	D	p23 m2_wait_rb

18.18 P24

ID	Finding	Class	Evidence / reference
M2-108	No register-write hazard: consumers at distance 0 see the new value (add, mov, ldi address, sti data, bnz)	D	p24 m2_alu_hazard 32/32
M2-109	JR with R field 0 and r0 = 0 does not jump, regardless of gap (29 cases); JR with all registers non-zero jumped	D	p24 m2_jr_hazard, m2_jr_width; p16 JR records; rule M2-110

18.19 P25

ID	Finding	Class	Evidence / reference
M2-110	JR jumps to r[arg] iff r[R] ≠ 0	D	p25 m2_jr_cond 16/16
M2-111	Among WAIT 5..10, 12..15 only WAIT 5 passed, after a status latch (7 states)	D	p25 m2_wait_sources
M2-112	FMC_STATUS bits 1/2/3/20/23 = command/address/data/DMA/status latch; C50 bits 0..5 the same plus ECC (bit 4); C50 bits 11..12 ECC idle	D	p25 m2_fmc_flags
M2-113	FMC_DMA_CTRL changes during a DMA (0xa07 in, 0x1e07 out sampled; the field is bits 12:6, M2-398)	D	p25 m2_fmc_flags
M2-114	512-byte buffer-to-memory DMA completes within 1..2 us; a second DMA started earlier never completes	D	p25 m2_dma_cut; per-direction timeline M2-117
M2-115	Start-word bits 7:4 = n: timeout step 2^3 cycles for n = 2, 2^5 for n = 3 and 2^(n+3) for n = 4..11 (saturating at n = 11; n = 2, 3 from M2-216); n = 0 faults at once (0..4 us for every tested count); n = 1 faults at 2 us for count 0x0f, 4 us for 0x40, 8 us for 0x80 and 14..16 us for 0xff, consistent with a step of 2 cycles per count (3.85, 7.70, 15.34 us predicted) within the 2-us resolution	D	p25 m2_timeout_low; p35 tmobits, tmofield; p38 m2_r3_prescaler_phase 3 (n = 1); p39 m2_r3_prescaler_phase 3
M2-116	Encode parity readable at 0x840..0x85c (word order reversed, big-endian words); 0x800 = 0x20070618, 0x808 = 0x700	D	p25 m2_ecc_regs

18.20 P26

ID	Finding	Class	Evidence / reference
M2-117	DMA in ≈ 2.7 us, out ≈ 1.1 us to 048 bit 20; 038 bits 12:6 change during the transfer (M2-398)	D	p26 m2_timeline
M2-118	ECC encode (BCH8 = BCH16) and erased decode ≈ 4.1 us; uncorrectable decode ≈ 5.8 us	D	p26 m2_timeline
M2-119	512-byte NAND data read ≈ 22 us at the setup timing word	D	p26 m2_timeline
M2-120	CPU writes to C0C, C10, C48, C54 release none of WAIT 6..10, 12..15	D	p26 m2_wait_cpu 36
M2-121	CPU C60 = 1 during a run holds the engine (C08 = 1) beyond STOP, the reset handshake and an emCORE reload of the same kernel	D	p26 cc60/wait06 and the following failures; p27 reload

18.21 P28 to P36

ID	Finding	Class	Evidence / reference
M2-122	JR targets: byte offsets, low 3 bits ignored, no wrap (as BNZ)	D	p28 m2_jr_width2 21/21
M2-123	Per-case reset zeroes FMC 00c..05c, ECC_CTRL, PRAM; FMC_DMA_CTRL reads 7 (the FMC words and 038 by FMCTRL1 bit 31, ECC_CTRL by 0x81c bit 0: M2-645)	D	p28 m2_reset_retention
M2-124	READ ID via FIFO 0x52 in 0x60/0x64 (AD D5 14 B6 44); ID repeats after 5 bytes; 0x080 reads 0	D	p28 m2_fifo_read
M2-125	CPU writes C64/C68/C6C release no WAIT 6..10, 12..15	D	p28 m2_wait_cpu2 27
M2-126	C60 hold released by C60 = 0; probe cleanup method 0x20 recovers it	D	p28 restore-2500, p29 restore-6000
M2-127	NAND read cycle ≈ (f14:12 + f18:16 + 2) × 7.06 ns per byte; f30:28 no read effect; f18:16 ≤ 1 misreads	D	p29 m2_nand_timing
M2-128	0x820.. = decode error locations, 13-bit fields byte × 8 + bit (bit 0 = LSB) in ascending byte order, within one byte the higher bit first (M2-714); rest 0	D	p30 m2_ecc_decode_regs 36/36; bit order within a byte: M2-714
M2-129	BCH8 decode time ≈ 5.8–6.0 us for 1..8 errors	D	p30 m2_decode_timeline
M2-130	One status latch polls until ready (WAIT5, WAIT12 40/40 pass at erase end); no latch or WAIT12 arg 0 never passes; on an idle chip WAIT5 passes for every tested ordering in which the 70h was waited for and acknowledged (one latch, two latches, the acknowledge or a NOP before the latch) and for a latch without 70h	D	p30 m2_status_wait
M2-131	Data-in uses the same cycle (f14:12 + f18:16 + 2) × 7.06 ns; f30:28 no transfer effect; f18:16 ≤ 1 shifts read data by one byte	D	p30 m2_nand_write_timing, m2_nand_timing
M2-132	Location numbering: data 0..4095, metadata/parity from 4096 (byte × 8 + LSB bit); BCH16 16 fields; uncorrectable/clean = 0; 818 total counts meta+parity	D	p31 m2_ecc_locations 15/15
M2-133	Decode ≈ 4.0 us with zero syndromes; +≈ 1.6 us BCH8 / +≈ 2.3 us BCH16 for any non-zero syndrome	D	p31 m2_decode_timeline2
M2-134	WAIT12 passes iff status & arg & mask ≠ 0	D	p31 m2_wait12_args 18/18
ID	Finding	Class	Evidence / reference
M2-135	Command/address cycles use the data cycle; f30:28 no	D	p31 m2_cmd_timing, m2_ctrl0_bits

ID	Finding	Class	Evidence / reference
	transfer effect; FMC_CTRL0 bit 0 required, bits 9..11,15,19..23,25..27,31 inert for reads		
M2-136	One status latch makes 04c follow the chip (0x80 busy → 0xe0 ready); no latch, no status read	D	p33 m2_status_poll
M2-137	WAIT12 on a busy chip releases only at ready (80/80 at erase end with 04c 0x80, bit 23 clear); 01/01 faults	D	p33 m2_status_poll
M2-138	Location space = data, metadata, parity (sector-0 parity from 4176)	D	p33 m2_ecc_locations2 4/4
M2-139	FMC_CTRL0 stores 9, 10, 25, 26; 15, 19..23, 27, 31 read 0; no bit enables +400; f30:28 no page-open effect	D	p33 m2_bank_enable, m2_rb_timing
M2-140	wait2 N ignores VIC SOFTINT (59 lines raised 2 ms, none released); hardware lines 5 (always), 8 (timer), 16 (DMAC0) release it	D	p34 m2_wait2_soft 192
M2-141	Source selector 10 = destination register in every MOV/ALU form	D	p16, p35 m2_regsrc
M2-142	FMC interrupt = VIC0 line 20, enabled by 0x044; wait2 20 passes 4–7 us after DMA/command/status; an ECC encode raises no VIC line (with 0x044 = 0 or all ones)	D	p35 m2_wait2_fmc
M2-143	CPU writes to C0C, C10, C48, C54, C60..C6C release no WAIT 6..10, 12..15	D	p35 m2_wait_cpu 72
M2-144	Full campaign reproduces earlier captures: 79 of 31,974 outcomes differ (USB/DMAC lines, fault-window TIMEOUT→FAULT, JR into SDRAM, newly run wait_cpu)	D	p35 vs p16..p34
M2-145	FMC_STATUS bit 27 is set after reading a programmed page and clear after reading an erased page (raw read 0x8041000 / 0x41000, ECC path 0x803c000 / 0x3c000); per sector it is set after the metadata/parity or data transfer of every written sector (all-zero and random pages, 16 of 16) and of no erased sector (0 of 8), also for a single 512-byte, parity-only or ECC-decode read; writing 0x0ff00ffe clears it	D	p35 fullpage, rawpage, pageorder, fmcbits, rawwrite, eraseseq, m2_bch8_correct, m2_ecc_threshold, m2_fmc_misc, m2_fmc_flags; p36 m2_m1_fmc_status_bit27; p37 m2_m1_fmc_status_bit27 6; p38, p39 m2_m1_fmc_status_bit27 6
ID	Finding	Class	Evidence / reference
M2-146	shl/shr are logical and give 0 for every count >= 32 in immediate and register form: counts 32, 33, 63, 64, 255, 256, 0xffffffff on 1, 0x80000000,	D	p35 m2_shifts, alu, m2_opcode_confirm; p36 m2_m1_shift_counts 36; p37 m2_m1_shift_counts 36; p38, p39 m2_m1_shift_counts 36

ID	Finding	Class	Evidence / reference
	0xffffffff (168 cases) and counts 32..255, 2^8..2^31, 0x101, 0x11f, 0x1001f, 0x8000001f, 0xfffffffe1 on 0x80000001 (1144 results); counts 0..31 shift normally; no modulo-32/64/256 or 8-bit-immediate aliasing		
M2-147	0xd80..0xdfc read 0 and drop program `wr/pst` writes (all 32 words, sequencer read-back and CPU read-back), no alias into 0xd00..0xd7c	D	p35 pram, access, m2_pram_alias
M2-148	C58 bit 0 is not stored (C58 = 1, 3, 5, 7 read back 0, 2, 4, 6) and an END with it written leaves C5C 0 and runs no fault vector	D	p35 c58b1, c58b2, resume, m2_c58_bits; p36 m2_m1_c58_end_divert 12; p37 m2_m1_c58_end_divert 12; p38, p39 m2_m1_c58_end_divert 12 each
M2-149	No C58 value diverts an END to C4C: C58 = 0..7, 0x12 and 0x16 with C4C written by the CPU (before or after C58) or by the program, including C58 = 2 and 6 in every M1 write order, end at the fall-through END with C5C = 0	D	p35 oneshot, c58b1, c58b2; p36 m2_m1_c58_end_divert 30; p37 m2_m1_c58_end_divert 30; p38, p39 m2_m1_c58_end_divert 30 each
M2-150	Consecutive tmwait/15 each wait a full C14 period; no immediate reload from 0	D	p35 tmwait2, c14
M2-151	After the ordinary reset the first start word runs a program whose instruction 0 is not END	D	p35 startseq, startseq2, startseq3
M2-152	C50 bit 0 follows a pending command flag: clear in the first two reads after the 70h write, set from the third instruction after it, kept through a timeout fault and a write of 1, cleared by the FMC_STATUS bit 1 acknowledgement; not set without a command or after an acknowledged one	D	p35 c50fault; p36 m2_m1_c50_bit0 6; p37 m2_m1_c50_bit0 6; p38, p39 m2_m1_c50_bit0 6 each
M2-153	Start-word bit 19 keeps the program from running like bits 1, 3 and 16, but with C00 reading 0x7f60 (bit 18 clear; bits 1/3/16 give C00 bit 18 set); bit 3 does not read back	D	p35 startbits, ctrlbits; p36 m2_m1_start_bits; p37 m2_m1_start_bits; p38, p39 m2_m1_start_bits
M2-154	C3C and C48 hold 32 bits and C54 mask 0x83f, from CPU and sequencer; a start keeps CPU-written values (the program reads 0xffffffff, 0xffffffff, 0x83f after CPU writes of all ones)	D	p35 access, rwmask; p36 m2_m1_window_matrix; p37 m2_m1_window_matrix; p38, p39 m2_m1_window_matrix 128
ID	Finding	Class	Evidence / reference
M2-155	Offset bit 9 is not decoded below 0x400 and in 0x800..0xbff (0x200..0x3fc and 0xa00..0xbfc are copies) but is decoded in 0xc00..0xfff: none of the 38 holding words of 0xc00..0xdff (C10, C3C, C48, C4C, C54, C58, 0xd00..0xd7c) appears at +0x200; the 0x100	D	p35 access, fmcmap; p36 m2_m1_window_matrix 128; p37 m2_m1_window_matrix 128; p38, p39 m2_m1_window_matrix 128

ID	Finding	Class	Evidence / reference
	block appears only at 0x100 and 0x300 (each of 0x1bc..0x1d8 copied to +0x200, none to +0x400/+0x600; 0x1b8 holds 0 in the pattern and is not decisive); 0x400..0x7fc read 0		
M2-156	FMC 0x028 does not advance over a 128-iteration pld/sti window dump, over 64 pld/pst/sti/ldi/wr/ld, or over 128 iterations of pld 0x080 + sti, pld 0x048 + sti or ldi + sti (M1: +1 per pld+sti iteration)	D	p35 fmcmap, fmc28; p36 m2_m1_fmc28; p37 m2_m1_fmc28; p38, p39 m2_m1_fmc28 11
M2-157	Start bits 1, 3, 16 hold the program at instruction 0 until a start without them (0xffb5 runs it once from instruction 0); no single-step; preloaded r2..r5 survive starts with and without bit 17 (0xfff5, 0x20001, 0x20000), so bit 17 does not clear the registers	D	p35 ctrlhold; p36 m2_m1_start_bits; p37 m2_m1_start_bits; p38, p39 m2_m1_start_bits
M2-158	Start bit 1 while running freezes the program exactly (C08 = 3) and a later start resumes it; bit 3 or 16 while running restarts it from instruction 0 after the release; a plain start while running is ignored	D	p35 pause; p36 m2_m1_start_bits; p37 m2_m1_start_bits; p38, p39 m2_m1_start_bits
M2-159	`wait` event numbers 16..255 never pass in 8 states (2640 cases, no modulo-16 alias)	D	p35 waitf, evsweep2, evsweep3
M2-160	Block 0x100: sequencer-writable words exactly 0x1b8 (1 bit), 0x1bc..0x1c8/0x1d4/0x1d8 (32 bits) and 0x1cc/0x1d0 (24 bits), reading 0, 0xffffffff and 0x8060/0x3010d0 before the program's write (each case restores the word), visible again at +0x200; other words keep nothing and read 0x12 from the sequencer; the CPU reads 0 at 0x100..0x1b8, the reset values at 0x1bc..0x1d8 (0xffffffff, 0x8060, 0x3010d0), 0xffffffff at 0x1dc and 0x12 at 0x1e0..0x1fc, the same at 0x300..0x3fc, at power-on and after every reset	D	p35 blk100; p36 m2_m1_blk100; p37 m2_m1_blk100; p37 m2_m1_poweron_values fullmaps; cold m2_cold_boot fullmaps; p38 m2_m1_blk100 17; p39 m2_m1_blk100 17
M2-161	The standard status latch (70h, wait 0, ack, latch) passes WAIT5 every time, with no intermittent timeout: 1337 of 1337 in repeated series, 2644 of 2644 first latches (WAIT5 or WAIT12, 0x044 = 0, 1, 2 or 0xffff) and 410 of 410 (11 forms, once and 40 times each) in each of four captures	D	p35 latchrep; p36 m2_m1_latch_arm; p37 m2_m1_latch_arm 25; p38, p39 m2_m1_latch_arm 25
M2-162	`wr/st` to 0xc38 jump to the written offset (fall-through not executed); `ld` of 0xc38 reads the instruction's own offset	D	p35 pcwrite; p36 m2_m1_pc_write; p37 m2_m1_pc_write; p38, p39 m2_m1_pc_write 2

ID	Finding	Class	Evidence / reference
M2-163	A status latch never sets FMC_STATUS bit 24: 0 of 2743 latch records, with 0x044 = 0, 1, 2, 0xffff (x40 each) or every single bit (x3) written	D	p35 latchrep, irq044
M2-164	end ignores non-zero R/arg/imm (no fault, END CS_IRQ 1; also R 0x80/0xff, arg 0x8000/0xffff, imm 0x80000000/0xffffffff, r6 not overwritten by the following instruction); C40 latches R<<16|arg and C44 the imm	D	p35 fields0; p37 m2_open_reserved_fields 7; p38, p39 m2_open_reserved_fields 7
M2-165	Non-zero fields that the zero form leaves 0 are ignored: imm of `st`, `ldi`, `mov`, `sti`, `pld`, `pst` (0xff, 4, 1, 0x10, 0x100, 0x80000000, 0xffffffff), R of `wr` (3, 7, 1, 0x80, 0xff), arg of `movi` (0x1234, 0xd70, 1, 0xa, 0x8000, 0xffff) and arg of `bnz`/`bz` (1, 0xff, 0xffff; the branch is still taken)	D	p35 fields0 16; p37 m2_open_reserved_fields 43; p38, p39 m2_open_reserved_fields 31
M2-167	No wait2 N (0..255) tracks NAND erase/program/read busy; only always-asserted 5 and CPU-device lines 8/16/19 pass	D	p35 op09nand
M2-168	Four-cycle erase (ANUM 3) erases the block on CE0/CE1, FAIL clear, same time as 3-cycle	D	p35 eraseseq
M2-169	With FMC_DATA_SEL = 0x100 the status latch returns the real status (0x80 busy, 0xe0 ready), as with DATA_SEL = 1 (idle CE0 latch 0xe0 with both; no CE selected 0x70 with both)	D	p35 m2_status_busy; p36 m2_m1_status_datasel; p37 m2_m1_status_datasel; p38, p39 m2_m1_status_datasel
M2-171	0x900..0x9fc (0x990 included) read 0 after BCH8 and BCH16 encodes of zero, ones and ramp data and after decodes; 0 or 0xffffffff written to 0x940..0x950 before an encode or decode changes neither parity nor the decode (data, 0x810, 0x818)	D	p35 m2_ecc_regs, eccregs; p36, p37 m2_m1_ecc_area 27; p38, p39 m2_m1_ecc_area 27
ID	Finding	Class	Evidence / reference
M2-172	C14 reads start-word bits 15:8 after a `wait` that passes before the first timeout step: after DMA and BCH8-encode waits (0xff for 0xffb5; 0xfe, 0xfd, 0xfb, 0xf7, 0xef, 0xdf, 0xbf, 0x7f with one of bits 8..15 cleared; SDRAM and SRAM) and after passing waits 0..5, 11, `wait2` 5 and `evjmp` departures ending 3..13 us after the start (0xff with 0xffb5, 0x07 with 0x07b5); with start word 0xefb5 the sequencer reads 0x7f before a passing `wait 0` or `wait 2` and 0xef right after it, about 1 ms later and (CPU) after the END, and 0xef stays through a later passing `wait 3` or `wait 4`, so C14 does not	D	p35 startbits, holdout; p37 evcons, op16, ev11, op09, irqsrc, holdout, m2_m1_latch_arm (after-maps and sequencer reads of C14); p38, p39 m2_r3_c14_reload 9 each

ID	Finding	Class	Evidence / reference
	change outside a wait; the same waits released 1.5 ms in by a CPU write leave 0x7f; runs without such a wait leave 0x7f; during a stalled wait or a `tmwait` C14 counts down by one per timeout step (0xbd 33 ms into a second wait after a fault, 0xbc 34 ms into a second `tmwait`, start word 0xffb5)		
M2-173	FMC 0x028 is a 9-bit storage word (0x5a5 written by the sequencer reads 0x1a5 from both sides) that FMC_FIFO 0x080 reads do not advance (1, 4 and 130 reads, READ ID latched or not; M1 +4 per read) and that does not drift between runs	D	p35 bufport, fmcmap, access; p36 m2_m1_fmc28; p37 m2_m1_fmc28; p38, p39 m2_m1_fmc28 11
M2-174	No miscorrection: 13440 BCH8 (9..64 errors) and 11520 BCH16 (17..64) random sectors all uncorrectable, data unchanged	D	p35 miscorr
M2-175	`pld`/`pst` use offset bits 11:0 only: offsets with any of bits 12..31 set (bits 12, 13, 16 or 20 in 100 cases; 0x1d64, 0x2d64, 0x8d64, 0xfd64, 0x10d64, 0x80000d64; 0xd64 and 0xd68 with each of bits 17..30) and the aliases 0x1c18, 0x1c1c, 0x1c28, 0x10c28, 0x1d68, 0x10d70, 0xfd6c reach the word at offset & 0xfff and change no other word; through 0xc18..0xc34 `pld` reads r0..r7 (through the pointer register r1 it reads the pointer itself) and `pst` writes r0..r7; unaligned `pld`/`pst` (low bits 1..3) read 0 and drop the write	D	p35 m2_pram_alias, indirect, m2_opcode_confirm; p36, p37 m2_m1_indirect_wrap 13; p37 m2_m1f_indirect_reach 45; p38, p39 m2_m1_indirect_wrap 13, m2_m1f_indirect_reach 45
M2-176	C14 ignores CPU and program writes (CPU 0x55, program 0x12/0/0xff/0x33 all leave 0x7f)	D	p35 c14, op15; p36, p37 m2_m1_c14_timer, m2_m1_poweron_values; cold m2_m1_poweron_values
M2-177	C14 reads 0x7f after a timeout fault, whatever the start word (143 cases; ten start words 0xffb5, 0xfeb5..0x7fb5, 0xff75 with fault times following bits 15:8), before the first wait of a run, before, at and after a `wait 0` released by a CPU 70h write 1..120 ms into it (start word 0xffb5), and after a WAIT5 that passed at the end of an erase 2.27 ms into it; a `wait 0` released by a CPU 70h write after 1, 2 or 4 timeout steps leaves 0x7f for counts 3, 0x10 and 0x40 (n = 0xb) and count 0x10 with n = 9, read right after the release and 1 ms later, while one released before the first step leaves the count (M2-172)	D	p35 tmobits, tmofield, startbits; p36, p37 m2_m1_c14_timer 15; p37 m2_m1_latch_arm; p38, p39 m2_m1_c14_timer 15 each; p38, p39 m2_r3_c14_reload (released waits); p42 m2_r4_c14_release 15

ID	Finding	Class	Evidence / reference
M2-178	A latch written straight after 70h (no wait 0) passes WAIT5 (41 of 41 in a series, in four captures) and returns 0x70, later in a series 0xe0; a latch one or two instructions after an unwaited 70h (an acknowledge, one `movi`, or one or two nop2 between: M2-593) faults with bit 23 clear (also straight after a reset, M2-390); status reads before a WAIT do not consume event 5	D	p35 m2_status_order, evclear, latchrep; p36, p37 m2_m1_latch_arm; p38, p39 m2_m1_latch_arm
M2-179	C3C = faulting offset + 8 in the handler (also nested: 0x30/0x88, 0x48/0xa0, 0x78/0xd0 for waits at 0x28/0x40/0x70), 0 after the run; C10, C3C, C48, C54 preset to 0 / all ones leave the WAIT timeout (4388..4393 us at count 7) and a DMA in + BCH8 encode + DMA out (16 us, parity, 0x810 and output identical) unchanged; the `tmwait` END stays at 3879..3900 us with C48 = 0 and every preset of C10, C3C and C54, but with C48 = 0xffffffff or 0x55555555 the `tmwait` faults at the WAIT timeout instead (4388..4390 us, CS_IRQ 5, C14 then 0x7f), in every run: 5 of 5 for each value in each of two captures, after one run with 0xffffffff in each of three earlier captures; single bits: M2-627	D	p35 m2_nested_fault, timeregs; p36, p37 m2_m1_ctrl_words 36 each; p38, p39 m2_m1_ctrl_words 36 each; p38, p39 m2_r3_control_words 15 each (C48)
M2-181	After a status latch a new command is refused (wait 0 times out, bit 1 clear, bit 26 not set) until an FMCTRL1 write; status clear, ECC kick and FFh do not release; after the release FMCTRL1 0x30020 + ack 0x1800000 the next 70h passes its wait 0 (11 of 11 forms, four captures); an address operation in that position fails with bit 26 set (M2-610)	D	p35 latchrep, ev5rule, ev5arm; p36, p37 m2_m1_latch_arm; p38, p39 m2_m1_latch_arm
M2-182	VIC1 raw bit 22 is set at the stop after a run that executed an END (CS_IRQ bit 0, a fault-handler END included) with C10 non-zero (C10 = 1 or 0xdf), and clear after an END with C10 = 0 and after every run that stops without an END (held, paused, looping, or faulted with no handler END); it is clear while a program runs	D	p35 3784 records (irqline, irq044, irqop, latchrep, m2_wait2_fmc, m2_status_order, ev5rule, ev5arm, op16b, c50fault, c14, timeregs, tmwait2); p36 1411 set, 23 END with C10 = 0 (25 families incl. m2_m1_irq044_routing, m2_fmc_irq_bits, m2_m1_ctrl_words, m2_m1_irq_end_pending); p37 4560 records of 66 families: 4496 set, 26 END with C10 = 0 and 38 without END clear (C10 from the after-maps); p38 3294 records of 76 families: 3205 set, 18 END with C10 = 0 and 71 without END clear; p39 3328 records of 79 families: 3240 set, 18 and 70 clear

ID	Finding	Class	Evidence / reference
M2-183	A 90h or 70h command written through +0x408 changes no +0x400 word and raises no first-set event 0 or C50 bit 6 (C50 0x00ff1800 in eight samples right after the write and after wait 6, which times out); a full READ ID through +0x400 (set-up word +0x404, CE in +0x400, 90h to +0x408, address, DNUM 7, FMCTRL1 0xe2 or 0x1c2) times out at waits 6, 7 and 9, leaves +0x424 and every +0x400 word 0 (sequencer view of 0x400..0x47c) and sets no C50 bit 6 or 8, while the same READ ID through the primary set passes waits 0, 1 and 3	D	p35 chan1, m2_bank_regs; p36 and p37 m2_m1_bank_channel 6; p37 m2_m1f_bank_channel 8; p38, p39 m2_m1_bank_channel 6, m2_m1f_bank_channel 8
M2-184	FMC_CTRL0 bit 24 clear until the END: the buffer->RAM DMA of a page read never starts (no byte in RAM, DMA pointer static, FMC_STATUS bit 20 clear, wait 2 faults) while the NAND->buffer transfers complete; bit 24 set: exact	D	p35 fmcbits, dmaloop
M2-185	The +0x400 set moves no DMA: a DMA programmed through +0x404 raises no first-set wait 2 and wait 8 never passes; a +0x400 RAM->buffer->RAM loopback (slot 0 and 1) leaves the +0x400 output untouched with both wait 8 timed out while the primary buffer keeps its content, and a +0x400 parity DMA leaves the first set's parity buffer 0 unchanged	D	p35 ch1buf, readev; p36 m2_m1_bank_channel 4; p37, p38, p39 m2_m1_bank_channel 6
M2-186	evjmp waits for an event arriving during the wait (command flag 100, 1000 or 5000 us into it) or already pending (command, DMA), jumps to C4C for C58 = 0, 2 and 4 with CS_IRQ bit 2 clear in the handler and leaves the flag set (FMC_STATUS 0xc3002 / 0x1c3000 at the target); an acknowledged flag is no event, and a 70h written through +0x408 1 ms into the wait raises none; with no event it times out after the full period, continuing with C58 = 0 and through the fault vector (CS_IRQ bit 2 set) with C58 = 4	D	p35 op16b; p36 m2_m1_evjmp_rules 10; p37 m2_m1_evjmp_rules 10, m2_m1f_evjmp 14; p38, p39 m2_m1f_evjmp 14 each
M2-187	wait2 ignores arg/imm: (0,0), (0x40,0x40), (0,0x40), (1,0xd64), (0xd60,0xffffffff) and (0xffff,0) give the same outcome (wait2 5 idle passes; wait2 20 after a DMA with 0x044 all ones passes, without the enable or idle it times out)	D	p35 op09b; p36 m2_m1_wait2_fields 10; p37 m2_m1_wait2_fields 10; p38, p39 m2_m1_wait2_fields 10 each
ID	Finding	Class	Evidence / reference
M2-188	pause continues after CPU PC = offset + 8 and a start word with	D	p35 op12cont, m2_pause_state, m2_opcode_m1, m2_recovery;

ID	Finding	Class	Evidence / reference
	or without CS_IRQ bit 3 ack (11 cases); no PC write -> stays paused (12); PC write followed by two start words: M2-512		p38, p39 m2_m1_pc_write 7
M2-189	A DMA moves 512 bytes for every FMC_DNUM tested (0..0x1fff; in, out and both directions; both slots)	D	p35 bufsize; p36 m2_m1_dma_length 22; p37 m2_m1_dma_length 22; p38, p39 m2_m1_dma_length 22; p39 m2_r3_dma_ctrl_low (FMC_DNUM 3, 15)
M2-191	Window matrix (CPU masks, sequencer masks equal): 000 77077fff, 018 f, 804 3f04, 808 1f81, 814 2f, 844 0, 940 0, 944 0, 948 0, 94c 0, 950 0, c10 df, c54 83f; 0x940..0x950 drop writes; M1-empty words holding bits: c68 7	D	p35 access; p36 m2_m1_window_matrix (13 p35 masks reproduced); p37 m2_m1_window_matrix; p38, p39 m2_m1_window_matrix 128
M2-193	With a 0x30020 release and bit-23 ack between polls, each new 70h+latch sets bit 23 exactly when the chip reports ready (0xe0); 0x80 busy leaves it clear	D	p35 m2_status_busy
M2-194	Programs on CE0/CE1 (single raw and ECC pages, 4/8 pages per run, one page on each CE in one run) and erases on CE0/CE1 read back exactly (erased pages all 0xff, status 0xe0 after each)	D	p35 writeseq, writeseq2, rawwrite, eraseseq, pageorder; p36 m2_m1_program_erase_matrix 7; p37 m2_m1_program_erase_matrix 7; p38, p39 m2_m1_program_erase_matrix 7
M2-195	A status poll on DATA_SEL = 0x100 reads 0x80 while a page programs and 0xe0 once ready	D	p35 m2_status_busy
M2-196	Multi-page program runs write every page with the status poll on FMC_DATA_SEL 0x200 (4 and 8 pages, two CEs), 0x100 and 1 (8 pages each, every poll 0xe0)	D	p35 writeseq, writeseq2; p36 m2_m1_status_datasel; p37 m2_m1_status_datasel; p38, p39 m2_m1_status_datasel
M2-197	No ECC_CTRL word selects a code other than BCH8/BCH16: single bits 3..7 and 16..31, every pair of them and all of them together with BCH8 and BCH16, and metadata lengths 0x80/0xff give BCH8 or BCH16 parity (of the data alone: M2-662) as set by bit 2, or none (438 encodes classified), never the M1 RS-8/12/16 parity; 0x003/0x007 encode nothing; length bits 15:14 ignored	D	p35 eccmode2, m2_ecc_modes(2), eccmode3; p36 m2_m1_ecc_alt_code 447; p37 m2_m1_ecc_alt_code 447; p38, p39 m2_m1_ecc_alt_code 447
M2-198	A latch issued during an erase completes WAIT5 when the chip becomes ready; a second chip's poll after an unreleased latch is refused at wait 0; with the release (0x30020 + ack) between polls, two-chip polls during an erase on CE0 and CE1 pass in both orders (4 polls each, status 0xe0, four captures)	D	p35 m2_status_wait, ev5two; p36, p37 m2_m1_latch_arm; p38, p39 m2_m1_latch_arm

ID	Finding	Class	Evidence / reference
M2-199	FMC 0x028 stays 0 through command, chip read, DMA, ECC encode (with set-up word) and 64x each instruction kind (without); after the set-up word and a READ ID it stays 0 through 32 instructions, 128 ldi+sti iterations and a 512-byte DMA in and out	D	p35 fmc28b, fmc28; p36 m2_m1_fmc28; p37 m2_m1_fmc28; p38, p39 m2_m1_fmc28 11
M2-200	0x044 bits 0, 1, 2, 11 and 12 route the command, address, transfer, DMA and status-latch flags: VIC0 line 20 and wait2 20 follow the routed flag only while it is pending, and of the 32 single bits only the operation's own bit raises line 20; CS_IRQ bit 4 is set by every routed flag raised during the run and stays set after the acknowledgement (after an address operation it also reflects the preceding 90h command)	D	p35 irq044, irqop; p36 m2_m1_irq044_routing 160, m2_fmc_irq_bits 96; p37 m2_m1_irq044_routing 174, m2_fmc_irq_bits 96; p38, p39 m2_m1_irq044_routing 174, m2_fmc_irq_bits 96
M2-201	Start-word bit 6 acts only as part of the timeout step field (bits 7:4); bits 17, 18, 20..31 have no observed effect; bit 19 inhibits the start	D	p35 startbits, startdiff, startregion, irqline
M2-202	A second 70h + latch with no FMCTRL1 write after the first latch is refused at its 70h wait 0 (342 of 342)	D	p35 latchrep
M2-203	FMC_STATUS bits 19:12 read 0xc3 (bits 12, 13, 18, 19) after reset and survive writing 1 (0xc3000); a short chip read (5, 16, 23 or 128 bytes) leaves 0x41 (bits 12, 18), also after a 512-byte read (a raw page read ending with the 128-byte spare read ends 0x41000 / 0x8041000); a 512-byte read leaves 0x3c (bits 14..17); a DMA in (data or parity buffer), a status latch and the set-up word restore 0xc3 from 0x41 and from 0x3c (set-up word: M2-497); command, address, metadata encode and DMA out do not	D	p35 fmcstat, fmcstat2, m2_fmc_flags; p36 m2_m1_fmc_status_low; p37 m2_m1_fmc_status_low 8; p38, p39 m2_m1_fmc_status_low 8; p38, p39 m2_m1f_setup_bits 3
M2-204	wait2 20 follows VIC0 line 20 (FMC flag pending and enabled), not the sticky CS_IRQ bit 4: with the command flag pending it passes for 0x044 bit 0 alone of the 32 single bits (address bit 1, transfer 2, DMA 11, status 12), and 0x844 does not release it	D	p35 m2_wait2_fmc; p36 m2_m1_irq044_routing; p37 m2_m1_irq044_routing; p38, p39 m2_m1_irq044_routing
M2-206	wait 8 never passes on the first set: it times out after a complete CE0 page read with DMA out, after a one-sector CE0 read (data exact) with C58 = 0 or 4 after WAIT5 or a WAIT12 ready poll, and idle	D	p35 readev; p36 m2_m1_event8 4; p37 m2_m1_event8 5; p38, p39 m2_m1_event8 5

ID	Finding	Class	Evidence / reference
M2-207	FMC_FIFO 0x080 reads 0 from DMA-filled data buffers 0/1 and parity buffer 0 (130 reads each, with and without FMCTRL1 = 0x340); 32 or 128 writes to it without an FMCTRL1 write leave the buffer unchanged and hold no written value; after FMCTRL1 0x340 with FMC_DATA_SEL 1 a DMA out of data buffer 0 no longer returns the buffer (M2-341), and writes after 0x340 do not reach buffer 0 either (M2-617)	D	p35 bufport, m2_fifo_access; p36 and p37 m2_m1_fifo_port 8; p37 m2_m1f_fifo_write 12; p38, p39 m2_m1_fifo_port 8
M2-208	An END (also a fault-handler END) with an enabled FMC flag pending (0x044 = 1, a single bit or all ones) leaves USB working under the probe's cleanup: every following marker case answers (5 of 5 after each)	D	p35 irqop, m2_wait2_fmc; p36 m2_m1_irq_end_pending 18; p37 m2_m1_irq_end_pending 18; p38, p39 m2_m1_irq_end_pending 18
M2-209	C50 bits 0..5 mirror pending events 0..5: the own event's bit is set while the flag is pending and clears with its acknowledgement (command, address, transfer, DMA, status); bits 6 and 8 never set, also after a +0x408 command	D	p35 op16b, chan1; p36 m2_m1_irq044_routing; p37 m2_m1_irq044_routing; p38, p39 m2_m1_irq044_routing, m2_m1_bank_channel 6, m2_m1f_bank_channel 8
M2-210	Writing 0x1b8..0x1d8 with 0/1/0xffffff does not change DMA, DMA+BCH8-encode, 4-byte data-read, 16-page open + 512-byte read or unmet-wait times, the data or FMC 0x048; 0x3a8..0x3b4 read 0x12 from the sequencer, 0 from the CPU	D	p35 blk100b; p36 m2_m1_blk100; p37 m2_m1_blk100; p38, p39 m2_m1_blk100 17
M2-211	wait2 32..63 never pass: in every state (620 cases), with a VIC1 soft interrupt raised, and for wait2 22, 53, 54, 55 after the sequencer polled VIC1 raw and free-running in a 32 ms window (28 runs in each of four captures, all timed out although the CPU sampled VIC1 raw bit 22 during 10, 13, 14 and 12 of them)	D	p35 w2vic, m2_wait2_soft; p36, p37 m2_m1_wait2_vic1 28 each; p38, p39 m2_m1_wait2_vic1 28 each
M2-212	No tested ECC enable raises an ECC interrupt: after BCH8 encodes and decodes with 0x044 = 0 or all ones and 0x800, 0x808, 0x814, 0x818, 0x844 and 0x860..0x9fc written all ones, single bits of 0x808, 0x814, 0x818 and 0x844, ECC_CTRL kick bits 3..7 and 16..31 or 810 bits 9..31 written, CS_IRQ bits 5/6 stay clear while 810 shows every completion, no VIC raw line rises beyond the no-candidate baseline, and wait2 20 and wait2 43 never pass while wait 4 passes	D	p35 irqop, m2_wait2_fmc; p36, p37 m2_m1_irq044_routing 14, m2_ecc_irq 21; p37 m2_m1f_ecc_csirq 288; p38, p39 m2_m1_irq044_routing 14, m2_ecc_irq 21, m2_m1f_ecc_csirq 288
ID	Finding	Class	Evidence / reference
M2-213	Original WRITE reads the DMA list pages x chunks + 1 times:	S	WRITE.s 0x048, 0x3f0, 0x580

ID	Finding	Class	Evidence / reference
	one prefetch (0x050) and one read per chunk (0x3f0) before the count test; the last entry is beyond the list		
M2-214	An all-zero sector-0 chunk (metadata, parity, data 0) decodes clean (0x810 bit 3 clear, 0x80c = 0) with BCH8 and BCH16; zero parity is not special; 0xff data with zero parity is uncorrectable	D	p36 m2_m1_zero_chunk
M2-216	Start-word bits 7:4 = n give a WAIT timeout step of 2^3, 2^5, 2^7, 2^8, 2^9 engine cycles for n = 2, 3, 4, 5, 6 (0.233..0.244, 0.961..0.964, 3.844..3.85, 7.697..7.70, 15.395..15.403 us; fault time linear in count + 1, five counts per n in each of four captures): 2^(n+3) holds for n = 4..6 but not for n = 2, 3	D	p36, p37 m2_m1_timeout_steps 25 each; p38, p39 m2_m1_timeout_steps 25 each
M2-217	VIC1 line 11 (M1 ECC source 43) and every other VIC line stay at the no-candidate baseline after BCH8 encode and decode completions for every tested ECC word (0x800, 0x808, 0x814, 0x860..0x9fc; values 0xffffffff, 1, 0xffff, 0xffff0000, 0x55555555, 0xaaaaaaaa; 0x044 = 0 or all ones; 751 cases)	D	p36, p37 m2_m1_ecc_irq_line, m2_ecc_irq; p38 m2_m1_ecc_irq_line 751; p39 m2_m1_ecc_irq_line 751 (one case faulted at the wait timeout, no line rose)
M2-232	A 512-byte DMA into parity buffer 0 (select 0x10) leaves the source's last 64 bytes (offsets 448..511), read back by a DMA out (the 64 bytes repeated over 512) and by programming 128 bytes from the buffer (the 64 bytes twice); an inbound DMA with select 0x1000 completes but leaves the buffer 0xff	D	p36 m2_m1_parity_buffer 5; p37 m2_m1_parity_buffer 5; p38, p39 m2_m1_parity_buffer 5
M2-237	After a latch, its WAIT5 and the release (FMCTRL1 0x30020 + ack 0x1800000), a second 0xca latch without a new 70h passes WAIT5 in 41 of 41 runs with no operation between and after each of: set-up word, FMCTRL1 0x100e0, 16-byte read (its wait 3 passing), DMA, address cycles, FMCTRL1 0x20, status clear, CE toggle, ECC kick (four captures); without the release see M2-494	D	p36 m2_m1_latch_arm 20; p37 m2_m1_latch_arm 20; p38, p39 m2_m1_latch_arm 20
M2-239	A latch after 90h sets no FMC_STATUS bit 23 (WAIT5 times out) and leaves the first READ ID byte 0xad in FMC_NSTAT; the next 70h after the release is accepted (its wait 0 passes)	D	p36, p37 m2_m1_latch_arm (one run each); p38, p39 m2_m1_latch_arm (one run each)

ID	Finding	Class	Evidence / reference
M2-251	With FMC_CTRL0 bit 24 cleared before each DMA and set again after its timed-out wait 2, RAM receives only the first 32 bytes of each buffer-to-RAM DMA (loopback: rest untouched; page read: 32 data bytes then zeros in sectors 0..4, shorter in the spare sectors) and no wait 2 passes; the held DMA starts when bit 24 is set (M2-338) and is cut by the next controller operation or the END that follows a few instructions later (cut inferred from M2-338, where 200 delay loops after the bit-24 write give the complete 512 bytes). An outbound DMA programmed right after the held inbound one has started (bit 24 set by that write) moves nothing, while its wait 2 passes	D	p36 and p37 m2_m1_ctrl0_dma_bit 10; p37 m2_m1f_ctrl0_held_dma 10; p38, p39 m2_m1_ctrl0_dma_bit 10
M2-278	CPU writes of C38 take effect outside a pause: written before the start, the program starts at the written offset; written 1 ms into a running loop, it jumps there (END at 1002 us); written during an unmet wait 3, the wait still times out and the run continues at the written offset (one run each)	D	p36 m2_m1_pc_write; p37 m2_m1_pc_write; p38, p39 m2_m1_pc_write 3
M2-281	The untouched window (after the kernel boot or the previous case's cleanup) differs from the window after the ordinary reset only in C00 (untouched 0x7f60; after the reset the last start word, 0xffb4) and C04 (0 versus the program address); C14 reads 0x7f in both	D	p36, p37 m2_m1_poweron_values 4; cold m2_m1_poweron_values 4
M2-282	All 228 values 0x1b..0xfe give exactly the outcome of 0x1a in every scanned state, operand form and C58 setting (why, fault time, CS_IRQ, r0..r7, the complete after-window and scratch: 0 differing words in 1368 comparisons); only 0xff (nop2) differs; no value in 1a..fe acts as an instruction in the tested forms	D	p36 m2_opcode_scan 1368 cases

18.22 P37 and the power-on capture

ID	Finding	Class	Evidence / reference
M2-332	FMCTRL1 0x100 (DMA out of buffer 0) replaced by 0xe2 (chip read into a buffer) before the DMA completes suppresses the DMA's completion flag but not its data: written at once, 497..509 of 512 bytes reach RAM (a run of 4..16 bytes ending at byte 23, from byte 8, 12 or 20, reads 0), FMC_STATUS bit 20 never sets and wait 2 times out; with 1, 4 or 16 delay loops before the replacement the DMA output is exact and wait 2 still times out; with 64 loops (after the DMA has completed) both operations complete. The replacing read completes (bit 3, wait 3 passes) only when FMC_DATA_SEL is switched to 2 before the 0xe2 write; with DATA_SEL left at 0x100 wait 3 also times out. A read (0xe2) replaced at once by the DMA loses the read's flag (wait 3 times out) while the DMA completes. The serial order (DMA, wait 2, ack, read, wait 3) is exact	D	p37 m2_m1f_dma_then_read 18 (two runs per variant); p38, p39 m2_m1f_dma_then_read 18
M2-336	After a primary-set READ ID (90h, address, FMC_DNUM 7, FMCTRL1 0xe2 or 0x1c2; waits 0, 1, 3 pass) FMC 0x024 reads 8, the number of bytes transferred; the same READ ID written through +0x400 leaves 0x424 and every +0x400 word 0	D	p37 m2_m1f_bank_channel 8
M2-338	A DMA programmed with FMC_CTRL0 bit 24 clear is held: its wait 2 times out, FMC_STATUS bit 20 stays clear, 0x03c stays at the start address and no byte moves in either direction, also 2000 delay loops later; setting bit 24 again starts it (0x03c advances by 0x200, bit 20 sets, the next wait 2 passes, all 512 bytes arrive). It runs with the FMC_DMA_ADDR value at that moment: rewritten while held (0x03c follows the write), the output lands at the new address and an inbound DMA reads the new source; the old area stays untouched	D	p37 m2_m1f_ctrl0_held_dma 10 (two runs per mode); p38, p39 m2_m1f_ctrl0_held_dma 10
ID	Finding	Class	Evidence / reference
M2-339	A two-plane program (80h, row of block 3000, data, 11h, ready, 81h, row of block 3001, data, 10h, ready) of page 0 of blocks 3000 and 3001 on one CE reads back exactly on both blocks (status 0xe0 after each half), and a two-plane erase (60h, row, 60h, row, D0h, ready) leaves both pages all 0xff	D	p37 m2_m1f_two_plane 5; p38, p39 m2_m1f_two_plane 5

ID	Finding	Class	Evidence / reference
	(status 0xe0); single-plane erases of both blocks and a single-plane program of block 3001 read back exactly		
M2-341	FMCTRL1 0x340 with FMC_DATA_SEL 1 (its wait 2 passing and acknowledged) writes 512 zero bytes to FMC_DMA_ADDR; a DMA out of data buffer 0 programmed after it, with 0, 32 or 128 writes to FMC_FIFO 0x080 in between, leaves its target holding the fill that was there (all 512 bytes, or all but 16 bytes that read 0) and no written value, and no written value reaches buffer 0 (read-backs after 0x340: M2-613; buffer 0 after 0x340: M2-617); with FMC_DATA_SEL 0 the same sequence with 128 writes reads back the buffer (in 3 of 4 runs with 16 bytes at offset 464 or 496 reading 0), as do 128 writes without 0x340 (exact); FIFO reads after the writes return 0	D	p37 m2_m1f_fifo_write 12 (two runs per variant); p38, p39 m2_m1f_fifo_write 12
M2-364	A register-form source field with any of bits 8..15 set reads 0 in these runs, whose preceding instructions read no register (the value read: M2-618): `mov r2` with source 0x103, 0x803, 0x8003, 0xff03, 0x10a or 0x1ff gives 0, `add r2` (register form) leaves r2 = 0x33333333, and `and`/`or`/`sub`/`shl`/`shr` with 0x104 act on 0, where source 3 gives r3 = 0x44444444 and sources 9 and 11..255 give r0 (M2-40); the source field is not decoded from its low byte alone; 0x108 after a `mov` that wrote 0x20 reads 0x20 (M2-614)	D	p37 m2_open_reserved_fields 8; p38, p39 m2_open_reserved_fields 8; p41 m2_r3_reserved_fields 8 (single probes)
M2-365	Broadcast destination r10 combined with an extended source: `mov r10` with source 8 writes the instruction offset (0x20) to every observed GPR (r0..r5, r7); with source 10 each GPR is its own source: `mov r10, sel10` leaves r0..r5, r7 at their values and `add r10, sel10, #5` adds 5 to each (0x11111116 .. 0x8888888d)	D	p37 m2_open_reserved_fields 3; p38, p39 m2_open_reserved_fields 3; p41 m2_r3_reserved_fields 4 (single probes; r10 loads and immediate sub: M2-616)
M2-366	After a broadcast `mov r10, r3` (every GPR 0x44444444) an extended `st` (R = 10) stores 0x11111111, the value r0 held before the broadcast, not the broadcast value	D	p37 m2_open_reserved_fields 1; p38, p39 m2_open_reserved_fields 1
ID	Finding	Class	Evidence / reference
M2-367	An all-ones (erased) instruction word (op 0xff, R 0xff, arg 0xffff, imm 0xffffffff) executes as `nop2`: 1, 8 and 64 consecutive words continue to the END, as does one word with a pending	D	p37 m2_open_erased_ops 11; p38, p39 m2_open_erased_ops 11

ID	Finding	Class	Evidence / reference
	command, address, DMA, ECC operation or status latch, and each word costs 30.2 ns (1 cycle at 33.25 MHz), the cost of a zero-form nop2 (30.1 ns)		
M2-370	sig (10) sets CS_IRQ (C0C) bit 1 and continues in the zero form and with R 1/0xff, arg 0xffff and imm 1/0x80000000/0xffffffff (fields ignored); writing 2 to C0C clears the bit (W1C) and the next sig sets it again; no VIC0 or VIC1 raw line beyond the run baseline rises (VIC0 line 5 only); cost 30.2 ns (1 cycle at 33.25 MHz)	D	p37 m2_open_operand_forms 10; p38, p39 m2_open_operand_forms 10
M2-373	tmwait (15) entered at the start of a run ends 492.747 us × count + 450 us after the start (within 3 us) for start-word count (bits 15:8) 1, 7, 64 and 255: 943, 3898, 31986 and 126103 us, one 2^14-cycle step less than the WAIT timeout of the same start word (M2-81); R 1/0xff, arg 1/0xffff and imm 1..0x10000 give the zero-form time (3897..3900 us at count 7), C58 = 0 the same END, and with C14 written 0x10 before it the time is unchanged (126103 us at count 255), with 0x1000 it was 24 us shorter in two captures (126079/126080 us) and not in a third (126102 us; three more runs: M2-629); C14 reads 0x7f before it and 0 after it	D	p37 m2_open_operand_forms 15; p38, p39 m2_open_operand_forms 15
M2-377	nop (08) with R = 8, 0xa or 0xff (also with arg 0xffff and imm 0xffffffff) does not continue: it holds at its own offset until the timeout fault (4226..4250 us with start word 0x07b5, CS_IRQ bit 2, in runs with the full-map pre-dump before the start, in which an unmet wait 3 also faults at 4242..4246 us; at 4388..4390 us, the unmet-wait time, without it: M2-625); with R = 0 or 1, arg 2/0x8000/0xffff or imm 1/0x80000000 it continues with the GPRs and the full window equal to the zero form	D	p37 m2_open_operand_forms 11; p38, p39 m2_open_operand_forms 11; p39 m2_m1_blk100 3 and m2_c60_hold 2 (unmet wait 3 with the full-map pre-dump)
ID	Finding	Class	Evidence / reference
M2-378	Instruction costs add linearly per copy (1..8 copies inside a 20000-iteration loop, residual ≤ 1.1 us): nop2 (ff), movi, an untaken bnz and a bnz taken to the next instruction 30.1 ns (1.00 cycle at 33.25 MHz), nop (08) 37.6 ns (1.25 cycles), sti to SDRAM 97.7 ns (3.25 cycles); the loop's sub + backward taken bnz costs 67.9 ns (2.25 cycles); 16 copies in a 30000-iteration loop give the same 1.00 cycle for nop2, movi,	D	p37 m2_open_cycle_timing 37; p38, p39 m2_open_cycle_timing 37; p42 m2_r4_c08_running 39 (loop durations)

ID	Finding	Class	Evidence / reference
	`mov` and `add`, 1.25 for `nop` (`08`) and a passing `wait 11`, 61.2 ns (2.03 cycles) for `ld`, `st`, `wr` and `pld` of PRAM or FMSS words, 97.8 ns (3.25 cycles) for `sti` and 199.5 ns (6.6 cycles) for `ldi` from SDRAM, with an empty loop at 67.7 ns per iteration (three runs per kind within 7 us, `ldi` within 33 us)		
M2-384	C08 read by the CPU 3 ms into a run (inside the state) is 4 in every unmet `wait` (0, 1, 2, 3, 4, 6, 13, 15), in a `wait2` 40, in a `tmwait` and in a `wait 3` held by start word 1, 0xb in a zero-field `evjmp` with C58 = 0, 0xa in a `pause`; while a delay loop runs it reads 1 or 2, also after a `sig`, while the program runs on after a timeout fault with C58 = 0 and while a fault handler runs after a vectored fault (C58 = 4; C5C = 1, C3C = faulting offset + 8), and 9 in one run-on and one handler run, the value differing between identical runs (run-on 9/1/2 and handler 1/9/2 in three captures; loop 1/2); C00 reads the start word with bit 18 set and bit 0 clear in every one of these states (0x4ffb4 for 0xffb5, 0x401b4 for 0x01b5; 0x40000 for start word 1); C08 is 0 after every END, also a handler END; the sequencer reads C08 = 5, C3C = 0, C5C = 0 and C14 = 0x7f at the start of each of the 18 runs	D	p37 m2_open_c08_states 18; p38, p39 m2_open_c08_states 18 each (after-maps read 3 ms into the run, inside the state)
M2-386	WAIT 6, 7, 8, 9, 10, 13, 14 and 15 time out (4.39 ms window, in every run: 160 cases in one capture, 192 in each of two later captures) after each of: an unacknowledged DMA out, an unacknowledged 512-byte data read and spare read of an open page, a FIFO READ ID (0x040 = 0x52), a metadata write, a DMA with 0x044 all ones, a BCH8 slot-1 and a BCH16 encode, 0x81c = 1, a 70h on CE1, CE2 or CE3, a pending `sig` (CS_IRQ bit 1), the completion of an uncorrectable and of an erased-sector BCH8 decode (810 = 0x1d / 0x55), a 70h with 0x044 all ones written by the CPU before the start (CS_IRQ bit 4); CPU writes 1 ms into the wait of C40 = 1, C44 = 1, C50 = 0xffffffff, C14 = 0, C5C = 1 or C3C = 0; all VIC0 or all VIC1 soft lines raised from 1 to 2.5 ms	D	p37 m2_open_wait_sources 160; p38, p39 m2_open_wait_sources 160 each; p38, p39 m2_r3_wait_sources 32 each
ID	Finding	Class	Evidence / reference
M2-387	WAIT11 passes at once (END 3..13 us after the start) idle with C58 = 0 or 4, with start word 1 (timeout disabled), during an unacknowledged DMA, during a	D	p37 m2_open_wait11 14; p38, p39 m2_open_wait11 14 each; p38, p39 m2_r3_wait11_busy 5 each

ID	Finding	Class	Evidence / reference
	BCH8 encode, with a command flag or a status latch pending, after acknowledging every flag, with C10 = 0 and with 0x044 all ones, and it passes on every iteration of loops running through NAND array busy periods (1000 after 00h/address/30h, 1000 after FFh, 30000 during a block erase); operands (0x40, 0x40) and (0xffff, 0xffffffff) give the same outcome; a passing WAIT11 costs 37.4..37.7 ns (1.25 cycles at 33.25 MHz), as `nop` (08) and as a passing `wait 0` whose flag is pending		
M2-390	The two faulting WAIT5 orderings of M2-68 (70h, then an acknowledge of bit 1 or one `movi`, then the latch, no wait 0) fault in 18 of 18 runs: straight after the per-case reset and after each of eight predecessor runs (latch with a WAIT15 fault, latch with a passing WAIT12, latch without a wait, two latches, unacknowledged command, unacknowledged DMA, unacknowledged BCH8 encode, FIFO READ ID); FMC_STATUS reads 0xc300a / 0xc3000 (bit 23 clear) before the WAIT5; the fault depends on the ordering, not on an earlier program	D	p37 m2_open_wait5_history 26; p38, p39 m2_open_wait5_history 26 each
M2-391	`evjmp` (16) with R = 1, 2, 4 or 0xff, arg = 1, 2 or 0xffff, or imm = 0xffffffff or a program label gives the outcome of the zero form for a pending command, DMA, BCH8 encode, status latch, command + DMA and DMA + encode (60 runs): it departs to C4C 3..25 us after the start, never to imm, with CS_IRQ 0 in the handler, which sees the source's flag: FMC_STATUS bit 1 and C50 bit 0 (command, command + DMA), FMC_STATUS bit 20 and C50 bit 2 (DMA, DMA + encode, ECC_STATUS 0), ECC_STATUS 0x13 and C50 bit 4 (encode), FMC_STATUS bit 23 and C50 bit 5 (status latch)	D	p37 m2_open_evjmp_forms 60; p38, p39 m2_open_evjmp_forms 60 each
M2-392	On a blocked start = 1 zero-field `evjmp` (C08 = 0xb, PC 0x38), CPU C00 writes at 1 ms and 2 ms leave: STOP (8) C08 0xb, C00 0x40000; abort (2) C08 3, C00 0x40002; STOP then abort C08 3; abort then STOP C08 0xb; 0x80000 alone or after STOP or abort C08 0, C00 0x7f60, PC 0; 0x80000 then STOP C08 0, C00 0; the probe cleanup recovers all eight	D	p37 m2_open_evjmp_forms 8 (reset orders); p38, p39 m2_open_evjmp_forms 8 each (reset orders)

ID	Finding	Class	Evidence / reference
M2-393	No candidate enable write of the 17 tested (C10 = 0/2/3/0xff, 018 = 1/2/0xff, 058 = 1/0xff, 05c = 0x200/0xffff, 01c = 2/0xff, C48 = 1, C54 = 1, 0x400 = 0x1077803, 0x438 = 7) makes a +0x400 word hold a value: 0x40c, 0x410, 0x434 written afterwards and 0x400, 0x438 read 0	D	p37 m2_open_bank_enable 17; p38, p39 m2_open_bank_enable 17
M2-394	A DMA programmed 0 or 16 nop2 after the FMCTRL1 write of a previous one (one run per case in each capture; FMC_DNUM 63..511 and a preceding encode shift the split points only). Out then out (buffer 0 to address P, then buffer 1 to address Q): the second wait 2 never passes (it faults at 126.6 ms, and is still waiting at the end of a 50 ms observation); buffer-1 bytes appear at their own offsets, first at P for 16..52 bytes and then at Q from that offset (56..120) to byte 511, its earlier bytes at neither address; P holds buffer-0 bytes before that run (8..72 bytes, with 4..16 bytes matching neither), after 16 nop2 also after it up to offset 196..236, and is untouched beyond; the running transfer follows the second DMA's FMC_DATA_SEL and FMC_DMA_ADDR writes (M2-603). In then in: both complete; buffer 1 holds the second input except its first 8..64 bytes, which hold first-input bytes at the same offsets or (in some runs) 24..32 bytes matching neither, and buffer 0 holds the first input except 0, 8, 12 or 32 leading bytes, which hold first-input bytes 84..95 or 64..95; the same case gives the same byte map in most runs, and different split points in others	D	p37 m2_open_dma_cut 32; p38, p39 m2_open_dma_cut 32; tagged inputs: M2-697, M2-698
M2-396	FMC_DMA_CTRL (038) reads back every written value 0x40..0x1000 (alone and with bits 2:0 = 7), 0x1fc7 and 0; with any of bits 12:6 set the next DMA never completes (wait 2 faults), with 0 the loopback is exact (presets before a DMA out, bits 5:3 and 31:13: M2-669)	D	p37 m2_open_dma_ctrl 16; p38, p39 m2_open_dma_ctrl 16
ID	Finding	Class	Evidence / reference
M2-398	During a DMA with 038 = 7, bits 12:6 count up from 0 in steps of 8 (0x200; single samples show values in between) and return to 0 (038 = 7) at completion: inbound 13..16 samples of 188 ns, outbound 5..6 samples, reaching 0x1c07..0x1e07 (bits 12:6 = 112..120), the same	D	p37 m2_open_dma_ctrl 6; p38, p39 m2_open_dma_ctrl 6; p39 m2_r3_dma_ctrl_low 10

ID	Finding	Class	Evidence / reference
	sequence for FMC_DNUM 3, 15, 63, 255 and 511 (a DMA moves 512 bytes whatever FMC_DNUM holds, M2-189); bits 5:3 stay 0 and bits 2:0 stay 7; the field and bits 2:0 together: M2-609 (word count and burst length, inferred)		
M2-399	A DMA into a buffer under its own BCH8 or BCH16 encode (0, 8 or 32 nop2 after the kick) completes, but 68..164 bytes of the old data stay in runs of 4..32 bytes; the encode parity matches neither the old nor the new data at gap 0 and 8 and the old data at gap 32; FMC_STATUS bit 25 sets. After 128 nop2, or with the DMA after WAIT4 (serial), the buffer holds the new data, the parity is that of the old data and bit 25 stays clear	D	p37 m2_open_ecc_collision 10; p38, p39 m2_open_ecc_collision 10
M2-401	A DMA out of a buffer under its own BCH8 or BCH16 encode (0, 8 or 32 nop2 after the kick) delivers 344..424 of the 512 bytes: runs of 12..32 bytes about every 96 bytes differ, while the encode parity is that of the buffer data and the buffer is unchanged; FMC_STATUS bit 25 sets. After 128 nop2 the output is exact and bit 25 stays clear	D	p37 m2_open_ecc_collision 8; p38, p39 m2_open_ecc_collision 8
M2-405	A 512-byte buffer loopback is exact with 018 = 1/2/3/0xff, 058 = 1/0x10/0x100/0xff or 05c = 0x100/0x200/0x1ff/0x3ff written before it; afterwards 018 reads the written value & 0xf, 058 reads 0 and 05c reads 0x100 whatever was written (058 and 05c are FMC_DATA_SEL bits 7:0 and 15:8, which the loopback's DMA out set to 0x100: M2-643). A DMA in with FMC_DATA_SEL 0 and 058 = 1, 0x10 or 0x100 leaves buffer 0 holding the data the preceding cases left there; with distinct contents in both buffers it changes neither data buffer (M2-597)	D	p37 m2_open_fmc_words 16; p38, p39 m2_open_fmc_words 16
M2-407	FMC_CTRL0 bits 30:28 = 0, 1, 3, 7 give identical FMC_STATUS timelines (188-ns samples) for a 70h command (bit 1 at the first sample), a status latch (bit 23 at the first sample), an address-to-data read (bit 3 at the fourth sample) and a command after the chip became ready (bit 1 at the first sample)	D	p37 m2_open_fmc_words 16; p38, p39 m2_open_fmc_words 16
ID	Finding	Class	Evidence / reference
M2-411	FMCTRL1 (004) single bits with FMC_DNUM 15, DATA_SEL 1 and	D	p37 m2_open_op_words 32; p38, p39 m2_open_op_words 32

ID	Finding	Class	Evidence / reference
	no CE selected: bit 0 starts an address operation (C50 bit 1 at once, FMC_STATUS bit 2 within 200 loops), bit 1 a data read (C50 bit 3, FMC_STATUS 0xc3000 → 0x8041008: bits 3 and 27 set, 13 and 19 cleared), bits 8 and 9 a DMA (038 counting at once, then FMC_STATUS bit 20 and C50 bit 2); the other 28 bits alone change none of 048, C50, 038, 04c, C08, 810		
M2-412	Writing all ones to FMC_STATUS (048) clears the pending DMA flag (bit 20) and leaves bits 12, 13, 18, 19 set; writing 0 changes nothing, and writing ones when idle sets no bit. FMC_NSTAT (04c) ignores writes of all ones and of 0 (reads 0 with no CE selected)	D	p37 m2_open_op_words 5; p38, p39 m2_open_op_words 5
M2-417	FMC 0x01c stores bit 0 only (written 2, 4, 8, 0x100, 0x80000000 read 0; 3, 7, 0xff read 1) and an encode uses the data buffer of slot (bit 0): the parity at 0x840.. equals the host parity of the slot-0 data for even values and of the slot-1 data for odd values (BCH8; BCH16 with 01c = 2: slot 0); WAIT4 passes in every case	D	p37 m2_open_ecc_slots 9; p38, p39 m2_open_ecc_slots 9
M2-418	Two encode kicks back to back (01c = 0, kick; 01c = 1, kick; then WAIT4) give one result: 0x840.. holds the slot-1 parity, 810 reads 0x13, and after the acknowledgement (810 = 1) 810 reads 0x12 with the same slot-1 parity; the second kick replaces the first and no slot-0 parity appears	D	p37 m2_open_ecc_slots arb/back-to-back; p38, p39 m2_open_ecc_slots arb/back-to-back
M2-419	Changing 01c from 0 to 1 while a slot-0 BCH8 encode runs gives a parity at 0x840.. that matches neither slot's data (810 0x13)	D	p37 m2_open_ecc_slots arb/slot-change-mid; p38, p39 m2_open_ecc_slots arb/slot-change-mid
M2-420	810 bits 7..12, 16, 24 and 31 are never set, neither by writing 1 to them nor by a BCH8 encode; bit 4 reads 1 when idle and after the encode (0x10 / 0x13) and writing 1 to it changes nothing	D	p37 m2_open_ecc_status_bits 10; p38, p39 m2_open_ecc_status_bits 10
M2-421	A write of 0x81c with bit 0 set (1 or 0xffffffff; slot 0 or 1) after a completed encode changes 810 from 0x13 to 0x10, and a following WAIT4 faults; values without bit 0 (0, 2, 4, 0x80, 0x100) change nothing and WAIT4 still passes; 818, 80c and 048 stay unchanged (0, 0, 0xc3000)	D	p37 m2_open_ecc_status_bits 8; p38, p39 m2_open_ecc_status_bits 8

ID	Finding	Class	Evidence / reference
M2-422	CPU C00 sequences 1 ms (and 2 ms) into a running loop: STOP (8), abort (2) and STOP then abort keep r0..r5, PRAM 0xd00/0xd70, C04, C10 and C4C (after STOP C08 0 and C00 0x40000, after abort C08 3 and C00 0x40002, C40/C44 holding the last instruction); 0x80000 alone or in any order with STOP or abort clears r0..r7, 0xd00/0xd70, C04, C10, C4C, C40 and C44 and leaves C00 0x7f60 (0 when STOP follows); no sequence changes FMC 0x00c..0x038, 0x058, 0x05c or ECC_CTRL 0x804	D	p37 m2_open_reset_stages 8; p38, p39 m2_open_reset_stages 8 each
M2-430	The power-on window (first FMSS access after a power cycle, DFU and the kernel boot, which does not touch the FMSS) equals the window after the ordinary reset in every word of 0x000..0xffc except C00 (0x7f60 at power-on, the preflight start word 0xffb4 after the reset) and C04 (0 at power-on, the preflight program address after the reset); it equals word for word the untouched window after a probe cleanup; power-on values include C14 0x7f, C08 0, C50 0xff1800, CS_IRQ 0, 0x004 0x40000000, 0x020 0x7f167f, 0x038 7, 0x048 0xc3000, 0x1cc 0x8060, 0x1d0 0x3010d0, 0x800 0x20070618, 0x808 0x700, 0x810 0x10, and every other GPR, PRAM and FMC/ECC storage word 0	D	cold m2_cold_boot 4, m2_m1_poweron_values 4
M2-431	After power-on the first start (an END program) ends at once, and from a window identical to the power-on window a program whose instruction 0 is not END (movi, st [0xd00], end) runs to its END on its first start (r2 and 0xd00 hold the marker); no priming start is needed	D	cold m2_cold_boot first-program
M2-432	`ldi` from 0x20000000 + off and from off return the same word for each of off = 0, 4, 0x60, 0x100, 0x1000, 0x8000, 0xfffc, but not the boot-ROM contents: 0xe1a0f00e for off = 0..0x1000, 0xe59f42b0 for 0x8000, 0 for 0xfffc (the ROM image holds 0xea000027 at 0 and 0xe1c120b0 at 0x8000)	D	p37 m2_open_low_bus 7; fw/ipod4g_bootrom.bin; p38, p39 m2_open_low_bus 7; per-quarter words: M2-682, M2-683
ID	Finding	Class	Evidence / reference
M2-433	With start word 0x00b5 (count 0) a `wait 3` entered 4..407 us after the start faults at 446..448 us and one entered at 477 or 544 us faults at 940/941 us; with 0x0075 (count 0, n = 7) a wait entered 3..17 us after the start faults at 16..18 us and one	D	p37 m2_open_cycle_timing 16; p38, p39 m2_open_cycle_timing 16 (phase cases)

ID	Finding	Class	Evidence / reference
	entered at 23/29 us at 46/48 us: the timeout fault falls on instants fixed relative to the start (447.5 + 492.747 k us for n = 0xb, period 2^14 cycles; ≈ 16.5 + 30.8 k us for n = 7, period 2^10 cycles), the first one after the wait entry for count 0, not a fixed time after the entry		
M2-434	With 0x808 bits 12:8 = 0 (0x808 written with a single bit outside 8..12; storage mask 0x1f81) 810 bit 5 is set after every BCH8 encode and clean decode (0x33 / 0x35); with one of bits 8..12 set it stays clear (0x13 / 0x15); the reset value 0x700 has bits 12:8 = 7, the count from which bit 5 is set (M2-94, M2-100), so bits 12:8 are inferred to be the bit-5 threshold	D,I	p37 m2_m1f_ecc_csirq bit/808/b00..b31 (27 of 32 set bit 5)
M2-435	CPU writes alone, with no sequencer program, drive the NAND controller and ECC engine with the programs' register sequences (each WAIT replaced by a bounded poll of its flag): raw page reads by DMA and by FIFO 0x080 return the same bytes as the sequencer raw read for nine pages (reserved region rows 0, 1, 2, 100; main rows 128, 129, 256000, 383872; a sequencer-written page); CPU-kicked decodes in the boot-ROM format (three sectors of 26 parity and 512 data bytes, BCH16, 0x808 = 0x80) and the main BCH8 format (meta[10], parity, data) equal the raw data and metadata and the firmware readers' output; a CPU erase leaves pages 0, 1, 127 all 0xff and a CPU program with host BCH8 parity reads back exactly by both raw paths, CPU decode (all ring results 0) and the sequencer READ (status 0xe0 after each)	D	p37 m2_cpu_nand 73 (68 checks match); p38, p39 m2_cpu_nand (68 checks match in each)
M2-436	CPU reads of FMC_FIFO 0x080 return page data in 32-byte transfers: per chunk FMC_DNUM = 31, FMC_DATA_SEL = 1, FMCTRL1 = 0x1c2, poll FMC_STATUS bit 3, then FMC_DATA_SEL = 0x100, FMCTRL1 = 0x340 and eight reads of 0x080; the result equals the DMA raw read for every tested page; CPU-kicked encodes give the host BCH8 parity (slots 0 and 1, with metadata 0xa02) and BCH16 parity (slot 0, with metadata 0xa06)	D	p37 m2_cpu_nand cpu/raw-fifo (10), cpu/encode (5); p38, p39 m2_cpu_nand

18.23 P38 and P39

ID	Finding	Class	Evidence / reference
M2-494	After a status latch (70h, wait 0, acknowledge, latch FMCTRL1 0xca, WAIT5) and only the acknowledge of bit 23 (0x800000), with no FMCTRL1 0x30020 release, a second latch (0xca, no new 70h) passes WAIT5 and returns 0xe0 with no operation between and after each of: FMCTRL1 0x20, a status clear 0x0ff00ffe, a CE toggle, a BCH8 encode with its WAIT4, the set-up word, FMCTRL1 0x100e0 and the release itself; a latch needs no re-arm. Of the nine operations only the address operation does not complete in that position (M2-610)	D	p38 m2_m1f_latch_rearm 9; p39 m2_m1f_latch_rearm 9 (one run per case each)
M2-497	The set-up word (FMCTRL1 0x33338e0) restores FMC_STATUS bits 19:12 to 0xc3 after a short read as after a 512-byte read: 0x8041000 after a 23-byte read and 0x803c000 after a 512-byte read read 0x80c3000 after the set-up word (bit 27 kept) and 0xc3000 after the 0x0ff00ffe acknowledge; with a write of 0 to 048 in its place 0x803c000 stays	D	p38 m2_m1f_setup_bits 3; p39 m2_m1f_setup_bits 3
M2-512	A CPU resume from pause written 1 ms after the start as PC = offset + 8 followed by two start words 0xffb5 continued the program to END in three of four runs and left it paused in one (C08 = 0xa, PC = the pause offset, no END within 5 ms); in the same runs the resume with one start word, with or without the acknowledgement, always continued (M2-188)	D	p36, p37, p39 m2_m1_pc_write (resumed); p38 m2_m1_pc_write (paused), 1 case each
M2-583	Every undefined opcode 0x1a..0xfe with a pending command, address, DMA, ECC or status flag acts as an unmet wait, as when idle (M2-282): the instruction after it is not reached and the timeout fault comes at the time of an unmet `wait 3` of the same start word: 4388..4390 us with 0x07b5 for 0x1a, 0x80 and 0xfe (M2-81), and 1416..1450 us with 0x01b5 (count 1; unmet `wait 3` 1432 us, M2-584) for each of the 226 values 0x1b..0xfd other than 0x80 in all five flag states, CS_IRQ 5 at the END; no flag state lets one pass	D	p38 m2_r3_undefined_pending 15; p39 m2_r3_undefined_pending 15; p42 m2_r4_undefined_scan 1130
ID	Finding	Class	Evidence / reference
M2-584	Timeout instants are counted from the wait's entry. With n = 0xb (instants 447.5 + 492.747 k	D	p38 m2_r3_tmwait_rule 15; p39 m2_r3_tmwait_rule 15

ID	Finding	Class	Evidence / reference
	us after the start, M2-433) a `tmwait` of count 1, 2, 3 entered at the start ends at 942, 1434, 1927 us and a `wait 3` of the same count faults at 1432, 1926, 2418 us; entered 474 us after the start (after the first instant) each ends one instant later (`tmwait` count 1 at 1436 us, `wait 3` count 1 at 1926 us). A count-0 `tmwait` does not end: like a count-0 wait it takes the timeout fault at the first instant after its entry (446 us entered at the start, 940 us entered at 474 us). Counted from the entry, a `tmwait` of count c ≥ 1 ends at the (c + 1)-th instant and a wait of count c ≥ 1 faults at the (c + 2)-th; with n = 7 and n = 9 (count 1) `tmwait` ends at 50 / 203 us and `wait 3` faults at 78 / 324 us, the second and third instants of those step fields (30.8 / 123.2 us apart). C14 reads 0 after every `tmwait` END		
M2-588	A count-0 `wait 3` entered at the start faults at the first timeout instant: 446..448 us for n = 0xb..0xf (the step saturates at n = 11), 202 us for n = 0xa, 78 us for n = 9, 16 us for n = 7 and 8, 4 us for n = 4 and 0..2 us for n = 5 and 6, the same in a repeat and in a second capture. For n = 7..0xb these instants are (2^(n+3) − φ mod 2^(n+3)) engine cycles after the start with one phase φ ≈ 1488 cycles (16.8, 16.8, 78.4, 201.6, 447.5 us predicted); the same phase predicts 1.4 us for n = 4..6, within the 2-us resolution of the measured 0..4 us (repeats: M2-631). With 64 extra CPU parameter writes before the start the first instant comes 4..6 us earlier relative to the start (72..74 us for n = 9, 444 us for n = 0xb), so the instants are fixed to an event before the start, not to the start itself (inferred: the prescaler runs before the start; the per-case reset sets its phase: M2-631)	D,I	p38 m2_r3_prescaler_phase 19; p39 m2_r3_prescaler_phase 19
ID	Finding	Class	Evidence / reference
M2-593	An unwaited 70h followed by the latch (FMCTRL1 0xca) with k nop2 between: k = 0 passes WAIT5 (FMC_STATUS 0x8c300a: bits 1, 3 and 23; status 0x70); k = 1 and 2 fault at the timeout, with bit 23 (and 24) never set in 24 samples of 188 ns after the latch and FMC_STATUS 0xc300a (bit 3 set) before the WAIT5; k = 3, 4, 6, 8, 12, 16 and 20 pass (0x8c3002, bit 3 clear, status 0xe0). One `nop` (08), one `movi`	D	p38 m2_r3_latch_gap 16; p39 m2_r3_latch_gap 16 (one run per case each)

ID	Finding	Class	Evidence / reference
	(three runs) or the acknowledge of bit 1 between fault like k = 1; the acknowledge followed by 20 nop2 passes. A latch written one or two instructions (30..60 ns of nop2) after the 70h write sets bit 3 instead of bit 23		
M2-596	A sequencer ld of 0x40c (a +0x400 word) costs the same as an ld of the decoded FMC word 0x00c and of the unused offset 0x60c: 60.6..60.9 ns (2 cycles at 33.25 MHz, the window access cost of M2-102), the three equal within the 1-us resolution of 20000-iteration loops; access time does not separate a decoded register slot from undecoded space	D	p38 m2_r3_bank_decode 6; p39 m2_r3_bank_decode 6
M2-597	A DMA in with FMC_DATA_SEL (014) = 0, after data buffers 0 and 1 were loaded with distinct data, completes (its wait 2 passes, 038 reads 7 and 048 0xc3000 after the acknowledge) but changes neither data buffer, for 058 = 0, 1, 2, 0x10 and 0x20 alike: the DMA's own 014 = 0 write follows the 058 write and replaces it, since 058 is 014 bits 7:0 (M2-643)	D	p38 m2_r3_select058 5; p39 m2_r3_select058 5
M2-599	CPU writes of 058 = 0x10 and 05c = 0x200 before the start are read back unchanged by the sequencer (after sequencer writes and a buffer loopback, 058 reads 0 and 05c 0x100: M2-405)	D	p38 m2_r3_select058 1; p39 m2_r3_select058 1
ID	Finding	Class	Evidence / reference
M2-603	A buffer-to-RAM DMA in flight follows FMC_DMA_ADDR (034) and FMC_DATA_SEL (014) as they are rewritten: 034 written at once after the FMCTRL1 0x100 write (or 16 nop2 later) sends bytes 32..511 (256..511) to the new address at the same offsets while bytes 0..31 (0..255) stay at the old one; 014 = 0x200 (data buffer 1) makes bytes 8..511 (232..511) come from buffer 1 at the same offsets. Rewriting FMC_DNUM or writing FMCTRL1 0x100 again changes nothing (one transfer, one completion). 038 = 7 written at once changes bytes 8..23 of the output, 16 nop2 later nothing, and both leave 038 = 0x187 after completion instead of 7. A complete second buffer-to-RAM DMA written over the running one (038, FMC_DNUM, 014 = 0x200, 034, FMCTRL1) gives the byte map of M2-394 with a single completion: no second transfer starts and the running one	D (burst and live-register reading I)	p39 m2_r3_dma_supersede 18

ID	Finding	Class	Evidence / reference
	continues with the new select and address. A RAM-to-buffer DMA follows the same registers: 034 rewritten at once fills buffer 0 bytes 32..511 from the new source; 014 = 2 written at once puts all 512 bytes into buffer 1 and leaves buffer 0 unchanged; with both rewritten buffer 1 receives bytes 0..31 of the old source and 32..511 of the new one. wait 2 passes once in every case (one run per case)		
M2-609	With FMC_DMA_CTRL (038) bits 2:0 = v written before a 512-byte RAM-to-buffer DMA, the DMA completes and the buffer is exact for every v = 0..7, and the transfer takes longer for smaller v (bits 12:6 still changing after 48 samples of 188 ns for v = 0, then 46, 32, 33, 25, 24, 16 and 17 samples for v = 1..7). After completion 038 reads v with bits 12:6 = (v + 1) × ceil(128 / (v + 1)) − 128: 0 for v = 0, 1, 3, 7 and 1, 2, 4, 5 for v = 2, 4, 5, 6 (038 = 0x42, 0x84, 0x105, 0x146; the 2 → 0x42 and 4 → 0x84 readbacks of M2-79). During the DMA bits 12:6 advance in multiples of 2 for v = 1 and of 3 for v = 2 and mostly by 8 for v = 7; bits 31:13 and 5:3 stay 0. With v = 7, FMC_DNUM 3 or 15 (in) and 3 (out) give the full 512-byte transfer and the field sequence of a 512-byte DMA. Inferred: bits 12:6 count the 4-byte words moved (128 per DMA, modulo 128) and bits 2:0 set the burst length to v + 1 words	D,I	p39 m2_r3_dma_ctrl_low 10
M2-610	After a status latch, its WAIT5 and only the acknowledge of bit 23 (no FMCTRL1 0x30020 release), an address operation (0x02c = 0, 0x00c = 0, FMCTRL1 = 1) does not complete: its wait 1 times out at the 4.39 ms timeout (PC held at the wait), FMC_STATUS bit 2 stays clear and bit 26 sets (0x40c3000); a command refused after a latch leaves bit 26 clear (M2-181)	D	p38 m2_m1f_latch_rearm address (1); p39 m2_m1f_latch_rearm address (1)
ID	Finding	Class	Evidence / reference
M2-611	USB communication was lost during the resident run of the reserved-field family (high source- and address-selector bits, r10 and r8 destinations of loads and ALU results), with no record of any case reaching the host and partly drawn characters on the device LCD; which case was running is not recorded. The same 17 forms, run one per probe with every GPR holding a valid SDRAM	I	p38 m2_r3_reserved_fields (no records; USB lost, LCD corrupted line); p40 m2_r3_reserved_fields (USB lost again, reported; no capture retained); p41 m2_r3_reserved_fields 17 (single probes, every register a valid address)

ID	Finding	Class	Evidence / reference
	address, all end normally (M2-614, M2-616): the r10 and r8 destinations never reach the PC, while the address selector 0x105 of `ldi` uses r0 (M2-615). In the lost run r0 held 0x11111111, so that `ldi` read bus address 0x11111110 (M2-05). Inferred, not measured: that load of an unmapped address hung the bus, which also stalled the CPU's LCD refresh (the partly drawn characters) and USB		
M2-612	C58 bits 1, 4, 6 and 7 are stored and change neither the timeout path nor the handler: with C58 = 0x40, 0x80 or 0xc0 an unmet `wait 3` (start word 0x07b5) takes the timeout fault at 4388..4390 us and the program continues (C5C 0), with 0x44 or 0x84 it vectors to C4C (C5C 1 in the handler); with C58 = 0, 2, 0x10 or 0x12 an unmet `wait 3`, a count-0 `tmwait` and the undefined opcode 0x1a (start word 0x00b5) take the fault at 446..448 us and continue (C5C 0, C08 5, C0C 4), with 4, 6, 0x14 or 0x16 they vector (C5C 1, C08 5, C0C 4 in the handler); the sequencer reads C58 back as written (0x2, 0x6, 0x10, 0x12, 0x14, 0x16, 0x40, 0x80, 0xc0, 0x44, 0x84); only bit 2 selects the path	D	p38 m2_r3_control_words 5; p39 m2_r3_control_words 5; p42 m2_r4_c58_bits 24
M2-613	After FMCTRL1 0x340 with FMC_DATA_SEL 1 (512 zero bytes written to FMC_DMA_ADDR = sink S, wait 2 passing and acknowledged, 038 reading 7 afterwards) no read-back DMA out of data buffer 0 returns the buffer: with 038 polled until its bits 12:7 (the upper six bits of the bits 12:6 field, M2-398) read 0 (at once), with 038 rewritten to 7, or read twice with 3000 delay loops between, each read-back (its wait 2 passing) leaves 038 at 0x207, 0x307 or 0x407 and delivers at its own address, or 0x200 above it, 0..512 bytes equal to the RAM content at S + 0x200 (the bytes following the 0x340 output), runs of zeros, or (second read-back, no writes) the original buffer data in its last 384..416 bytes; 128 FIFO writes (0x5a000000 + k) before the read-back give the same kinds of output and no written word appears anywhere. The same RAM bytes from S + 0x200 on enter buffer 0 after 0x340 (M2-617); inferred: the read-backs continue that transfer rather than reading the buffer	D (continuation I)	p39 m2_r3_fifo_readback 6; p40b m2_r3_fifo_readback 6 (one run per case each; the same kinds of output, byte counts differing between the runs)

ID	Finding	Class	Evidence / reference
M2-614	A register-form source selector with any of bits 8..15 set reads 0 in these runs (preceded by `wr`; the value read: M2-618) also for `and`, `or`, `sub`, `shl` and `shr` (selector 0x104: `and r2` gives 0, `or`/`sub`/`shl`/`shr r2` leave r2 = 0x33333333) and for `mov` with 0x10a and 0x1ff (r2 = 0), so neither the low byte (r4, or the destination for 10) nor the r0 fallback of selectors 9 and 11..255 (M2-40) is used; one exception: `mov r2` with selector 0x108 straight after `mov r3` with selector 8 (which wrote its own offset 0x20) gives 0x20, not its own offset 0x28 and not 0 (the value the preceding `mov` read through selector 8, M2-618)	D	p41 m2_r3_reserved_fields 8 (single probes)
M2-615	An address selector with bits 8..15 set uses r0, the fallback of the undecoded address selectors (M2-48), not its low byte: `ldi r2` with selector 0x105, with each GPR pointing at its own SDRAM marker word, returns r0's marker (0x5e1000; r5's is 0x5e1005); `pld r2` and `pst r3` with selector 0x101 and r1 = 0xd68/0xd64 neither read 0xd64 (r2 = 0, not 0xcafe0064) nor write 0xd68 or offset 0 (FMC_CTRL0 still 0), as an access through r0 = 0x11111111 predicts (offset 0x111, unaligned: read 0, write dropped, M2-175)	D	p41 m2_r3_reserved_fields 3 (single probes); sti: M2-680, M2-681
M2-616	Loads and an immediate ALU result with destination r10 broadcast like `mov r10` (M2-43, M2-365): `ld r10,[0xd70]`, `pld r10,[r1]`, `ldi r10,[r5]` and `sub r10,r0,#1` write the loaded or computed value (a program offset) to r0..r5 and r7 and do not write the PC: the code at that offset (a store of 0x7ad to 0xd74) is not executed and the program runs on to its END. Loads with destination r8 do not jump: `ld r8,[0xd70]` and `pld r8,[r1]` with 0xd70 holding the offset of a later instruction leave every GPR unchanged, the next instruction (`movi r2,#0xbad`) executes and the program ends normally (0xd74 = 0xbad); unlike `movi`/`mov`/`add` (M2-43) a load into r8 is discarded	D	p41 m2_r3_reserved_fields 6 (single probes)

18.24 P42

ID	Finding	Class	Evidence / reference
M2-617	After FMCTRL1 0x340 with FMC_DATA_SEL 1 (FMC_DMA_ADDR = S; its wait 2 passing and acknowledged; S..S + 0x1ff written with zeros, M2-341) a transfer from RAM S + 0x200 on into data buffer 0, at the same offsets, follows with no further FMCTRL1 write. A page programmed from buffer 0 by a 512-byte data-out transfer (80h, address, FMC_DATA_SEL 0x100, FMCTRL1 0xe4, 10h, ready) holds RAM S + 0x200..S + 0x37f in words 0..95 and the buffer's earlier content in words 96..127 when no FIFO write comes between (038 bits 12:6 = 24 seven instructions after the acknowledge and 96 after the data-out transfer, FMC_STATUS bit 20 not set again, 03c = S + 0x200), and RAM S + 0x200..S + 0x3ff in all 128 words with 32 or 128 writes of 0x5a000000 + k to FMC_FIFO 0x080 between (038 bits 12:6 = 88 after the 32 writes; bit 20 set again; 03c = S + 0x400 at the end). With a DMA out of buffer 0 to S' programmed right after the acknowledge, words 8..15 hold S + 0x220..S + 0x23f, words 0..7 S + 0x240..S + 0x25f and words 16..127 the earlier content, and S' is not written (03c = S' + 0x400 at the end). With FMC_DATA_SEL 0 the 0x340 also zeroes S, and after 128 writes the page holds the earlier content. Without 0x340, 0 or 128 FIFO writes leave the page equal to the earlier content. No written value appears in any page. Within the 16-KiB scratch only S + 0x200 holds the 512 bytes of the 128-word pages. Inferred: the second transfer runs at DMA speed, not driven by the FIFO writes, and stops when the data-out set-up rewrites FMC_DATA_SEL (M2-603); the DMA out's 038 write restarts its buffer offset at 0	D,I	p42 m2_r4_fifo_page_readback 16 (one run per case; each page raw-read in a separate case); completion and stop: M2-703..M2-705

ID	Finding	Class	Evidence / reference
M2-618	A register-form source selector with any of bits 8..15 set (0x104, 0x108 and 0x1ff alike) reads the value that the last source selector before it read: after `movi r3` (source field 0) r0's 0x11111111, after `mov r3` with selector 5 r5's 0x66666666, after `add r3,r3,#1` the old r3 (0x44444444, not the sum), after `mov r3` with selector 8	D	p42 m2_r4_bus_latch 30 (one run per case); ld/st/wr between: M2-674..M2-677

ID	Finding	Class	Evidence / reference
	that instruction's offset 0x20 (M2-614); a `nop2` between `movi` and the read keeps 0x11111111; with `ld`, `st` or `wr` as the only instruction before it after the prologue's `wrs`, or none, it reads 0. `add`, `or` and `sub` with selector 0x104 after `movi` use the same value (r2 = 0x33333333 gives 0x44444444, 0x33333333, 0x22222222). Whether `ld`, `st` and `wr` set the value to 0 or keep the value from the reset is not separated by these runs		
M2-622	A write to a discarded extended destination does not change the latch that the extended `st` stores (M2-47): after `movi r3,#0x1a2b3c4d`, a `movi r12,#0x5e5e1212` or a `mov r12` with selector 5 (r5 = 0x66666666) followed by the extended `st` (R = 9) stores 0x1a2b3c4d	D	p42 m2_r4_bus_latch 2
M2-625	`nop (08)` continues for R = 0..7 and holds until the timeout fault for each R = 8..0xff (248 values, start word 0x07b5: fault at 4368..4390 us, 246 of them at 4388..4390 us, the time of an unmet `wait 3`, CS_IRQ 5); with a pending command, address, DMA, ECC or status flag R = 8..15 hold in the same way (40 cases, 4374..4390 us), so none of these flags releases them. A loop of `nop` (R = 0) with start word 0xffb5 leaves C14 = 0xff (start-word bits 15:8) after the END, as a loop of passing `wait 11` does, while loops without a wait leave 0x7f; `nop` costs 1.25 cycles, as a passing wait (M2-378, M2-387). Inferred: `08` is a wait whose condition holds for R < 8	D,I	p42 m2_r4_nop_operand 296; p42 m2_r4_c08_running 9 (nop, wait 11, nop2 loops); C08 during the hold: M2-685, M2-686
M2-627	With a count-1 `tmwait` (start word 0x01b5) and C48 written by the CPU: C48 = 0 and each single bit 8..31 let it end at 941..943 us with C14 = 0; C48 = 1 ends it one timeout instant earlier, at 449 us, with C14 = 1; each single bit 1..7, 0x55555555, 0xaaaaaaaa and all ones make it take the wait timeout fault (1434..1436 us, C14 0x7f). Inferred: a `tmwait` ends at the instant at which C14 equals C48 bits 7:0, and never when that value exceeds its count; bits 31:8 do not act on it	D,I	p42 m2_r4_c48_bits 36 (one run per value); count 7: M2-687, M2-688
ID	Finding	Class	Evidence / reference
M2-629	Program writes of C14 (0, 0x10, 0x100, 0x1000, 0x2000, 0xffff, 0xffffffff) before a count-255 `tmwait` leave its end at	D	p42 m2_r4_c14_write 17; repeat runs: M2-689, M2-690

ID	Finding	Class	Evidence / reference
	126101..126102 us, within the three unwritten controls (126099..126101 us); 0x1000 gives 126102 us in three runs and changes nothing at count 0x10 (8333 us) and count 1 (943 us); one run with 0x800 ended at 126088 us. The 24-us earlier end with 0x1000 (M2-373) does not recur		
M2-631	A count-0 `wait 3` entered at the start takes its fault at the first timeout instant at the same time in three runs after the per-case reset: 16/16/18 us for n = 7, 78/78/78 us for n = 9, 448/446/446 us for n = 0xb, 4/0/0 us for n = 4 and 0 us for n = 5 and 6. With 30, 60 and 90 extra CPU parameter writes before the start the instant comes earlier, by about 0.08 us per write (n = 9: 76/74/70 us; n = 0xb: 444/442/440 us). Without the per-case reset before the run it comes elsewhere: 26/10 us for n = 7, 98/96 us for n = 9, 96/100 us for n = 0xb. The per-case reset sets the prescaler phase, and the prescaler runs while the parameters are written (inferred: from a step of that reset)	D,I	p42 m2_r4_prescaler_origin 30 (no-reset cases skip the reset by design); single reset steps: M2-691, M2-692
M2-641	After a status latch (70h waited, FMCTRL1 0xca, wait 12, acknowledge 0x1800000) without the 0x30020 release, a DMA in, a DMA out, a 512-byte data read, a 16-byte parity read, an 8-byte FIFO read (FMC_DATA_SEL 0x10), a BCH8 encode and a second latch complete as after the release (no timeout; 048, 810, 038 and 04c the same), while a 70h command and an address operation time out (4388 us, C58 = 0): the address operation with FMC_STATUS bit 26 set (0x40c3000), the command without (M2-181, M2-610)	D	p42 m2_r4_latch_refusal 18
M2-642	FMC_STATUS bits 19:12 (0xc3 before) after a data read of L bytes from an open page: 0x41 (bits 12, 18) for L = 1, 2, 3, 4, 5, 8, 16, 31, 32, 33, 64 and 128, 0x14 (bits 14, 16) for 256 and 384, 0x3c (bits 14..17) for 511 and 512; 0x41 after parity reads of 13, 16, 23 and 64 bytes; 0xc3 after a DMA in or out; 32 writes to FMC_FIFO 0x080 leave 0x41 and 128 writes 0x3c (bit 27 set after each data read, not after FIFO writes)	D	p42 m2_r4_status_bits 22; p42 m2_r4_fifo_page_readback (048 after the writes); lengths 160..508: M2-706, M2-707

ID	Finding	Class	Evidence / reference
M2-643	FMC 0x058 is FMC_DATA_SEL (014) bits 7:0, and 0x05c reads 014 bits 15:8 in place: after 014 = 0x5541, 058 reads 0x41 and 05c 0x5500; 058 = 0xd written after it gives 014 = 0x550d, 058 = 0xff gives 0x55ff; 058 = 0xd alone gives 014 = 0xd; 014 = 0xffff then 058 = 0 gives 0xff00; 05c = 0x200 written after 014 = 0x0101 and 058 = 0x20 gives 014 = 0x220, 058 = 0x20, 05c = 0x200, and the low byte of a 05c write is not stored; a 058 write before 014 is replaced by it; 018 is separate (reads 0xd, 014 unchanged). Sequencer writes, CPU parameter writes and CPU writes during the run give the same values, so the 0x550d of M2-83 is a 058 = 0xd write after 014 = 0x5541	D	p42 m2_r4_data_sel_alias 19; p42 m2_r4_reset_steps (no-stage readbacks)
M2-645	FMCTRL1 bit 31 (0x80000000 alone, after 0x800 or as 0x80000800) zeroes FMC 00c, 010, 014, 018, 01c, 02c, 030, 034, 058 and 05c and sets FMC_DMA_CTRL (038) to 7 (1 written before), and leaves ECC_CTRL (0x804); 0x81c = 1 zeroes ECC_CTRL and none of the FMC words; FMCTRL1 0x800 changes none of them; CPU parameter writes and sequencer writes give the same result. These two steps account for the FMC and ECC_CTRL zeroing of the complete per-case reset (M2-123); the C60/C64 handshake is not written	D	p42 m2_r4_reset_steps 16
M2-660	0x814 = 1, 2, 4, 8, 0x20 or 0x2f (read back as written) leaves an eight-sector BCH8 decode of a host-coded page with 0, 1, 4, 8, 9, 0, 2 and 12 bit errors per sector identical to one with 814 = 0 (per-sector 810 0x15/0x35/0x1d, 818 totals, the 4096 decoded bytes and 644..645 us), and the BCH8 parity of a 512-byte buffer encode identical (810 0x13)	D	p42 m2_r4_ecc_814 16 (scratch block programmed with host BCH8 code words); BCH16: M2-708..M2-710
M2-662	With metadata words 0x060..0x068 holding 0xa0..0xa9, 0xff, 0xff and ECC_CTRL bits 15:8 = L, an encode's parity equals the host code of the 512 data bytes followed by L metadata-stream bytes (the twelve register bytes, then 0xff) for L = 0, 10, 12, 13, 16, 20 and 32 with BCH8 and 10 and 32 with BCH16, and that of the data alone for L = 33, 64, 128 and 255 with BCH8 and 255 with BCH16 (810 0x13 after each)	D	p42 m2_r4_ecc_metalen 14

ID	Finding	Class	Evidence / reference
M2-664	A DMA's RAM address is aligned down to a multiple of 4 bytes: a DMA in from a + 1, a + 2, a + 3 reads from a, from a + 4, 8, 16, 32 exactly there; a DMA out to b + 1, 2, 3 writes b..b + 511, to b + 4, 8, 16, 32 exactly there; 512 bytes in each case, nothing outside	D	p42 m2_r4_dma_align 14
M2-668	A DMA in (FMCTRL1 0x2e0) completes (048 bit 20, 038 7) for every FMC_DATA_SEL value tested; it writes data buffer 0 only with 1 and parity buffer 0 (the last 64 source bytes) only with 0x10, and with 0, 3, 4, 8, 0x40, 0x80, 0x11, 0x100, 0x400, 0x4000 or 0x8000 none of data buffers 0 and 1 and parity buffers 0 and 1 changes; the source and the RAM below the read-back areas stay unchanged	D	p42 m2_r4_data_sel_targets 13; DMA out by select: M2-702
M2-669	A DMA out of buffer 0 after an 038 write w: w = 0, 3, 7, 0x2007, 0x80000007 and 0xffffe007 complete exactly (038 reads 7 after the last three: bits 31:13 are not stored); any non-zero bits 12:6 (k = 1, 4, 8, 16, 64, 120, 127 with bits 2:0 = 7; 1, 4, 64 with 0; 2, 4, 8 with 3) or bits 5:3 = 2, 4 or 7 (0x17, 0x27, 0x3f) keep it from completing (wait 2 faults, bit 20 clear, 038 still w), and the target receives only its first 32 bytes: buffer words 0 and 1 and six zero words (words 0..3 and four zero words for k = 120); with bits 5:3 = 1 (0xf) it completes and writes each group of four words as four copies of the group's first word	D	p42 m2_r4_dma_ctrl_count 23; bits 5:3 = 3, 5, 6 and DMA in 0xf: M2-699..M2-701
M2-671	An FMC_DMA_ADDR (034) rewrite g nop2 after the FMCTRL1 0x100 write of a buffer-to-RAM DMA takes effect at byte 32 (g = 0, 2), 64 (4), 128 (8), 256 (16) and 480 (32), and not at all for g = 64 (the DMA has completed): bytes before the switch go to the old address, the rest to the new one at the same offsets; the offsets are multiples of 32 bytes and equal in both runs per gap	D	p42 m2_r4_supersede_offset 14 (two runs per gap)
ID	Finding	Class	Evidence / reference
M2-672	An FMC_DATA_SEL rewrite to 0x200 (data buffer 1) in the same position takes effect at byte 8 (g = 0), 32 (2), 44 (4), 120 (8), 256 or 228 (16), 476 or 492 (32), and not at all for g = 64; from there on the bytes come from buffer 1 at the same offsets; in 7 of 14 runs (both runs of g = 4 and 8) the word at the switch holds 0xd5fa1f44,	D,I	p42 m2_r4_supersede_offset 14 (two runs per gap); tagged word at the switch: M2-694..M2-696

ID	Finding	Class	Evidence / reference
	the buffer-1 word at offset 224 (equal to the word at 480 in this fill), whatever the switch offset; the select switches no later than the address does for the same gap. Inferred: buffer reads lead the RAM writes by up to one 32-byte burst		
M2-673	`jr` (condition r1, every GPR holding the offset of its own landing pad) with a target selector with any of bits 8..15 set (0x100, 0x101, 0x105, 0x1ff, 0x8005) jumps to r0's pad, with selector 5 to r5's: high selector bits make the target r0, as for the `ldi` address selector (M2-615)	D	p42 m2_r4_jr_selector 6

18.25 P43

ID	Finding	Class	Evidence / reference
M2-674	A high-bit source selector (`mov r2` with selector 0x104) reads the value that the last source selector read also when one `ld`, `st` or `wr` stands between it and that instruction: r0's seed after `movi r3`, r5's after `mov r3` with selector 5, with each of the three instructions between and with none (refines M2-618)	D	p43 m2_r5_latch_clear
M2-675	An `ld r3,[0xd60]` between gives the high-bit source neither the loaded word (0x0d600d60, which reaches r3) nor 0: a load does not set the value a high-bit source selector reads	D	p43 m2_r5_latch_clear
M2-676	A `st [0xd64],r4` between gives the high-bit source neither r4's value nor 0: the store's register read does not set the value a high-bit source selector reads (it does set the extended-`st` latch, M2-47)	D	p43 m2_r5_latch_clear
M2-677	A `wr [0xd68],#0x1234` between gives the high-bit source neither the immediate nor 0: `ld`, `st` and `wr` all leave this value, so after a reset it stays 0 until the first source selector (M2-614)	D	p43 m2_r5_latch_clear
M2-678	`mov` to r9, r11, r12, r13, r14 or r15 with source selector 8 (the instruction offset) or 10 (the destination) writes no GPR: in all twelve forms r0..r5 and r7 keep their values (r3 the preceding `movi` value) and the program runs to its END (no jump)	D	p43 m2_r5_ext_dest
M2-679	The same twelve writes leave the register-bus latch: an extended `st` (R = 9) after each stores the preceding `movi r3` value. Extended destinations other than 8 and 10 discard with extended sources as with ordinary ones and the immediate (M2-43, M2-622)	D	p43 m2_r5_ext_dest
M2-680	`sti r1` with address selector 0x105, every GPR holding the address of its own marker word, replaces r0's marker (selector 5, the control, replaces r5's): straight after `movi` instructions a high-bit `sti` address selector uses r0, as `ldi` does (M2-615)	D	p43 m2_r5_sti_high
ID	Finding	Class	Evidence / reference
M2-681	After `mov r2` with selector 5 the same `sti` with selector 0x105 replaces r5's marker: a high-bit `sti` address selector uses the value the last source selector read (r5's), the rule of the	D	p43 m2_r5_sti_high

ID	Finding	Class	Evidence / reference
	high-bit register sources (M2-618), so the r0 of M2-680 is r0 as the source the preceding `movi` read		
M2-682	`ldi` of 0x20000000 + k x 0x1000 (k = 0..15) returns 0xe1a0f00e for k = 0..3, 0x1affffef for 4..7, 0xe59f42b0 for 8..11 and 0 for 12..15, never the boot-ROM word at the addressed offset (0xea000027 at 0, 0xe1500003 at 0x1000, 0xe1c120b0 at 0x8000, ...; refines M2-432)	D	p43 m2_r5_rom_ldi
M2-683	Each of the three non-zero words is a boot-ROM word from the 16-KiB quarter of the sampled offsets: 0xe1a0f00e (`mov pc, lr`) occurs at 18 ROM offsets in 0..0x3fff, 0x1affffef (a bne loop back) only at 0x4d9c, 0xe59f42b0 only at 0x9b5c; 0xc000..0xffff gives 0. The sequencer reads one fixed word per quarter, not the addressed word. Inferred: each quarter returns a word that the ROM's own data path last held, plausibly the last word the CPU fetched from that quarter before leaving the ROM	D,I	p43 m2_r5_rom_ldi
M2-684	Opcode 0x1a with 64 random R/arg/imm words gives the zero-form outcome in every case (an unmet wait to the timeout fault, GPRs unchanged): its operand fields have no effect	D	p43 m2_r5_op1a_operands
M2-685	With the timeout disabled (start word 1), C08 read 3 ms into the run is 0x4 for `nop` with R = 8, for `nop` with R = 0xff and for an unmet `wait 3`; `nop` with R = 0 runs to its END	D	p43 m2_r5_nop_stall_c08
M2-686	The `nop` (`08`) hold for R >= 8 is the wait state of an unmet wait (C08 = 4), not a halt or a pause (refines M2-625, which inferred `08` to be a wait whose condition holds for R < 8)	D,I	p43 m2_r5_nop_stall_c08
M2-687	A count-7 `tmwait` ends at 3896 us with C48 = 0 (C14 0 at the end) and with C48 = v = 1..7 at 3406, 2914, 2421, 1928, 1435, 942 and 449 us, 493 us (one timeout interval) earlier per unit, with C14 = v; with C48 = 8 it takes the wait timeout fault at 4390 us (C14 0)	D	p43 m2_r5_c48_count7
M2-688	The C48 compare rule of a count-1 `tmwait` (M2-627) holds for count 7: the wait ends at the timeout instant at which the C14 count reaches C48 bits 7:0, and a C48 value above the count is never reached, so the wait takes the timeout fault	D	p43 m2_r5_c48_count7

ID	Finding	Class	Evidence / reference
M2-689	Count-255 `tmwait` end times in five runs each: without a C14 write 126099, 126102, 126100, 126108 and 126079 us; after a program write of C14 = 0x800 126102, 126102, 126101, 126102 and 126101 us (median of all ten 126102 us)	D	p43 m2_r5_tmwait_repeat
M2-690	One of the ten count-255 runs ends 23 us early (126079 us), and it is one without a C14 write; the five C14 = 0x800 runs do not repeat the 126088 us end of M2-629: the isolated early ends occur with and without a C14 write and are run-to-run spread, not an effect of the write	D	p43 m2_r5_tmwait_repeat
M2-691	The first fault instant of a count-0 `wait 3` (n = 9), two runs each: 76/78 us after the full per-case reset; 24/34 us with the reset skipped; with only one reset step 82/82 us (C00 = 0x80000), 30/32 us (FMCTRL1 0x80000000) and 28/32 us (0x81c = 1)	D	p43 m2_r5_prescaler_step
M2-692	Of the three reset steps only C00 = 0x80000 moves the first fault instant to near the reset value (82 versus 76..78 us); FMCTRL1 bit 31 and 0x81c leave it near the no-reset value. No single step reproduces the reset instant: the prescaler phase also depends on another reset step or on their combination (refines M2-631)	D	p43 m2_r5_prescaler_step
M2-693	C08 read once 3 ms into a 400 ms loop of one instruction kind, three runs each: `nop2` 0x9, 0x9, 0x2; `ld` 0x1, 0x1, 0x1; `sti` 0x8, 0x1, 0x8. The value differs between runs for `nop2` and `sti`: each run gives one sample, taken at an instant whose phase in the loop is not controlled	D	p43 m2_r5_c08_long
M2-694	An FMC_DATA_SEL rewrite to 0x200 16 `nop2` after the FMCTRL1 0x100 write of a DMA out (both buffers tagged per word) switches the output at word 63, 57 and 58 in three runs; from the switch on the words come from buffer 1 at their own offsets, except the word at the switch itself in the first two runs (refines M2-672)	D	p43 m2_r5_select_tag
M2-695	That word at the switch is buffer-1 word 120 (byte offset 480) in both runs, at output word 63 and 57: the 0xd5fa1f44 word of M2-672 is buffer-1 word 120, a fixed word independent of the switch offset	D	p43 m2_r5_select_tag

ID	Finding	Class	Evidence / reference
M2-696	With the rewrite 32 nop2 after the start the switch falls at word 114 in all three runs and every word from it on comes from buffer 1 at its own offset	D	p43 m2_r5_select_tag
M2-697	Overlapping DMAs with tagged inputs (output word ranges: source; init = not written). Out then out, buffer 0 to P = 0x2000 then buffer 1 to Q = 0x2400, gap 0: P 0..1 buffer 0, 2..5 buffer-0 words 0..3, 6..8 buffer 0, 9 buffer-1 word 120, 10..13 buffer 1, 14..127 init; Q 0..13 init, 14..127 buffer 1. Gap 16: P 0..7 buffer 0, 8 buffer-1 word 120, 9..13 buffer 1, 14..57 buffer 0, 58..63 buffer-0 words 0..5, 64..127 init; Q 0..13 init, 14..127 buffer 1. In then in (source 1 into buffer 0, then source 2 into buffer 1), gap 0: buffer 0 all source 1, buffer 1 words 0..7 source 1, 8..127 source 2; gap 16: buffer 0 words 0..7 source-1 words 16..23, 8..127 source 1, buffer 1 words 0..15 source 1, 16..127 source 2 (words at their own offsets unless a source word is named; refines M2-394)	D	p43 m2_r5_overlap_tags
M2-698	Every output word of the four overlaps is a word of one of the two inputs, the initial content or zero; in the out-then-out overlaps the buffer-1 word at the select switch is buffer-1 word 120, as in a single DMA with a select rewrite (M2-695)	D	p43 m2_r5_overlap_tags
M2-699	A DMA after an 038 write w (038 read after the write and after wait 2 reads w each time): out w = 7 completes exactly; out w = 0x1f, 0x2f and 0x37 do not complete (wait 2 faults) and write source words 0 1 0 1 2 3 4 5 to the first 32 bytes, nothing after; in w = 0xf completes	D	p43 m2_r5_dma_ctrl_sel
M2-700	038 bits 5:3 = 3, 5 or 6 (with bits 2:0 = 7) stall a DMA out after 32 bytes holding source words 0 1 0 1 2 3 4 5; with bits 5:3 = 2, 4 and 7 stalling likewise (M2-669) only bits 5:3 = 0 and 1 let a DMA out complete	D	p43 m2_r5_dma_ctrl_sel
ID	Finding	Class	Evidence / reference
M2-701	A DMA in (RAM to buffer) with 038 = 0xf completes and stores buffer word 4g + j (j = 0..3) from source word 4g + 5j; where 4g + 5j passes the 128-word source it takes the RAM after the source. Inferred from the word order: the transfer runs as four-beat bursts 16 bytes apart whose beats step 20 bytes. With 0xf a	D,I	p43 m2_r5_dma_ctrl_sel

ID	Finding	Class	Evidence / reference
	DMA out repeats each burst's first word (M2-669)		
M2-702	A DMA out (FMCTRL1 0x100) completes for every FMC_DATA_SEL value tested and delivers data buffer 0 only with 0x100; with 0, 1, 0x300, 0x400 and 0x1100 it writes 512 zero bytes (all four buffers tagged; refines M2-668 for the out direction)	D	p43 m2_r5_out_select
M2-703	After FMCTRL1 0x340 (FMC_DATA_SEL 1, wait 2 passing and acknowledged) a page programmed from data buffer 0 holds per word either RAM S + 0x200 + 4k or the tag the buffer held before: all 128 words RAM with 2000 delay loops before the program, 0, 16 and 32 RAM words (the rest tags) with an 014 = 0x100 write 0, 8 and 16 nop2 after the acknowledge (refines M2-617)	D	p43 m2_r5_x340_window
M2-704	The second transfer of FMCTRL1 0x340 (RAM S + 0x200 on into data buffer 0, at the same offsets) runs to its end with no further CPU or sequencer write: time alone completes it	D	p43 m2_r5_x340_window
M2-705	An FMC_DATA_SEL write stops the second transfer of 0x340 where it is: the words moved grow with the time to the write, 16 words per 8 nop2, and none move after it in the following 2000 delay loops	D	p43 m2_r5_x340_window
M2-706	FMC_STATUS bits 19:12 after a data read of L bytes from an open page: 0x41 for L = 160, 192 and 224; 0x14 for L = 255, 257, 320, 383, 385, 448, 480 and 508	D	p43 m2_r5_status_lengths
M2-707	Together with M2-642, bits 19:12 after a data read are 0x41 for L = 1..224, 0x14 for 255..508 and 0x3c for 511 and 512: the value changes between 224 and 255 bytes and between 508 and 511 bytes	D	p43 m2_r5_status_lengths
M2-708	A BCH16 buffer encode with 0x814 = 0x2f gives the same parity and 810 (0x13) as with 0x814 = 0; 0x814 reads back 0x2f	D	p43 m2_r5_bch16_814
M2-709	A BCH16 buffer decode of a host code word with 12 data errors gives the same 810 (0x35), 80c (0), 818 (0x6001: 12 corrections, 1 sector) and decoded data with 0x814 = 0x2f as with 0x814 = 0	D	p43 m2_r5_bch16_814
ID	Finding	Class	Evidence / reference
M2-710	0x814 is stored but changes neither BCH16 encode nor	D	p43 m2_r5_bch16_814

ID	Finding	Class	Evidence / reference
	BCH16 decode, as for BCH8 (M2-660); the parity equals the host BCH16 code and the decoded data the clean data in both settings		
M2-711	The CPU alone (no sequencer program) reads raw pages through DMA and through the FIFO, and decodes them with the CPU-kicked ECC engine and with software BCH, with results equal to the sequencer readers: boot-ROM reserved-region pages (CE0 pages 0, 1, 2, 100; 3 x BCH16 sectors) and main-format pages (BCH8; rows 128, 129, 256000, 383872 and a sequencer-written page); data, metadata and sector status agree in every comparison	D	p43 m2_cpu_stack
M2-712	CPU-only erase and program (engine-encoded, software-encoded and raw images; main and boot formats) give pages that read back exactly through raw DMA/FIFO, the engine decoder, the software decoder and the sequencer readers; the engine's CPU-kicked encode equals host BCH8 and BCH16	D	p43 m2_cpu_stack
M2-713	With 1, 2, 4, 7 and 8 injected data errors, 3 errors in metadata/parity and one 9-error sector per page, the CPU-kicked engine decode and the software decoder return the same data, metadata, correction counts and location sets; the 9-error sector is uncorrectable in both and left as stored	D	p43 m2_cpu_stack
M2-714	The 0x820 location fields list the error bits byte by byte in ascending byte order, and within one byte the higher bit first: a sector with errors at bits 2 and 3 of byte 512 reports 4099 before 4098 (refines M2-128)	D	p43 m2_cpu_stack
M2-715	CPU-only job times including the reset before and after: raw page 586 us DMA, 795 us FIFO; engine-decoded main page 648 us, boot page 270 us; erase 2274 us; program 899..1831 us; software BCH8 decode 1956 us per clean page and 18.4 ms with 33 corrections, software BCH16 boot page 1370 us	D	p43 m2_cpu_stack
M2-716	Every CPU-only job that touched the controller (89) leaves C08 = 0 and FMC_CTRL0 = 0 after its closing reset	D	p43 m2_cpu_stack

18.26 Reboot sessions

ID	Finding	Class	Evidence / reference
M2-717	A DMA from the unmapped bus address 0x50000000 (FMC_DMA_ADDR 0x50000000, direction RAM to buffer) completes: wait 2 passes, the program reaches its END 9 us after the start, 034 keeps 0x50000000, and the device stays fully responsive; the same holds with the CPU reading no FMSS register during the run (stall-safe)	D	reboot m2_reboot_dma_hole 2+2
M2-718	An ldi from the unmapped bus address 0x50000000 hangs the whole system: USB is lost during the case even when the CPU reads no FMSS register during the run (stall-safe mode), and only a power cycle recovers the device; the control ldi from SDRAM before it ends normally in both modes	D	reboot m2_reboot_ldi_hole 2

18.27 P48

ID	Finding	Class	Evidence / reference
M2-719	CPU samples of C08 taken while a program runs read 1, 2 and 9 in ALU and nop2 loops, also 7 in an SDRAM-access loop and 5 in a window-access loop, and 4 in an unmet wait; the value changes within one loop, so C08 while running is a fast-changing state, not one value per instruction kind	D	p48 m2_r6_c08
M2-720	Sampled during a data read, FMC_STATUS bits 19:12 go from 0xc3 (set-up) to 0x41; reads longer than 254 bytes then pass 0x14 (with a transient 0), and 511/512-byte reads end at 0x3c; the next set-up restores 0xc3: the field tracks the progress of the read, not only its length	D	p48 m2_r6_read_status
M2-721	FMC_STATUS bits 3 and 23 around a status latch change in every order (0 -> 8, 0 -> 0x800000, 0 -> 0x800008 and back) at the CPU's microsecond sampling; edges inside one sample interval stay unordered	D	p48 m2_r6_latch
M2-722	While a DMA of 32, 128 or 512 bytes runs (038 = 7), FMC_DMA_CTRL reads non-zero bits 12:6 (e.g. 0x807, 0xd87, 0x1c07) and 7 again afterwards; every transfer delivers its bytes exactly: bits 12:6 are live during the transfer	D	p48 m2_r6_dma
M2-723	wait2 32 is released when VIC line 32 is asserted by hardware (GPIO interrupt group, set through its controller) and wait2 33 by line 33; wait2 31 and 33 (resp. 32 and 34) are not, and with the line not asserted none is: wait2 condition n follows VIC line n, VIC1 line n - 32	D	p48 m2_r6_vic1
M2-724	With the GPIO interrupt controller set for the opposite level, and for line 52 in every tried set-up, the VIC raw bit never set, so no wait2 release can be tested there	D	p48 m2_r6_vic1
M2-725	bnz and jr reach targets 2^16..2^24 and 2^n + 0x100 exactly, each landing on its own tagged END pad with none of the 2^17..2^32 alias pads taken: the sequencer PC is at least 25 bits wide	D	p48 m2_r6_pc
ID	Finding	Class	Evidence / reference
M2-726	Apple's original ERASE and WRITE programs (instruction stream unmodified) with their parameter blocks erase and program blocks 3000 and 3001: the erased read is all 0xff and	D	p48 m2_r6_apple

ID	Finding	Class	Evidence / reference
	the readback data and metadata are exact		
M2-727	The resident CPU campaign completes 1,850 sequence jobs in 15 worker RUNs, with 1,850 verified resets, 41.38 s of worker time and 58.06 s of family execution; completion does not establish data correctness: all 12 required round-trip cases fail their three payload comparisons	D	p48 m2_cpu_direct; validation/m2-p48-cpu-gate-review.json
M2-728	The generic CPU sequence leaves FMC_CTRL0 bit 24 set when entering its FIFO input path, unlike the earlier qualified CPU raw reader. All 12 round trips retain corrupted earlier DMA buffers and a failing final FIFO buffer; comparable DMA-only partial-page and overwrite controls match. Unintended DMA during FIFO access is the inferred cause, consistent with M2-617 and M2-703..705; individual stray writes are not timed by this capture. The 44 cases containing FIFO input cannot qualify optional NAND responses	D,I	p48 m2_cpu_direct roundtrip/ce0..3/0..2; cpu_nand_ll.c reviewed in docs/149
M2-729	Baseline and gates 0, 1 and 2 return records with exact clock restoration and successful cleanup. Gate0 was already disabled; gates1/2 were enabled. VIC0 raw remains 0x20 and VIC1 raw 0 before, during and after every 28-us window. USB is lost on the next case, gate3, with no returned record: neither the gate identity nor the stalled instruction is established. Later gate3 returns with the corrected pulse (M2-742)	D	p48 m2_r6_gate5; validation/m2-p48-cpu-gate-review.json
M2-730	The corrected native harness selects FIFO/DMA mode per read and restores the caller's control word. Its clock pulse uses only SRAM instructions, registers and PWRCON writes until clocks are restored; it makes no during-window peripheral measurements. The old image exposes both defects under native regressions; the rebuilt image passes 75 regressions, 1,850 CPU templates under an explicit command reference profile and 161 gate cases. These checks do not establish optional Hynix commands, physical gate dependencies or hardware recovery from gate3	M	validation/m2-p48-fixed-validation.json; validation/m2-p48-fixed-preflight.json; docs/149

18.28 P49

ID	Finding	Class	Evidence / reference
M2-731	CPU-direct random-column-read matched all 5 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-732	CPU-direct random-column-write matched its programmed-data readback for the captured command variant	D	p49 m2_cpu_direct
M2-733	CPU-direct partial-page matched all 3 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-734	CPU-direct overwrite/more-zeroes matched all 2 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-735	CPU-direct overwrite/attempt-zero-to-one matched all 2 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-736	CPU-direct out-of-order/3-1-2-0 matched all 4 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-737	CPU-direct out-of-order/127-0-64 matched all 3 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-738	CPU-direct copyback/same-block matched all 2 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-739	CPU-direct two-plane-program/81 matched all 2 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-740	CPU-direct two-plane-erase matched all 2 programmed-data readbacks for the captured command variant	D	p49 m2_cpu_direct
M2-741	CPU-direct read and write timing qualified in three patterns at minimum measured setting cycle 28.24 ns (fields 0/2); qualification is specific to this unit and operating conditions	D	p49 m2_cpu_direct

18.29 P49 gate completion and worker dispatch

ID	Finding	Class	Evidence / reference
M2-742	Baseline and gates0..3 all return, with exact clock restoration and successful initial reset/cleanup. Gate3 was initially enabled. VIC0 raw 0x20 and VIC1 raw 0 before/after; no during-window interrupt sample. The next BATCH_FREE (job63, kind19) remains QUEUED with jobs done62 while USB status and the microsecond counter remain live. Gate4 was not submitted; the physical cause of the dispatch stall is not established	D	p49 m2_r6_gate5; validation/m2-p49-gate-worker-review.json; docs/150
M2-743	Masked timer IRQ8 and a non-idle current thread reproduce queued FREE on the previous native image. Selecting a READY worker for USB IRQ return completes FREE without changing IRQ enables; the updated host retains live scheduler diagnostics on queued timeout. Forty-one distinct regressions and all161 gate preflight cases pass. Gate peripheral effects are not modelled; no device capture validates this dispatch change or identifies p49's starvation cause	M	validation/m2-p49-worker-validation.json; validation/m2-p49-worker-preflight.json; docs/150

18.30 P50

ID	Finding	Class	Evidence / reference
M2-744	Baseline and gates0..108 return END, worker rc0, successful initial reset/cleanup and exact restoration of all five clock words (110 cases). Of 109 gate entries, 67 were enabled and 42 already disabled. VIC0 raw0x20 and VIC1 raw0 before and after restoration; during-pulse fields are unavailable, so no gate is excluded as the source of line5. The queued FREE stall after gate3 does not recur; the exact image hash is absent	D	p50 m2_r6_gate5; validation/m2-p50-gate-review.json; docs/151
M2-745	USB is lost submitting BATCH_RUN after gate108. The journal records gate108 FREE330 DONE, then the gate109 manifest and allocation331 DONE. No acceptance, execution or restoration of the next RUN is observed. Gate109 targets PWRCON3 bit13; its physical identity and the stalled instruction are unknown. The later gate3 loss under different entry history is separate evidence (M2-746, M2-747)	D	p50 summary and journal; validation/m2-p50-p51-journal-review.json; docs/152

18.31 P51 journals and checkpoint validation

ID	Finding	Class	Evidence / reference
M2-746	Baseline and gates0..2 return END with worker rc0, successful initial reset/cleanup and exact clock restoration. The journal then records the gate3 manifest and allocation13 DONE before USB is lost on command140. Gate3's manifest is byte-identical to the one that returned in p50; acceptance/execution of the lost RUN is not observed	D	p51 summary and journal; validation/m2-p51-gate-review.json; validation/m2-p50-p51-journal-review.json
M2-747	P50 capabilities0xfffe3000 include the EXECFIRMWARE entry marker; p51 capabilities0xdffe3000 omit it, with no other capability difference. The five shared manifests, 160000-us observation floor and sampled starting clock words agree. Neither capture has a firmware hash or ARM cache/MMU snapshot, so entry history is observed but is not established as the cause of the different loss points	D	p50/p51 journal sessions and manifests; docs/152
M2-748	The checkpoint build publishes masks, clocks, job/case identity and CPU state before candidate start, then waits for an automatic host ACK after durable collection. Native packet tests with cache on/off establish ordering; wrong-token and missing-ACK tests prevent premature pulse execution. Running code fingerprints and pending submission phases are saved. All53 diagnostic cases and59 distinct relevant regressions pass; no physical clock dependency or hardware recovery is established	M	validation/m2-p51-checkpoint-validation.json; validation/m2-p51-checkpoint-preflight.json; docs/152

Chapter 19 — Differences between M1 and M2

Summary: the behaviour of the M1 sequencer (S5L8702, iPod nano 3G) set against the M2 sequencer (S5L8720, iPod nano 4G), first as a table of the substantive differences by topic, then finding by finding for all 175 M1 findings. M2-nn refers to chapter 18; M1-Fnnn to the M1 ledger. The same comparison in machine-readable form, with the device experiments of section 19.4, is `validation/m1-m2-crosswalk.json`.

Verdicts: **same** — M2 behaves as the M1 claim states, over the claim's whole scope; **differs** — at least one part of the claim is different on M2 (both sides are stated); **m2-only-context** — the claim concerns a nano 3G board, chip or firmware detail, and the M2 cell gives the nano 4G counterpart; **unresolved** — the M2 cell gives the measured state and states which part of the claim is not measured on M2. "Source" marks a fact read from the original M2 programs or CPU driver rather than measured on the device.

19.1 Substantive differences

Instruction set

On M1	On M2
`jr` (`0f`) jumps unconditionally to r[arg], unrounded; R ignored; a target above `0xfff` does not jump (M1-F053, M1-F097)	`jr` jumps to r[arg] only if r[R] ≠ 0; the low three target bits are ignored; no wrap up to 2^20, a target above `0xfff` jumps (M2-110, M2-122)
bz/bnz take unaligned targets as given (M1-F070)	branch targets ignore their low three bits; PC ≥ 25 bits (M2-105, M2-725)
source selector 10 is the destination in MOV/immediate forms, r0 in the register form (M1-F069)	source selector 10 is the destination in every MOV and ALU form, register form included (M2-141)
`08` costs the same as `movi` (M1-F052)	`08` costs 1.25 cycles (37.6 ns), `ff` and `movi` 1 cycle; `08` with R = 8..`0xff` stalls until the timeout fault; an all-ones word runs as `ff`; every value `1a..fe` acts as an unmet wait also with a pending flag (M2-102, M2-378, M2-625, M2-367, M2-583)
`wait` ignores arg/imm (M1-F070)	`wait  12` uses arg and imm as status operand and mask; it passes iff status & arg & mask ≠ 0 (M2-134)
the original programs use `pld`/`pst` on `0xd60..0xd7c` (M1-F001, M1-F003)	no original program uses `pld`/`pst`; the engine executes them (M2-08)
15 toggles `C14`: consecutive `tmwaits` alternate a full wait and an immediate reload (M1-F047, M1-F171)	each `tmwait` ends at the (count + 1)-th timeout instant after its entry (492.747 us × count + 450 us from start-word bits 15:8 when entered at the start), faults for count 0, ignores its fields and `C14` writes and leaves `C14` = 0; the next one waits again (M2-373, M2-584, M2-629, M2-150)
end faults when R or arg is non-zero (M1-F105)	end ignores non-zero R, arg and imm and latches its word and imm in `C40`/`C44` (M2-164)

Registers and sources

On M1	On M2
C58 holds 8 bits; bit 0 diverts ENDs (M1-F044, M1-F045)	C58 keeps only the bits of 0xd6; bit 0 reads back 0 (M2-70)
0x400 is a second register set with writable words, own buffers and DMA (M1-F056, M1-F082)	the +0x400 storage words read 0 after any write and alias nothing; no FMC_CTRL0 bit enables them (M2-82, M2-139)
0x800 reads 0x20060808 (M1-F056)	0x800 reads 0x20070618 (M2-116)
C08 codes 0, 1, 3, 4, 0xa (M1-F067)	0 idle, 3 paused or aborted, 4 in every unmet wait, wait2 and tmwait, 0xa at pause, plus 0xb for a zero-field evjmp with the timeout disabled, which survives STOP; while a program runs (also in a fault handler or after a C58 = 0 fault) 1, 2 or 9, differing between identical runs (M2-57, M2-62, M2-384)
C58 = 2 or 6 with a CPU-written C4C diverts an END, C58 = 3 every END after the first (M1-F046, M1-F162)	no C58 value diverts an END; C5C stays 0 (M2-148, M2-149)
C38 is read-only for the CPU, C54 holds 12 bits, 0x940..0x950 are read/write and a start clears C3C/C48/C54 (M1-F055, M1-F103)	a CPU write of C38 moves the PC, C54 keeps mask 0x83f, 0x940..0x950 hold no bits and a start keeps C3C/C48/C54 (M2-154, M2-191, M2-278)
offset bit 9 is not decoded; the 0x100 block is mirrored four times (M1-F056)	bit 9 is decoded in 0xc00..0xfff only; the 0x100 block appears at 0x100 and 0x300 (M2-155)
0x3a8..0x3b4 read 0xffffffff from the sequencer (M1-F172)	they read 0x12 from the sequencer and 0 from the CPU (M2-210)

Events and waits

On M1	On M2
event 4 = ECC_DONE (any bit but 4); events 6..9 = second-set flags (M1-F004, M1-F075)	event 4 = 0x810 bit 1 (encode) or bit 2 (decode), not cleared by writing 0x810 = 1; events 6..10 never pass (M2-97, M2-99, M2-111)
wait 15 passed after a command byte in the NAND-state sweep (M1-F062)	wait 15 never passes, after any of five stimuli in either order (M2-69)
status bit 23 set after every poll, ready or not (M1-F006); waits on status use wait 5 (M1-F007)	one latch polls until the chip is ready; the originals use wait 12 (0x40/0x40, 0x20/0x20) and no wait 5 (M2-130, M2-136, M2-137)
wait 5 after a status latch timed out in one of two identical runs and once straight after 70h + latch (M1-F063, M1-F087)	the standard latch never times out; WAIT5 faults only for a latch one or two instructions after an unwaited 70h, which sets FMC_STATUS bit 3 instead of bit 23, also straight after a reset (M2-161, M2-178, M2-68, M2-390, M2-593)
a refused command sets FMC_STATUS bit 26; a latch after 90h leaves 0x90 in FMC_NSTAT (M1-F066)	a refused command leaves bit 26 clear (an address operation after an unreleased latch sets it); FMC_NSTAT holds the first READ ID byte 0xad (M2-181, M2-610, M2-239)
event 5 is set by a latch only while armed; FMCTRL1 operations, address cycles and a chip's busy-to-ready transition re-arm it (M1-F121, M1-F135)	a latch needs no arm: after only the bit-23 acknowledge a second latch passes wait 5 with or without an operation between; address cycles after an unreleased latch do not run (M2-494, M2-610)
a latch with 0x044 or 0x844 written raises FMC_STATUS bit 24 instead of 23 (M1-F104)	a latch never raises bit 24 (M2-163)
16 counts the second set's flags (M1-F088)	evjmp catches arriving and pending primary flags (tested C58 = 0, 2, 4); no +0x400 operation raises a flag (M2-186, M2-391, M2-183)
wait 8 passed after a CE0 page read; the first program after a power cycle passed wait 4 (M1-F163)	wait 8 never passes (M2-206)

Interrupts and `wait2`

On M1	On M2
`wait2 N` matches the raw VIC status of line N, 64 of 64 (M1-F174)	`wait2 N` follows VIC0 hardware lines; a software interrupt visible in the raw status releases none of 59 lines (M2-140)
in idle/ECC/DMA states only conditions 5 and 33 passed; of 0..15 only 5, also with the chip busy (M1-F092, M1-F147, M1-F091, M1-F152)	5 always, 8 (timer), 16 (DMAC0), 19 (USB) with their devices; no VIC1 line passed (M2-74, M2-140)
VIC0 line 20 never for DMA sources in `irqsrc` (M1-F140)	with `0x044` all ones line 20 rises after a DMA, a command and a status latch (M2-142)
ECC sources enabled in `0x844` raise source 43 (M1-F175)	`0x844` is an encode-parity word; an ECC encode raises no line (M2-116, M2-142)
`CS_IRQ` bits 6:4 are FMC and ECC sources gated by `0x044` and `0x844`, bit 6 `ECC_DONE` pending (M1-F065, M1-F076)	`CS_IRQ` bit 4 is the sticky OR of the routed FMC flags; bits 5 and 6 never rise for any tested ECC enable (M2-200, M2-212)
VIC0 line 20 per single source only for the command (bit 0) and address (bit 1); VIC1 line 22 never (M1-F102, M1-F146)	line 20 for each of `0x044` bits 0, 1, 2, 11, 12 with its own flag; END raises VIC1 raw bit 22 while `C10` ≠ 0 (M2-200, M2-182)
an END with an enabled FMC flag pending stops USB (M1-F167)	USB keeps answering (M2-208)

Control registers and reset

On M1	On M2
timeout 4.87 ms for `0xfff5`; step 2^s with s = 3, 5, 7, 9, 10, 11 for n = 1..5, ≥ 6; 4868 us for count `0xff` at n = `0xf` (M1-F043, M1-F111, M1-F107)	about 126.6 ms for the same count; step 2^3, 2^5, 2^7, 2^8, 2^9 engine cycles for n = 2..6, 2^(n+3) from n = 4 up to the saturation at 2^14 (n = 11); n = 0 immediate, n = 1 2 cycles per count; faults fall on instants fixed before the start, counted from the wait's entry (M2-81, M2-115, M2-216, M2-433, M2-584, M2-588)
CPU reset writes `0x30000` (M1-F030)	STOP 8, abort 2 when `C08` = 4, buffer handshake `C60`/`C64`, `C00` = `0x80000`, FMC `0x800`/`0x80000000`, ECC `0x81c` = 1; `C60` = 1 while running holds the engine (M2-20, M2-121)
start-word bits 1, 3, 16 hold the program, the other bits have no effect, and clearing bit 4, 5 or 7 leaves the timeout unchanged (M1-F054, M1-F153)	bit 19 also holds the program, bit 3 does not read back, and bits 7:4 are the timeout step field (M2-153, M2-201, M2-115)
after a reset a program whose instruction 0 is not end needs a second start (M1-F049)	the first start runs it, also the first program after power-on (M2-151, M2-431)
the power-on window differs from a soft reset in status words and `C14` (M1-F078, M1-F080)	the power-on window equals the window after the ordinary reset except `C00` (`0x7f60`) and `C04` (0) (M2-430, M2-281)
`C14` reads `0xff` after power-on and after a timeout (M1-F048, M1-F051)	`C14` reads `0x7f` at power-on, after a timeout and after a stalled wait; it holds start-word bits 15:8 after a wait that passes before the first step and counts down per step during a stall (M2-430, M2-172, M2-177)
`C10`, `C3C`, `C48`, `C54` change no timeout, wait, DMA or encode (M1-F064)	none changes a wait timeout, DMA or encode, but any of `C48` bits 1..7 makes a `tmwait` fault at the wait timeout and bit 0 ends a count-1 `tmwait` one timeout instant early (M2-179, M2-627)
the CPU can write the PC only in a pause (M1-F094)	a CPU write of `C38` also takes effect before the start and while running (M2-278)

NAND controller

On M1	On M2
FMC_DMA_CTRL bit 1 or 2 makes every later DMA time out; the value persists between runs (M1-F158)	page reads are exact with 0, 1, 2, 4, 7, 8 and a DMA completes with every value of bits 2:0, more slowly for smaller values; any of bits 12:6 set, or bits 5:3 = 2, 4 or 7, makes the next DMA fault after its first 32 bytes; bits 12:6 count up during a DMA and keep a residue after it (2 → 0x42, 4 → 0x84); the reset leaves 7 (M2-76, M2-79, M2-396, M2-398, M2-609, M2-669, M2-123)
timing word edo (bits 29:28) is the read-capture delay (M1-F031)	cycle = (CTRL0[14:12] + CTRL0[18:16] + 2) × 7.06 ns; [18:16] ≤ 1 shifts read data; [30:28] has no effect (M2-127, M2-131, M2-139)
READ ID through FMC_FIFO 0x080 (M1-F032)	READ ID through latch 0x040 = 0x52 into 0x060/0x064; 0x080 reads 0 (M2-124)
128 FMC_FIFO writes after FMCTRL1 = 0x340 change 26 of 128 words of data buffer 0 (M1-F164)	FMC_FIFO writes never reach a data buffer; after 0x340 with FMC_DATA_SEL 1 buffer 0 is filled from the RAM that follows the 0x340 output (M2-207, M2-617)
FMCTRL1 0x100 replaced at once by 0xe2 loses the DMA (M1-F036)	the DMA data reach RAM (all but a run of 4..16 bytes when replaced at once); only its completion flag is lost, so its wait 2 times out (M2-332)
FMC 0x028 is the FIFO read position: +4 per 0x080 read, +1 per pld + sti iteration (M1-F155, M1-F060)	0x028 is a 9-bit storage word that neither FIFO reads nor loop iterations advance (M2-156, M2-173)
the second set runs a READ ID, has its own buffers and DMA, and C50 bit 6 follows its command (M1-F079, M1-F169, M1-F170)	nothing written through +0x400 runs, moves data or raises a flag; C50 bits 6 and 8 never set (M2-183, M2-185)
a status poll on FMC_DATA_SEL = 0x100 returns 0x70 and the next page's 80h is ignored (M1-F123, M1-F128)	it returns the chip status (0x80 busy, 0xe0 ready) and multi-page programs write every page (M2-169, M2-196)

ECC

On M1	On M2
registers ECC_KICK/ECC_RESULT/ECC_MODE/ECC_DONE at 0x80c/0x810/0x814/0x840, parity in 0x900..0x90c (M1-F013, M1-F098)	ECC_CTRL 0x804 (command + mode), result ring 0x80c, ECC_STATUS 0x810, totals 0x818, reset 0x81c, locations 0x820.., parity 0x840..0x85c (M2-89, M2-91, M2-116, M2-128)
BCH8 with 3 metadata bytes per sector; RS-8/12/16 via bit 15; length field 20:16; byte swap bit 4 (M1-F109, M1-F133, M1-F132, M1-F141)	BCH8 (same code) and BCH16 (t = 16, 26 bytes); 10 metadata bytes in sector 0 only; bits 15:8 metadata length; no RS, length or swap bit (M2-77, M2-86, M2-98, M2-101)
stream [16-byte chunk][512] × 4 per 2 KiB unit (M1-F017)	meta[10] | par | data[512], then par | data[512] per sector (M2-77, M2-86)
ECC_DONE 0x33/0x35/0x3d; bit 3 survives the 0x1ff clear (M1-F119, M1-F139)	0x810 0x13/0x15/0x1d/0x35/0x55; bit 5 = ≥ 7 corrected, bit 6 erased; 0x1ff clears bits 3/5 (M2-88, M2-94, M2-95)
no mode corrects more than 8 scattered bits (M1-F143)	BCH16 corrects 16 (M2-92)
the kick word selects data buffer, parity buffer and slot; ECC_MODE bit 8 enables the slot path and bits 20:16 the data length (M1-F014, M1-F118, M1-F125)	ECC_CTRL holds decode, encode, BCH16 and the metadata length (bits 15:8); the slot is FMC 0x01c bit 0; the other bits have no encode effect (M2-98, M2-101, M2-417)
a 3-byte metadata fragment per chunk, placed by FMC_METACFG and coded in separate passes, in both 2 KiB units (M1-F018, M1-F019, M1-F020, M1-F112, M1-F113)	ten metadata bytes once per page, inside sector 0's code word; one encode or decode covers data and metadata (M2-77, M2-98)
9..12 errors report ECC_RESULT 0x11; an all-zero chunk is uncorrectable (M1-F108, M1-F116)	beyond the radius the data stay unchanged with ring entry 1 and 0x810 bit 3; an all-zero sector-0 chunk decodes clean (M2-84, M2-88, M2-214)
RS-8/12/16 correct 4/6/8 bytes; modes without a code write a fixed sequence into 0x900..0x90c (M1-F134, M1-F136, M1-F137, M1-F138, M1-F142, M1-F144, M1-F148, M1-F165)	no Reed-Solomon or code-less mode; 0x900..0x9fc read 0 after encodes (M2-72, M2-171, M2-197)
an encode in the same code as the last decode leaves ECC_RESULT unchanged (M1-F166)	0x80c is an eight-slot ring of decode results; each read, by the sequencer or the CPU, advances it (M2-90, M2-91)

Timing

On M1	On M2
216 ns per instruction, fetch-bound; SRAM faster than SDRAM (M1-F039, M1-F071, M1-F154)	ALU 30.1 ns (1 cycle at 33.25 MHz, inferred), window 60.6 ns, `ldi` 211 ns; SRAM = SDRAM (M2-102, M2-103)
512-byte transfer 49 us, DMA 14 us (M1-F072, M1-F083)	transfer ≈ 22 us at the setup timing word, DMA ≈ 2.7 us in / 1.1 us out (M2-117, M2-119)

Original programs and CPU interface

On M1	On M2
prologue FMCTRL1 = `0x0ff3f8e0` (M1-F009)	prologue FMCTRL1 = `0x033338e0` (ch. 9.3)
ERASE: staged ready wait in three tries with C58 = 0 (M1-F008, M1-F028)	ERASE: one `wait 12` per check, also before reusing a CE, under C58 = 4 (source)
READ batches all 30h before the transfers (M1-F021)	READ is a three-stage pipeline with up to three pages on distinct CEs (source)
program FAIL ends the run at once (M1-F027)	1.0.4 WRITE records the CE in `0xd04`, sets r6 bit 28 and runs the final sweep (source)
erased page from FMC_STATUS bit 27 (M1-F022)	erased sector from ECC_STATUS bit 6 (M2-95)
FMC_CTRL0 bit 24 is set around the DMA phases (M1-F010)	READ sets bit 24 for the whole run, WRITE after its first DMA address (source)
READID resets all CEs, uses FMC_CTRL0 = `0x33201` and reads 8 ID bytes through the FIFO (M1-F011, M1-F029)	a separate RESET program; READID uses `0x77001` per CE and reads 5 bytes through the `0x52` latch into `0x060` (M2-124)
the FMCTRL1 code set of M1 chapter 9 (M1-F012)	adds `0x42`, `0x12`, `0x180`, `0x10020`, `0x30020` and FMC `0x058` transfer targets (source)
the page result word takes bit 30 and the count bit n − 1 from ECC_RESULT (M1-F015)	READ adds the erased bit (bit 29) and a 5-bit count that sets bit n (source)
the DMA address is written once per unit and FMC_DMA_CTRL = 7 before the last DMA (M1-F024)	READ writes the DMA address once per 2 KiB chunk and never writes FMC_DMA_CTRL (source)
WRITE reads the DMA list one entry ahead (M1-F025)	WRITE reads pages × chunks + 1 entries, the last one unused (M2-213)
every fault handler puts the CE mask in r6 bits 7:0 (M1-F033)	only RESET and READID do; READ sets the page summary with bit 27 (source)
78h variants and WRITECACHE (15h, 71h) are separate programs (M1-F035)	one WRITE program serves every variant through CPU-built descriptors (source)

19.2 Finding by finding

M1 ID	On M1	On M2	verdict	evidence
M1-F001	On M1 the original programs use 8-byte instructions with 17 base opcodes plus 18/19 (`pld`/`pst`), byte-offset branch targets, r0..r7 and r6 as the result.	On M2 the original programs use the same 8-byte format, byte-offset targets, r0..r7 and r6 result with the same 17 base opcodes, but none of the ten M2 programs contains 18/19; the engine still executes `pld`/`pst` (M2-08).	differs	M2-01; M original p code/boo code/roo
M1-F002	On M1 a non-zero `imm` puts the source in `arg` and `imm` = 0 selects the register form, so an immediate of 0 cannot be encoded.	On M2 the same operand rule holds in the original-program dataflow and on the device (immediate-zero ADD uses the register form).	same	M2-06; cl
M1-F003	On M1 `pld`/`pst` access a window word at an offset held in a register and the originals use them only on `0xd60..0xd7c`.	On M2 `pld`/`pst` access the GPR/PRAM window through a register on the device, and no original program uses them.	differs	M2-08; M sources code/boo code/roo
M1-F004	On M1 `wait` events 0..5 follow FMC_STATUS bits 1, 2, 20, 3, ECC_DONE bit 0 and FMC_STATUS bit 23.	On M2 events 0..3 and 5 follow FMC_STATUS bits 1, 2, 20, 3 and 23, but event 4 follows ECC_STATUS (0x810) bit 1 after an encode and bit 2 after a decode, and writing bit 0 does not clear it.	differs	M2-66; M chapter 7
M1-F005	On M1 the flags stay set until acknowledged, so a stale flag lets the next wait pass early.	On M2 command, address and DMA flags are levels that waits do not consume; an unacknowled ged DMA flag lets the next `wait 2` pass at once and cuts the following DMA.	same	M2-66; M

M1 ID	On M1	On M2	verdict	evidence
M1-F006	On M1 FMC_STATUS bit 23 is set after every status poll whether or not the chip is ready (third-party measurement).	On M2 one status latch keeps polling: FMC_NSTAT reads 0x80 while busy and `wait 5` passes only at the erase end, so the ready condition, not the poll, completes the latch.	differs	M2-130;
M1-F007	On M1 13 of the 19 `wait 5` uses test FMC_NSTAT & 0x40 first and wait only when it is clear; ERASE arranges its 6 differently.	On M2 the original programs contain no `wait 5`; every status check is 70h, FMCTRL1 = 0xca and `wait 12` with arg = mask = 0x40 (nine uses) or 0x20 after 78h (four uses), without a pre-test.	differs	M2-19; M program code/boo code/roo
M1-F008	On M1 ERASE waits for ready in up to three tries, with C58 = 0 before the first two `wait 5`, reloading C14 and acknowledging the fault between them.	On M2 ERASE waits once per check with `wait 12` (0x40/0x40) under the prologue's C58 = 4, before reusing a CE and in the final sweep, with no retry and no C14 write.	differs	original E `mem:0x08` store/fms chapter 1
M1-F009	On M1 every prologue writes CS_IRQ = 0xff, C10 = 1, C58 = 4, C4C = handler, FMCTRL1 = 0x0ff3f8e0 and FMC_STATUS = 0x0ff00ffe.	On M2 the prologue writes the same words except FMCTRL1 = 0x033338e0; BLREAD/BLWRITE place one marker instruction before it.	differs	chapter 9 program code/boo code/roo
M1-F010	On M1 FMC_CTRL0 = 0x801 | timing, CE bit 1 << (CE+1), CE changes mask 0xfffffe01, and bit 24 is set around the DMA phases.	On M2 the CE bit and the 0xfffffe01 mask are the same, but READ writes P_TIMING | 0x1000801 at its start (bit 24 set for the whole run, cleared with 0xfefffe01 at the end), WRITE sets bit 24 after its first DMA address, and RESET/READID use 0x77201/0x77001.	differs	original F `bl_NandF` `mem:0x64` fmss_asn WRITE re `mem:0x08` store/fms original F and RESE (code/boo

M1 ID	On M1	On M2	verdict	evidence
M1-F011	On M1 READID writes FMC_CTRL0 = 0x33201 (bit 9 set, bit 11 clear) and the third-party word 0x43803 also reads the ID.	On M2 READID writes 0x77001 | 2 << CE (bits 9 and 11 clear) and RESET 0x77201; READ ID returns AD D5 14 B6 44 on four CEs, and bits 9 and 11 change neither read time nor data.	differs	M2-124; READID m RESET me (code/bo
M1-F012	On M1 the FMCTRL1 codes are 0x1, 0x20, 0x32, 0x34, 0xca, 0xe2, 0xe4, 0x100, 0x1c2, 0x1e0, 0x2e0, 0x2e4, 0x340, 0x100e0, 0x0ff3f8e0, with the FMC_DATA_SEL buffer fields of M1 chapter 9.	On M2 the originals use 0x1, 0xe2, 0x42, 0x12, 0x100, 0x180, 0xca, 0x20, 0x10020, 0x30020, 0x2e0, 0x2e4, 0x34, 0xe4 and 0x033338e0, with data selections 0x10, 0x102/0x201 and parity 0x1000/0x2000 plus FMC 0x058 for transfer targets.	differs	chapter 9 bl_NandF mem:0x64 fmss_asn WRITE re mem:0x08 store/fms
M1-F013	On M1 READ writes ECC_DONE = 0x1ff and ECC_MODE = 0x180, kicks, waits on event 4, writes ECC_DONE = 1 and reads ECC_RESULT.	On M2 READ writes ECC_STATUS (0x810) = 0x1ff, selects the slot in FMC 0x01c, starts the operation and mode in one write of ECC_CTRL (0x804), waits on event 4, writes 0x810 = 1 (which clears only bit 0) and reads the 0x80c result ring.	differs	M2-91; M original F bl_NandF mem:0x64 fmss_asn
M1-F014	On M1 the kick word has bit 0 decode, bit 1 encode, bits 9:8 data buffer, bits 13:12 parity buffer, bits 17:16 slot.	On M2 ECC_CTRL has bit 0 decode, bit 1 encode, bit 2 BCH16 and bits 15:8 metadata length; the buffer is FMC 0x01c (0/1) and bits 3..7 and 16..31 have no encode effect.	differs	M2-98; M

M1 ID	On M1	On M2	verdict	evidence
M1-F015	On M1 ECC_RESULT bit 0 sets page-word bit 30 and the count n = (ECC_RESULT >> 16) & 0xf sets bit n - 1.	On M2 READ forms v = ECC_RESULT | (ECC_STATUS & 0x40): uncorrectable sets bit 30, erased (0x40) sets bit 29, otherwise the 5-bit count n = (v >> 16) & 0x1f sets bit n.	differs	original F bl_NandF mem:0x64 fmss_asm 10.9
M1-F016	On M1 a decode corrects the data buffer in place and the DMA follows the decode.	On M2 a decode corrects data, metadata and parity bits in place and READ DMAs each sector after its decode; corrected data is returned through that sequence.	same	M2-84; M bl_NandF mem:0x64 fmss_asm
M1-F017	On M1 the chip stream is [16-byte chunk][512 data] four times per 2 KiB unit, two units per 4 KiB page, WRITE sending the parity chunk first.	On M2 the stream is meta[10] | parity | data[512] for sector 0, then parity | data[512] for each further sector (13 or 26 parity bytes), with no 2 KiB unit structure on the chip.	differs	M2-77; M
M1-F018	On M1 each chunk is 13 parity bytes plus a 3-byte metadata fragment, placed by FMC_METACFG nibble slot numbers.	On M2 there is no per-sector metadata fragment: ten metadata bytes precede sector 0's parity, their length is ECC_CTRL bits 15:8, and no FMC_METACFG counterpart is used.	differs	M2-77; M
M1-F019	On M1 the metadata encode runs once per page before the first sector's encode and the metadata decode once after the last sector's decode.	On M2 the metadata is part of sector 0's code word: one encode (0xa02/0xa06) or decode (0xa01/0xa05) covers data and metadata, with no separate metadata pass.	differs	M2-77; M

M1 ID	On M1	On M2	verdict	evidence
M1-F020	On M1 both 2 KiB units of a 4 KiB page carry the same three metadata bytes per chunk.	On M2 the page carries its ten metadata bytes once, in sector 0.	differs	M2-77; cl
M1-F021	On M1 READ issues every 30h of a batch of distinct CEs before the transfers, ends the batch at the first repeated CE and returns results in request order.	On M2 READ is a three-stage pipeline (issue, ready wait, transfer) with up to three pages on distinct CEs in flight; a request for a CE still waiting is postponed to the next pass.	differs	original F bl_NandF mem:0x64 fmss_asm 11.2
M1-F022	On M1 the page word becomes 0x20000000 when FMC_STATUS bit 27 stayed clear, the last sector's verdict is added afterwards and bit 27 is acknowledged at page end.	On M2 READ marks an erased sector from ECC_STATUS bit 6 (erased, 0x55) by setting page-summary bit 29 (0x20000000) per sector; FMC_STATUS bit 27 is not used.	differs	M2-95; o bl_NandF mem:0x64 fmss_asm
M1-F023	On M1 FMC_STATUS bit 27 means a non-blank chunk was read: every programmed page sets it, every erased page does not.	On M2 FMC_STATUS bit 27 is set after reading a programmed page (all-zero or random data: every sector after its metadata/parity or data transfer) and clear after reading an erased page (no sector); writing 0x0ff00ffe clears it.	same	M2-145;
M1-F024	On M1 the DMA address is written once per unit, advances 512 per sector, and FMC_DMA_CTRL = 7 is written before the last DMA.	On M2 READ writes FMC_DMA_ADDR once per 2 KiB chunk, never writes FMC_DMA_CTRL (it reads 7 after reset) and writes FMC 0x05c = 0x200 and FMCTRL1 = 0x180 for the final DMA.	differs	M2-79; M original F bl_NandF mem:0x64 fmss_asm

M1 ID	On M1	On M2	verdict	evidence
M1-F025	On M1 WRITE reads the DMA list one entry ahead and reads pages x units entries in total.	On M2 WRITE reads P_DMALIST at two places only: once before the transfer loop (0x050) and once per sector chunk (0x3f0), before the chunk-count test; it therefore reads pages x chunks + 1 entries, the last one beyond the list and never used for a transfer.	differs	M2-213; restore m /root-rest .s)
M1-F026	On M1, after 11h the chip is polled (70h, ready check, FAIL check) before 81h.	On M2 a page for a CE that already has one in flight carries a status byte in its descriptor, so with the two-plane builder WRITE sends 70h, latches, waits on wait 12 (0x40) and tests FAIL before 81h (source; the nano 4G uses the single-plane builder).	same	original V mem:0x08 store/fms chapter 1
M1-F027	On M1 a program FAIL ends the run at once with r6 bits 7:0 = selected CE mask and bit 28 set.	On M2 the 1.0.4 WRITE sets r6 bit 28, stores the failing CE mask in 0xd04 and still runs the final status sweep before END, r6 low byte holding the touched CEs in CTRL0 form; the older WRITE generation branches to the fault tail with bit 28 and the CE mask in r6.	differs	original V mem:0x08 store/fms chapter 1
M1-F028	On M1 ERASE issues 60/A3/D0 per entry without waiting, then sweeps status with the staged ready wait; r6 = mask of the CEs used.	On M2 ERASE issues 60h, three row cycles (FMC_ANUM = 2) and D0h per entry, first waiting for a previous erase on the same CE with one wait 12, then sweeps the CEs with one wait 12 each; r6 = mask of the CEs used.	differs	original E mem:0x08 store/fms

M1 ID	On M1	On M2	verdict	evidence
M1-F029	On M1 READID resets all eight CEs together, loops P_CEMAP times, reads 8 ID bytes through the FIFO and stores the first word.	On M2 the reset of all eight CEs is a separate RESET program; READID loops over the CE count in 0xd0c, reads 5 ID bytes (FMC_DNUM = 4) into the 0x52 FIFO latch and stores FMC 0x060 per CE.	differs	M2-124; mem:0x75 mem:0x76 (code/boo
M1-F030	On M1 the CPU writes FMSS_PROG, CS_IRQ = 0xff, start word 0xfff5, bounds the wait by 2 s and resets the sequencer with 0x30000.	On M2 the CPU writes FMSS_PROG, CS_IRQ = 0xff, completes the C60/C64 buffer handshake, starts with 0xffb5 (bootloader) or 0xfff5 (restore OS), bounds the wait by 2 s and resets with STOP (8), abort (2) when C08 = 4, the handshake, C00 = 0x80000, FMCTRL1 0x800/ 0x80000000 and 0x81c = 1.	differs	M2-20; M original C _resetFM _startFM _waitFor mem:0x51
M1-F031	On M1 the timing word is setup << 16 | hold << 12 | edo << 28, with edo (bits 29:28) the read-capture delay.	On M2 the cycle is (CTRL0[14:12] + CTRL0[18:16] + 2) x 7.06 ns for every bus byte; CTRL0[18:16] <= 1 shifts read data by one byte, and CTRL0[30:28] has no measured effect on any operation.	differs	M2-127; M2-139
M1-F032	On M1 READID reads FMC_FIFO (0x080) twice; once is enough.	On M2 READID latches the ID with FMC 0x040 = 0x52 and reads FMC 0x060; the CPU port 0x080 reads 0 with or without the latch.	differs	M2-124; mem:0x75 mem:0x76 (code/boo

M1 ID	On M1	On M2	verdict	evidence
M1-F033	On M1 every fault handler puts the selected CE mask in r6 bits 7:0; READ, READID and RAWREAD always set bit 27, PROGRAM and ERASE set it only if C50 bit 0 is set and always set bit 28.	On M2 RESET/READID put the CE mask in bits 7:0 and set bit 27; READ sets r6 = page summary | bit 27 without a CE mask; ERASE and WRITE set bit 27 only if C50 bit 0 is set and always set bit 28.	differs	chapter 1 bl_NandF mem:0x64 fmss_asm ERASE re mem:0x08 store/fms original W mem:0x08 store/fms
M1-F034	On M1 RAWREAD (no ECC) loads FMC_DATA_SEL = 0x102 in all steps and the CPU driver does not need it.	On M2 no RAWREAD sequencer program exists; the diag/restore raw page read issues 00h/30h and FMCTRL1 = 0xe2 from the CPU.	m2-only-context	chapter 1 docs/04-f section 2 docs/11-f section 2
M1-F035	On M1 the 78h variants test FMC_NSTAT & 3 with three row cycles, and WRITECACHE confirms with 15h while pages remain and ends with 71h.	On M2 one WRITE program serves every variant through CPU-built descriptors: after 78h (three row cycles) it waits on wait 12 arg = mask = 0x20 and tests FAIL bit 0; cache (15h) and 71h forms come from the descriptor builders, and the nano 4G uses 80h/10h with 70h.	differs	original W mem:0x08 store/fms docs/11-f section 3

M1 ID	On M1	On M2	verdict	evidence
M1-F036	On M1 FMCTRL1 takes one operation at a time: 0x100 immediately replaced by 0xe2 loses the DMA, whose `wait 2` times out.	On M2 FMCTRL1 0x100 replaced by 0xe2 before the DMA completes does not lose the DMA's data: its bytes reach RAM (written at once, all but a run of 4..16 bytes ending at byte 23; after 1..16 delay loops exactly), but FMC_STATUS bit 20 never sets, so its wait 2 times out; the read completes only when FMC_DATA_SEL is switched to 2 before 0xe2; after 64 loops both complete, and the serial order is exact.	differs	M2-332;
M1-F037	On M1 the 78h write forms of release 1.1.3 sweep four rows per CE with mask 3 and every other program is unchanged from 1.0.1 to 1.1.3.	On M2 the survey of 73 program instances (1.0..1.0.4, bootloader, restore, osos, diag) finds eight distinct programs: two WRITE generations (426 and 456 instructions, 1.0.4 and restore) with different failure reporting, every other program byte-identical.	m2-only-context	docs/29-f y.md; docs/11-f section 4
M1-F038	On M1 r0..r7 are 0xc18 + 4n with r6 = SEQ_RESULT; a CPU write before the start is the starting value, a start keeps them and the reset clears them.	On M2 the same: CPU-preloaded GPRs survive the start, STOP preserves them and the clearing step C00 = 0x80000 zeroes them.	same	M2-02; M
M1-F039	On M1 an instruction takes about 216 ns.	On M2 an ALU instruction takes 30.1 ns (one cycle of the inferred 33.25 MHz engine clock) and a two-instruction loop 33.8 ns per instruction.	differs	M2-09; M

M1 ID	On M1	On M2	verdict	evidence
M1-F040	On M1 `shr` is logical, every count >= 32 gives 0, arithmetic wraps at 32 bits, immediate 0 selects the register form and `ld` ANDs its mask.	On M2 shl/shr are logical and every count >= 32 gives 0 in the immediate and the register form (all counts up to 255, powers of two up to 2^31, 0xffffffff and wrap probes such as 0x101, 0x11f, 0x8000001f); counts are not taken modulo 32, 64 or 256 and the immediate is not truncated to 8 bits; add/sub wrap at 32 bits, immediate 0 selects the two-register form and ld ANDs its mask.	same	M2-146;
M1-F041	On M1 `ldi/sti` are little-endian with unaligned addresses rounded down, and the `pld/pst` offset wraps at 12 bits and reaches r0..r7.	On M2 `ldi/sti` are little-endian with unaligned addresses rounded down, and `pld/pst` use offset bits 11:0 only (every bit 12..31 ignored) and reach r0..r7: through 0xc18..0xc34 `pld` reads and `pst` writes the eight GPRs, through the pointer register r1 `pld` reads the pointer itself.	same	M2-175;
M1-F042	On M1 parameter RAM is 0xd00..0xd7c and 0xd80..0xdfc read 0 and drop CPU and program writes.	On M2 0xd80..0xdfc read 0 from both sides and drop CPU writes and program writes: 32 `wr` (distinct values), `wr` of all-ones to each of the 32 words followed by `ld`, and `pst` to 0xd80/0xdfc (also through offsets with bits 12/13/16/20) all read back 0 to the sequencer and the CPU, and no write aliases into 0xd00..0xd7c.	same	M2-147;

M1 ID	On M1	On M2	verdict	evidence
M1-F043	On M1 the wait timeout is about 4.87 ms (start word 0xfff5), independent of C58; at idle only event 11 is set.	On M2 the same start-word count gives about 126.6 ms (2^14 engine cycles per count step), also independent of C58 bit 2; at idle only event 11 passes.	differs	M2-10; M
M1-F044	On M1 C58 bit 2 sends a timeout or illegal-opcode fault to the vector (else the next instruction), bits 0 and 2 set C5C and C58 holds 8 bits.	On M2 bit 2 selects vector or continuation in the same way and C5C reads 1 after a vectored timeout, but C58 keeps only the bits of 0xd6 (bit 0 reads back 0) from both CPU and sequencer.	differs	M2-11; M 8.16
M1-F045	On M1 C58 bit 0 at an END sets C5C and makes every later END run the fault vector once until C58/C4C/C5C are rewritten.	On M2 C58 bit 0 is not stored (C58 = 1, 3, 5, 7 read back 0, 2, 4, 6) and has no effect on the fault policy: an END with it written (CPU- or program-written C4C) leaves C5C = 0 and runs no fault vector; the vector itself works (C58 bit 2 faults vector and set C5C = 1).	differs	M2-148;
M1-F046	On M1 C58 = 2 with a CPU-written C4C diverted one END in `oneshot`, bit 1 alone diverted none in c58b1/c58b2, and C58 = 6 diverted the first END once.	On M2 no END runs the fault vector: C58 = 2 written by the CPU before a program that writes C4C, C58 = 2 in every write order of C58 and C4C, and C58 = 6 with a CPU-written C4C all end at the fall-through END with C5C = 0.	differs	M2-149;

M1 ID	On M1	On M2	verdict	evidence
M1-F047	On M1 `0f` is an unconditional indirect jump, `10` sets CS_IRQ bit 1, `12` stops with CS_IRQ bit 3, C08 = 0xa and the instruction in C40/C44 until the CPU writes PC and a start word, `15` toggles C14, `08`/`ff` have no effect and `1a..1f`, `20`, `40`, `80` fault after about 4.87 ms.	On M2 `15` does not toggle C14: `tmwait` waits from start-word bits 15:8 until the (count + 1)-th timeout instant after its entry (492.747 us × count + 450 us when entered at the start, 126.1 ms for 0xff), takes the timeout fault for count 0, ignores its fields and C14 writes and leaves C14 = 0, and a following `tmwait` waits again instead of reloading at once; `0f` jumps only if r[R] != 0; `10` sets CS_IRQ bit 1 and continues; `12` pauses with CS_IRQ bit 3, C08 = 0xa and the PC kept until the CPU writes the PC and a start word; `ff` and `08` with R = 0..7 continue, but `08` with R = 8..0xff stalls until the timeout fault; every value 0x1a..0xfe faults at the timeout like 1a.	differs	M2-373; M2-14; M M2-377; M2-629;
M1-F048	On M1 C14 reads 0xff after power-on and ignores CPU and program writes.	On M2 C14 reads 0x7f at power-on (first FMSS access after a power cycle), in the untouched window after a cleanup and after the ordinary reset, and ignores CPU and program writes (CPU 0x55; program 0x12, 0, 0xff, 0x33).	differs	M2-430; chapter 8

M1 ID	On M1	On M2	verdict	evidence
M1-F049	On M1, after a reset, the first START fetches instruction 0 and executes it only if it is end; a second START is needed otherwise.	On M2 the first start word after the ordinary reset runs a program whose instruction 0 is not END (movi, 0x08, wr CS_IRQ) to its END within a few microseconds, and so does the first such program after power-on (from a window identical to the power-on window); no second start is needed.	differs	M2-151; driver _r mem:0x50 mem:0x51 _waitFor mem:0x51
M1-F050	On M1 C50 bit 0 is set while a command flag is pending; a timeout or illegal opcode does not change it and a handler write of 1 does not clear it.	On M2 C50 bit 0 is set while the command flag (FMC_STATUS bit 1) is pending: it rises by the third instruction after the 70h write, stays set through a timeout or illegal-opcode fault (in the handler and after the END) and through a handler write of 1, and clears with the bit 1 ac knowledgement; a fault without a pending command does not set it.	same	M2-152; M2-112

M1 ID	On M1	On M2	verdict	evidence
M1-F051	On M1 C14 is the timeout timer: loaded 0xff by a waiting `wait`, counting down, 0xff after a timeout, frozen at END, writes ignored; 15 waits from non-zero and reloads from 0.	On M2 C14 ignores CPU and program writes and counts down by one per timeout step from start-word bits 15:8 during a stalled wait and a `tmwait`, but reads 0x7f, not the reload value, after a timeout fault whatever the start word, after a wait released after one or more timeout steps and before the first wait of a run; a completed `tmwait` leaves 0, and a second `tmwait` started from 0 waits again instead of reloading at once (0xbc about 34 ms into it).	differs	M2-177; M2-150;
M1-F052	On M1 `1a..fe` fault after about 4.87 ms with no other effect, `09`/16 time out only without their condition, the full opcode byte is decoded and `08`/`ff` cost the same as `movi`.	On M2 every `1a..fe` value (including `20`, `40`, `80`) gives exactly the outcome of 1a, a fault at the timeout with no register or scratch change, also with a pending command, address, DMA, ECC or status flag, and `09`/16 time out only without their condition, but `08` costs 37.6 ns (1.25 cycles) against 30.1 ns for `movi` and `ff`, and an all-ones word executes as `ff`.	differs	M2-56; M M2-583; M2-367
M1-F053	On M1 `0f` jumps to the unrounded byte offset in r[arg] (arg >= 8 other than 8 gives r0), ignores R and imm and keeps no return address.	On M2 `0f` jumps to r[arg] only if r[R] != 0, ignores the low three target bits, uses r0 for arg 9..255 and loops on itself for arg 8, ignores imm and keeps no return address.	differs	M2-46; M M2-122

M1 ID	On M1	On M2	verdict	evidence
M1-F054	On M1 start-word bits 1, 3 and 16 each keep the program from running (C00 bit 18 set), the other bits have no visible effect, and bits 15:0 read back with bit 0 cleared.	On M2 start-word bits 1, 3 and 16 keep the program at instruction 0 with C00 bit 18 set, as on M1, but bit 19 also keeps it from running (C00 then reads 0x7f60, bit 18 clear); bit 2, bits 4..15, 17, 18 and 20..31 do not change the run; bits 15:1 read back with bit 0 clear except bit 3, which does not read back, and bits 16..31 read 0.	differs	M2-157; chapter 8
M1-F055	On M1 the CPU write masks are those of M1 chapter 16.2, C38 is read-only, C3C and C48 hold 32 bits and C54 12 bits, and the +0x400 words are writable.	On M2 C3C and C48 hold 32 bits from both sides and C54 holds 0x83f (M1 12 bits), and a start does not clear them; C38 is writable by the CPU (the write moves the PC); further CPU masks depart from M1 chapter 16.2: 000 0x77077fff, 018 0xf, 804 0x3f04, 808 0x1f81, 814 0x2f, 844 0, 940..950 0, c10 0xdf.	differs	M2-191; M2-70; M
M1-F056	On M1 window offset bit 9 is not decoded, 0x400 is a second register set with the same layout, 0x100..0x1fc is a block mirrored four times and 0x800 reads 0x20060808.	On M2 window offset bit 9 is not decoded below 0x400 and in 0x800..0xbff but is decoded in 0xc00..0xfff (no holding word of 0xc00..0xdff appears at +0x200; 0xd80..0xffc read 0); 0x400..0x7fc read 0, so 0x400 is no second register set; the 0x100 block appears at 0x100 and 0x300 only (mirrored twice, not four times); 0x800 reads 0x20070618.	differs	M2-155; M2-116;

M1 ID	On M1	On M2	verdict	evidence
M1-F057	On M1 event 11 is not the ready line: it passes during an erase.	On M2 `wait 11` passes at once in every tested state (idle, timeout disabled, DMA in flight, encode running, command or status latch pending, every flag acknowledged, C10 = 0, 0x044 all ones), also right after an erase confirm with the chip busy, and whatever its operands.	same	M2-10; M M2-387
M1-F058	On M1 `wait 5` passes when the polled chip becomes ready (about 155 us into an erase on that chip).	On M2 `wait 5` after one status latch passes at the erase end (about 2.27 ms on the Hynix part), when FMC_NSTAT changes from 0x80 to 0xe0.	same	M2-130;
M1-F059	On M1 a DMA to 0x50000000 never finishes while an `ldi` from that address does not fault.	On M2 a DMA from 0x50000000 completes (wait 2 passes, END after 9 us, the device stays responsive), while an `ldi` from that address hangs the whole system (USB lost even in stall-safe mode; power cycle needed): the opposite of M1 on both accesses.	differs	M2-717;
M1-F060	On M1 FMC 0x028 advances by one per `pld+sti` loop iteration (128 over 128).	On M2 FMC 0x028 does not count pld+sti loop iterations: 128 iterations of pld 0x080 + sti, pld 0x048 + sti and ldi + sti leave it unchanged (M1: +128).	differs	M2-156; restore m /root-rest .s)

M1 ID	On M1	On M2	verdict	evidence
M1-F061	On M1 start-word bit 1 pauses before instruction 0 until a start without it, bits 3 and 16 give the post-reset hold, there is no single-step and bit 17 does not clear registers.	On M2 start-word bit 1 holds the program before instruction 0 (C00 0x40002, C08 = 3) until a start word without it (0x1, 0xfff5 or 0xffb5) runs it once from 0; bits 3 and 16 give the hold at PC 0 (C00 0x40000), released the same way; repeated 0x9, 0x8 or 0x2 writes execute no instruction (no single-step); preloaded r2..r5 survive starts with and without bit 17 (0xfff5, 0x20001, 0x20000), so bit 17 does not clear the registers.	same	M2-157; chapter 8
M1-F062	On M1 only events 0, 11 and 15 passed in the five NAND states (15 after a command byte).	On M2 event 0 passes after a command and event 11 always, but event 15 never passed, also after a command, address, DMA, ECC or status stimulus in either order and after the further FMC, ECC, CPU-write and VIC soft-line stimuli of M2-386.	differs	M2-69; M M2-386

M1 ID	On M1	On M2	verdict	evidence
M1-F063	On M1 a sequencer read of FMC_STATUS/FMC_NSTAT does not consume an event, and `wait 5` straight after `70h`+latch timed out once although bit 23 read set.	On M2 a sequencer read of FMC_STATUS/FMC_NSTAT does not consume event 5, as on M1, but wait 5 straight after 70h + latch never timed out (41 of 41 in a series in each of four captures, returning 0x70 or 0xe0); the only WAIT5 faults are latches one or two instructions after an unwaited 70h (one or two `nop2`, one `nop`, one `movi` or the acknowledge), which set FMC_STATUS bit 3 instead of bit 23, also straight after a reset; three or more instructions between pass.	differs	M2-178; M2-68; M
M1-F064	On M1 C10, C3C, C48, C54 change no timeout, wait, DMA or encode; C3C reads 0 after a run and holds the faulting offset + 8 inside a handler.	On M2 C3C holds the faulting offset + 8 inside the handler (also for a nested fault) and reads 0 after the run, and C10, C3C, C48, C54 preset to 0 or all ones leave the wait timeout, a DMA in, the BCH8 parity and a DMA out unchanged; but with C48 = 0xffffffff or 0x55555555 a `tmwait` does not end after its period (3898 us at count 7) and instead faults at the wait timeout (4390 us) in every run, while C48 = 0 lets it end; of the single bits, bits 1..7 make a count-1 `tmwait` fault, bit 0 ends it one timeout instant early and bits 8..31 change nothing.	differs	M2-179; chapter 8

M1 ID	On M1	On M2	verdict	evidence
M1-F065	On M1 CS_IRQ bits 6:4 are FMC and ECC sources gated by 0x044 and ECC 0x844, bit 6 being ECC_DONE pending.	On M2 CS_IRQ bit 4 is the sticky OR of the 0x044-routed FMC flags (bits 0, 1, 2, 11, 12); bits 5 and 6 never rise after FMC operations or after BCH8 encode and decode completions with any tested ECC enable candidate (0x800, 0x808, 0x814, 0x818, 0x844, 0x860..0x9fc all ones, their single bits, ECC_CTRL and 810 extra bits; 0x044 = 0 or all ones), so no ECC source of CS_IRQ exists among the tested words.	differs	M2-200; chapter 8
M1-F066	On M1 `wait 5` is FMC_STATUS bit 23 as a level; after a latch a new command is refused until FMCTRL1 = 0x20 (bit 1 stays clear, bit 26 set), and a latch after 90h sets no bit 23 and leaves 0x90 in FMC_NSTAT.	On M2 wait 5 is FMC_STATUS bit 23 and after a latch a new command is refused until an FMCTRL1 write (status clear, ECC kick and FFh do not release), as on M1, but the refused command leaves bit 26 clear, while an address operation after a latch without the release fails with bit 26 set; a latch after 90h sets no bit 23 and leaves the first READ ID byte 0xad in FMC_NSTAT, not 0x90.	differs	M2-181; M2-31; M

M1 ID	On M1	On M2	verdict	evidence
M1-F067	On M1 C08 = 1 running, 4 waiting or stalled, 3 paused, 0xa blocked at 12, 0 held or idle.	On M2 the CPU reads C08 = 0 idle or ended, 3 paused by start-word bit 1 or after abort, 4 in every unmet wait, `wait2` and `tmwait`, 0xa at `pause`, 0xb for a zero-field `evjmp` with the timeout disabled, 1 when held by C60 = 1, and 1, 2 or 9 while a program runs a loop (also in a fault handler or running on after a C58 = 0 timeout fault), the running value differing between identical runs; 2, 9 and 0xb are codes M1 does not show.	differs	M2-384; M2-62; M
M1-F068	On M1 start-word bit 1 pauses a running program exactly and a later start resumes it; bit 3/16 while running aborts to the post-reset hold and the release restarts from 0; a plain start while running is ignored.	On M2 start-word bit 1 written while a loop runs freezes it exactly (r0 stops at the write, C08 = 3, C00 0x4fff6) and a later 0xfff5 resumes at the same point (END at the predicted 5356 us); bit 3 or bit 16 while running stops at PC 0 and the release restarts the program from instruction 0; a plain start (0xfff5 or 0xfff4) while running is ignored.	same	M2-158;
M1-F069	On M1 register numbers 8..255: as destination only 10 acts besides the PC; as argument 9 and 11..255 read r0 and 10 reads the destination in the immediate form and r0 in the register form; `st` stores the register-bus latch; `bz`/`bnz` never branch; `jr`/`ldi`/`pld` use r0.	On M2 every role matches M1 except that source 10 reads the destination register also in the register form, for all six ALUs and every GPR destination.	differs	M2-43; M M2-52; M

M1 ID	On M1	On M2	verdict	evidence
M1-F070	On M1 `bz`/`bnz` ignore arg, test 32 bits and take unaligned targets as given; `wait` ignores arg/imm; event numbers > 15 never pass.	On M2 branch targets ignore their low three bits and `wait` 12 uses arg and imm as status operand and mask (both unlike M1); wait event numbers above 15 never pass, as on M1: events 16, 27, 0x8b and 255 time out with C58 = 0 and all 2640 `wait` `16..255` cases in 8 states (idle, erase, program, read, DMA, ECC, metadata encode/decode) fault, including 150 whose low four bits name an event that passes in the same state (no modulo-16 alias).	differs	M2-159;
M1-F071	On M1 instruction time is fetch-bound: 217 ns from SDRAM, 138 ns from SRAM, window access +35..37 ns, `ldi`/`sti` +120..218 ns.	On M2 SRAM and SDRAM give the same times: ALU 30.1 ns, window `ld`/`wr` 60.6 ns, `sti` to SDRAM 97.8 ns, `ldi` from SDRAM 211 ns.	differs	M2-102
M1-F072	On M1 a page read waits on events 1 (address), 0 (command), 3 (transfer, 512 B in 49 us) and 4 (ECC).	On M2 a page read waits on the same events 1, 0, 3 and 4, and the 512-byte transfer takes about 22 us at the setup timing word.	differs	M2-19; M
M1-F073	On M1 the 0x400 set answers no READ ID with only channel 0's pins configured, and four CEs on the 0x000 set answer ec d5 14 b6 74.	On M2 four CEs on the 0x000 set answer AD D5 14 B6 44 (Hynix, ID repeating after five bytes), and the +0x400 storage words read 0.	m2-only-context	M2-82; M

M1 ID	On M1	On M2	verdict	evidence
M1-F074	On M1 block 0x100 has writable words at 0x1b8..0x1d8 (values 0xffffffff, 0x008060, 0x3010d0).	On M2 the sequencer-writable words of 0x100..0x1fc are 0x1b8..0x1d8 (0x1b8 one bit, read before the write 0; 0x1cc/0x1d0 24 bits, read before the write 0x8060/0x3010d0; the others 32 bits, read before the write 0xffffffff); the +0x200 copy follows every write and the other words keep nothing (the sequencer reads 0x12).	same	M2-160;
M1-F075	On M1 events are levels; event 4 is any ECC_DONE bit but bit 4; events 6, 7, 9 are the second set's bits 1, 2, 3 and event 8 its bit 20; `wait 15` did not pass with the enables on and a flag pending.	On M2 events 0..2 are levels that waits do not consume and event 15 never passes, but event 4 follows 0x810 bit 1/bit 2 only, and events 6..10 never passed in any tested state.	differs	M2-66; M M2-107;
M1-F076	On M1 only enabled FMC and ECC sources reach CS_IRQ bits 4 and 6 (0x044 bit 0 command; 0x844 bits 0, 2, 5 ECC); END, faults, `sig` and `pause` never raise the CPU line.	On M2 only 0x044-enabled FMC flags reach CS_IRQ bit 4, and no ECC enable candidate sets CS_IRQ bit 5 or 6 or raises a VIC line after an encode or decode (0x844 is an encode-parity word); `sig` raises no VIC line, but END raises a CPU line: VIC1 raw bit 22 (source 54) is set after every END while C10 is non-zero and stays clear with C10 = 0 and in runs that stop without an END.	differs	M2-212; M2-200;

M1 ID	On M1	On M2	verdict	evidence
M1-F077	On M1 r8 is the PC (read = current offset, write = jump) and a write to r10 sets r0..r7.	On M2 the same: `movi r8` with a local target jumps, `mov r2, r8` returns the instruction offset, and MOVI/MOV/ADD to r10 write all eight GPRs.	same	M2-38; M
M1-F078	On M1 every window has known power-on values and a soft reset of the writable words leaves status words and C14 different.	On M2 the power-on window equals the window left by the ordinary reset in every word except C00 (0x7f60 at power-on, the last start word after the reset) and C04 (0 at power-on, the program address after the reset); status words and C14 (0x7f) do not differ.	differs	M2-430; chapter 8
M1-F079	On M1 the second set's controller runs a READ ID (command, address, 8-byte transfer counted in 0x424) with no chip answering.	On M2 the +0x400 set runs no READ ID: with set-up word, CE, 90h, address cycles, DNUM 7 and FMCTRL1 0xe2 or 0x1c2 written through it, waits 6, 7 and 9 time out and 0x424 and every other +0x400 word stay 0; the same READ ID on the primary set passes waits 0, 1, 3 and leaves 0x024 = 8.	differs	M2-183; M2-139
M1-F080	On M1 a soft reset of the writable words to their power-on values plus status acknowledges gives power-on behaviour, and 7 words keep last-written values or ignore writes.	On M2 the ordinary reset alone reproduces every power-on word except C00 and C04, which hold the start word and program address of its preflight; no soft reset of writable words to power-on values is needed.	differs	M2-430;

M1 ID	On M1	On M2	verdict	evidence
M1-F081	On M1 FMC_CTRL0 bit 24 enables DMA: without it nothing moves in either set or direction.	On M2 nothing moves with FMC_CTRL0 bit 24 clear (wait 2 times out, FMC_STATUS bit 20 clear, 0x03c static, in both directions and slots); the DMA is held and starts when bit 24 is set again, with the FMC_DMA_ADDR value at that moment; the +0x400 set moves no DMA at all (M2-185).	same	M2-338; M2-76; M `bl_NandF` `mem:0x64` fmss_asm
M1-F082	On M1 the second set has its own data buffer and DMA engine and event 8 is its DMA flag.	On M2 the +0x400 set has no working buffer or DMA engine: its RAM-to-buffer-to-RAM loopback leaves the output untouched with wait 8 timed out in both slots while the primary buffer keeps its content, and wait 8 never passes on the first set either.	differs	M2-185; M2-111
M1-F083	On M1 a page-read sector takes 49 us NAND transfer, 14 us DMA to RAM and 123 us from command to RAM.	On M2 a 512-byte NAND transfer takes about 22 us at the setup timing word, a DMA in about 2.7 us and out about 1.1 us, and a page open about 44 us.	differs	M2-117;
M1-F084	On M1 16 timed out in the near-power-on sweep but went on at once in every operand form with an event pending; `09` timed out in all sixteen forms, including register-and-mask conditions that held.	On M2 `evjmp` vectors at once with a pending command, address, data, DMA, ECC or status event in all three tested operand forms and faults when idle; `wait2` with an unmet condition times out like an illegal opcode.	same	M2-34; M

M1 ID	On M1	On M2	verdict	evidence
M1-F085	On M1 C50 bits 5:0 are events 0..5 pending, bit 0 the command flag.	On M2 C50 bits 0..5 are command, address, DMA, data/parity transfer, ECC and status-latch completion, the same order as events 0..5.	same	M2-112;
M1-F086	On M1 16 goes to the fault vector at once while any event is pending, else times out and faults.	On M2 the `evjmp` continues at C4C without a fault (CS_IRQ = 1 at the handler's END) for six FMC/ECC stimuli, in every tested R/arg/imm form, and faults with CS_IRQ = 5 when idle.	same	M2-58; M
M1-F087	On M1 the status latch's `wait 5` timed out in one of two identical runs.	On M2 the first status latch (70h, wait 0, acknowledge, latch) never timed out: 1337 of 1337 in repeated series, 2644 of 2644 first latches and 410 of 410 in 11 forms in each of four captures (status 0xe0).	differs	M2-161
M1-F088	On M1 16 also catches an event arriving during the wait, jumps to C4C, ignores C58, leaves the flag set, and counts the second set's flags.	On M2 evjmp catches an event that arrives during the wait (a command flag 100, 1000 or 5000 us in) or is already pending (command, DMA, ECC, status), jumps to C4C for C58 = 0, 2 and 4 with CS_IRQ bit 2 clear and leaves the flag set; an acknowledged flag is no event and with no event it times out after the full period; a 70h written through +0x408 during the wait raises no event, and no +0x400 operation raises a flag, so there are no second-set flags to count.	differs	M2-186; M2-183;

M1 ID	On M1	On M2	verdict	evidence
M1-F089	On M1 `08`, `10`, `15`, `ff` do not react to pending flags and 12 blocks in every state.	On M2 the same: the M1-comparison forms match in the idle, DMA and ECC states, and the opcode-state matrix with command, address, DMA, ECC and status stimuli shows the same signatures.	same	M2-34; M
M1-F090	On M1 `09` continued with condition 5 while the status flag was pending and timed out with other flags, numbers or masks alone, like an unmet wait.	On M2 `wait2 5` passes in every state, FMC flags alone (0x044 = 0) raise no line, and an unmet `wait2` faults at the timeout like 1a.	same	M2-65; M M2-142
M1-F091	On M1 `09` ignores arg/imm and of 0..15 only field 5 passes, also with the status flag acknowledged and the chip busy.	On M2 wait2 ignores arg/imm (six (arg, imm) forms give the same outcome), but of conditions 0..15 not only 5 passes: 5 passes at once in every state and 8 passes with the CPU timer line, also while the chip is busy.	differs	M2-187; M2-140
M1-F092	On M1, from a clean state, only `wait2` condition 5 of 0..31 passes.	On M2 condition 5 always passes and condition 19 (USB) passes depending on USB traffic, with 8 and 16 passing on timer or DMAC0 activity.	differs	M2-74; M
M1-F093	On M1 the 0x400 set is NAND channel 1 with its own GPIO pins and the boot ROM can boot from channel 0 or 1.	On M2 the boot ROM's NAND boot configures one pin set and reads bank 0 of the 0x000 controller only; the +0x400 words hold no values and no channel-1 pin map is known for S5L8720.	m2-only-context	M2-82; M docs/01-B (`rom_boo` FUN_200

M1 ID	On M1	On M2	verdict	evidence
M1-F094	On M1 the CPU continues pause by writing the PC (offset + 8) and a start word, with or without acknowledging, and can write the PC only then.	On M2 a program stopped at pause continues after the CPU writes the PC (offset + 8) and a start word, with or without acknowledging CS_IRQ bit 3, and stays paused after a start word or a PC write alone, as on M1 (with two start words after the PC write it stayed paused in one of four runs); but the CPU can also write the PC outside a pause: before the start (the program starts at the written offset), while a loop runs (it jumps at once) and during an unmet wait (the run continues there after the timeout).	differs	M2-278; M2-26; M
M1-F095	On M1 the program can write the PC (0xc38 = r8) to jump.	On M2 a program write of the PC is a jump: `wr [0xc38],#lab` and `st [0xc38],r1` land on `lab` without executing the fall-through, as do `movi`/`mov`/`add` with destination r8; `ld [0xc38]` reads the offset of the `ld` itself and does not jump.	same	M2-162; M2-616
M1-F096	On M1 start-word bit 2 enables the timeout; without it waits and illegal opcodes wait forever; bits 15:8 and 7:4 set the length.	On M2 with start word 1 an unmet `wait 3` and illegal 1a stay at C08 = 4 without a fault, and bits 15:8 (count) and 7:4 (step) set the length.	same	M2-56; M

M1 ID	On M1	On M2	verdict	evidence
M1-F097	On M1 window offsets are 12 bits, branch and r8 targets with bit 12, 16 or 24 set go elsewhere, and `jr` with a target above 0xfff does not jump.	On M2 branch and `jr` targets do not wrap between 2^15 and 2^24 (both land exactly up to 2^24 + 0x100: the PC has at least 25 bits) and `jr` to a target above 0xfff jumps.	differs	M2-105;
M1-F098	On M1 the 13 parity bytes appear in 0x900..0x90c after an encode, zero data gives non-zero parity, and 0x9a4 counts decodes.	On M2 the parity of the last encode is at 0x840..0x85c (word order reversed, big-endian words), 0x900..0x910 read 0 after encodes, and 0x818 bits 10:0 count decodes.	differs	M2-72; M
M1-F099	On M1 a DMA always moves 512 bytes and FMC_DNUM sets only NAND transfer lengths.	On M2 RAM-to-buffer-to-RAM loopbacks with FMC_DNUM from 0 to 0x1fff moved exactly 512 bytes in every case (in, out and both; the output beyond byte 512 untouched); a RAM-to-buffer length below 512 is not separable from the buffer's earlier content.	same	M2-189;
M1-F100	On M1 no chip answers on channel 1 of this board with its pins switched.	On M2 the nano 4G's four Hynix CEs answer on the 0x000 controller and the +0x400 words hold no values; no channel-1 wiring is known.	m2-only-context	M2-82; M
M1-F101	On M1 `wait2` conditions are not the second register set's flags.	On M2 `wait2` conditions are VIC0 hardware interrupt lines (5, 8 timer, 16 DMAC0, 19 USB, 20 NAND controller), not controller flags.	same	M2-140;

M1 ID	On M1	On M2	verdict	evidence
M1-F102	On M1 C50 bits 5:0 follow their events, 0x044 bits 0, 1, 2, 11, 12 route flags into CS_IRQ bit 4, VIC0 line 20 appears with FMC enables and VIC1 line 22 never.	On M2 C50 bits 5:0 follow their events (set while the flag is pending, clear after the acknowledgement) and 0x044 bits 0, 1, 2, 11, 12 route the command, address, transfer, DMA and status flags into CS_IRQ bit 4 and VIC0 line 20, as on M1; but VIC1 raw bit 22 rises at every END while C10 is non-zero.	differs	M2-200; M2-112;
M1-F103	On M1 the whole window has a measured read/write matrix from both sides, mirrors are confirmed by writes, 0x940..0x950 are read/write and C3C/C48/C54 clear at a start.	On M2 the window matrix differs from M1: 0x940..0x950 hold no bits (M1 read/write), the 0x100 block is mirrored twice (0x100, 0x300) instead of four times, offset bit 9 is decoded in 0xc00..0xfff, and a start does not clear C3C/C48/C54 (the program reads 0xffffffff, 0xffffffff, 0x83f after CPU writes of all ones).	differs	M2-154; M2-03; M M2-83
M1-F104	On M1 a status latch raised FMC_STATUS bit 24 instead of 23 (no `wait 5`) in every run with 0x044 or 0x844 written.	On M2 a status latch with 0x044 written never raised FMC_STATUS bit 24: 969 WAIT5 latches with 0x044 = 1, 2 or 0xffff all passed with bit 23, and 99 latches with 0x044 = 0 or each single bit 0..31 set bit 23 99 times and bit 24 0 times.	differs	M2-163;

M1 ID	On M1	On M2	verdict	evidence
M1-F105	On M1 end faults when R or arg is non-zero, ignores imm and latches its word and imm in C40/C44.	On M2 end with non-zero R or arg (R 1; arg 1; R 1 + arg 7 + imm 0xff) does not fault: it stops like the zero-field end (END, CS_IRQ 1, next instruction not executed); imm is ignored, and the word's low 24 bits (R << 16 | arg) and imm are latched in C40/C44.	differs	M2-164;
M1-F106	On M1 the fields the originals leave 0 (imm of `st`/`ldi`/`mov`/`sti`/`pld`/`pst`, R of `wr`, arg of `movi`) are ignored.	On M2 the fields the original programs leave 0 are ignored when set: imm 0xff and 4 in `st`, `ldi`, `mov`, `sti`, `pld` and `pst`, R 3 and 7 in `wr`, and arg 0x1234 and 0xd70 in `movi` give exactly the zero-field result (registers, PRAM 0xd00..0xd7c, scratch RAM).	same	M2-165;
M1-F107	On M1 start-word bits 7:4 = 0 make the timeout immediate, and with bits 7:4 = 0xf the length grows with bits 15:8 (16 / 310 / 4868 us for 0x00 / 0x0f / 0xff).	On M2 bits 7:4 = 0 fault at once (0..4 us for every tested count), bits 7:4 = 1 give about 2 cycles per count (4, 8 and 14..16 us for counts 0x40, 0x80, 0xff), and with n >= 11 the length is 492.747 us x (count + 1) + 447.5 us (about 126.6 ms for count 0xff).	differs	M2-81; M
M1-F108	On M1 the ECC corrects up to 8 data bits per sector, and 9..12 are reported uncorrectable (ECC_RESULT 0x11) with the data left as sent.	On M2 BCH8 corrects 1..8 bit errors and leaves the data unchanged for 9, 10 and 16, reporting ring entry 1 (0x80c) and ECC_STATUS bit 3 instead of 0x11.	differs	M2-84; M

M1 ID	On M1	On M2	verdict	evidence
M1-F109	On M1 the code is binary BCH t = 8 over GF(2^13)/0x201b with the M1 generator; the message is 512 data bytes plus 3 slot metadata bytes, MSB first, parity in slot bytes 3..15.	On M2 BCH8 is the same code (field, generator, MSB-first), but sector 0's message is the data followed by ten metadata bytes and the other sectors' only the data, with parity after the metadata in the stream.	differs	M2-77; cl
M1-F110	On M1 the bit-24 failure did not recur in fifteen runs with and without enables, so the enables are not shown to cause it.	On M2 the bit-24 -instead-of-23 failure never occurred: none of 2743 latch records shows bit 24, with and without 0x044 enables.	same	M2-163;
M1-F111	On M1 the timeout is (count + 1) x 2^s bus cycles with s = 3, 5, 7, 9, 10 for bits 7:4 = 1..5 and 11 from 6 up (0: immediate).	On M2 the step for bits 7:4 = n is 2^3, 2^5, 2^7, 2^8, 2^9 engine cycles for n = 2..6 (33.25 MHz; fault time linear in count + 1) and 2^14 cycles from n = 11 (492.747 us x (count + 1) + 447.5 us); n = 0 faults at once and n = 1 gives 2 cycles per count within the 2-us resolution (4, 8, 14..16 us for counts 0x40, 0x80, 0xff); the exponents differ from the M1 table, while consecutive n from 2 to 6 change them by 2, 2, 1, 1 as on M1.	differs	M2-216; M2-115
M1-F112	On M1 original-firmware pages are code words of the BCH code with their metadata in slot bytes 0..2.	On M2 original-firmware pages on CE0 and CE1 are code words of the BCH8 code (352 of 352 sectors) with ten metadata bytes at the start of sector 0.	differs	M2-77

M1 ID	On M1	On M2	verdict	evidence
M1-F113	On M1 the metadata path writes FMC_META0..2 into slots 0..3, bytes in order, and the encode includes them.	On M2 WRITE loads FMC 0x060..0x068 with the 12-byte record and latches it with 0x040 = 0xc1; sector 0's encode covers ten of those bytes and the last two are not stored.	differs	M2-77; M restore m /root-rest .s)
M1-F114	On M1 a parity buffer is 64 bytes and a 512-byte DMA into it wraps, leaving the source's last 64 bytes.	On M2 a 512-byte DMA into parity buffer 0 leaves the source's last 64 bytes, read back by a DMA out (the 64 bytes repeated) and by programming 128 bytes from the buffer (the 64 bytes twice): the buffer is 64 bytes and wraps.	same	M2-232;
M1-F115	On M1 FMC_DNUM limits a write from a data buffer, so a raw page with host-built chunks is written without the ECC engine.	On M2 raw pages with host-built code words (metadata, parity and data of their own lengths) are written without the engine and verified by raw read-back.	same	M2-84; M
M1-F116	On M1 stored errors give 8 data bits corrected, 9 uncorrectable, 2 metadata bits corrected and counted, and an all-zero chunk uncorrectable.	On M2 stored BCH8 errors give 1..8 corrected including metadata and parity (counted in 0x818) and 9 or more data unchanged; an all-zero sector-0 chunk (metadata, parity and data 0) decodes as a clean code word with BCH8 and BCH16 (0x810 = 0x15, 0x80c = 0, zero data returned): zero parity is the valid parity of zero data and is not treated specially.	differs	M2-214;

M1 ID	On M1	On M2	verdict	evidence
M1-F117	On M1 FMC_STATUS bit 27 is set for every written sector read and clear for erased ones.	On M2 FMC_STATUS bit 27 is set after the metadata/parity or data transfer of every written sector (16 of 16) and clear for every erased sector (0 of 8), also for single 512-byte, parity-only and ECC-decode reads.	same	M2-145;
M1-F118	On M1 ECC_MODE bit 8 enables the slot path, bits 0, 6, 7, 9, 10 make no difference with it, and there are 144 modes without bit 8.	On M2 ECC_CTRL has no slot-path bit: bit 8 is part of the metadata length (bits 15:8) and bits 3..7 and 16..31 have no encode effect.	differs	M2-98; M
M1-F119	On M1 ECC_DONE reads 0x33 after an encode, 0x35 after a decode and 0x3d after an uncorrectable one, bit 3 staying set.	On M2 ECC_STATUS reads 0x13 after an encode, 0x15 after a decode with 0..6 corrections, 0x35 with 7 or more, 0x1d uncorrectable and 0x55 erased; bit 5 means at least 7 corrected bits.	differs	M2-88; M chapter 1
M1-F120	On M1 no wait2 condition N in 0..255 follows a chip's erase, program or read, and condition 33 passed only in some runs.	On M2 no wait2 condition follows chip busy: after erase, program and read starts, 804 cases of N 0..255 pass only lines 5, 8, 16 and 19, the same as in idle and controller states (5 at once; 8, 16, 19 = CPU timer, DMAC0, USB at times unrelated to the about 2273 us erase busy); 32..255 never pass and 33 never passed (23 cases).	same	M2-167;

M1 ID	On M1	On M2	verdict	evidence
M1-F121	On M1 event 5 is set by a latch only while armed, the latch uses the arm, and FMCTRL1 operations re-arm it while 0x20, status clear, CE toggle and ECC kick do not.	On M2 a status latch needs no arm: after a latch, its wait 5 and only the acknowledge of bit 23, a second latch (FMCTRL1 0xca, no new 70h) passes wait 5 with no operation between and after FMCTRL1 0x20, a status clear, a CE toggle, an ECC encode, the set-up word, FMCTRL1 0x100e0 or the 0x30020 release; an address operation in that position does not complete (wait 1 times out, FMC_STATUS bit 26 set). A new command after a latch is refused until an FMCTRL1 write.	differs	M2-494; M2-237; M2-68; M
M1-F122	On M1 single-page programs on CE0/CE1, two CEs in one run, two-plane program and erase and erase on CE1 read back correctly.	On M2 raw and ECC single-page programs on CE0 and CE1, multi-page runs, one page on each CE in one run and erases on both CEs read back exactly, and a two-plane program (80h/11h/81h/10h) of blocks 3000/3001 and a two-plane erase (60h/60h/D0h) read back exactly (both pages all 0xff after the erase, status 0xe0).	same	M2-339;
M1-F123	On M1 a status poll on FMC_DATA_SEL = 0x100 returns 0x70, ends while the chip programs, and the next page's 80h is ignored.	On M2 with the status poll on FMC_DATA_SEL = 0x100 the poll returns the chip status (0x80 busy, 0xe0 ready) and eight pages programmed in one run are all written.	differs	M2-196;

M1 ID	On M1	On M2	verdict	evidence
M1-F124	On M1 reprogramming a page without an erase stores old AND new.	On M2 this is a property of the attached Hynix part with no measurement; the M2 controller writes raw pages exactly after an erase.	m2-only-context	M2-84
M1-F125	On M1 ECC_MODE bits 20:16 give the number of data bytes encoded (the rest counted as 0xff).	On M2 ECC_CTRL bits 16..23 have no effect on the encode parity.	differs	M2-98
M1-F126	On M1 metadata and parity errors are corrected and counted with data errors, up to 8 in the whole code word.	On M2 BCH8 corrects 1..8 errors in data, metadata, parity and mixtures, and 0x818 counts metadata and parity corrections.	same	M2-84; M
M1-F127	On M1 an erase with four address cycles erases the block.	On M2 an erase with four row-address cycles (FMC_ANUM = 3, fourth byte 0) erases the block on CE0 and CE1: the programmed page reads back all 0xff, the status FAIL bit is clear and the erase time equals the three-cycle erase.	same	M2-168; restore m /root-rest .s)
M1-F128	On M1 a status latch with FMC_DATA_SEL = 0x100 returns the last command byte (0x70).	On M2 a status latch with FMC_DATA_SEL = 0x100 returns the chip status as with DATA_SEL = 1 (idle CE0 0xe0, 0x80 while busy); only with no CE selected does it return 0x70.	differs	M2-169;
M1-F129	On M1 multi-page programs with the status poll on FMC_DATA_SEL = 1 write every page.	On M2 eight pages programmed in one run with the status poll on FMC_DATA_SEL = 1 are all written (every poll 0xe0), as with 0x100 and 0x200.	same	M2-196

M1 ID	On M1	On M2	verdict	evidence
M1-F130	On M1 the (Samsung) status byte is 0xc0 when ready and 0x81 right after 10h, and reprogramming reports no FAIL.	On M2 the Hynix part reads 0xe0 ready and 0x80 busy through the latch.	m2-only-context	M2-87; M
M1-F131	On M1 program time follows the chip's page pairing (about 478 us versus 728..873 us) and an erase takes 1.1 ms with data, 0.18 ms blank.	On M2 the Hynix part stays busy about 325 us for a program and about 2.27 ms for an erase in the measured cases; page pairing has no measurement.	m2-only-context	M2-87; M
M1-F132	On M1 the ECC_MODE length field is bits 20:16 (1..31, 0 = 512), and a decode with it checks only the first L bytes.	On M2 there is no data-length field: ECC_CTRL bits 16..23 have no effect and a sector is 512 bytes.	differs	M2-98; M
M1-F133	On M1 ECC_MODE bit 15 selects Reed-Solomon over GF(2^10), byte symbols, 8 roots, 10 parity bytes, no metadata.	On M2 every ECC_CTRL encode word measured yields BCH8 or BCH16 parity (as bit 2 selects) or none, never the M1 Reed-Solomon parity: single bits 3..7 and 16..31, every pair of them and all together with BCH8 and BCH16, and metadata lengths 0x80/0xff; metadata lengths 33..0xff give the parity of the data alone, as length 0, while lengths up to 32 cover that many meta data-stream bytes.	differs	M2-197; chapter 1
M1-F134	On M1 the 8-symbol Reed-Solomon code corrects up to 4 bad bytes and 5 or more are uncorrectable.	On M2 no Reed-Solomon mode is established; the measured codes are BCH8 (8 bits) and BCH16 (16 bits).	differs	M2-84; M

M1 ID	On M1	On M2	verdict	evidence
M1-F135	On M1 a chip's busy-to-ready transition re-arms `wait 5` (two-chip polls in both orders) and address cycles re-arm it.	On M2 a second latch needs no re-arm (after only the bit-23 acknowledge it passes wait 5), and address cycles after a latch without the 0x30020 release do not run (wait 1 times out, FMC_STATUS bit 26 set), so they re-arm nothing; two-chip polls during an erase pass in both orders with the release between the polls, while a second chip's 70h after an unreleased latch is refused at wait 0.	differs	M2-494; M2-237;
M1-F136	On M1 bit 15 with L = 31 encodes RS over 31 bytes, and ECC_MODE bits 2, 3, 4 change the result.	On M2 bit 2 selects BCH16, bits 3 and 4 have no encode effect, and no Reed-Solomon mode is established.	differs	M2-98; M
M1-F137	On M1 modes without a code (bits 8 and 15 clear) write the n-th value of a fixed sequence into 0x900..0x90c and 0x990 per encode.	On M2 every encode is BCH8 or BCH16 and 0x900..0x910 read 0 after encodes.	differs	M2-72; M
M1-F138	On M1, with bit 15, 0x90a..0x913 hold the RS encoder's trace.	On M2 0x900..0x910 read 0 after encodes and no Reed-Solomon mode is established.	differs	M2-72; M
M1-F139	On M1 ECC_DONE bit 3 stays set across encodes, programs and the 0x1ff clear until the next good decode; bit 5 is cleared by ECC_MODE bits 21..24.	On M2 ECC_STATUS bit 3 (uncorrectable) and bit 5 (at least 7 corrected) accumulate across decodes until 0x1ff is written, and 0x81c = 1 sets 0x810 to 0x10.	differs	M2-88; M 10.10

M1 ID	On M1	On M2	verdict	evidence
M1-F140	On M1 VIC0 line 20 appeared in 4 of 102 runs (command with 0x044 bit 0 or 0xffff, status latch with bit 13 or 0xffff), never for address, transfer or DMA sources.	On M2 with 0x044 = all ones VIC0 line 20 rises after a DMA, a command and a status latch (`wait2 20` passes 4-7 us later), and with 0x044 = 0 never.	differs	M2-142
M1-F141	On M1 ECC_MODE bit 4 swaps the bytes of each 32-bit word of the data read and slot write.	On M2 ECC_CTRL bit 4 has no effect on the encode parity.	differs	M2-98
M1-F142	On M1 bits 3:2 with bit 15 select RS-8/12/16 (10/15/20 bytes), correcting 4/6/8 bytes.	On M2 bit 2 selects BCH16 (26 parity bytes, 16 bits), bit 3 has no effect and no Reed-Solomon mode is established.	differs	M2-86; M
M1-F143	On M1 no mode corrects more than 8 scattered bits per sector; RS-16 corrects 8 bad bytes.	On M2 BCH16 corrects up to 16 scattered bit errors per sector in data, metadata and parity.	differs	M2-92
M1-F144	On M1 the no-code sequence has 61 values, no short LFSR, and nothing within a run restarts it.	On M2 no ECC_CTRL value without a code is established and 0x900..0x910 read 0 after encodes.	differs	M2-72; M
M1-F145	On M1 FMC 0x028 stays 0 after the set-up word through instructions, `sti`, `ldi`, DMA, ECC and chip reads; 0x1cc/0x1d0 are not NAND timing.	On M2 FMC 0x028 stays 0 after the set-up word and a READ ID through 32 plain instructions, 128 ldi+sti iterations, a 512-byte DMA in and out, a BCH8 encode and chip reads; setting 0x1b8..0x1d8 (0x1cc/0x1d0 included) to 0 or 0xffffff changes no DMA, ECC, page-read or timeout time.	same	M2-199; WRITE re `mem:0x08` store/fms

M1 ID	On M1	On M2	verdict	evidence
M1-F146	On M1 VIC0 line 20 per single source: a command with 0x044 bit 0 and an address with bit 1 only; VIC1 line 22 never.	On M2 VIC0 line 20 rises, per single 0x044 bit, for that bit's own flag only (command bit 0, address 1, transfer 2, DMA 11, status 12), and VIC1 raw bit 22 rises at every END while C10 is non-zero.	differs	M2-182;
M1-F147	On M1 only `wait2` conditions 5 and 33 passed while an ECC operation or a DMA ran (N = 0..63).	On M2 during ECC and DMA states condition 5 and condition 19 (USB) passed, 16 with DMAC0 activity, and no line 32..63 passed.	differs	M2-74; M
M1-F148	On M1 Reed-Solomon ignores the ECC_KICK slot field, writing parity at byte 0 of the parity buffer, and the ECC area holds it reversed.	On M2 no Reed-Solomon mode is established; BCH parity is read from 0x840..0x85c with reversed word order.	differs	M2-98; M
M1-F149	On M1 0x990 reads 0 after a BCH encode, a data-dependent value with RS, and 0x940..0x950 change neither parity nor decode.	On M2 0x900..0x9fc (0x990 included) read 0 after BCH8 and BCH16 encodes of zero, ones and ramp data and after BCH8/BCH16 decodes (clean, erased-free, two corrected errors); writing 0 or 0xffffffff to 0x940..0x950 before the encode or decode leaves parity, data, 0x810 and 0x818 unchanged.	same	M2-171;
M1-F150	On M1 0x044 routes one flag per bit (0 command, 1 address, 2 transfer, 11 DMA, 12 status) into CS_IRQ bit 4 and VIC0 line 20; CS_IRQ bit 4 stays set after the acknowledgement, line 20 follows the flag.	On M2 0x044 routes one flag per bit (0 command, 1 address, 2 transfer, 11 DMA, 12 status) into CS_IRQ bit 4 and VIC0 line 20; CS_IRQ bit 4 stays set after the acknowledgement while line 20 and wait2 20 follow the flag.	same	M2-200;

M1 ID	On M1	On M2	verdict	evidence
M1-F151	On M1 C14 reloads with start-word bits 15:8 (0xfe..0x7f with one of bits 8..15 cleared).	On M2 a `wait` loads C14 with start-word bits 15:8, which C14 still holds after a wait that passes before the first timeout step (0xff for 0xffb5, 0xfe..0x7f with one of bits 8..15 cleared, 0x07 for 0x07b5, 0xef for 0xefb5 after `wait` 0, 2, 3 and 4) and keeps outside a wait, also after the END, and counts it down by one per timeout step while the wait stalls; a wait that ends after a stall of one or more steps leaves 0x7f.	same	M2-172;
M1-F152	On M1 `wait2` passed only for 5 in every state and 20 with every enable on and a command flag pending; `wait` events 11, 4, 2, 0 and 6 passed in their states.	On M2 `wait2` 5 passes always, 20 after DMA/command/status with 0x044 = all ones, 8, 16 and 19 with timer, DMAC0 and USB activity; `wait` events 11, 4, 2, 0 pass in their states and 6 never.	differs	M2-66; M2-140;
M1-F153	On M1 start-word bit 6 and bits 17..31 change nothing in fetch, memory access, `sig`, DMA+encode, faults, reads, erases and programs, and clearing bit 4, 5 or 7 leaves the timeout unchanged.	On M2 start-word bit 6 changes nothing in fetch, ldi/sti, DMA+BCH8 encode, sig/pause, vectored faults, page reads and a page program except as part of the timeout step field (bits 7:4), and bits 17, 18, 20..31 change nothing; but bit 19 keeps the program from running, and clearing bit 4, 5 or 7 shortens the timeout (63130, 31592, 246 us versus 126434 us).	differs	M2-153;

M1 ID	On M1	On M2	verdict	evidence
M1-F154	On M1 program fetch from internal SRAM is faster than from SDRAM (497 us against 811..828 us for the same loop).	On M2 SRAM and SDRAM placement give the same instruction times within 0.5 ns.	differs	M2-102
M1-F155	On M1 FMC 0x028 is the FIFO read position: each 0x080 read advances it by 4 wrapping at 512, a DMA and the set-up word set it to 0.	On M2 reads of FMC_FIFO 0x080 do not advance FMC 0x028: after 1, 4 and 130 sequencer reads, with and without a READ ID latched, 0x028 keeps its value (0); a DMA and the set-up word leave it 0; it is a 9-bit storage word (0x5a5 written reads 0x1a5).	differs	M2-173; original W `mem:0x08` store/fms
M1-F156	On M1 a status latch passed 160 of 160, a second latch without an FMCTRL1 write timed out 40 of 40 and `wait 3` on CE0 passed 40 of 40.	On M2 the first status latch passed every time (2644 of 2644 first latches, 410 of 410 in 11 forms in each of four captures), a second 70h + latch with no FMCTRL1 write between failed 342 of 342 (its 70h refused at wait 0), and the 16-byte data read on CE0 passed wait 3 in 64 of 64 runs and in a 41-run series.	same	M2-161; M2-31; M
M1-F157	On M1 CS_STATUS read by the program itself is 5 in every state.	On M2 the program reads 5 in every tested state, also with a DMA, ECC operation or command in progress, while the CPU reads the state codes.	same	M2-70; M

M1 ID	On M1	On M2	verdict	evidence
M1-F158	On M1 on a page read CTRL0 bits 9/11/24 change nothing, FMC_DMA_CTRL bit 1 or 2 makes every later DMA time out (readback +bit 6/7), bits 4..7 every DMA, and the value persists between runs.	On M2 page reads are exact with FMC_DMA_CTRL 0, 1, 2, 4, 7 and 8, and a 512-byte DMA completes with every value 0..7 of bits 2:0, more slowly for smaller values; after the DMA bits 12:6 hold a residual count, so 2 and 4 read back 0x42/0x84 (5 → 0x105, 6 → 0x146); any of bits 12:6 set before a DMA (0x40..0x1000, 0x1fc7) keeps it from completing, the written value reading back unchanged, as do bits 5:3 = 2, 4 or 7, while bits 31:13 are not stored; bits 12:6 count up during a DMA (4-byte words, inferred); the reset leaves 7 at every case start, and CTRL0 bits 9/11 change neither time nor data.	differs	M2-76; M M2-398; M2-135;
M1-F159	On M1 FMC_STATUS bits 12, 13, 18, 19 are set after a soft reset, survive writing 1, are cleared by data reads and set again by DMA, latch and set-up word.	On M2 FMC_STATUS bits 12, 13, 18, 19 are set after reset and survive writing 1; a short chip read (5, 16, 23 or 128 bytes) clears 13 and 19 and leaves 12 and 18 set, also setting them again after a 512-byte read, which clears all four (and sets bits 14..17); a DMA in, a status latch and the set-up word set all four again after a short and after a 512-byte read (command, address, metadata encode and DMA out do not).	same	M2-203; chapter 9

M1 ID	On M1	On M2	verdict	evidence
M1-F160	On M1 only WRITECACHE reads the bank count 0xd04, confirming with 15h while pages remain.	On M2 no program reads a bank count; WRITE uses 0xd04 as the failing-CE mask and the 15h cache confirm comes from CPU-built command descriptors.	m2-only-context	original W `mem:0x08` store/fms docs/11-f section 3
M1-F161	On M1 `wait2 20` is true while a command flag is pending with 0x044 bit 0 set, any other 0x044 bit or 0x844 leaves it false, and condition 33 was false in all 34 states.	On M2 wait2 20 with the command flag pending passes with 0x044 bit 0 and with no other single bit; 0x844 does not release it, and after an ECC encode with 0x844 = 0 or all ones it times out; wait2 33 never passed (23 cases).	same	M2-204;
M1-F162	On M1 C58 = 6 with a CPU-written C4C diverts the first end once, C58 = 3 every end after the first, C58 = 2 none.	On M2 C58 = 6 with a CPU-written C4C (either write order) does not divert the first END, and neither C58 = 3 nor C58 = 2 diverts any END.	differs	M2-149;
M1-F163	On M1 `wait 8` passed during a second-set DMA and after a CE0 page read through the first set; the first program after a power cycle passed `wait 4`.	On M2 wait 8 never passes: idle, after a one-sector CE0 page read through the first set (data exact; C58 = 0 or 4, after WAIT5 or a WAIT12 ready poll), after a complete page read with DMA out, and after a DMA through +0x404.	differs	M2-206; M2-111

M1 ID	On M1	On M2	verdict	evidence
M1-F164	On M1 FMC_FIFO returned 0 in 130 reads from DMA-filled buffers and 128 writes after FMCTRL1 = 0x340 changed 26 of 128 words of data buffer 0.	On M2 FMC_FIFO 0x080 reads return 0 from DMA-filled buffers, as on M1, but writes to it never reach data buffer 0: without 0x340, 128 writes leave the buffer unchanged; after FMCTRL1 0x340 with FMC_DATA_SEL 1 buffer 0 is filled from the RAM that follows the 0x340's zero output (FMC_DMA_ADDR + 0x200 on): 384 bytes when the data-out set-up follows at once, all 512 when 32 or 128 FIFO writes come between, and no written value appears.	differs	M2-617; M2-613;
M1-F165	On M1 RS pages written by the engine keep the slot as it was and only sector 0 carried parity, decoding as in the buffers.	On M2 no Reed-Solomon mode is established; engine-written pages carry BCH8 or BCH16 parity for every sector.	differs	M2-77; M
M1-F166	On M1 an encode in the same code as the last decode leaves ECC_RESULT unchanged, while switching between RS and BCH sets it to 0 or 0x30000.	On M2 0x80c is an eight-slot ring of decode results whose read pointer advances on every read, encode stages read 0x80c = 0, and there is no Reed-Solomon code.	differs	M2-72; M

M1 ID	On M1	On M2	verdict	evidence
M1-F167	On M1 programs ending with an FMC flag pending and its 0x044 enable set stopped the device from answering USB; acknowledged and cleared versions ran normally.	On M2 an END with an enabled FMC flag pending (0x044 = 1 or all ones) does not stop USB: every following marker case answered (5 of 5 after each, as after the acknowledged control); the probe resets the engine after the END, so the pending state lasts only until that cleanup.	differs	M2-208;
M1-F168	On M1 no decode reported success with wrong data in 240 BCH sectors with 9..64 bad bits and 240 RS-8 sectors with 5..40 bad bytes.	On M2 no decode reports success with wrong data: 13440 BCH8 sectors with 9..64 and 11520 BCH16 sectors with 17..64 random bit errors (240 per count) all end with 0x810 bit 3 (uncorrectable) and the data unchanged.	same	M2-174;
M1-F169	On M1 the second set has its own data and parity buffers 0 and 1, shares parity buffer 0 with the first set, and its set-up word left the first set's parity buffer 0 all 0xff.	On M2 the +0x400 set has no own data or parity buffer: its loopback moves nothing in either slot, and a +0x400 parity DMA leaves the first set's parity buffer 0 unchanged.	differs	M2-185;
M1-F170	On M1 C50 bit 6 is `wait 6` pending after a second-set command, and bit 8 never appeared.	On M2 C50 bits 6 and 8 never appear, also in eight samples right after a 70h or 90h written through +0x408 and after a full +0x400 READ ID, whose waits 6, 7 and 9 time out.	differs	M2-183; M2-111;

M1 ID	On M1	On M2	verdict	evidence
M1-F171	On M1 four `tmwait` in one program alternate: a full wait from 0xff leaving 0, then an immediate reload (about 9.7 ms for four).	On M2 four `tmwait` do not alternate: the first `tmwait/15` waits the full period from bits 15:8 (126.1 ms from 0x7f, i.e. 256 counts) and leaves C14 = 0, and the next one started from 0 waits again instead of reloading at once (still waiting >= 33900 us later, C14 counting down from 0xff, 7 runs).	differs	M2-150;
M1-F172	On M1 no writable 0x100-block word changes DMA, ECC, chip-read or timeout times or `wait2` 5/33, and 0x3a8..0x3b4 read 0xffffffff from the sequencer and 0 from the CPU.	On M2 setting 0x1b8..0x1d8 to 0 or 0xffffff changes no DMA, BCH8 encode, page-open + read or unmet-wait time and no data, but 0x3a8..0x3b4 read 0x12 from the sequencer (M1 0xffffffff) and 0 from the CPU.	differs	M2-210;
M1-F173	On M1 the unit's boot-ROM straps P3.6 and P3.7 read 0 and 0, selecting SPI-flash boot on SPI port 0.	On M2 the boot ROM reads straps GPIO 3.5/3.6/3.7 as sel = p5 << 2 | p6 << 1 | p7 (0..3 SPI NOR, 4..6 NAND bank 0, 7 DFU); the test unit boots its bootloader from NAND, which in this ROM is sel 4..6 (inferred).	m2-only-context	docs/01- (`rom_mai`

M1 ID	On M1	On M2	verdict	evidence
M1-F174	On M1 `wait2 N` is true while VIC0 line N (0..31) or VIC1 line N - 32 (32..63) is raised; 64..255 never pass; all 64 matched the raw VIC status.	On M2 wait2 N follows VIC0 hardware lines only: lines raised by software never release it, wait2 32..63 never passed in any state (also with a VIC1 soft interrupt, and wait2 22, 53, 54, 55 while the CPU sampled VIC1 raw bit 22 raised, in four captures), and 64..255 never pass; whether wait2 32 + k follows a VIC1 line k asserted by a peripheral is not measured.	differs	M2-211; M2-142
M1-F175	On M1 all FMC sources share VIC0 line 20, the ECC sources enabled in 0x844 raise source 43 (VIC1 line 11), and sources 5 and 33 are outside devices.	On M2 the FMC sources share VIC0 line 20, but no ECC interrupt source was found: after BCH8 encodes and decodes with every tested ECC word written (0x800..0x9fc, six values, 0x044 = 0 or all ones) no VIC line rises beyond the baseline, source 43 (VIC1 line 11) never rises, wait2 20 and 43 never pass, and 0x844 is an encode-parity word; line 5 is always asserted and 33 never passed.	differs	M2-212; M2-140;

19.3 Counts

verdict	findings
same	45
differs	120
m2-only-context	10
unresolved	0
total	175

19.4 Experiment keys

Each key names one device experiment in the registry; its starting state, stimulus, observable and decision rule are in `validation/m1-m2-crosswalk.json`.

- `m2_m1_bus_error` — DMA and ldi to an unmapped bus address
- `m2_m1_wait2_vic1` — wait2 conditions 32..63 against VIC1 hardware lines (M1-F174)
- `m2_m1_ecc_alt_code` — Multi-bit ECC_CTRL combinations for any code other than BCH8/BCH16 (M1-F133)